

임베디드 시스템을 위한 SW 구조 설계

(file #2 / 10) ver 0.2

Yongseok Chi

Course Objectives

1. Develop an understanding of technologies

about the micro controller & processor systems using evaluation kit

2. Skill up a **design ability the micro controller application systems**

(1) technology of **the hardware and software** components

(2) **debugging** technology about the micro controller

(3) understanding of a **circuit design** skill

3. Develop an ability of design about the micro controller

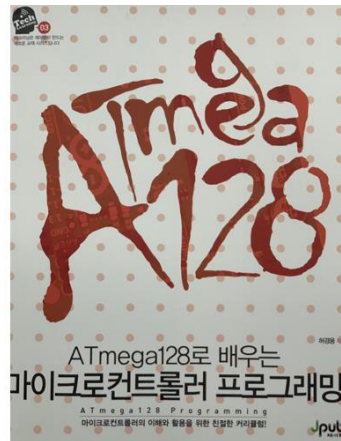
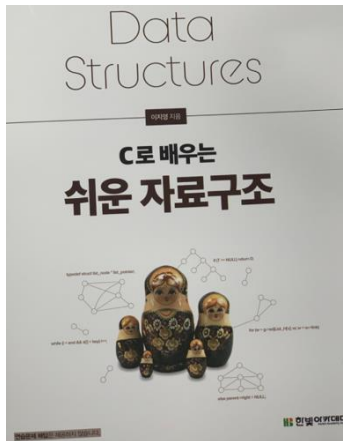
Reference

1. Reference information

(1) Atmel datasheet, <http://atmel.com> (→ microchip.com)

(2) ARM Architecture Reference Manual, <http://arm.com>

2. Books



3. 실험KIT (Evaluation board)

한백전자 IOT 실험 KIT



KUT-128_Com 실험 키트

Course Objectives

Ref. <https://youtu.be/r2ZLAu4mTSI>

World robot summit

: Standard Disaster Robot Challenge

<https://youtu.be/r2ZLAu4mTSI>

: Plant Disaster Prevention Challenge

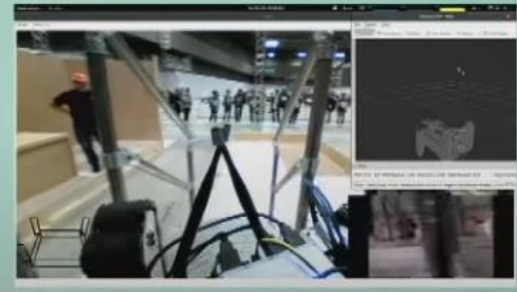
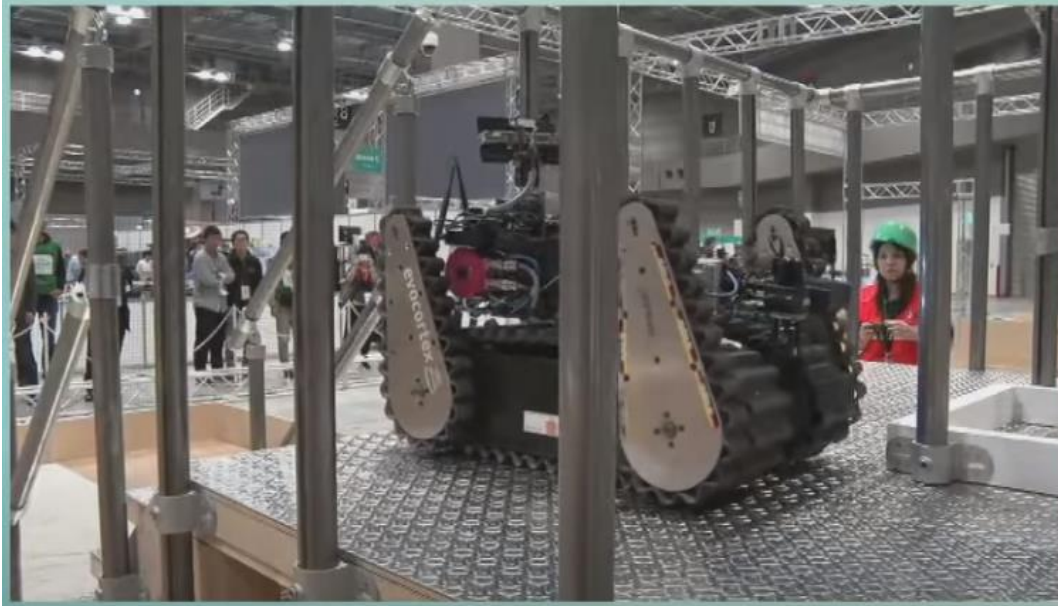
https://youtu.be/GoUN_k90h_A



Course Objectives

Ref. <https://youtu.be/r2ZLAu4mTSI>

World robot summit





atmega128



My Account ▾



PRODUCTS

SOLUTIONS

TOOLS AND RESOURCES

SUPPORT

EDUCATION

ABOUT

ORDER NOW

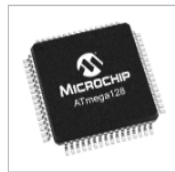
< Hide Filters

Refine your search

[All Microchip Content](#)[Application Center](#)[Products](#)[^ Product Documents](#)[Data Sheets](#)[Errata](#)[Package Drawings](#)[Reference Manuals](#)[User Guides](#)[Application Notes](#)[Product Change Notification](#)[Corporate Information](#)

Results 1 - 10 of 48 for "atmega128"

ATmega128 - 8-bit AVR Microcontrollers



The high-performance, low-power Microchip 8-bit AVR® RISC-based microcontroller combines 128 KB of programmable flash memory, 4 KB SRAM, a 4 KB EEPROM, an 8-channel 10-bit A/D converter, and a JTAG interface for on-chip debugging. The d...[\[More Info\]](#)



Buy



Samples



Document

ATmega1284RFR2 - Wireless Modules

An IEEE 802.15.4 compliant single chip combines an industry-leading AVR microcontroller and best-in-class 2.4GHz RF transceiver. It offers the industry's highest RF performance for single chip devices, with a link budget of 103.5dBm while c...[\[More Info\]](#)



Buy



Samples



Document

Project 1

제출 : 프로그램

- : 아래 내용을 하나의 프로그램을 제출(main 함수가 1개)
- : 주어진 조건 외에는 자율로 프로그래밍 가능
- : 모든 프로그램에 주석 넣기(주석이 없으면 감점)

제출 : 영어이름. C를 제출할 것 (예, hongkildong.c)

프로그램 발표 및 동작 시연

- : 수업시간 발표

주제 : 프로그램(GPIO 기능으로 프로그램하기)

- : IIC BUS WRITE - 1byte write mode
- : IIC BUS READ - 1byte read mode

Project 2

• Stabilization of SW - MISRA (Motor Industry Software Reliability Association)

: MISRA

“Guidelines for the use of the C language in vehicle based software” – 1998, 영국산업신뢰성협회

S.No	MISRA Rule#	Description
1	MISRA rule 5	Only those characters and escape sequences which are defined in the ISO C standard shall be used.
2	MISRA rule 7	Trigraphs shall not be used.
3	MISRA rule 8	Multibyte characters and wide string literals shall not be used.
4	MISRA rule 9	Comments shall not be nested.
5	MISRA rule 10	Sections of code should not be 'commented out'
6	MISRA rule 13	The basic types of char, int, long, float and double should not be used, but specific length equivalents should be typedefed for the specific compiler (and microprocessor).
7	MISRA rule 14	The type char shall always be declared as unsigned char or signed char.
8	MISRA rule 16	The underlying bit representations of floating point numbers shall not be used in any way by the programmer.
9	MISRA rule 17	typedef names shall not be reused.
10	MISRA rule 19	Octal constants (other than zero) shall not be used.
11	MISRA rule 20	All object and function identifiers shall be declared before use.
12	MISRA rule 21	Identifiers in an inner scope shall not use the same name as an identifier in an outer scope and therefore hide that identifier.
13	MISRA rule 22	Declarations of objects should be at function scope unless a wider scope is necessary.
14	MISRA rule 23	All declarations at file scope should be static where possible.
15	MISRA rule 25	An identifier with external linkage shall have exactly one external definition.
16	MISRA rule 26	If objects or functions are declared more than once they shall have compatible declarations
17	MISRA rule 27	External objects should not be declared in more than one file.
18	MISRA rule 29	The use of a tag shall agree with ist declaration.
19	MISRA rule 30	All automatic variables shall have been assigned a value before being used.
20	MISRA rule 31	Braces shall be used to indicate and match the structure in the non-zero initialisation of arrays and structures.
21	MISRA rule 32	In an enumerator list, the "=" construct shall not be used to explicitly initialise members other than the first, unless all items are explicitly initialised.
22	MISRA rule 33	The right hand operand of a && or operator shall not contain side effects.
23	MISRA rule 34	The operands of a logical && or shall be primary expressions
24	MISRA rule 35	Assignment operators shall not be used in expressions which return Boolean values.
25	MISRA rule 37	Bitwise operations shall not be performed on signed integer types.
26	MISRA rule 38	The right hand operand of a shift operator shall lie between zero and one less than the width in bits of the lefthand operand (inclusive).
27	MISRA rule 39	The unary minus operator shall not be applied to an unsigned expression.
28	MISRA rule 40	The sizeof operator should not be used on expressions that contain side effects.
29	MISRA rule 43	Implicit conversions which may result in a loss of information shall not be used.
30	MISRA rule 45	Type casting from any type to or from pointers shall not be used.
31	MISRA rule 46	The value of an expression shall be the same under any order of evaluation that the standard permits.
32	MISRA rule 48	Mixed precision arithmetic should use explicit casting to generate the desired result.
33	MISRA rule 50	Floating point variables shall not be tested for exact equality or inequality.
34	MISRA rule 52	There shall be no unreachable code.
35	MISRA rule 56	The goto statement shall not be used.
36	MISRA rule 59	The statements forming the body of an if, else if, else, while, do... While or for statement shall always be enclosed in braces.
37	MISRA rule 61	Every non-empty case clause in a switch statement shall be terminated with a break statement.
38	MISRA rule 62	All switch statements should contain a final default clause.
39	MISRA rule 64	Every switch statement shall have at least one case.
40	MISRA rule 65	Floating point variables shall not be used as loop counters.
41	MISRA rule 68	Functions shall always be declared at file scope.
42	MISRA rule 69	Functions with variable numbers of arguments shall not be used.
43	MISRA rule 70	Functions shall not call themselves, either directly or indirectly.
44	MISRA rule 71	Functions shall always have prototype declarations and the prototype shall be visible at both the function definition and call.
45	MISRA rule 75	Every function shall have an explicit return type.
46	MISRA rule 76	Functions with no parameters shall be declared with parameter type void.
47	MISRA rule 78	The number of parameters passed to a function shall match the function prototype.
48	MISRA rule 79	The values returned by void functions shall not be used.
49	MISRA rule 80	Void expressions shall not be passed as function parameters.
50	MISRA rule 81	const qualification should be used on function parameters which are passed by reference, where it is intended that the function will not modify the parameter.
51	MISRA rule 83	For functions with non-void return type
52		i, there shall be one return statement for every exit branch (including the end of program)
53		ii, each return shall have an expression
54		iii, the return expression shall match the declared return type.
55	MISRA rule 84	For functions with void return type, return statements shall not have an expression.
56	MISRA rule 85	Functions called with no parameters should have empty parentheses.
57	MISRA rule 87	#include statements in a file shall only be preceded by other preprocessor directives or comments.
58	MISRA rule 88	Non-standard characters shall not occur in header file names in #include directives.
59	MISRA rule 89	The #include directive shall be followed by either a <filename> or "filename" sequence.
60	MISRA rule 91	Macros shall not be #ifndef and #undef within a block.
61	MISRA rule 94	A function-like macro shall not be 'called' without all of its arguments.
62	MISRA rule 95	Arguments to a function-like macro shall not contain tokens that look like preprocessing directives.
63	MISRA rule 96	In the definition of a function-like macro the whole definition, and each instance of a parameter, shall be enclosed in parentheses.
64	MISRA rule 98	There shall be at most one occurrence of the # or ## pre-processor operators in a single macro definition.
65	MISRA rule 99	All uses of the #pragma directive shall be documented and explained.
66	MISRA rule 100	The defined pre-processor operator shall only be used in one of the two standard forms.
67	MISRA rule 102	No more than 2 levels of pointer indirection should be used.
68	MISRA rule 103	Relational operators shall not be applied to pointer types except where both operands are of the same type and point to the same array, structure or union.
69	MISRA rule 104	Non-constant pointers to functions shall not be used.
70	MISRA rule 105	All the functions pointed to by a single pointer to function shall be identical in the number and type of parameters and the return type.
71	MISRA rule 106	The address of an object with automatic storage shall not be assigned to and object which may persist after the object has ceased to exist.
72	MISRA rule 107	The null pointer shall not be de-referenced.

Abbreviation	description	Criteria
N1	Total Number of operator	
N2	Total number of operands	
STMT	Number of statement	< =50
VG	Cyclomatic Number (VG)	< =20
AVGS	Average size of statement	< =9
GOTO	Number of GOTO statement	0
RETU	Number of Return Statement	1
NBCALLING	Number of caller	< =10
PATH	Number of path	< =1000
PARA	Number of function parameter	< =5
ap_cg_cycle	call Graph recursions	0
	dead code	
DRCT_CALLS	Number of calls direct	
LEVL	Number of Level	



Project 3

Project 4

제출 : 프로그램

: 아래 내용을 하나의 프로그램을 제출(main 함수가 1개)

: 주어진 조건 외에는 자율로 프로그래밍 가능

: 모든 프로그램에 주석 넣기(주석이 없으면 감점)

제출 : 영어이름. C를 제출할 것 (예, hongkildong.c)

프로그램 발표 및 동작 시연

: 수업시간 발표

주제 : UART 모니터링 프로그램

제출 : 프로그램

: 아래 내용을 하나의 프로그램을 제출(main 함수가 1개)

: 주어진 조건 외에는 자율로 프로그래밍 가능

: 모든 프로그램에 주석 넣기(주석이 없으면 감점)

제출 : 영어이름. C를 제출할 것 (예, hongkildong.c)

프로그램 발표 및 동작 시연

: 수업시간 발표

주제 : DAC 프로그램

1. The Principle of Micro Processor

마이크로 프로세서

- 마이크로(**micro**)와 프로세서(**processor**)가 결합된 용어
- ‘매우 작은’(**micro**)이라는 의미와 ‘처리기’(**processor**)
- 크기가 매우 작고, 뛰어난 계산 능력을 가진 장치
- **IC 집적기술, 컴퓨터 구조기술, 시스템 프로그래밍 기술**을 함께 묶어 단일 칩으로 **집적화한 반도체 소자**
- 재료, 수학적 개념, 전자 집약기술, 사회적 요구를 수렴한 다양한 마이크로프로세서가 사용되고 있으며, 앞으로도 꾸준한 연구를 통해 더욱 향상된 마이크로프로세서가 등장
- 프로그램을 신속하게 실행하기 위한 목적으로, 내부 구조를 최적화

1. The Principle of Micro Processor

마이크로 컨트롤러

- 마이크로(micro)와 컨트롤러(controller)가 결합된 용어
- ‘매우 작은’(micro)이라는 의미와 ‘제어기’(controller)라는 의미
- 마이크로 프로세서의 연산 처리 기능에 제어 기능 추가
- 프로그램을 실행하면서 장치를 효과적으로 제어하기 위한 목적으로, 내부 구조를 최적화
- 값싼 장난감에서부터 산업용 장치에 이르기까지 넓은 범주를 대상
- 주변 장치

: 외부 디지털 전압에 대한 입출력 기능

: 메모리 기능(플래시 메모리, SRAM, EEPROM 등)

: 타이머 기능, PWM(Pulse Width Modulation) 펄스 생성 기능

: A/D(Analog Digital) 변환 기능, 통신 기능 등

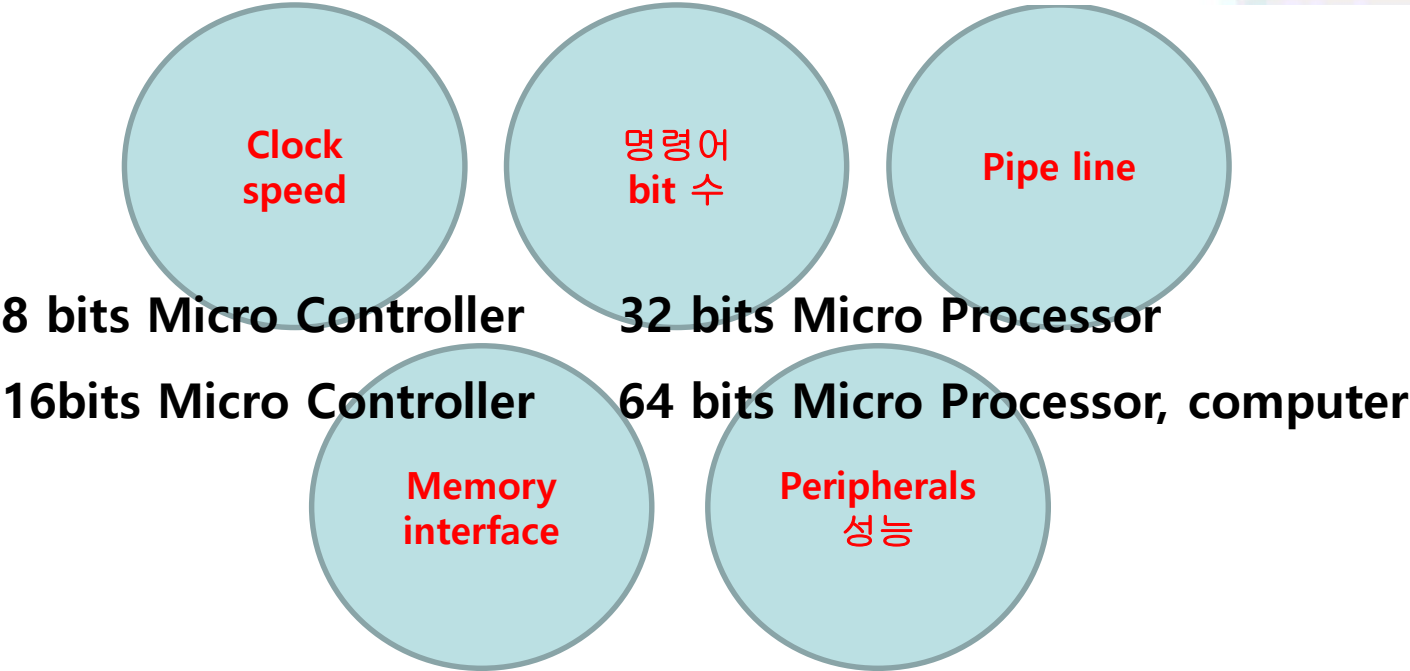
1. The Principle of Micro Processor

마이크로컴퓨터

- 마이크로(**micro**)와 ‘컴퓨터(**computer**)’가 결합된 용어
- ‘매우 작은(**micro**)’ 컴퓨터 시스템
- 마이크로컴퓨터는 상대적으로 대형 슈퍼컴퓨터와 대비
- (사례) 데스크톱 컴퓨터, 휴대용 노트북, 간단한 제어용 컴퓨터, **PMP**, 스마트 폰 등

1. The Principle of Micro Processor

Definition :



Frequency			1clock(sec)					1000 × usec	
10MHz	10	1000000	1/10MHz	1/10	0.000001	0.100	usec	nsec	100
25MHz	25	1000000	1/25MHz	1/25	0.000001	0.040	usec	nsec	40
30MHz	30	1000000	1/30MHz	1/30	0.000001	0.033	usec	nsec	33
60MHz	60	1000000	1/60MHz	1/60	0.000001	0.017	usec	nsec	17
100MHz	100	1000000	1/100MHz	1/100	0.000001	0.010	usec	nsec	10
200MHz	200	1000000	1/200MHz	1/200	0.000001	0.005	usec	nsec	5

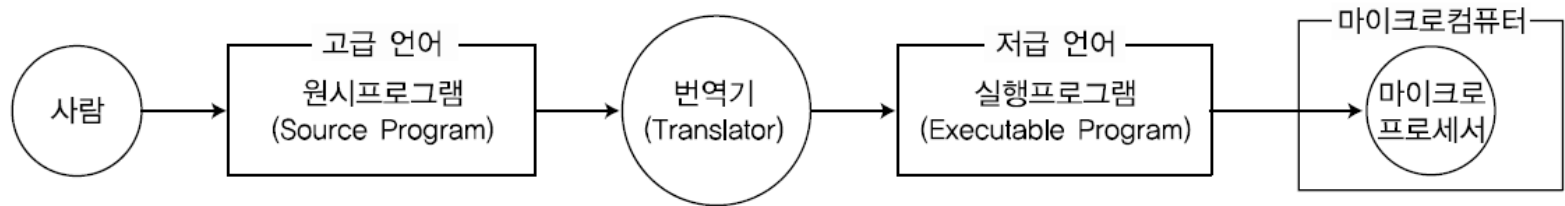
1. The Principle of Micro Processor

Compiler 필요성

: 컴퓨터에 내장된 마이크로프로세서는 **2진(0 또는 1)값을 갖는 명령어**가 차례대로 실행되면서 작업을 처리

: 프로그램

- 작업을 처리하기 위해 명령어를 차례로 배열한 것



: 원시프로그램 : 사람이 작성한 프로그램(고급 언어로 보통 작성)

: 번역기(Translator)

- 프로그램을 **Low level language**인 **Assembler** 또는 **2진 명령어로 변환**
- 컴퓨터가 인식할 수 있는 **기계어(Machine language)**의 **실행프로그램(Executable Program)**으로 변환

: 기계어 ➡ 마이크로프로세서 내부의 실행 동작을 명령어 단위로 표현

- 명령어를 이해하면 마이크로프로세서 내부 구조를 쉽게 이해

1. The Principle of Micro Processor

기계어와 어셈블리어

: 기계어(Machine Language)

- 특정 비트에, 특정 의미가 있는 **2진 값**을 설정하는 명령어를 나열

: 어셈블리어(Assembly Language)

- 기계어를 사람이 연상하기 쉬운 **니모닉(Mnemonic; 기억의)**과 **연산 대상**이라는 영문 단어로 바꿔 표현

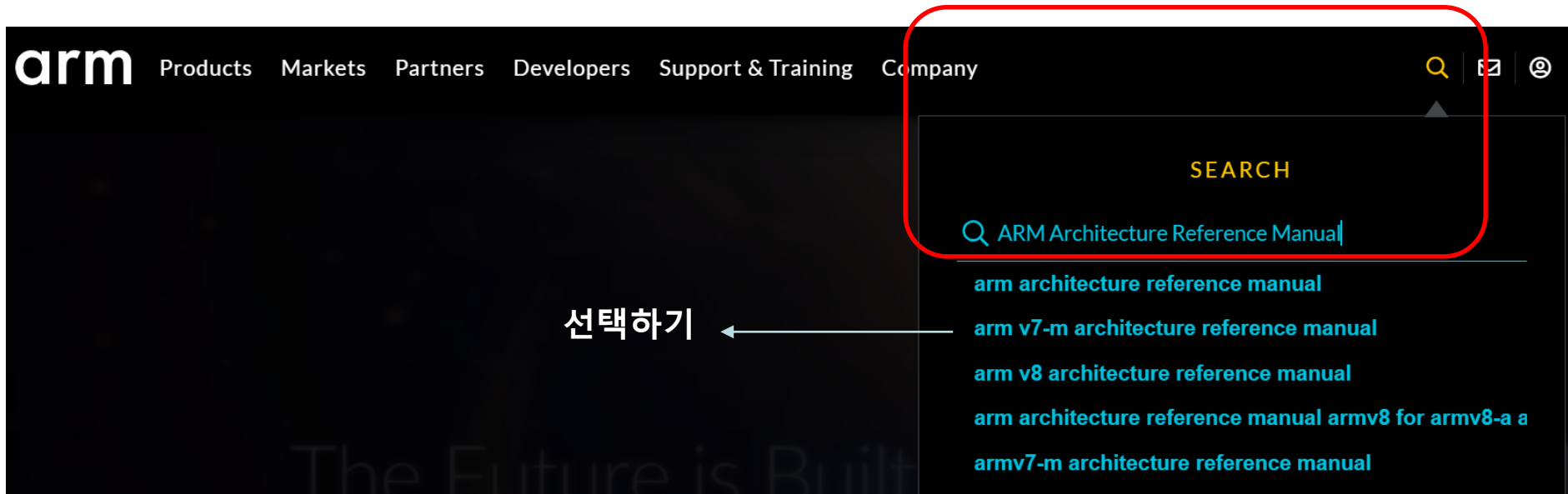
어셈블리어	기계어
movw r30, r22	1111 1101 0000 0001
subi r30, 0x00	1110 0000 0101 0000

: 어셈블러(Assembler)

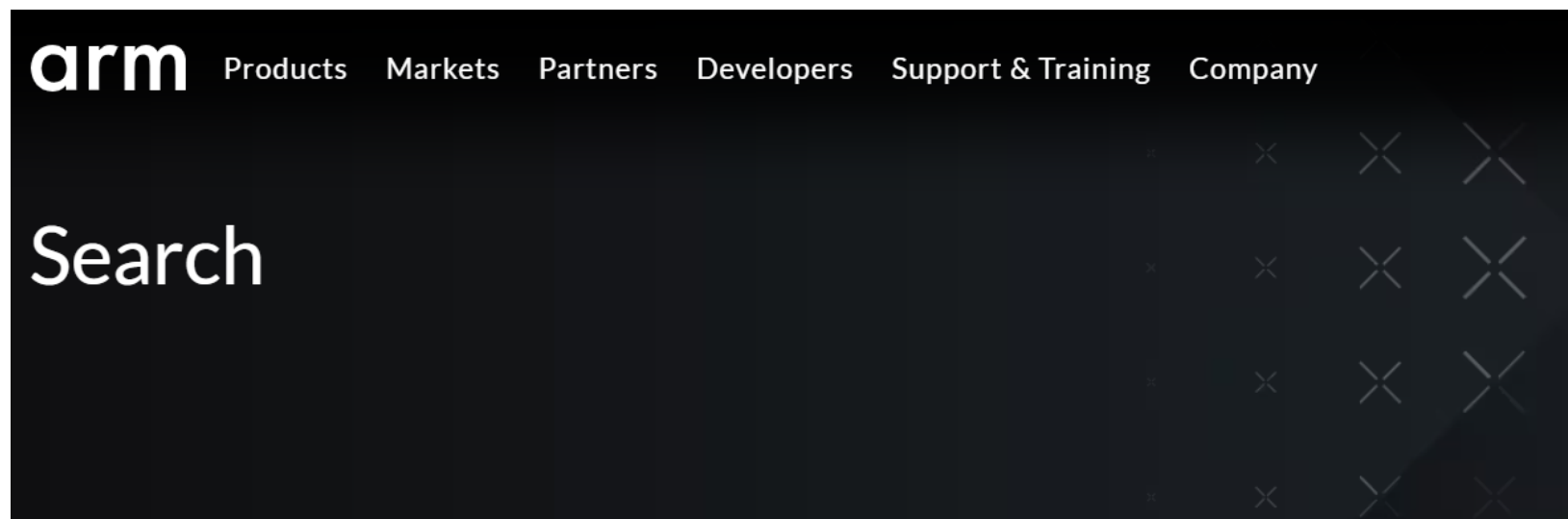
- 어셈블리어로 작성된 원시프로그램을 기계어로 변환시키는 번역기

1. The Principle of Micro Processor

arm.com 에서 “**ARM Architecture Reference Manual**” 입력하기



1. The Principle of Micro Processor



Products

- | | | |
|--------------------------|-----------|-----|
| ☐ | Cortex-M | 207 |
| ☐ | Cortex-A | 201 |
| ☐ | Mali GPUs | 97 |
| ☐ | Cortex-R | 81 |
| ☐ | System IP | 50 |

Markets and Technologies

- | | | |
|--------------------------|--------------------------|-----|
| ☐ | Internet of Things (IoT) | 251 |
| ☐ | Artificial Intelligence | 226 |
| ☐ | Automotive | 222 |

arm v7-m architecture reference manual

Results 1-10 of 1,409 for arm v7-m architecture reference manual

Cortex-M7 – Arm® 선택하기

<https://www.arm.com/products/silicon-ip-cpu/cortex-m/cortex-m7>

Key Documentation Cortex-M7 Technical **Reference Manual** Cortex-M7 Processor D and flexible enough to run any type of ML workload, today or in the ...

Arm Architecture – Arm®

<https://www.arm.com/architecture>

ARCHITECTURE The Basis for Innovation in the Digital World ... Arm views security as the design and it provides security **architectures** and the ...

Arm CPU Architecture – Arm®

1. The Principle of Micro Processor



arm

Products

Markets

Partners

Developers

Support & Training

밑으로 drag

Specifications

The Cortex-M7 enables partners to build the most sophisticated variety of MCUs and embedded SoCs. Its industry leading high-performance and flexible system interfaces are ideal for a wide variety of application areas including automotive, industrial automation, medical devices, high-end audio, image and voice processing, sensor fusion and motor control.

[Get Developer Resources for more details.](#)

Key Documentation

[Cortex-M7 Technical Reference Manual](#)

[Cortex-M7 Processor Datasheet](#)

Compare the specifications of Cortex-M processors:

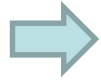
[Download Comparison PDF](#)

선택하여 download

"DDI0489F_cortex_m7_trm.pdf"

Adobe acrobat reader에서 열기

1. The Principle of Micro Processor



- 🔖 **Arm Cortex-M7 Processor Technical Reference Manual**
- 🔖 [Contents](#)
- > 🔖 Preface
- > 🔖 **1: Introduction**
- > 🔖 2: Programmers Model
- > 🔖 3: System Control
- > 🔖 4: Initialization
- > 🔖 5: Memory System
- > 🔖 6: Memory Protection Unit
- > 🔖 7: Nested Vectored Interrupt Controller
- > 🔖 8: Floating Point Unit
- > 🔖 9: Debug
- > 🔖 10: Cross Trigger Interface
- > 🔖 11: Data Watchpoint and Trace Unit
- > 🔖 12: Instrumentation Trace Macrocell Unit
- > 🔖 13: Cortex-M7 Trace Port Interface Unit
- > 🔖 14: Fault detection and handling
- 🔖 A: Revisions

1. The Principle of Micro Processor

프로그램에서 실행 파일 생성

➤ 컴파일(Compile), 컴파일러

- 고급언어로 작성된 원시프로그램을 기계어 목적 파일로 번역하는 과정
- 컴파일러 : 컴파일을 수행하는 번역기

➤ 어셈블(Assemble), 어셈블러

- 어셈블리어로 작성된 원시프로그램을 기계어 목적 파일로 번역하는 과정
- 어셈블러 : 어셈블을 수행하는 번역기

➤ 링크(Link), 링커

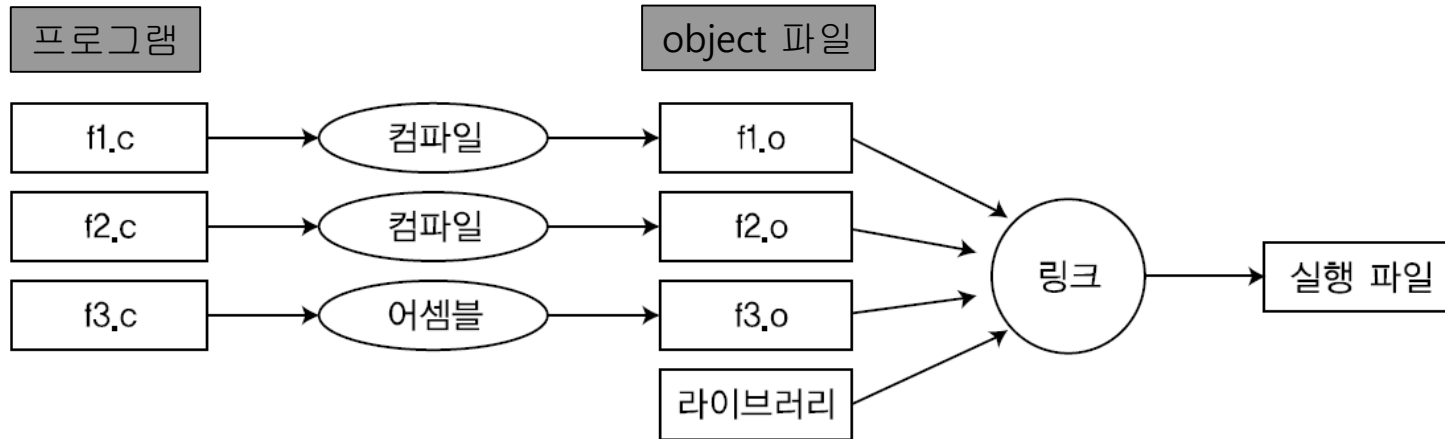
- 여러 개의 목적 파일을 연결하여 통합된 실행 파일로 만드는 과정
- 링커 : 링크를 전담하는 프로그램

➤ 라이브러리

- 공통으로 자주 사용하는 기능은 표준화된 이름과 매개 변수가 있는 함수로 작성한 후 , 컴파일하여 목적 파일을 생성
- 컴파일된 목적 파일을 모아놓은 파일을 '라이브러리 파일'이라 함
- (예시) sin, cos, tan, printf 등과 같이 자주 사용하는 함수

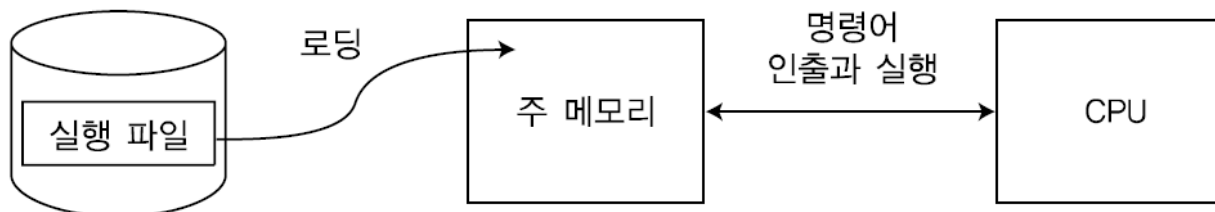
1. The Principle of Micro Processor

➤ 실행 파일이 생성되는 과정



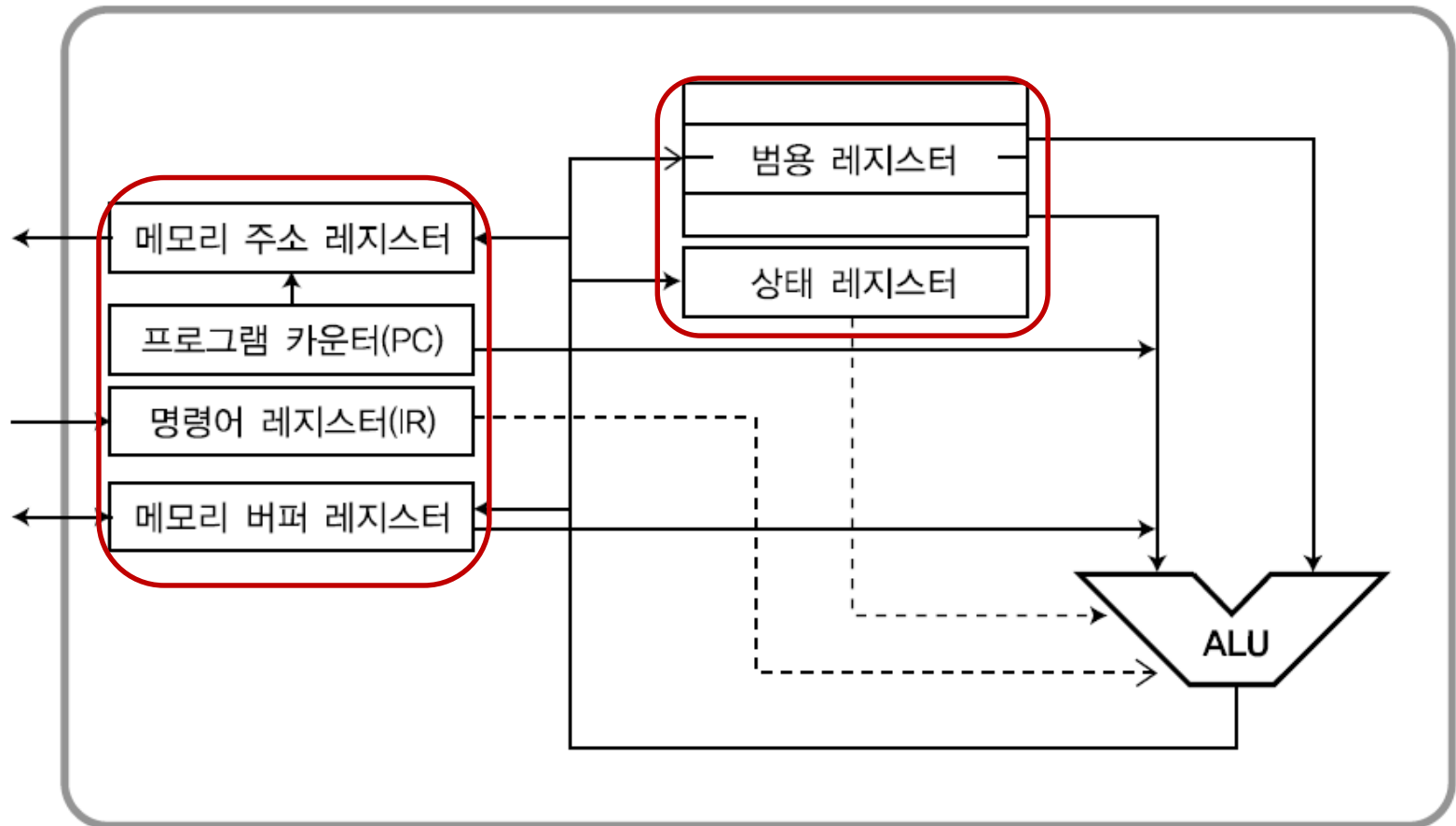
➤ 로딩

- 실행 파일이 중앙 처리 장치(CPU, Central Processing Unit)에서 실행되려면, 주 메모리에 탑재되는 과정 필요
- 로더 : 로딩을 전담하는 프로그램 (**Boot loader**)



1. The Principle of Micro Processor

- 특수 레지스터, 범용 레지스터, CPU 외부의 메모리 사이에
ALU(Arithmetic Logic Unit; 산술로직)를 통한 연산을 통해 명령어 실행



1. The Principle of Micro Processor

특수 레지스터와 범용 레지스터

➤ General Register (범용 레지스터)

- CPU 연산을 빠르게 처리하기 위해 ALU와 직접 연결 (R0, R1...)
- 연산 대상이 되는 오퍼랜드 값을 가짐

➤ SFR(Special Function Register) 특수 레지스터

- 명령어를 실행할 때 필요한 범용 데이터가 아닌 특수한 데이터를 처리하기 위한 레지스터
- 프로그램 카운터, 명령어 레지스터, 상태 레지스터, 메모리 주소 레지스터, 메모리 버퍼 레지스터 등

➤ PC(Program Counter) 프로그램 카운터

- 인출할 명령어가 있는 메모리의 주소를 갖는 특수 레지스터
- 프로그램 메모리에서 한 개의 명령어 인출이 끝나면, 그 명령어의 크기가 더해진 값으로 자동 변경되어 다음 명령어 인출을 위한 주소를 가짐

1. The Principle of Micro Processor

➤ 명령어 레지스터(IR, Instruction Counter)

- 프로그램 메모리에서 인출된 명령어를 기억하고 있는 특수 레지스터
- 인출된 명령어에는 특정 비트에 연산 동작(opcode), 연산 대상(operand), 연산 결과(result), 어드레싱 모드 등의 정보를 설정

➤ 상태 레지스터(Status Register)

- 명령어를 실행한 후의 연산 결과 정보를 기록
- 기록된 상태 레지스터의 각 비트는 연속되는 다음 명령어 실행에 영향을 미침

1	if(a >= 100)	cpi	r24, 0x64
2	a = 0x10;	brcs	.+6
3		ldi	r24, 0x10

- (예시) **brcs .+6** 명령은 **cpi** 명령 실행 후,

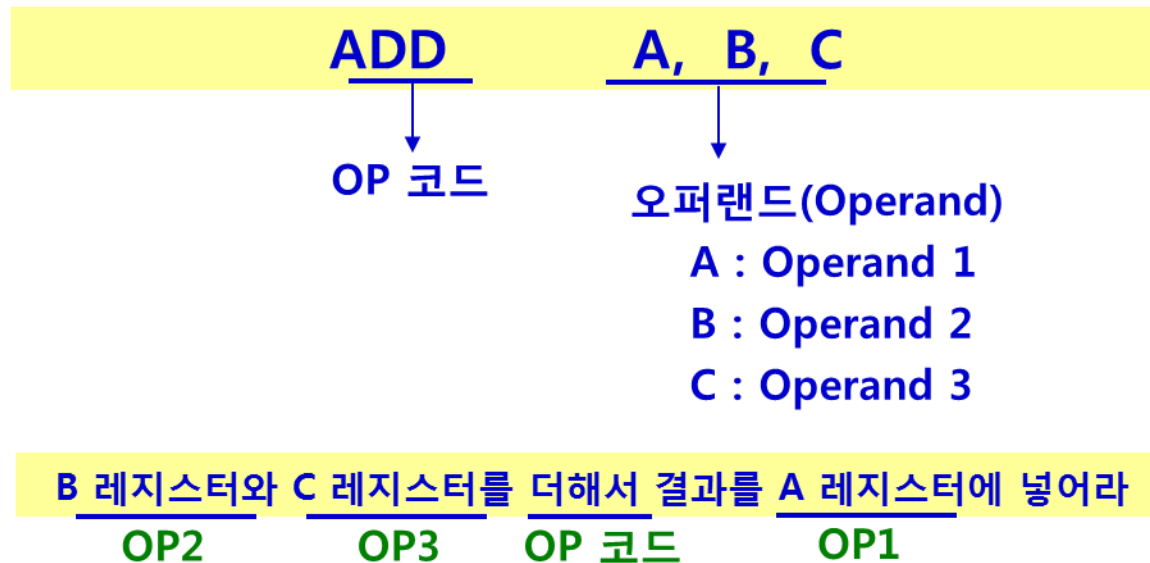
상태 레지스터의 캐리 비트 값이 1이면, 현재의 프로그램 카운터보다

6바이트 후의 분기된 곳을 실행

Architecture of ARM Processor

Architecture : Instruction

- 명령어(Instruction)의 구성
 - ❖ OP 코드(Operation code)
 - 프로세서가 실제로 취해야 하는 동작
 - ❖ 오퍼랜드 (Operand)
 - OP 코드가 명령을 수행하기 위한 대상



1. The Principle of Micro Processor

명령어 실행

① 명령어 인출 단계

- 프로그램 메모리에 있는 명령어를 **인출(Fetch)**하여, 명령어 레지스터(IR)에 넣는 단계
- 명령어는 **연산 동작(opcode)**, **연산 대상(operand)**, **연산 결과(result)**를 지칭하는 비트 정보로 구성

② 명령어 분석 단계

- 명령어 레지스터(IR) 정보를 **Decoding(디코딩)**하여 레지스터 파일, 버스 라인 등에 필요한 신호를 생성

1. The Principle of Micro Processor

③ 오퍼랜드 인출 단계

- 디코딩 결과에 따라 연산에 사용될 오퍼랜드(**operand**) 위치가 결정
- 오퍼랜드가 메모리에 있는 경우, 메모리로부터 인출하여 **CPU** 내부의 레지스터로 옮기는 단계
- 메모리로부터 **CPU** 내부로 옮기기 위해서는 부가의 **clock**이 요구되고 명령어 실행 시간이 길어짐
- 오퍼랜드가 명령어에 포함되어 있거나 레지스터에 있는 경우, 이 단계는 생략

④ 연산 실행(**Execute**) 및 결과 생성 단계

- 명령어 레지스터의 연산 동작(**opcode**)에 따라 오퍼랜드 값 등을 사용하여 산술 또는 논리연산
- 조합 논리 회로로 구성된 **ALU**에서 오퍼랜드를 사용하여 연산 결과는 어큐뮬레이터(**Accumulator**) 또는 범용 레지스터로 기록

⑤ 결과 기록 단계

- 명령어를 디코딩한 결과 연산된 결과를 저장하는 위치가 메모리이면, 이 단계가 수행

1. The Principle of Micro Processor

3주소, 2주소, 1주소, 0주소 마이크로프로세서

➤ 명령어에 포함되는 비트 필드(bit field) 형식

명령코드 (opcode)	오퍼랜드 (operand)	연산 결과 (result)
------------------	-------------------	-------------------

- 명령코드 : 명령어로 실행되어야 할 연산 동작을 표시
- 오퍼랜드 : 연산에 필요한 데이터 표시
- 연산 결과 : 명령코드, 오퍼랜드로 얻은 연산 결과 값을 저장할 위치 표시

➤ 명령어에 포함되는 주소방식에 따라 CPU 내부 구조가 달라짐

➤ 3주소 마이크로프로세서 명령어

- 명령어에 3개의 오퍼랜드 주소가 포함
- (예) **ADD a, b, c**

$c \leftarrow a + b$

1. The Principle of Micro Processor

➤ 2주소 마이크로프로세서 명령어

- 명령어에 2개의 오퍼랜드 주소가 포함
- (예) **ADD a, b** $b \leftarrow a + b$

➤ 1주소 마이크로프로세서 명령어

- 특수 레지스터 어큐뮬레이터(**AC, Accumulator**)를 묵시적으로 이용하여 연산 수행
- (예) **ADD a** $AC \leftarrow a + AC$

➤ 0주소 마이크로프로세서 명령어

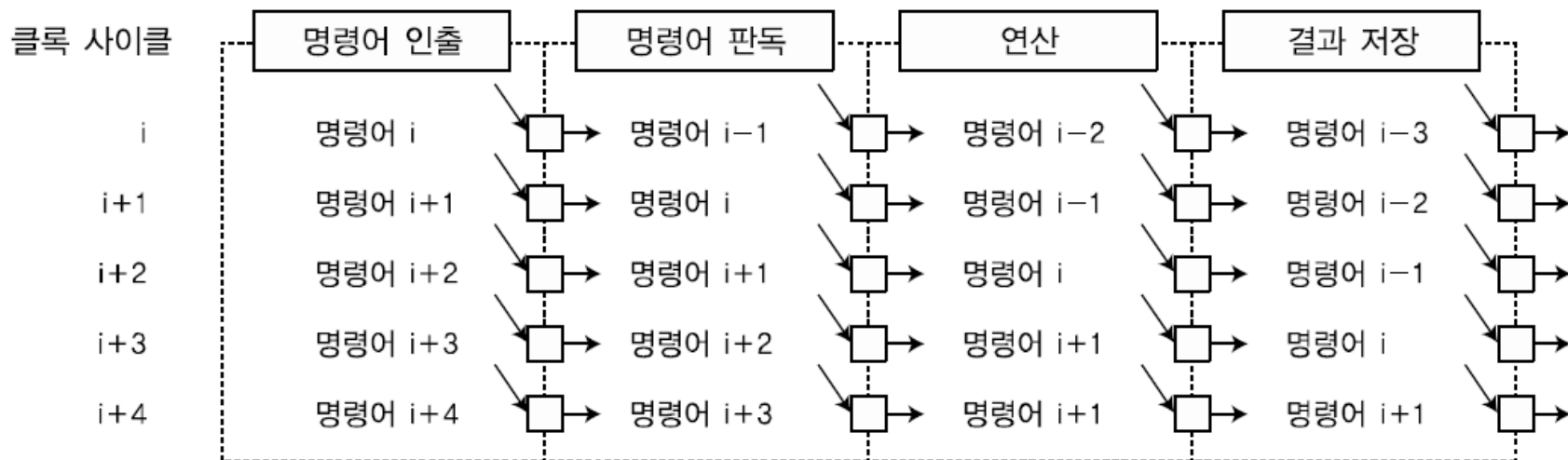
- 오퍼랜드용 주소를 따로 두지 않음
- 스택(**stack**)에 있는 연산 대상의 데이터를 사용
- (예) **ADD**

명령 순서	연산 동작
PUSH #a	$SP \leftarrow SP + 1, [SP] \leftarrow \#a$
PUSH #b	$SP \leftarrow SP + 1, [SP] \leftarrow \#b$
ADD	$AC \leftarrow [SP], SP \leftarrow SP - 1$ $AC \leftarrow AC + [SP]$ $[SP] \leftarrow AC$

1. The Principle of Micro Processor

RISC

- **RISC(Reduced Instruction Set Computer)**는 CISC와 상대되는 개념
- 기존의 컴퓨터 구조를 **CISC(Complex Instruction Set Computer)**로 규정하고,
- 명령어 셋을 줄임으로써 **CPU** 하드웨어 구조를 CISC와 비교하여 간단히 만들 수 있다는 취지에서 시작
- 파이프라인 기법(**Pipelining**)을 이용하여, 한 사이클에 한 명령어 실행

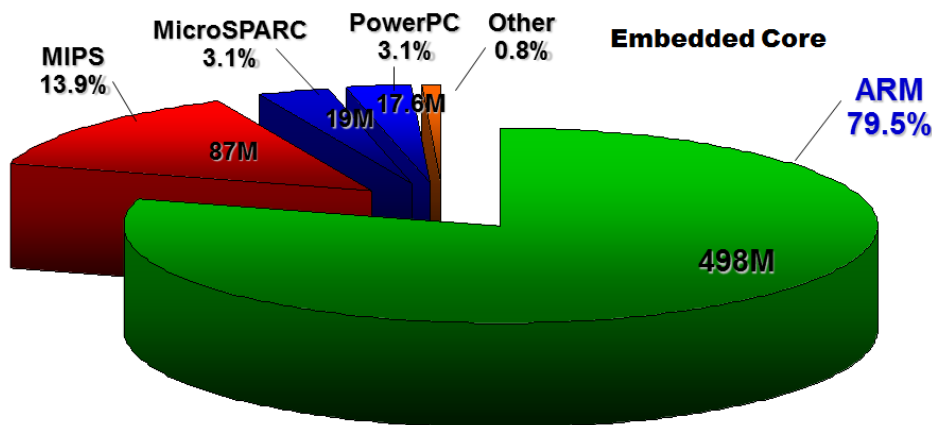


Architecture of ARM Processor

(1) Definition

- **ARM(Advanced RISC Machines)**

- : ARM is a family of **instruction set architectures** for **computer processors** based on a **RISC architecture** developed by British company ARM Holdings.
- : ARM사에서는 프로세서(CORE) 제공, 각 반도체 업체에서 코어와 여러 주변장치를 추가하여 IC 제작.
- : 사용 용도 및 목적에 따라 다양화될 수 있으며, 집적도가 증가함에 따라 SoC 시킴.



※ RISC : Reduced Instruction Set Computer, CISC : Complex Instruction Set Computer

Architecture of ARM Processor

(1) Definition

□ 1990년 설립

- ❖ Acorn Computer에서 독립
- ❖ 12 engineers & CEO
- ❖ Cambridge, UK

□ 32-bit RISC Intellectual Property 제공

- ❖ ARM은 silicon을 제조 판매 하지는 않는다.

□ ARM사는 직접 반도체를 제조하여 판매하는 것이 아니라 설계한 프로세서를 반도체 회사에 Hard Macrocell 또는 Synthesizable core로 제공

□ 반도체 제조회사 또는 SoC 제조사에서는 ARM사로부터 제공받은 ARM core와 주변장치를 추가하여 SoC(System on a Chip)를 만들어 사용자에게 판매 하거나 자체 제품에서 사용

Architecture of ARM Processor

- **2004년** ARM v7 코어 개발. **Cortex-M3** 모델 발표. ※ **MCU** 시장
- **2009년** **Cortex-M0** 모델 발표. ※ **8051이나 AVR과 경쟁**
- **2010년** **Cortex-M4/M4F** 모델 발표. ※ **DSP** 기능
- **2012년** **Cortex-M0+** 모델 발표. ※ **사물간인터넷(IoT)** 구현에 적합
- **2014년** **Cortex-M7** 발표 ※ **고성능 DSP** 기능

★ **2016년 7월** 일본 소프트뱅크가 **ARM**을 인수 합병함.

	ARM7	ARM9	ARM10	ARM11
발표 연도	1993	1996	1998	2002
파이프라인	3단계	5단계	6단계	8단계/9단계
최고 동작 속도	133 MHz	250 MHz	325 MHz	550 MHz
소비전력	0.25 mW/MHz	0.2 mW/MHz	0.5 mW/MHz	0.6 mW/MHz
기본 아키텍처	폰노이만 구조 온칩 캐시	수정된 하버드 구조 명령캐시/데이터캐시	수정된 하버드 구조 명령캐시/데이터캐시	수정된 하버드 구조 명령캐시/데이터캐시
곱셈기	32×8	32×8	32×16	32×16
적용 제품 코어	ARM720T ARM7EJ-S ARM7TDMI ARM7TDMI-S	ARM920T ARM922T ARM926EJ-S ARM946E-S	ARM1020E ARM1022E ARM1026EJ-S	ARM1136J-S ARM1136JF-S ARM1176JZ-S ARM1176JZF-S

Cortex-A



servers



set top boxes



netbooks



mobile applications

Cortex-R



disk drives



digital cameras



mobile baseband

Cortex-M



appliances

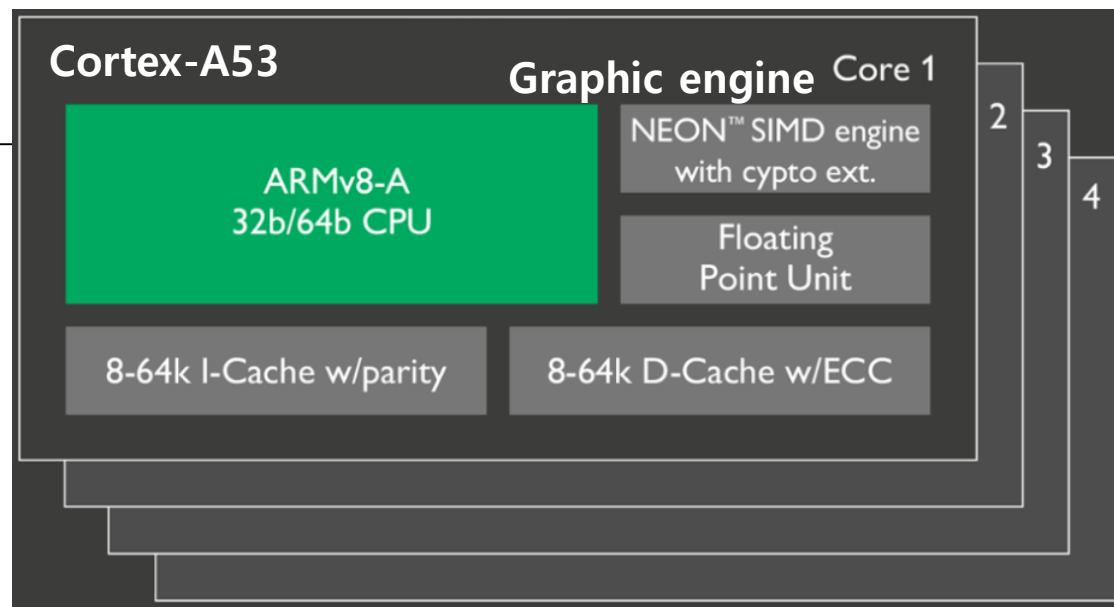


motors

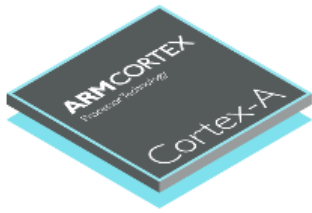


audio

ARM Cortex- Processor



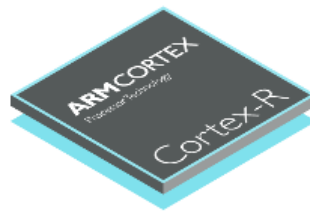
<http://www.arm.com/products/processors>



Cortex-A

Highest performance
Optimized for rich operating systems

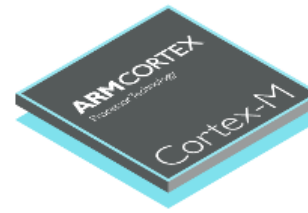
[Learn more about the Cortex-A series processors](#)



Cortex-R

Fast response
Optimized for high-performance, hard real-time applications

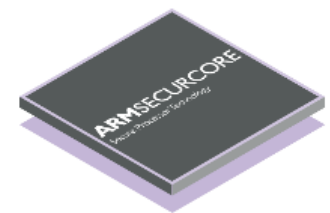
[Learn more about the Cortex-R series processors](#)



Cortex-M

Smallest/lowest power
Optimized for discrete processing and microcontroller

[Learn more about the Cortex-M series processors](#)



SecurCore

Tamper resistant
Optimized for security applications

[Learn more about SecurCore processors](#)

infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.home/index.html

The screenshot displays the ARM Developer website. At the top, the ARM logo is on the left, and a blue banner reads "arm Developer" with the text "Arm's technical content has moved to developer.arm.com" and a yellow "Click to view" button. A search bar is on the right. Below the banner, navigation links for Products, Support, Community, Markets, About, and Careers are listed. A breadcrumb trail shows "You are here: Home > Support > Documentation". A search bar and "Advanced Search" link are present. The left sidebar contains a "Contents" menu with categories like "Using this site", "ARM Forums and knowledge articles", "Frequently asked questions", "Giving feedback", "ARM Technical Support Knowledge Articles", "ARM architecture", "ARM Software development tools", "Keil embedded development tools", "Development boards", "Developer Guides and Articles", "Application Notes and Tutorials", "Research Papers", "Cortex-A series processors", "Cortex-R series processors", "Cortex-M series processors", "ARM11 processors", "ARM9 processors", "ARM7 processors", "AMBA", "CoreLink controllers and peripherals", "CoreSight on-chip trace and debug", and "Mali graphics processors". The main content area, titled "Using this site", features the "ARM Infocenter" banner and a welcome message: "Welcome to the ARM Infocenter. The Infocenter contains all ARM non-confidential[†] Technical Publications, including:". A list of publications follows: "ARM Architecture Reference Manuals", "Cortex-A, Cortex-R, Cortex-M, ARM11, ARM9, and ARM7 Technical Reference Manuals", "AMBA specifications and design tools and CoreLink peripherals and controllers product manuals", "CoreSight on-chip debug and trace TRMs and Architecture documentation", "ARM Software Development tools and Modeling tools documentation", and "Application Notes and Technical Support Knowledge Articles (FAQs)". Below this are links for "ARM Forums and knowledge articles" and "Frequently asked questions", and a "Feedback" link. On the right, a "Related information" section promotes "arm MALI Visual Technology" and "KEIL™ An ARM® Company", with links to the Mali Developer Center and the Keil website for more information on embedded software development.

arm Developer
Arm's technical content has moved to developer.arm.com [Click to view](#)

Search our site

Contact ARM | English

Products Support Community Markets About Careers

You are here: Home > Support > Documentation

All documents Search our documentation Advanced Search

Contents

- Using this site
- ARM Forums and knowledge articles
- Frequently asked questions
- Giving feedback
- ARM Technical Support Knowledge Articles
- ARM architecture
- ARM Software development tools
- Keil embedded development tools
- Development boards
- Developer Guides and Articles
- Application Notes and Tutorials
- Research Papers
- Cortex-A series processors
- Cortex-R series processors
- Cortex-M series processors
- ARM11 processors
- ARM9 processors
- ARM7 processors
- AMBA
- CoreLink controllers and peripherals
- CoreSight on-chip trace and debug
- Mali graphics processors

Using this site

ARM Infocenter

arm Developer
Arm's technical content has moved to developer.arm.com [Click to view](#)

Welcome to the ARM Infocenter. The Infocenter contains all ARM non-confidential[†] Technical Publications, including:

- ARM Architecture Reference Manuals
- Cortex-A, Cortex-R, Cortex-M, ARM11, ARM9, and ARM7 Technical Reference Manuals
- AMBA specifications and design tools and CoreLink peripherals and controllers product manuals
- CoreSight on-chip debug and trace TRMs and Architecture documentation
- ARM Software Development tools and Modeling tools documentation
- Application Notes and Technical Support Knowledge Articles (FAQs).

► **ARM Forums and knowledge articles**

► **Frequently asked questions**

[Feedback](#)

Related information

arm MALI
Visual Technology

Visit the Mali Developer Center for more information on the ARM Mali graphics processors.
[Find out more »](#)

KEIL™
An ARM® Company

See the Keil website for more information on the Keil range of embedded software development.

ARM® Cortex®-R,M Portfolio

Cortex-R4

Real-time
performance

Cortex-R5

Real-time
performance
with functional
safety

Cortex-R7

High
performance
4G modem and
storage

Cortex-R8

Highest
performance
5G modem and
storage

Cortex-R

Cortex-M0

Lowest cost,
low power

Cortex-M0+

Highest energy
efficiency

Cortex-M3

Performance
efficiency

Cortex-M4

Mainstream
control & DSP

Cortex-M7

Maximum
performance
control & DSP

Cortex-M

SC000

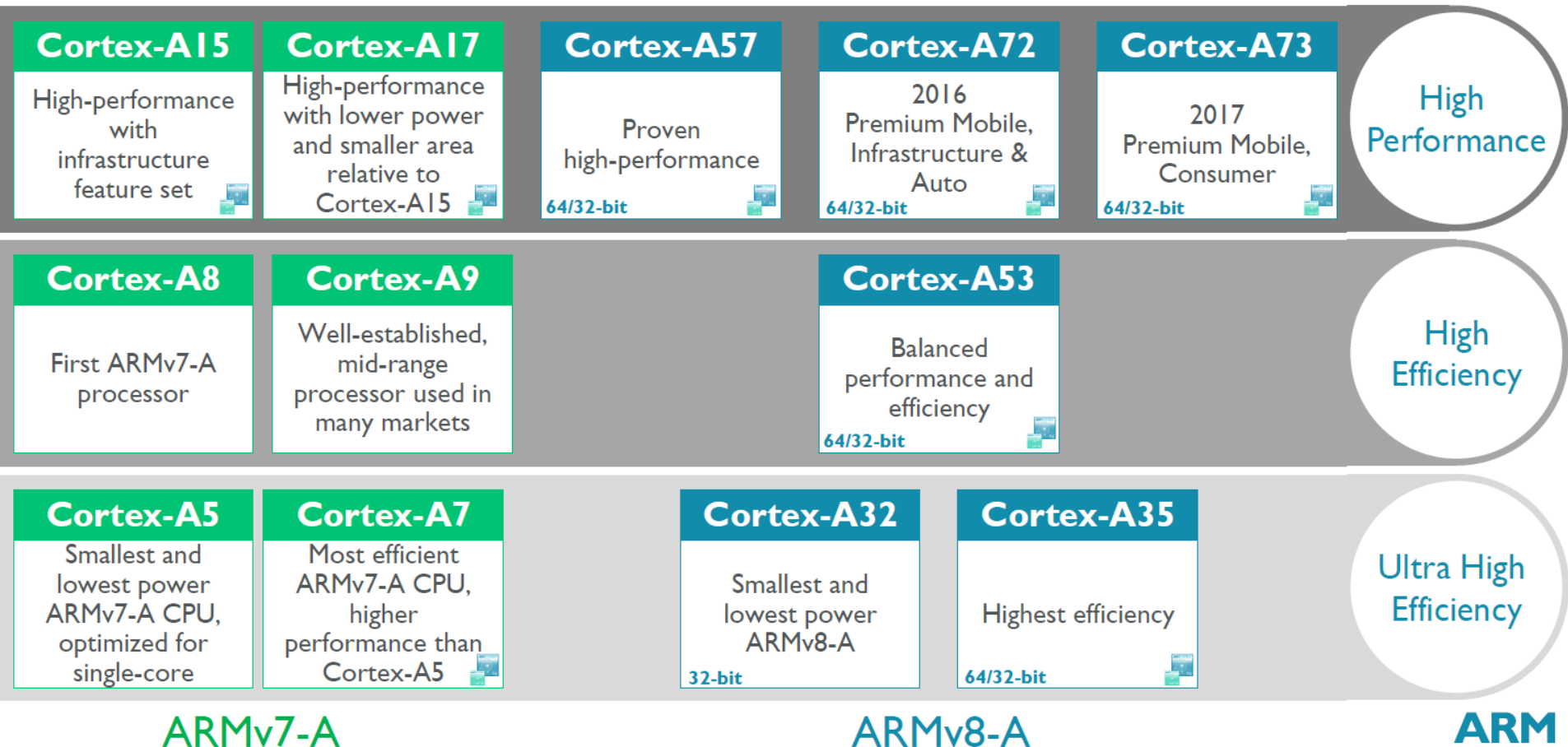
Optimized area,
anti-tampering

SC300

Performance,
anti-tampering

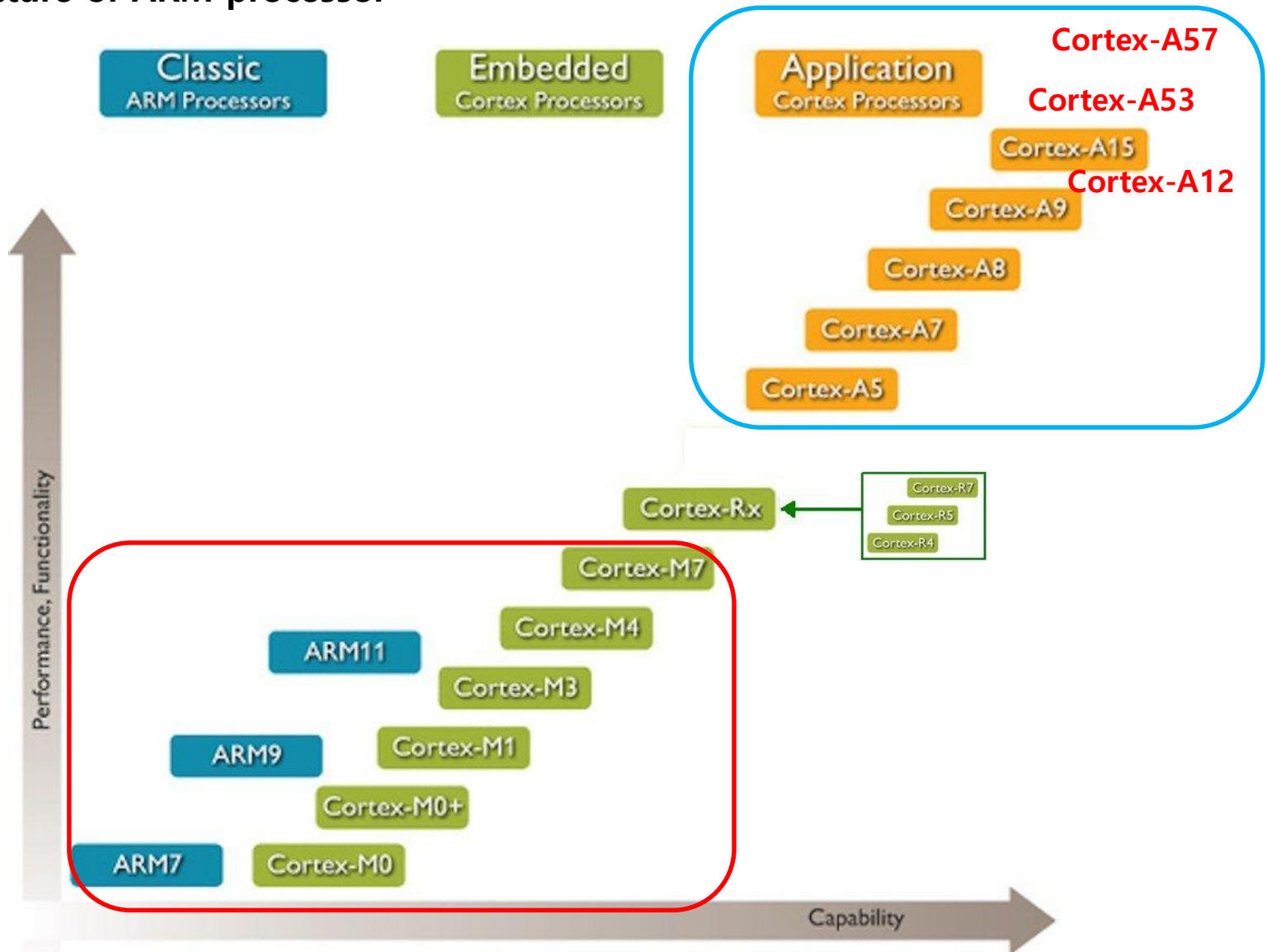
SecurCore

ARM® Cortex®-A Portfolio 2016



Architecture of ARM Processor

Architecture of ARM processor



ARM 마이크로프로세서 제품군의 분류

Architecture of ARM Processor

- ※ MAC(Multiply–Accumulate operation)
- ※ SIMD(Single Instruction Multiple Data)
- ※ DMIPS(Dhrystone Million Instruction per Second)

Architecture of ARM processor

ARM 패밀리	ARM 아키텍처	ARM 코어	특징	처리 속도
Cortex-A	ARMv7-A	Cortex-A5	VFP, NEON, Jazelle RCT, Thumb/Thumb-2, 1~4 cores.	1.57 DMIPS/MHz per core
		Cortex-A8	(2005년) VFP, NEON, Jazelle RCT, Thumb/Thumb-2, 13-stage superscalar pipeline.	2.0 DMIPS/MHz
		Cortex-A9 MPCore	(2007년) Application profile, VFPv3 FPU, NEON, Thumb/Thumb-2, Jazelle RCT/DBX, out-of-order speculative issue superscalar, 1~4 SMP cores.	2.50 DMIPS/MHz per core
		Cortex-A15 MPCore	(2010년) Application profile, VFPv4 FPU, NEON, Thumb-2, Jazelle RCT/DBX, out-of-order speculative issue superscalar, Large Physical Address Extensions (LPAE), Hardware virtualization, 1~4 SMP cores.	3.5~4.1 DMIPS/MHz per core
Cortex-R	ARMv7-R	Cortex-R4 Cortex-R4F	Real-time profile, Thumb-2, FPU.	1.50 DMIPS/MHz
Cortex-M	ARMv6-M	Cortex-M0 Cortex-M0+	(2009년) Microcontroller profile, Thumb-2 subset. Hardware multiply instruction.	0.9 DMIPS/MHz
		Cortex-M1	(2007년) FPGA targeted, Microcontroller profile, Thumb-2 subset.	0.8 DMIPS/MHz
	ARMv7-M	Cortex-M3	(2004년) Microcontroller profile, Thumb/Thumb-2. Hardware multiply and divide instruction.	1.25 DMIPS/MHz
	ARMv7E-M	Cortex-M4 Cortex-M4F	(2010년) Microcontroller profile, both Thumb/Thumb-2, Single precision FPU. Hardware MAC, SIMD and divide instructions.	1.25 DMIPS/MHz 1.27 DMIPS/MHz
	ARMv7E-M	Cortex-M7	(2014년) Single and double precision FPU. 0~64KB instruction cache and data cache, 6-stage superscalar	2.14 DMIPS/MHz

	Cortex-M0	Cortex-M0+	Cortex-M3	Cortex-M4	Cortex-M7
Architecture Version	ARMv6-M (Von Neuman)	ARMv6-M (Von Neuman)	ARMv7-M (Harvard)	ARMv7E-M (Harvard)	ARMv7E-M (Harvard)
Instruction Set	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2 DSP, SIMD, FPU	Thumb, Thumb-2 DSP, SIMD, FPU
Dhrystone DMIPS/MHz	0.87	0.95	1.25	1.27	2.14
CoreMark/MHz	2.33	2.46	3.34	3.40	5.04
Bus Protocol	AHB Lite	AHB Lite	AHB Lite, APB	AHB Lite, APB	AHB Lite, AXI, APB
Pipeline Stage	3	2	3	3	6
Instruction/Data Cache	No	No	No	No	4~64KB/4~64KB
TCM(Instruction/Data)	No	No	No	No	0~16KB/0~16KB
Number of Interrupts	1~32 + NMI	1~32 + NMI	1~240 + NMI	1~240 + NMI	1~240 + NMI
Interrupt Priority	4	4	8~256	8~256	8~256
Debug	JTAG, SWD	JTAG, SWD	JTAG, SWD	JTAG, SWD	JTAG, SWD
Memory Protection Unit	No	No	Yes(option)	Yes(option)	Yes(option)
Bit Banding Support	No	No	Yes	Yes	Yes
Single Cycle Multiply	Yes(option)	Yes(option)	Yes	Yes	Yes
Hardware Divide	No	No	Yes	Yes	Yes
Single Cycle DSP/SIMD	No	No	No	Yes	Yes
Floating Point Hardware	No	No	No	single precision	single/double precision
CMSIS Support	Yes	Yes	Yes	Yes	Yes
Power Consumption	12.5μW/MHz	9.8μW/MHz	31μW/MHz	32.82μW/MHz	53μW/MHz
Announcement Year	2009	2012	2004	2010	2014

MCU 계열	MCU 모델	연산시간	상대 속도	테스트 조건
8051	AT89S52-24PC (24MHz로 동작)	1.32 ms	0.11	Keil C컴파일러 V6.14 사용, 최고 speed(레벨 8)로 최적화함. 컴파일러 내장 sin(x) 함수 사용.
AVR	ATmega128A-AU (16MHz로 동작)	143.348 μ s	1	WinAVR-20100110 C컴파일러 사용, speed(-Os)로 최적화함. 컴파일러 내장 sin(x) 함수 사용.
Cortex-M3	STM32F103VCT6 (72MHz로 동작)	32.376 μ s	4.4	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. 컴파일러 내장 sin(x) 함수 사용.
Cortex-M4	STM32F407VET6 (168MHz로 동작)	10.753 μ s	13.3	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. 컴파일러 내장 sin(x) 함수 사용. FPU를 사용하지 않음.
		3.460 μ s	41.4	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. CMSIS의 arm_sin_f32(x) 함수 사용. FPU를 사용하지 않음.
		11.072 μ s	12.9	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. 컴파일러 내장 sin(x) 함수 사용. FPU를 사용함.
		0.524 μs	273.6	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. CMSIS의 arm_sin_f32(x) 함수 사용. FPU를 사용함.
Cortex-M7	STM32F767VGT6 (216MHz로 동작)	5.666 μ s	25.3	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. 컴파일러 내장 sin(x) 함수 사용. FPU를 사용하지 않음.
		2.008 μ s	71.4	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. CMSIS의 arm_sin_f32(x) 함수 사용. FPU를 사용하지 않음.
		0.954 μ s	150.3	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. 컴파일러 내장 sin(x) 함수 사용. FPU를 사용함.
		0.292 μs	490.9	IAR EWARM V7.70.1 C컴파일러 사용, High/Size로 최적화함. CMSIS의 arm_sin_f32(x) 함수 사용. FPU를 사용함.

42/200

* 8051 needs at least one cycle per instruction byte fetch as they only have an 8-bit interface

Architecture of ARM Processor

- **STMicroelectronics**

- : 가장 많은 제품 출시, **Coretex-M0/M0+/M3/M4/M7** 풀 라인업 생산.

- **Freescale**

- : 매우 많은 모델 출시, 2015년 **NXP**에 인수 합병됨.

- **NXP Semiconductors**

- : 2015년 **Freescale**을 인수하여 **Coretex-M0/M0+/M3/M4/M7** 풀 라인업 생산.

- NXP**는 2016년 10월에 다시 **퀄컴**에 인수 합병됨.

- **Texas Instruments**

- : 많은 모델 출시, 2009년에 **Luminary Micro**를 인수 합병함.

- **Atmel**

- : **Cortex-M** 시리즈 생산은 매우 늦게 출발하였으나 최근에는 적극적으로 모델을 늘림.

STMicroelectronics사

Cortex-M0/M0+/M3/M4/M7

모델의 제품 분류

Common core peripherals and architecture:
Communication peripherals: USART, SPI, I ² C
Multiple general-purpose timers
Integrated reset and brown-out warning
Multiple DMA
2x watchdogs Real-time clock
Integrated regulator PLL and clock circuit
Up to 3x 12-bit DAC
Up to 4x 12-bit ADC (Up to 5 MSPS)
Main oscillator and 32 kHz oscillator
Low-speed and high-speed internal RC oscillator
-40 to +85 °C and up to 125 °C operating temperature range
Low voltage 2.0 to 3.6 V or 1.65/1.7 to 3.6 V (depending on series)
Temperature sensor

High-performance

STM32F7 series – Very high performance with DSP and FPU (STM32F7x6)

200 MHz Cortex-M7 CPU	Up to 1-Mbyte Flash	Up to 336-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN CEC FMC	SDIO 2x I ² S audio Camera IF	Crypto Ethernet IEEE 1588 2x SAI	LCD-TFT SDRAM I/F Quad SPI SPDIF input
-----------------------	---------------------	----------------------	----------------------	-----------------------------	----------------	--	----------------------------------	--



STM32F4 series – High performance with DSP and FPU (STM32F401/411/405-415/407-417/427-437/429-439 and STM32F446)

Up to 180 MHz Cortex-M4 DSP/FPU	Up to 2-Mbyte Flash	Up to 256-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN CEC F(S)MC	SDIO 3x I ² S audio Camera IF	Crypto Ethernet IEEE 1588 2x SAI	LCD-TFT SDRAM I/F Quad SPI SDIF input
---------------------------------	---------------------	----------------------	----------------------	-----------------------------	-------------------	--	----------------------------------	---------------------------------------



STM32F2 series – High performance (STM32F2x5 and 2x7)

120 MHz Cortex-M3 CPU	Up to 1-Mbyte Flash	Up to 128-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN 2.0B FSMC	SDIO 2x I ² S audio Camera IF	Crypto Ethernet IEEE 1588
-----------------------	---------------------	----------------------	----------------------	-----------------------------	------------------	--	---------------------------



Mainstream

STM32F3 series – Mixed-signal with DSP (STM32F301/302/303/334/373/3x8)

72 MHz Cortex-M4 with DSP/FPU	Up to 512-Kbyte Flash	Up to 80-Kbyte SRAM CCM-RAM	USB 2.0 FS	3x 16-bit advanced MC timer	CAN CEC FSMC	7x comparator 4x PGA	HR-Timer	3x 16-bit $\Sigma\Delta$ ADC
-------------------------------	-----------------------	-----------------------------	------------	-----------------------------	--------------	----------------------	----------	------------------------------



STM32F1 series – Mainstream (STM32F100/101/102/103 and 105-107)

Up to 72 MHz Cortex-M3 CPU	Up to 1-Mbyte Flash	Up to 96-Kbyte SRAM	USB 2.0 OTG FS	2x 16-bit advanced MC timer	2x CAN CEC FSMC	SDIO 2x I ² S audio	Ethernet IEEE 1588
----------------------------	---------------------	---------------------	----------------	-----------------------------	-----------------	--------------------------------	--------------------



STM32F0 series – Entry-level (STM32F0x0/0x1/0x2 and 0x8)

48 MHz Cortex-M0 CPU	Up to 256-Kbyte Flash	Up to 32-Kbyte SRAM 20-byte backup data	USB 2.0 FS device Crystal less	CAN CEC	DAC Comparator
----------------------	-----------------------	---	--------------------------------	---------	----------------



Ultra-Low-Power

STM32L4 series – Ultra-Low-Power (STM32L4x6)

80 MHz Cortex-M4 CPU	Up to 1-Mbyte Flash	Up to 128-Kbyte SRAM	USB 2.0 OTG FS	2x 16-bit advanced MC timer	LCD up to 8x40	Op-amps comparator	FSMC SDIO CAN DFSDM	AES 256-bit T-RNG 2 x SAI
----------------------	---------------------	----------------------	----------------	-----------------------------	----------------	--------------------	---------------------	---------------------------



STM32L1 series – Ultra-Low-Power (STM32L100/151-152/162)

32 MHz Cortex-M3 CPU	Up to 512-Kbyte Flash	Up to 80-Kbyte SRAM	Up to 16-Kbyte EEPROM	USB 2.0 FS Device	LCD up to 8x40	Op-amps comparator	FSMC SDIO	AES 128-bit
----------------------	-----------------------	---------------------	-----------------------	-------------------	----------------	--------------------	-----------	-------------

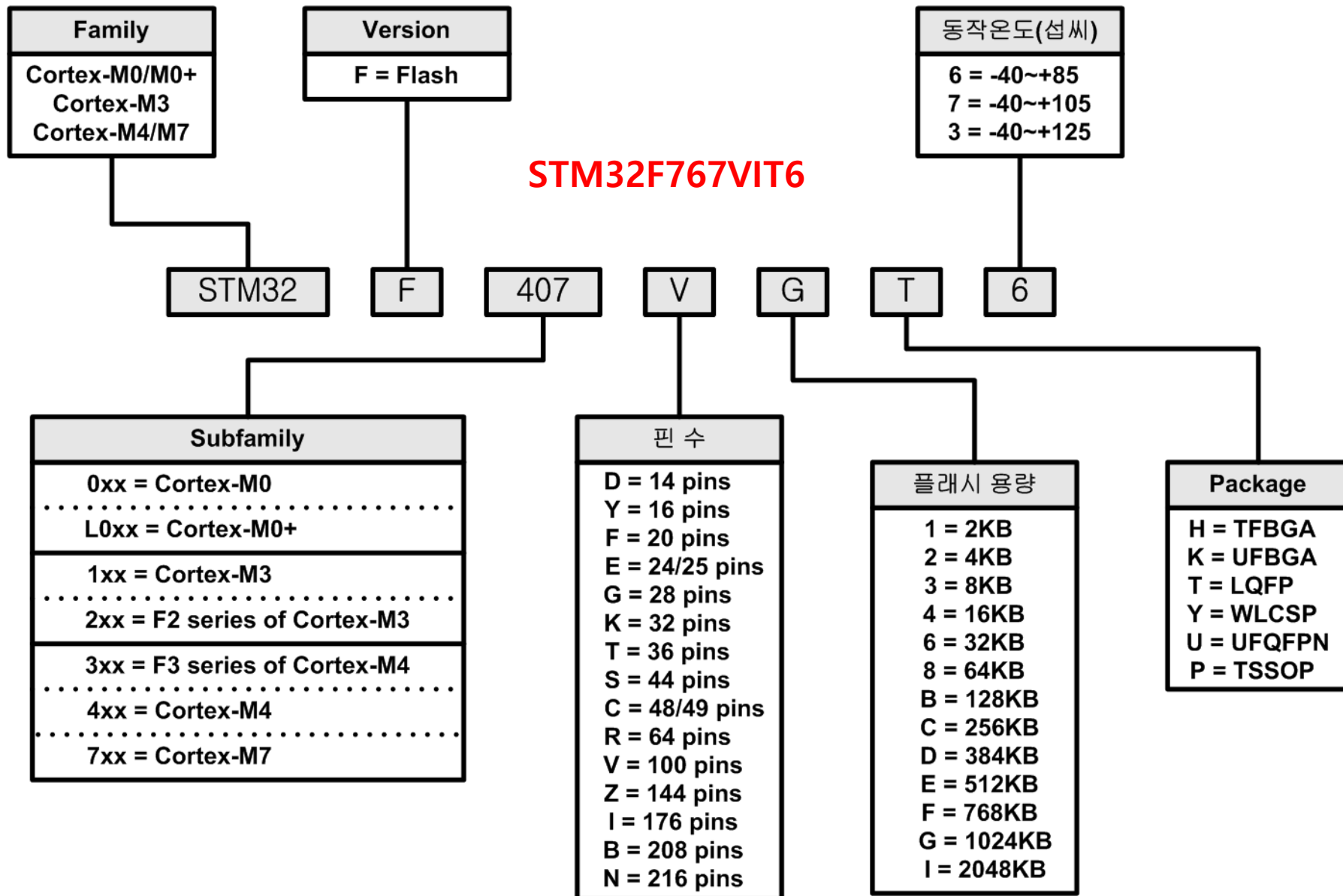


STM32L0 series – Ultra-Low-Power (STM32L0x1/0x2/0x3)

32 MHz Cortex-M0+ CPU	Up to 192-Kbyte SRAM	Up to 20-Kbyte SRAM	Up to 6-Kbyte EEPROM	USB 2.0 FS device Crystal less	LCD 8x40 4x52	T-RNG comparator	LP Timer LP UART LP 12-bit ADC	AES 128-bit
-----------------------	----------------------	---------------------	----------------------	--------------------------------	---------------	------------------	--------------------------------	-------------



Architecture of ARM Processor



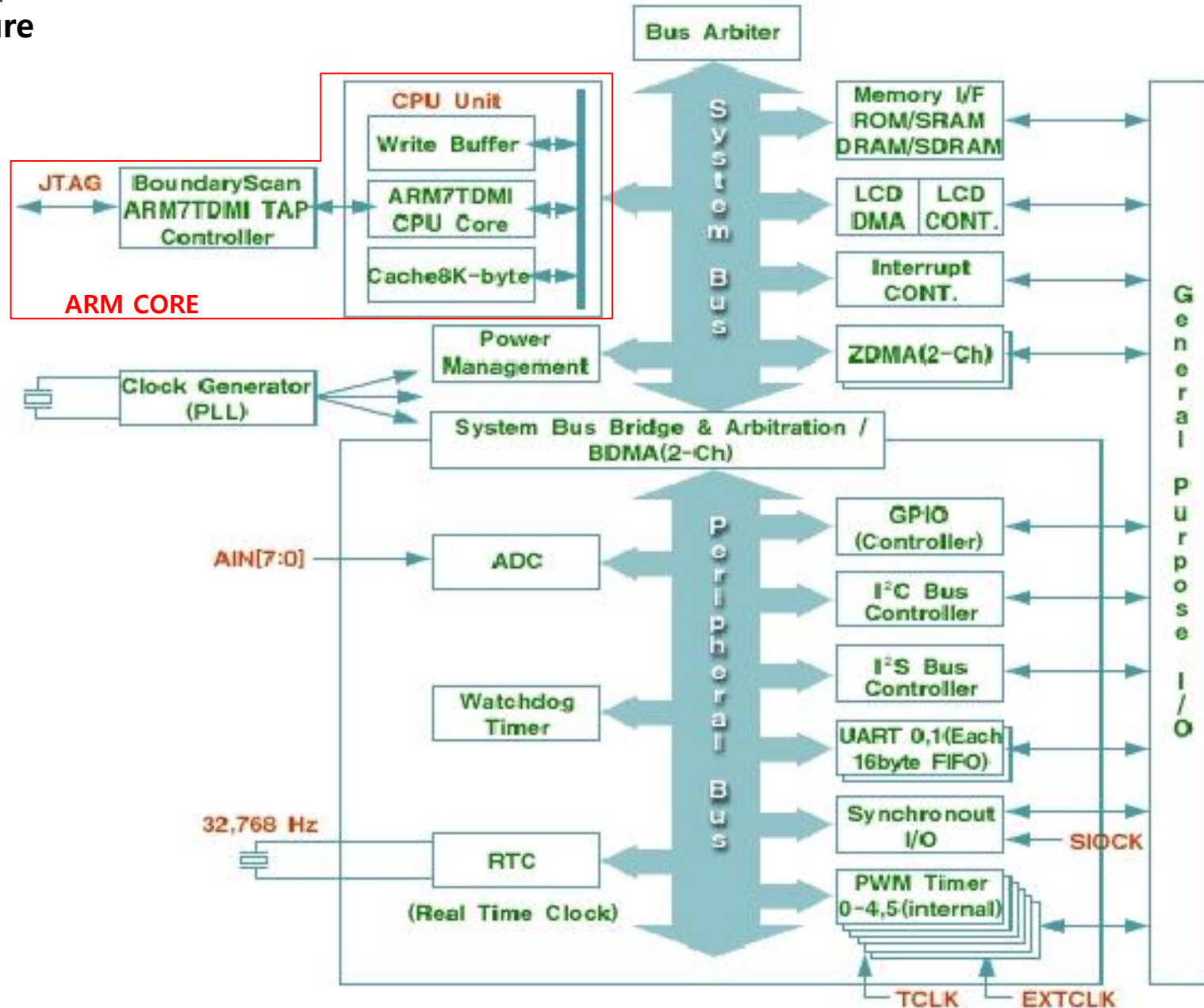
Architecture of ARM Processor

STMicroelectronics사 Cortex-M0/M0+/M3/M4/M7 모델의 제품 분류

모델명	패키지	속도 [MHz]	FPU/ 캐시 메모리	플래시 [KB]	SRAM [KB]	I/O 핀	16/32 비트 타이머	WDT/ RTC	camera int./ TFT	12 비트 ADC	12 비트 DAC	USART /UART	SPI	I2C	SDIO	USB	CAN/ Ethernet	Crypto /hash	전원 [V]	비 고
STM32F745VE STM32F745VG STM32F745ZE STM32F745ZG STM32F745IE STM32F745IG STM32F745BE STM32F745BG STM32F745NE STM32F745NG	LQFP100 LQFP100 LQFP144 LQFP144 LQFP176 LQFP176 LQFP208 LQFP208 TFBGA216 TFBGA216	216	single prec./ 4KB(IC) 4KB(DC)	512 1024 512 1024 512 1024 512 1024 512 1024	320	82 82 114 114 140 140 168 168 168 168	13/2	2/1	Y/-	3/16 3/16 3/24 3/24 3/24 3/24 3/24 3/24 3/24 3/24	2	4/4	4 4 6 6 6 6 6 6 6 6	4	1	2x OTG	2/Y	-	1.8/ 3.3	Base Series
STM32F765VG STM32F765VI STM32F765ZG STM32F765ZI STM32F765IG STM32F765II STM32F765BG STM32F765BI STM32F765NG STM32F765NI	LQFP100 LQFP100 LQFP144 LQFP144 LQFP176 LQFP176 LQFP208 LQFP208 TFBGA216 TFBGA216	216	double prec./ 16KB(IC) 16KB(DC)	1024 2048 1024 2048 1024 2048 1024 2048 1024 2048	512	82 82 114 114 140 140 168 168 168 168	13/2	2/1	Y/-	3/16 3/16 3/24 3/24 3/24 3/24 3/24 3/24 3/24 3/24	2	4/4	4 4 6 6 6 6 6 6 6 6	4	2	2x OTG	3/Y	Y	1.8/ 3.3	
STM32F746VE STM32F746VG STM32F746ZE STM32F746ZG STM32F746IE STM32F746IG STM32F746BE STM32F746BG STM32F746NE STM32F746NG	LQFP100 LQFP100 LQFP144 LQFP144 LQFP176 LQFP176 LQFP208 LQFP208 TFBGA216 TFBGA216	216	single prec./ 4KB(IC) 4KB(DC)	512 1024 512 1024 512 1024 512 1024 512 1024	320	82 82 114 114 140 140 168 168 168 168	13/2	2/1	Y/Y	3/16 3/16 3/24 3/24 3/24 3/24 3/24 3/24 3/24 3/24	2	4/4	4 4 6 6 6 6 6 6 6 6	4	1	2x OTG	2/Y	-	1.8/ 3.3	TFT-LCD 제어기
STM32F756VE STM32F756VG STM32F756ZE STM32F756ZG STM32F756IE STM32F756IG STM32F756BE STM32F756BG STM32F756NE STM32F756NG	LQFP100 LQFP100 LQFP144 LQFP144 LQFP176 LQFP176 LQFP208 LQFP208 TFBGA216 TFBGA216	216	single prec./ 4KB(IC) 4KB(DC)	512 1024 512 1024 512 1024 512 1024 512 1024	320	82 82 114 114 140 140 168 168 168 168	13/2	2/1	Y/Y	3/16 3/24 3/24 3/24 3/24 3/24 3/24 3/24 3/24 3/24	2	4/4	4 6 6 6 6 6 6 6 6 6	4	1	2x OTG	2/Y	Y	1.8/ 3.3	

Architecture of ARM Processor

(2) Architecture



Architecture of ARM Processor

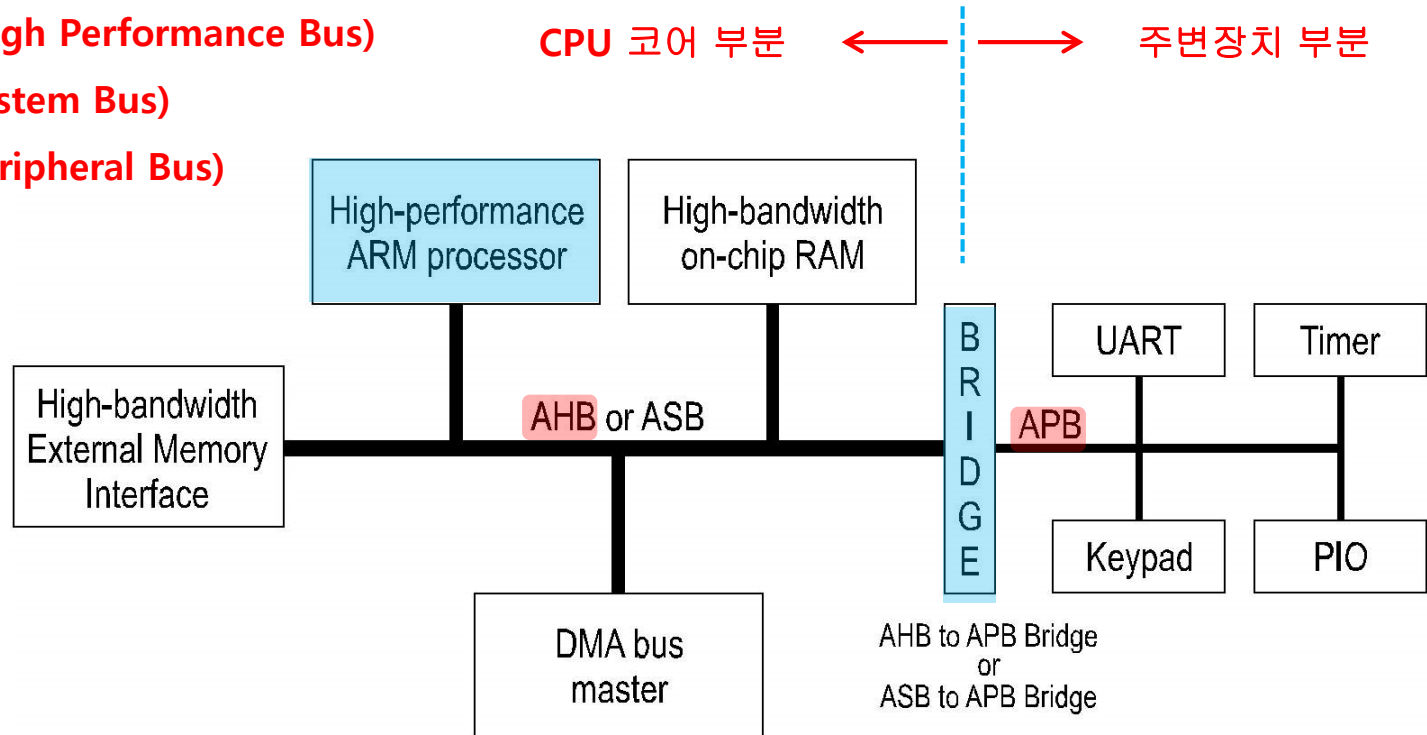
(3) Architecture

※ AMBA (Advanced Microcontroller Bus Architecture)

: AHB (Advanced High Performance Bus)

: ASB (Advanced System Bus)

: APB (Advanced Peripheral Bus)



AMBA AHB

- * High performance
- * Pipelined operation
- * Multiple bus masters
- * Burst transfers
- * Split transactions

AMBA ASB

- * High performance
- * Pipelined operation
- * Multiple bus masters

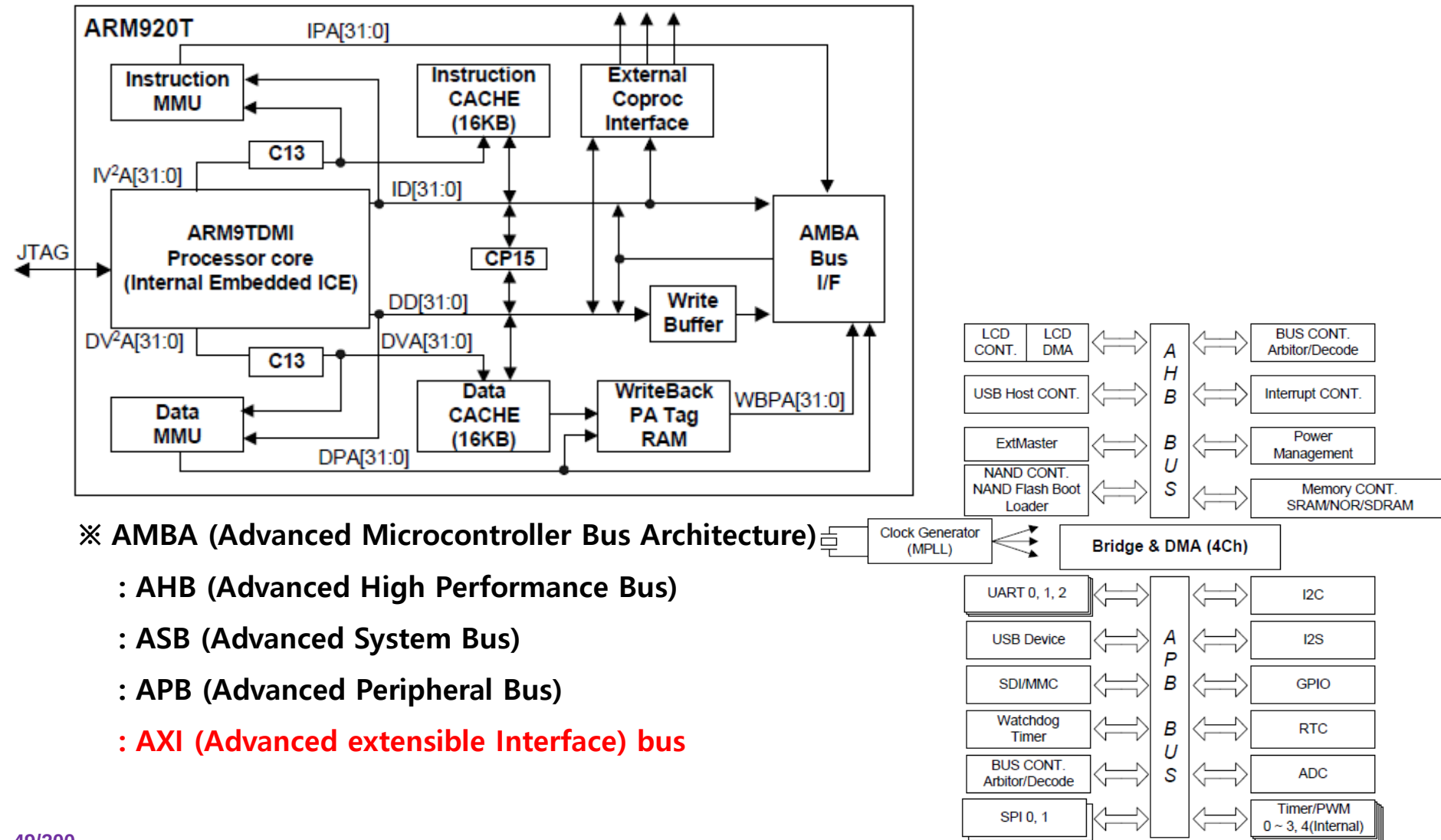
AMBA APB

- * Low power
- * Latched address and control
- * Simple interface
- * Suitable for many peripherals

Architecture of ARM Processor

Architecture of ARM9 processor

(3) Architecture



※ AMBA (Advanced Microcontroller Bus Architecture)

: AHB (Advanced High Performance Bus)

: ASB (Advanced System Bus)

: APB (Advanced Peripheral Bus)

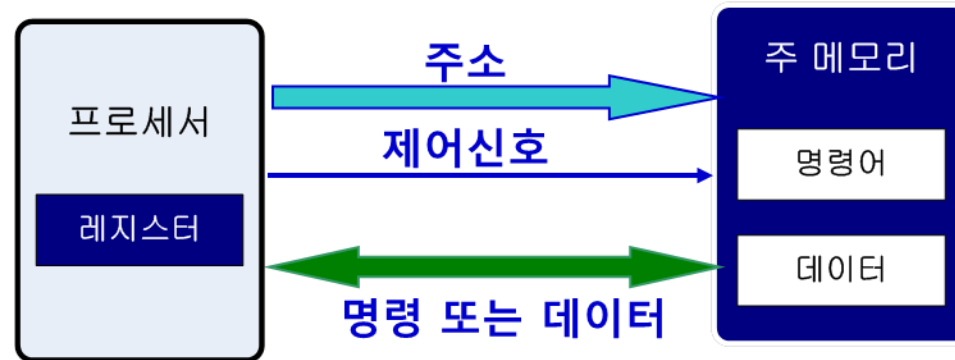
: AXI (Advanced extensible Interface) bus

Architecture of ARM Processor

(3) Architecture : Bus

□ 버스(BUS)란 ?

- ❖ 컴퓨팅 시스템의 각 모듈에서 발생한 신호를 공유해서 사용할 수 있도록 만든 신호의 집합
- ❖ 구동 주체(CPU 등)에 의해서 해당 소자에 데이터를 읽거나 쓸 수 있도록 구성된다.
- ❖ 어드레스 버스(address bus), 제어버스(control bus), 그리고 데이터 버스(data bus)로 구성된다.



Architecture of ARM Processor

(3) Architecture : Bus

□ 버스

- ❖ 디지털 회로에서 시스템의 여러 장치들을 연결하는 경로

□ 내부 버스 (Internal Bus)

- ❖ 프로세서 내부에서 레지스터와 ALU 사이의 신호를 교환하고, 그 결과를 다시 레지스터에 전달하는 경로

□ 외부 버스 (external bus)

- ❖ 프로세서와 외부의 기억장치 사이, 그리고 프로세서와 I/O 장치 사이에 존재하는 버스

❖ 외부버스 구성

➤ 데이터 버스(data bus)

- ✓ 데이터를 외부 장치에 전달하거나 외부 장치로부터 읽어오는 경로

➤ 어드레스 버스(address bus)

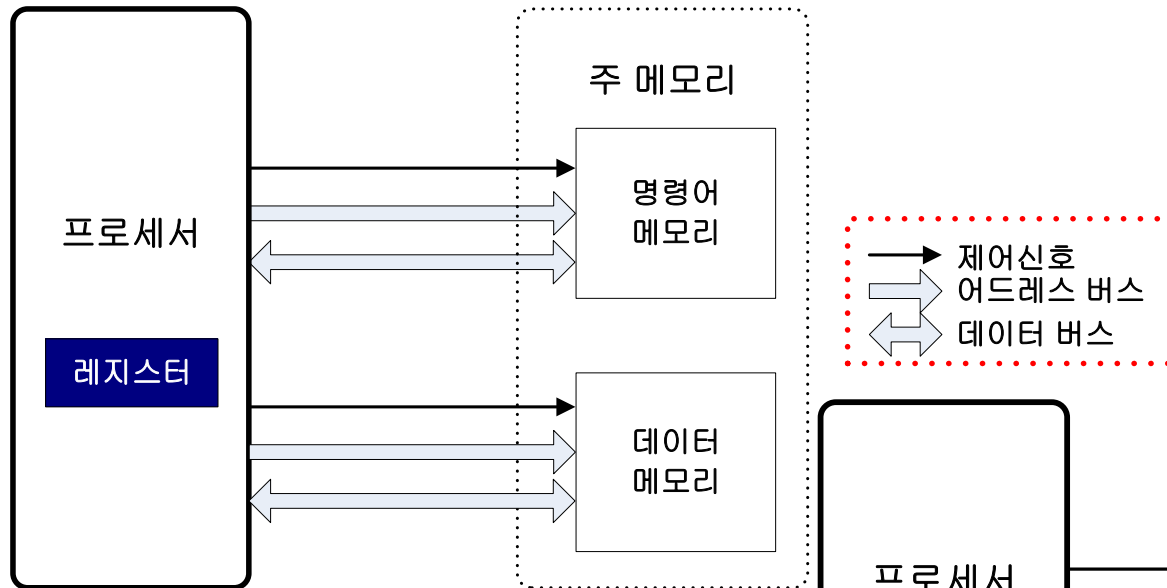
- ✓ 프로세서에서 기억장치나 I/O 장치의 주소 정보 전송 경로

➤ 제어 버스(control bus)

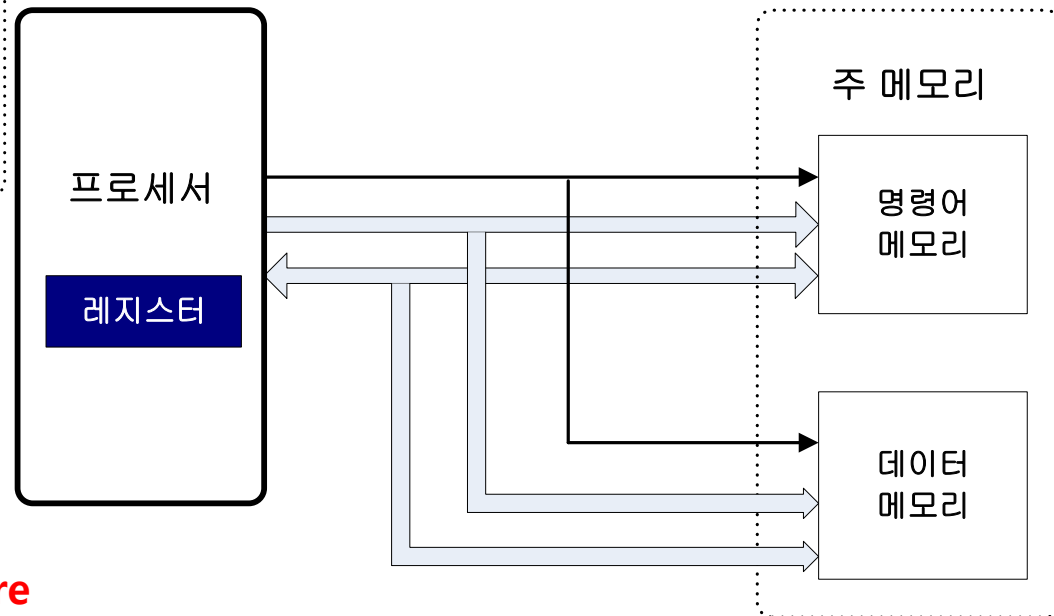
- ✓ 프로세서에서 기억장치나 I/O 장치에 입출력 동작을 지시하는 제어신호를 전송하는 경로

Architecture of ARM Processor

Harvard architecture



Von-Neumann architecture

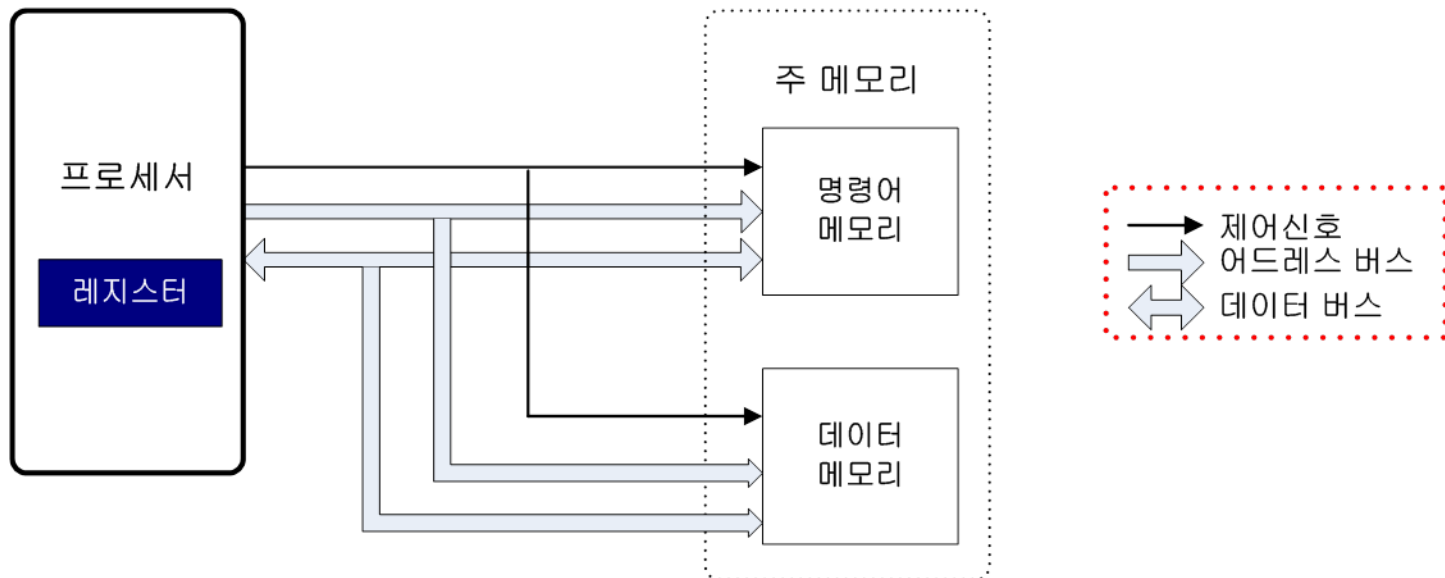


Architecture of ARM Processor

(3) Architecture : Bus

□ 폰 노이만(Von-Neumann) 아키텍처

- ❖ 명령어와 데이터를 위한 메모리 인터페이스가 하나이다.
- ❖ 명령어를 읽을 때 데이터를 읽거나 쓸 수 없다.
- ❖ IBM 계열 PC(개인용 PC), ARM7 등

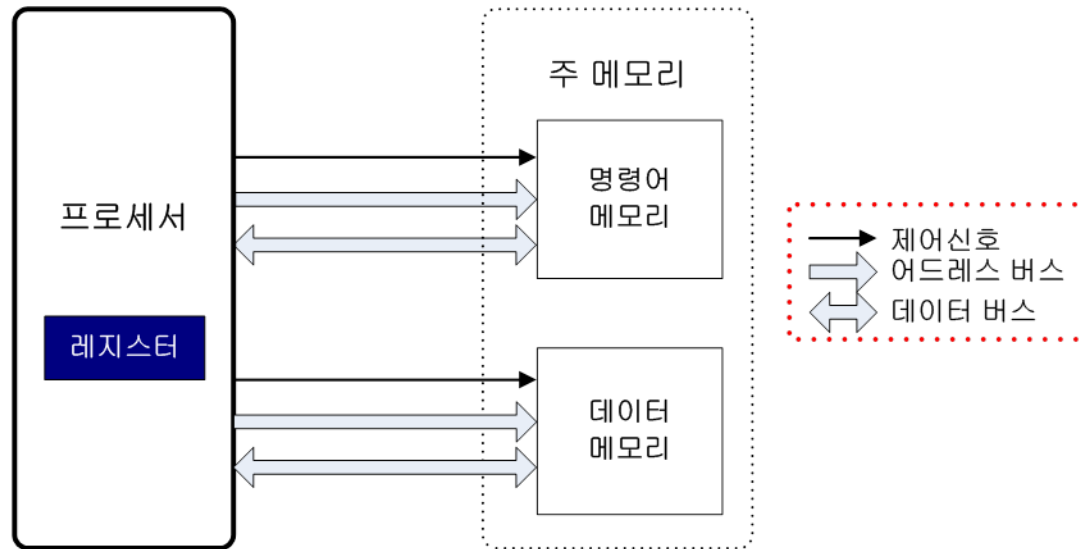


Architecture of ARM Processor

(3) Architecture : Bus

□ 하버드(Havard) 아키텍처

- ❖ 명령어를 위한 메모리 인터페이스와 데이터를 위한 메모리 인터페이스가 분리되어 있다.
- ❖ 명령어를 읽을 때 데이터를 읽거나 쓸 수 있어 성능이 우수하다.
- ❖ 버스 시스템이 복잡하여 설계가 복잡하다
- ❖ ARM9, ARM10, XScale 등



Architecture of ARM Processor

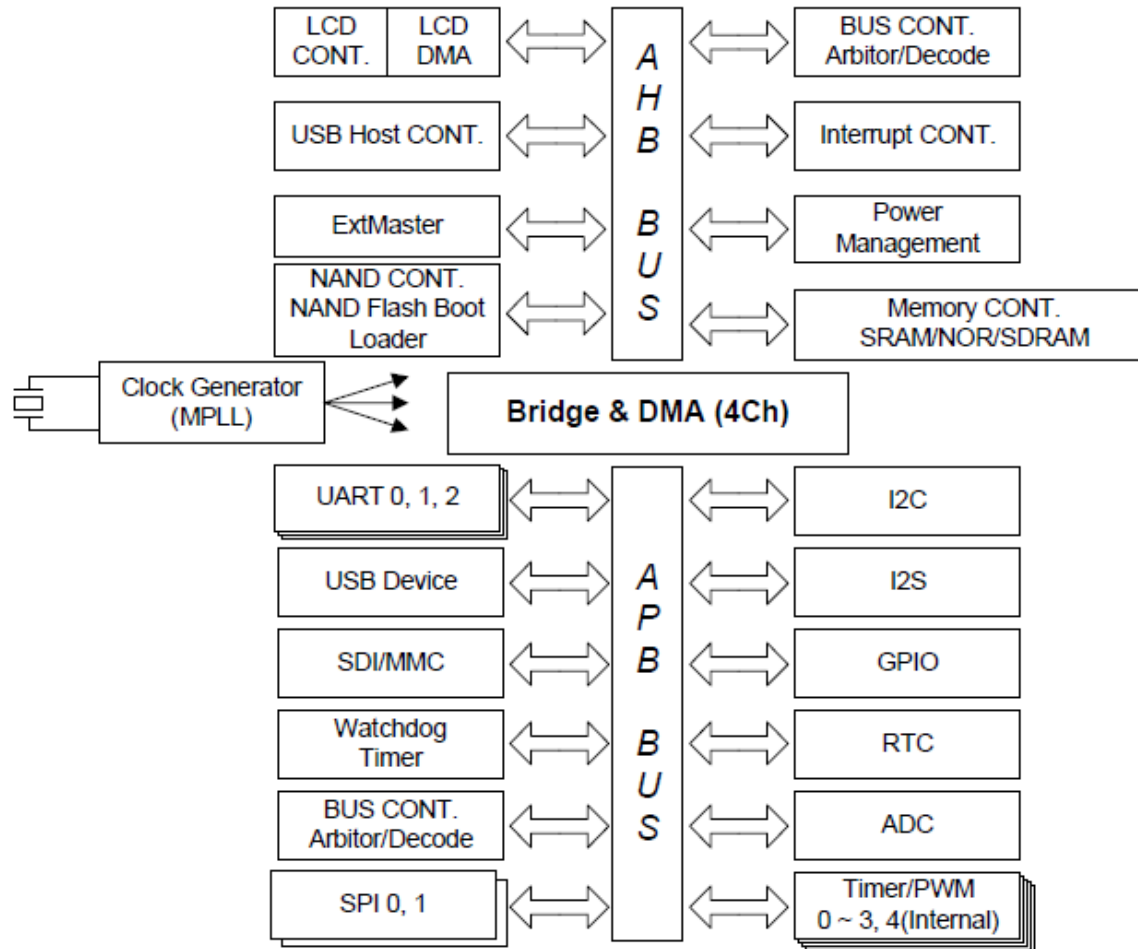
(3) Architecture : **AMBA Bus**

※ **AMBA** (Advanced Microcontroller Bus Architecture)

: **AHB** (Advanced High Performance Bus)

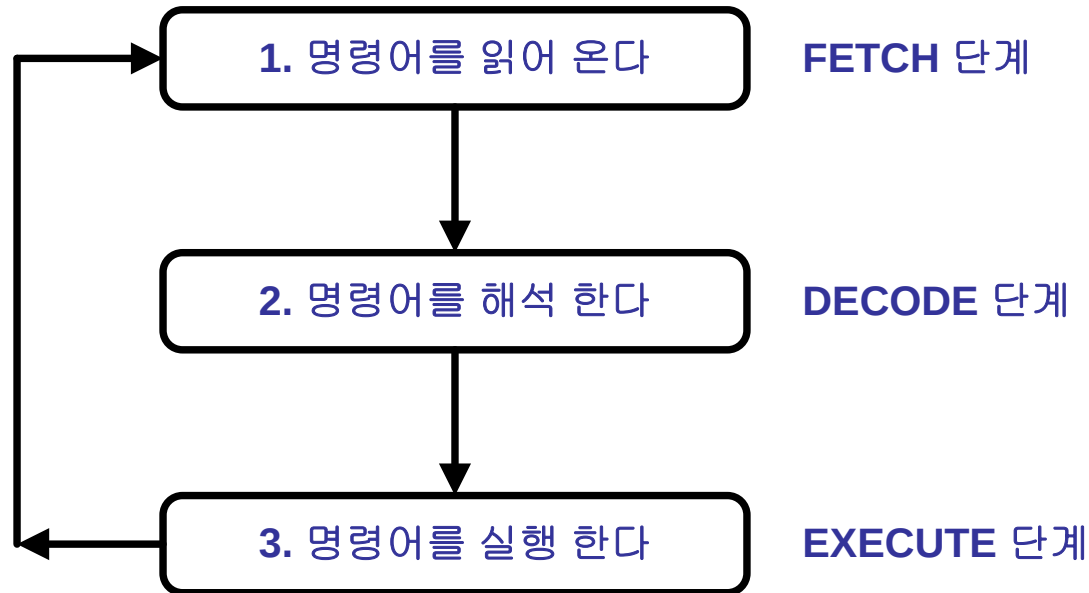
: **ASB** (Advanced System Bus)

: **APB** (Advanced Peripheral Bus)



Architecture of ARM Processor

(3) Architecture : 명령어 수행과정



Architecture of ARM Processor

(3) Architecture : Pipeline

□ 파이프라인

- ❖ 프로세서로 가는 명령어들의 움직임, 또는 명령어를 수행하기 위해 프로세서에 의해 취해진 산술적인 단계가 연속적이고 겹치는 것을 말한다.
- ❖ 파이프라인이 없으면 하나의 명령을 읽어와서 요구하는 연산을 수행하고, 다음 명령을 메모리에서 가져온다.
- ❖ 파이프라인을 사용하면 프로세서가 산술 연산을 수행하는 동안에 다음 명령어를 메모리에서 가져온다.

□ 파이프라인의 장점

- ❖ 명령어를 가져오고 실행하는 단계가 끊임없이 계속 된다.
- ❖ 주어진 시간동안에 수행될 수 있는 명령어의 수가 증가한다.

Architecture of ARM Processor

(3) Architecture : Pipeline

- ARM7 3steps pipeline



- ARM9 5steps pipeline



- Little/Big Endian

: 8bit 이상의 데이터에서 바이트 단위의 정렬방식.

(PC: Little Endian)

※ Cortex-M 시리즈는 항상 Little Endian 방식을 사용함

- Little Endian

: 상위 어드레스에 상위 데이터, 하위 어드레스에 하위 데이터가 저장

32bits data 0x12345678

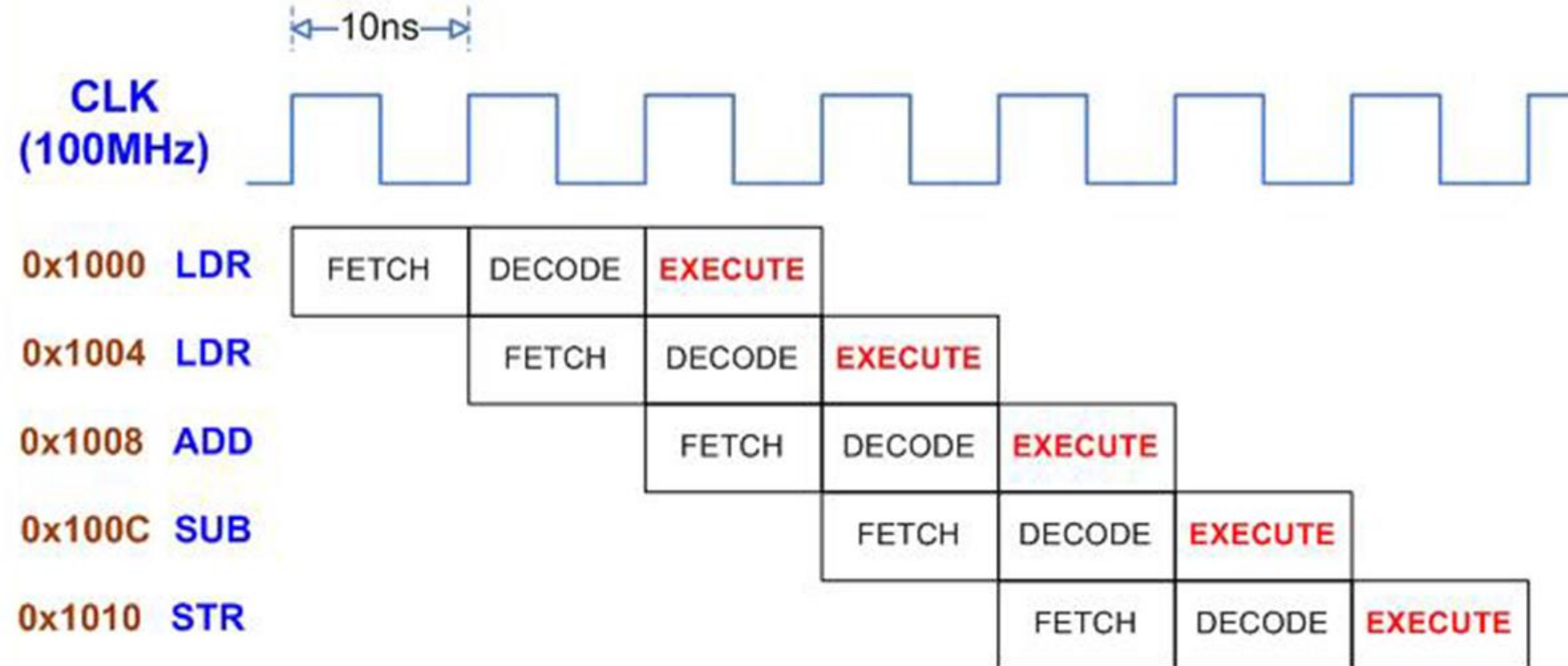
➔ 상위 address "0x12", 하위 address "0x78"

※ MSB (Most Significant Bit), LSB (Least Significant Bit)



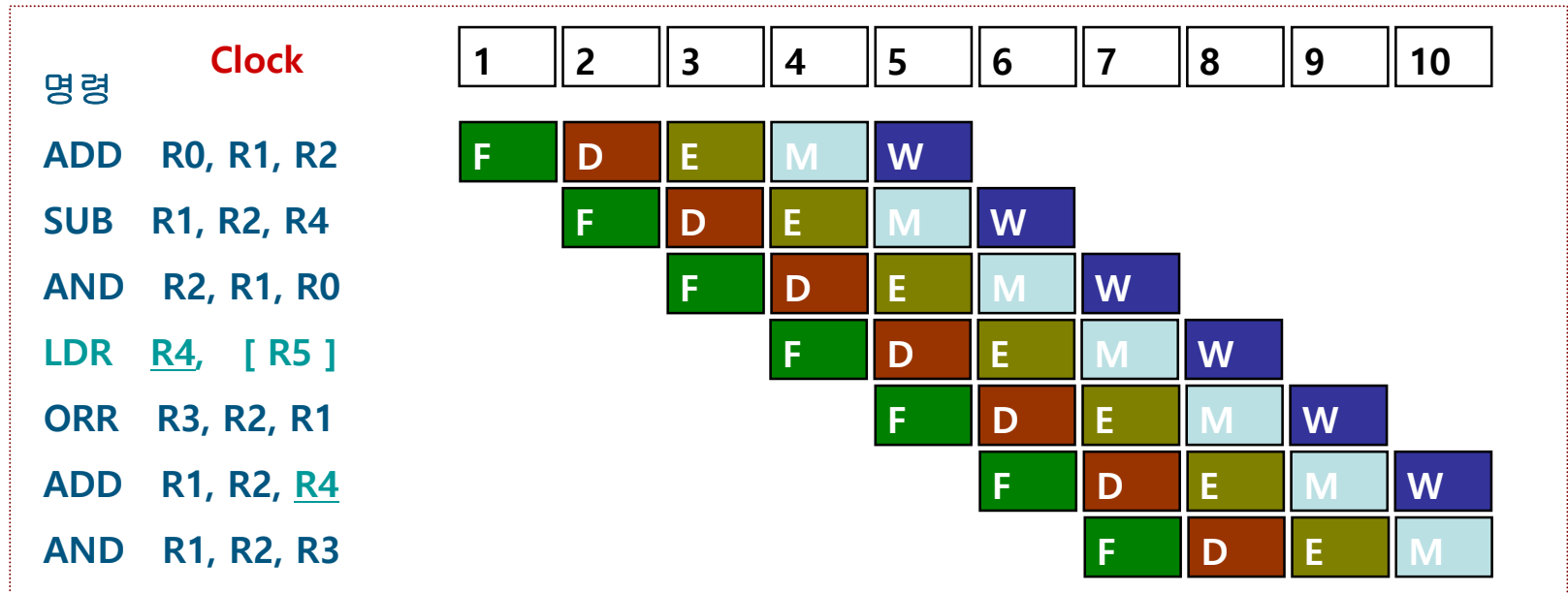
Architecture of ARM Processor

(3) Architecture : 3 Pipeline



Architecture of ARM Processor

(3) Architecture : 5 Pipeline



F : Fetch
 D : Decode
 E : Execute
 M : Memory
 W : Write
 I : Interlock



Architecture of ARM Processor

(3) Architecture : Pipeline

- **Very High Performance**

8BIT	16BIT	ARM7	ARM9
12MHz	33MHz	60MHz	200MHz

Clock
=Frequency/2

32bit

1clock = Pipeline (3steps,
5steps)

Cache	Write buffer	DMA	MMU(OS)
-------	--------------	-----	---------

ARM7TMI	Von-Neumann architecture
ARM9	Harvard architecture

- **Very Low Power Consumption**

ARM7TDMI	S3F445H	3.3V 150mA	5uA
ARM9	S3C2400	1.8V 3.3V 150mA	20uA

- **Very Small Die Size**

ARM7 0.18um process silicon wafer



Low cost

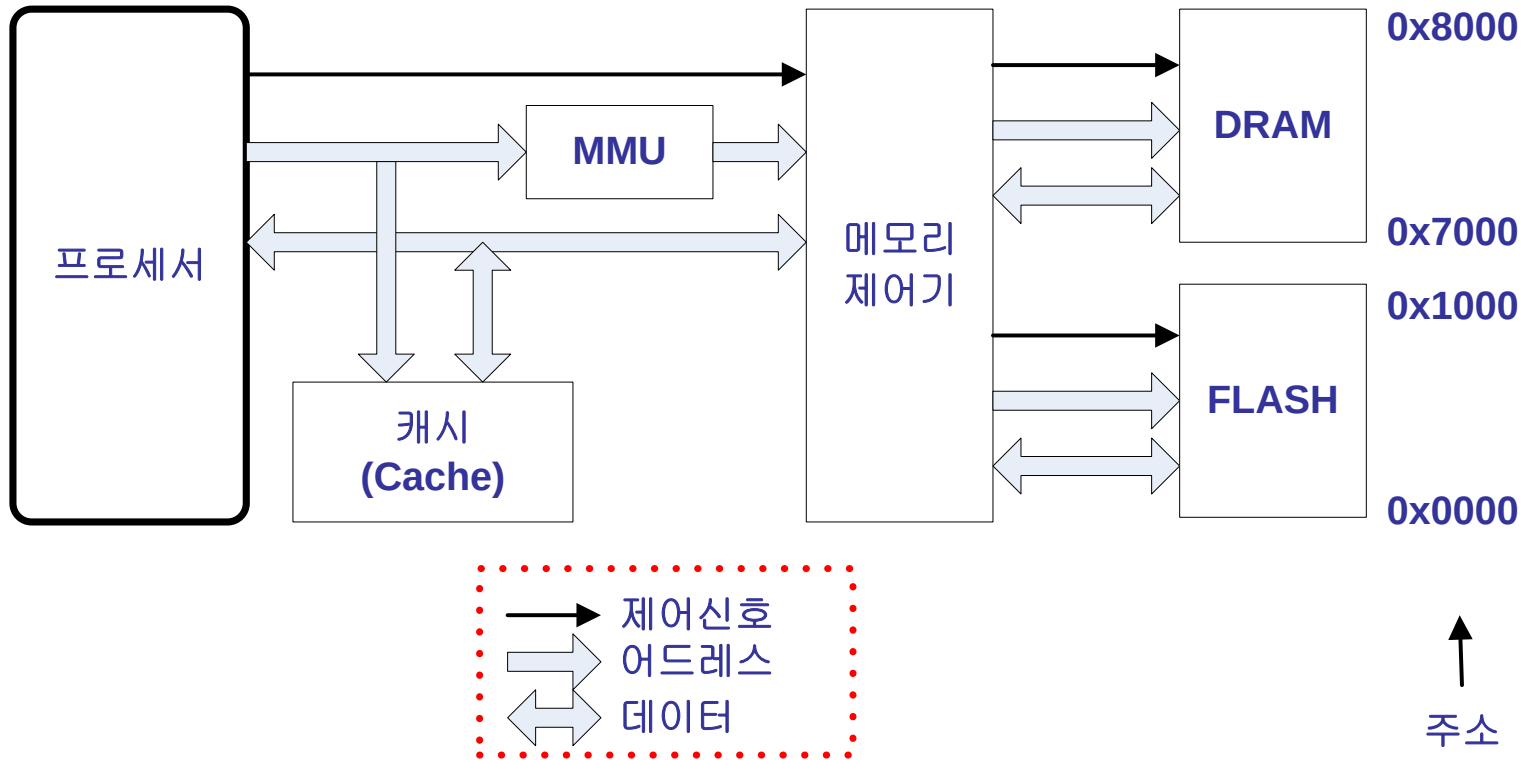
- **Very Small Code Size,
Good Code Density**

ARM, Thumb Instruction

- **Same development tool and CPU**

Architecture of ARM Processor

(3) Architecture : 메모리 시스템 구조



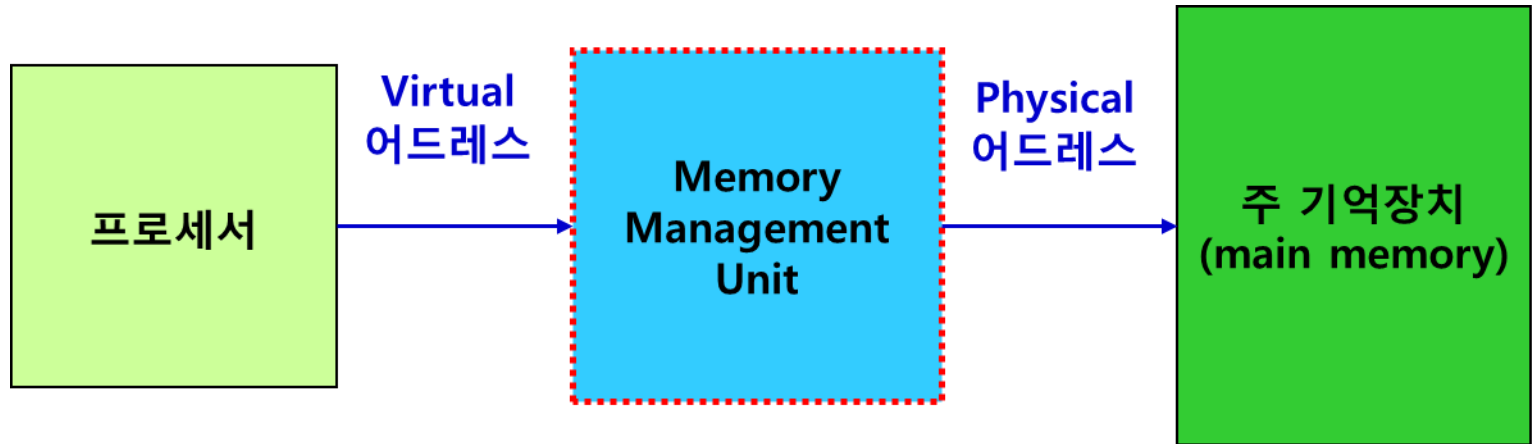
Architecture of ARM Processor

(3) Architecture : MMU (Memory Management Units)

□ 어드레스 변환(translation) 기능

❖ CPU에서 사용되는 logical 한 Virtual 어드레스를 physical 어드레스로 변환

□ 메모리 보호(protection) 기능



Architecture of ARM Processor

(3) Architecture : MMU (Memory Management Units)

Data	8bits format	$2 \times 2 \times 2 = 8$	2^3	HW	Hex
		256 (1Byte)	2^8	Data line 8개	
		1024	2^{10}	Data line 10개	
Address	8bits format	256개 공간	2^8	Address line 8개	물리 공간 (address) $2^8 = 255 = 0xFF = 11111111_{(2)}$ $256 = 0x100 = 000100000000_{(2)}$ $1024 = 0x010000000000_{(2)}$ $2^{16} = 2^8 \times 2^8 = 0xFFFF$ $2^{32} = 2^8 \times 2^8 \times 2^8 \times 2^8 = 0xFFFFFFFF$
Size	address x data	$(D)2^8 \times (A)2^8 = 2^{16}$	$2^{16} = 2^8 \times 2^8 = 2^8 \text{Byte}$	Address line 8개	
		address 16bits 경우 $(D)2^8 \times (A)2^{16}$	$2^8 \times 2^{16} = 2^{22} \text{bits}$	Address line 16개	

0x1FF4000	SFR 48kByte	0x1FF3FFF
0x1FF0000	Internal 16kB SRAM	0x1FEFFFF
0x1C00000	External 64k x 16bit SRAM	0x1C1FFFF
0x1800000	External 16k x 16bit FPGA	0x1BFFFFF
	Undefined Region	0x1807FFF
0x1400000	Undefined Region	0x17FFFFF
0x1000000	Undefined Region	0x13FFFFF
0x0C00000	Undefined Region	0x0FFFFF
0x0800000	Undefined Region	0x0BFFFFF
0x0400000	Undefined Region	0x07FFFFF
0x0000000	Flash ROM	0x03FFFFF
		0x007FFFF

0x7FFFF = 0x11111111111111111111₍₂₎ 19bits (address line)
0x3FFFF = 0x111111111111111111111111₍₂₎ 22bits => $2^2 \times 2^{10} \times 2^{10}$ => $4 \times K \times K = 4\text{Mbits}$

Architecture of ARM Processor

(3) Architecture : 명령어

□ ARM 프로세서의 명령어

- ❖ 32 비트 ARM instruction set
- ❖ 16 비트 Thumb instruction set

Architecture of ARM Processor

(3) Architecture : 명령어

□ 32 비트 ARM 명령어의 특징

- ❖ 모든 ARM 명령어는 조건부 실행이 가능하다.
- ❖ 모든 ARM 명령은 32 비트로 구성되어 있다.
 - Load/Store와 같은 메모리 참조 명령이나 Branch 명령에서는 모두 상대주소(Indirect Address)방식을 사용한다
- ❖ ARM 명령은 크게 11개의 기본적인 Type으로 구분된다.

□ 32 비트의 고정된 명령 길이를 사용하는 이유

- ❖ Pipeline 구성이 용이
- ❖ 명령 디코더의 구현이 쉽다
- ❖ 고속으로 처리 가능

Architecture of ARM Processor

(3) Architecture : 명령어

□ Thumb 명령어

- ❖ 32비트의 ARM 명령을 16비트로 재구성한 명령

□ Thumb 명령어의 장점

- ❖ 코드의 크기를 줄일 수 있다

- ARM 명령의 65%

- ❖ 8비트 나 16비트와 같은 좁은 메모리 인터페이스에서 ARM 명령을 수행할 때 보다 성능이 우수하다

□ Thumb 명령어의 단점

- ❖ 조건부 실행이 안됨.

Architecture of ARM Processor

(3) Architecture : 명령어 요약

	Instruction Type	Instruction (명령)
1	Branch, Branch with Link	B, BL
2	Data Processing 명령	ADD, ADC, SUB, SBC, RSB, RSC, AND, ORR, BIC, MOV, MVN, CMP, CMN, TST, TEQ
3	Multiply 명령	MUL, MLA, SMULL, SMLAL, SMULL, UMLAL
4	Load/Store 명령	LDR, LDRB, LDRBT, LDRH, LDRSB, LDRSH, LDRT, STR, STRB, STRBT, STRH, STRT
5	Load/Store Multiple 명령	LDM, STM
6	Swap 명령	SWP, SWPB
7	Software Interrupt(SWI) 명령	SWI
8	PSR Transfer 명령	MRS, MSR
9	Coprocessor 명령	MRC, MCR, LDC, STC
10	Branch Exchange 명령	BX
11	Undefined 명령	

Architecture of ARM Processor

(3) Architecture : ARM & Thumb state

□ ARM state와 Thumb state

❖ ARM state

- 32 비트 ARM 명령을 수행하는 상태
- Reset 및 SWI, IRQ, FIQ, UNDEF, ABORT와 같은 Exception 발생시 프로세서는 무조건 ARM state가 된다.

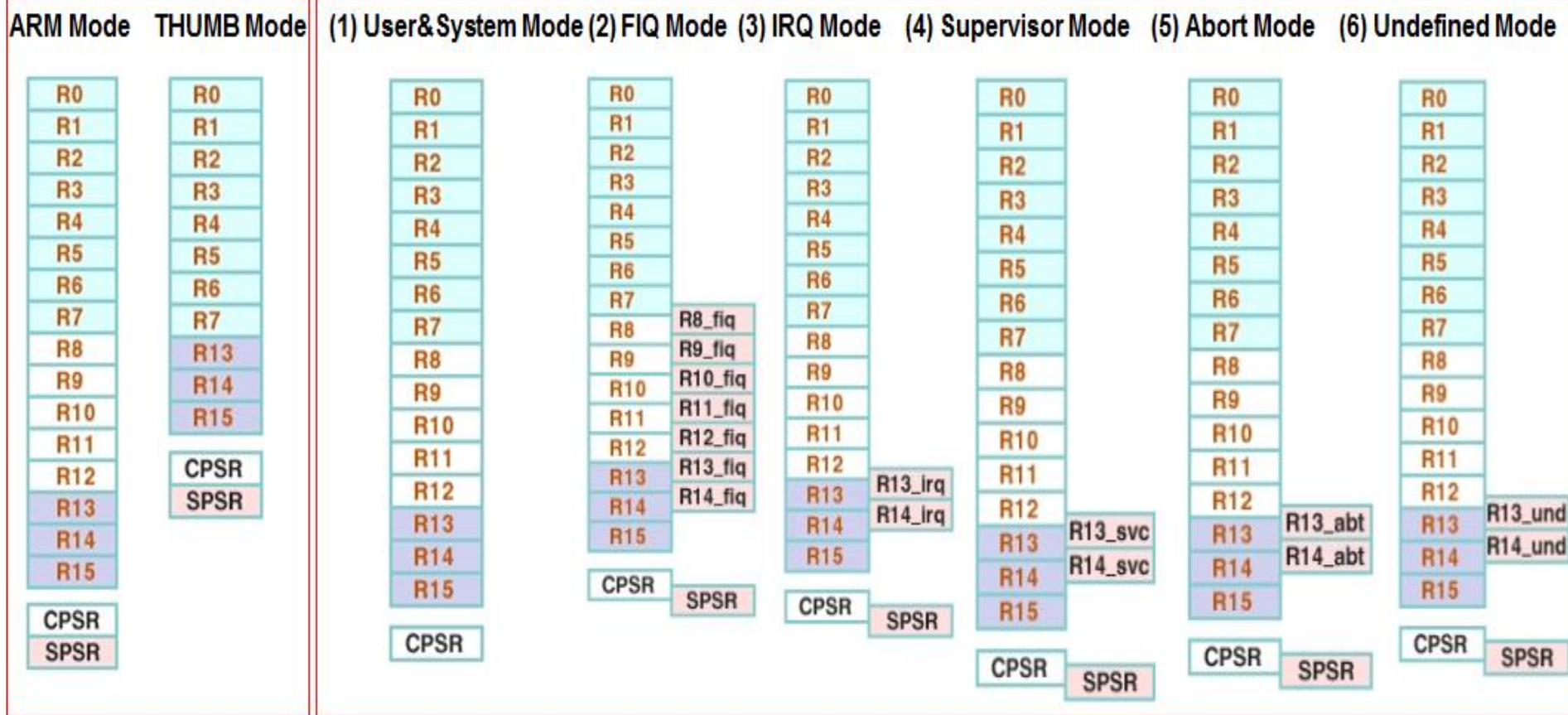
❖ Thumb state

- 16 비트 Thumb 명령을 수행하는 상태

□ ARM / Thumb Interwork

- ❖ ARM state에서 Thumb state로 Thumb state에서 ARM state로 상태가 전환 되는 작업
- ❖ BX 명령이 의해서 이루어 진다.

2. Architecture of ARM Processor



R0-15 : General register

CPSR(Current Program Status Register)

SPSR(Saved Program Status Register)

R13 : SP(Stack Pointer)

R14 : LR(Link Register: PC값을 임시로 저장, "BL"의 실행시 사용)

R15 : PC(Program Counter: 다음에 수행이 되어야 할 명령어의 주소)

Architecture of ARM Processor

(4) Operating mode

□ 현재 프로세서가 어떤 권한을 가지고 어떤 종류의 작업을 하고 있는지를 나타낸다.

❖ User Mode

➤ User task 또는 어플리케이션을 수행할 때의 프로세서의 동작 모드

❖ FIQ(Fast Interrupt Request) Mode

➤ 빠른 인터럽트의 처리를 위한 프로세서의 동작 모드

❖ IRQ(Interrupt Request) Mode

➤ 일반적으로 사용되는 인터럽트를 처리하기 위한 프로세서의 동작 모드

❖ SVC(Supervisor) Mode

➤ 시스템 자원을 관리할 수 있는 프로세서의 동작 모드

➤ Operating 시스템의 커널이나 드라이버를 처리하는 모드

➤ Reset이나 소프트웨어 인터럽트(SWI)가 발생하면 ARM은 Supervisor 모드가 된다.

Architecture of ARM Processor

(4) Operating mode

❖ Abort Mode

- 명령이나 데이터를 메모리로부터 읽거나 쓸 때 오류가 발생하면 ARM은 Abort 모드가 된다.

❖ Undefined Mode

- ARM이 정의되지 않은 명령을 수행하려고 하면 수행되는 프로세서의 동작 모드

❖ System Mode

- User 모드와 동일한 용도로 사용
- ARM Architecture v4이후부터(ARM7) 지원된다.

Architecture of ARM Processor

(4) Operating mode

□ 예외처리(Exception)

- ❖ IRQ, FIQ를 비롯한 대부분의 Operating 모드는 외부에서 발생하는 조건에 의해서 ARM 프로세서가 하드웨어적으로 변경한다.
- ❖ 외부에서 발생하는 물리적인 조건에 의해서 정상적인 프로그램의 실행을 미루고 예외적인 현상을 처리하는 것을 Exception이라 한다.
- ❖ Operating 모드의 변경은 소프트웨어에 의하여 제어 할 수도 있다.

□ 시스템 콜

- ❖ User 모드에서 수행되는 프로그램에서 Supervisor 모드로 전환하여 시스템 자원을 사용할 수 있게 해주는 인터페이스
- ❖ OS에서는 소프트웨어 인터럽트(SWI)를 사용하여 시스템 콜을 구현

Architecture of ARM Processor

(4) Operating mode

- 30개의 범용(General Purpose) 레지스터
 - ❖ 대부분 데이터 연산 등에 사용
 - ❖ 프로세서의 동작(Operating) 모드에 따라 사용되는 레지스터가 제한된다
- 1개의 프로그램 카운터(PC : Program Counter)
 - ❖ 프로그램을 읽어올 메모리의 위치를 나타낸다.
- 1개의 CPSR(Current Program Status Register)
 - ❖ 프로세서가 수행하고 있는 현재의 동작 상태를 나타낸다.
- 5개의 SPSR(Saved Program Status Register)
 - ❖ 이전 모드의 CPSR의 복사본으로
 - Exception이 발생하면 ARM이 하드웨어적으로 이전 모드의 CPSR 값을 각각의 모드에 해당하는 SPSR에 복사

Architecture of ARM Processor

(4) Operating mode

□ ARM state의 경우

❖ 16개의 범용 레지스터

- R0에서 R15의 키워드를 사용하여 관리
- R0에서 R12는 연산 명령과 같은 범용으로 사용
- R13(SP), R14(LR), R15(PC)는 특수한 목적으로 사용

❖ 1개의 CPSR

❖ Privilege 모드의 경우 각각 1개의 SPSR

□ Thumb state의 경우

❖ 8개의 범용 레지스터

- R0에서 R7

❖ R13(SP), R14(LR), R15(PC) 레지스터

❖ 1개의 CPSR

❖ Privilege 모드의 경우 각각 1개의 SPSR

Architecture of ARM Processor

(4) Operating mode : Stack Point (SP, R13)

- 프로그램에서 사용하는 스택의 위치를 저장하는 레지스터
- 프로세서의 동작 모드마다 별도로 할당된 SP 레지스터를 가지고 있다
- ARM은 별도의 스택 명령이 없다
 - ❖ Push나 Pop 과 같은 별도의 스택 명령을 제공하지 않는다
 - ❖ LDR/STR, LDM/STM 같은 데이터 전송 명령을 사용하여 스택을 처리

Architecture of ARM Processor

(4) Operating mode : Link Register (LR, R14)

- 서브루틴에서 되돌아 갈 위치 정보를 저장하고 있는 레지스터
 - ❖ 서브루틴을 호출하는 BL 명령을 사용하면 PC 값을 자동으로 LR에 저장
 - ❖ 되돌아 갈 때는 MOV 명령을 이용하여 LR 값을 PC에 저장

MOV PC, LR

- ❖ ADD나 SUB와 같은 연산 명령을 이용하여 되돌아갈 위치를 조정 할 수도 있다.
- 프로세서의 동작 모드마다 별도로 할당된 LR 레지스터를 가지고 있다

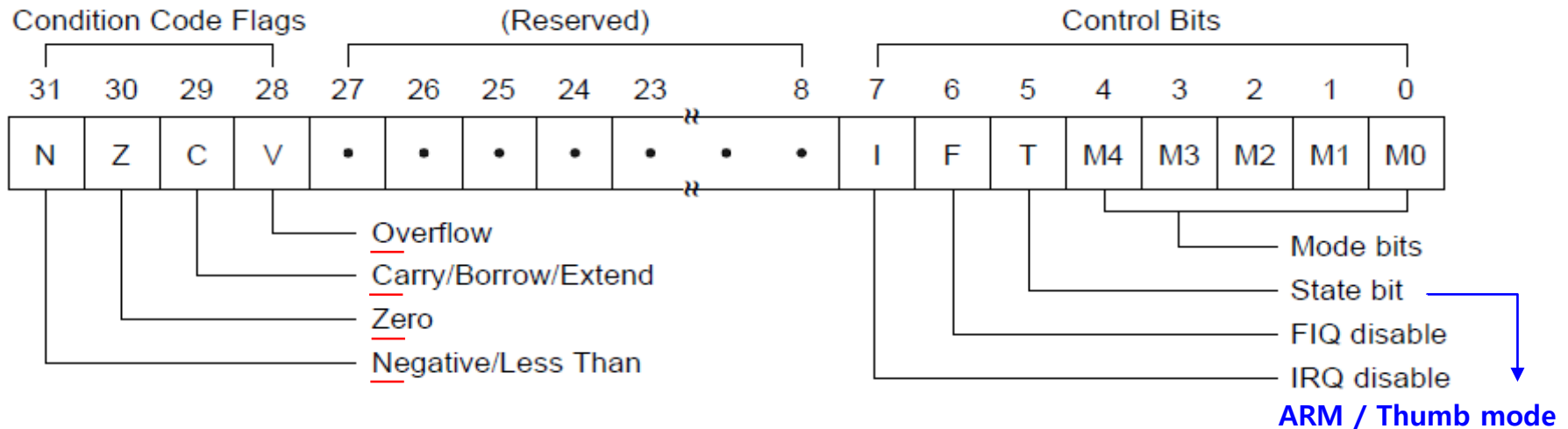
Architecture of ARM Processor

(4) Operating mode : Program Counter (PC, R15)

- 프로그램을 수행하는 위치를 저장하고 있는 레지스터
 - ❖ ARM state의 경우 위치 정보 비트 [31:2] 저장
 - ❖ Thumb state의 경우 위치 정보 비트 [31:1] 저장
- 다른 범용 레지스터와 마찬가지로 ADD, SUB와 같은 연산 명령을 사용하여 프로그램이 분기할 위치를 조정 가능
- 프로세서의 모든 동작 모드에 대하여 하나만 존재한다.

Architecture of ARM Processor

(4) Operating mode : Program Status Register (PSR)



M[4:0]	Mode	Accessible registers
0b10000	User	PC, R14 to R0, CPSR
0b10001	FIQ	PC, R14_fiq to R8_fiq, R7 to R0, CPSR, SPSR_fiq
0b10010	IRQ	PC, R14_irq, R13_irq, R12 to R0, CPSR, SPSR_irq
0b10011	Supervisor	PC, R14_svc, R13_svc, R12 to R0, CPSR, SPSR_svc
0b10111	Abort	PC, R14_abt, R13_abt, R12 to R0, CPSR, SPSR_abt
0b11011	Undefined	PC, R14_und, R13_und, R12 to R0, CPSR, SPSR_und
0b11111	System	PC, R14 to R0, CPSR (ARM architecture v4 and above)

Architecture of ARM Processor

(4) Operating mode : Program Status Register (PSR)

□ Flag bits는 ALU를 통한 명령의 실행 결과를 나타내는 부분이다.

❖ Negative flag ('N' 비트)

➤ ALU 연산 결과 마이너스가 발생한 경우 세트

❖ Zero flag ('Z' 비트)

➤ ALU 연산 결과 0가 발생한 경우 세트

❖ Carry flag ('C' 비트)

➤ ALU 연산 결과 자리올림이나 내림이 발생한 경우 세트

➤ shift 연산에서 carry가 발생한 경우 세트

❖ oVerflow flag ('V' 비트)

➤ ALU 연산 결과 overflow가 발생하면 세트

Architecture of ARM Processor

(4) Operating mode : Program Status Register (PSR)

□ 프로세서의 모드, 동작 state 와 인터럽트를 제어

❖ I/F 비트

- IRQ('I' 비트) 또는 FIQ('F' 비트)를 disable 또는 enable

❖ T 비트

- Thumb state 임을 나타낸다.
- 상태를 나타낼 뿐 프로세서는 이 비트에 강제로 값을 write 할 수 없다.

❖ Mode bits

- ARM의 7 가지의 동작 모드를 나타낸다.

M[4:0]	Operating Mode	M[4:0]	Operating Mode
10000	User 모드	10111	Abort 모드
10001	FIQ 모드	11011	Undefined 모드
10010	IRQ 모드	11111	System 모드
10011	SVC 모드		

Architecture of ARM Processor

Processor mode		Description
User	usr	Normal program execution mode
FIQ	fiq	Supports a high-speed data transfer or channel process
IRQ	irq	Used for general-purpose interrupt handling
Supervisor	svc	OS Mode이며 SWI exception 발생할 때 진입, RESET exception 발생할 때 진입
Abort	abt	Data read, write時 메모리 시스템이 읽거나 쓸 수 없다고 하는 경우에 발생
Undefined	und	명령어를 fetch했으나, ARM 가 모르는 명령어인 경우에 발생
System	sys	Runs privileged operating system tasks - 특권 mode로서 OS kernel에서 사용되고 - User mode와 동일 register 사용되나 stack에 의해 원활히 동작

※ SWI : Software Interrupt (OS 호출하기 위해 만들어짐)

Privileged Mode (특권 모드)

: IRQ나 FIQ등의 Interrupt의 사용 가능 유무를 직접 설정 할 수 있음.

: IRQ, FIQ, SVC, Abort, Undefined, SYS

: Privileged Mode 는 서로 Mode 변경이 가능 (SYS ↔ FIQ, IRQ ↔ SVC)

: Privileged Mode → Normal Mode (USR)은 가능하지만, USR → Privileged Mode로의 변경은 불가능.

Architecture of ARM Processor

(5) Exception

□ 예외처리(Exception)

- ❖ 외부의 요청이나 오류에 의해서 정상적으로 진행되는 프로그램의 동작을 잠시 멈추고 프로세서의 동작 모드를 변환하고 미리 정해진 프로그램을 이용하여 외부의 요청이나 오류에 대한 처리를 하도록 하는 것
- ❖ Exception의 예
 - I/O 장치에서 인터럽트를 발생시키면 IRQ Exception이 발생하고, 프로세서는 발생한 IRQ Exception을 처리하기 위해 IRQ 모드로 전환되어 요청된 인터럽트에 맞는 처리 동작 수행

□ ARM의 Exception

- ❖ Reset
- ❖ Undefined Instruction
- ❖ Software Interrupt
- ❖ Prefetch Abort
- ❖ Data Abort
- ❖ IRQ(Interrupt Request)
- ❖ FIQ(Fast Interrupt Request)

Architecture of ARM Processor

(5) Exception Priority

Exception	Vector Address	우선순위	전환되는 동작모드
Reset	0x0000 0000	1 (High)	Supervisor(SVC)
Undefined Instruction	0x0000 0004	6 (Low)	Undefined
Software Interrupt(SWI)	0x0000 0008	6	Supervisor(SVC)
Prefetch Abort	0x0000 000C	5	Abort
Data Abort	0x0000 0010	2	Abort
Reserved	0x0000 0014		
IRQ	0x0000 0018	4	IRQ
FIQ	0x0000 001C	3	FIQ

Architecture of ARM Processor

(5) Exception

- Reset Exception

: 전원이 최초 공급이 된 상황, 혹은 하드웨어적으로 리셋이 걸리는 경우 발생.

- Data Abort Exception

: 버스를 통해 명령어가 아닌 데이터(데이터)를 읽으려고 하지만 동작이 실패하였을 경우 발생.

- Prefetch Abort Exception

: CPU 는 명령어를 읽지만(prefetch), 시스템 메모리에서 명령어 코드를 읽을 수 없는 경우 발생.

- Undefined Instruction Exception

: 명령어를 읽었으나, ARM이 수행할 수 없는 명령어인 경우 발생.

- FIQ, IRQ Exception

- SWI(Software Interrupt) Exception

: OS상의 여러 프로그램들이 공유해서 사용하는 루틴에서 소프트웨어 인터럽트를 주로 사용함.

OS가 통상 supervisor 모드에서 동작하므로, SWI 발생시 이 모드로 전환됨.

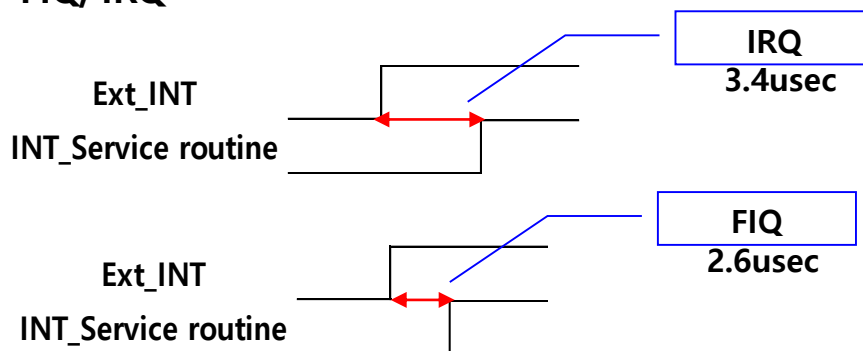
Architecture of ARM Processor

(5) Exception

Exception	Priority	Mode	Vector
Reset	1	Supervisor	0x00000000
Data Abort	2	Abort	0x00000010
FIQ	3	FIQ	0x0000001c
IRQ	4	IRQ	0x00000018
Prefetch Abort	5	Abort	0x0000000c
Underfined Instruction	6	Undef	0x00000004
SWI	7	Supervisor	0x00000008

```
B ResetHandler ; 0x00 Reset Exception
B HandlerUndef ; 0x04 Undefined Exception
B HandlerSWI ; 0x08 SWI Exception
B HandlerPabort ; 0x0c Prefetch Memory Abort
B HandlerDabort ; 0x10 Data Memory Abort
B . ; 0x14 Reserved
B HandlerIRQ ; 0x18 IRQ Exception
B HandlerFIQ ; 0x1c FIQ Exception
```

- **Handler**
: ResetHandler, Handler- (HandlerIRQ, HandlerFIQ, ...)
- **FIQ, IRQ**



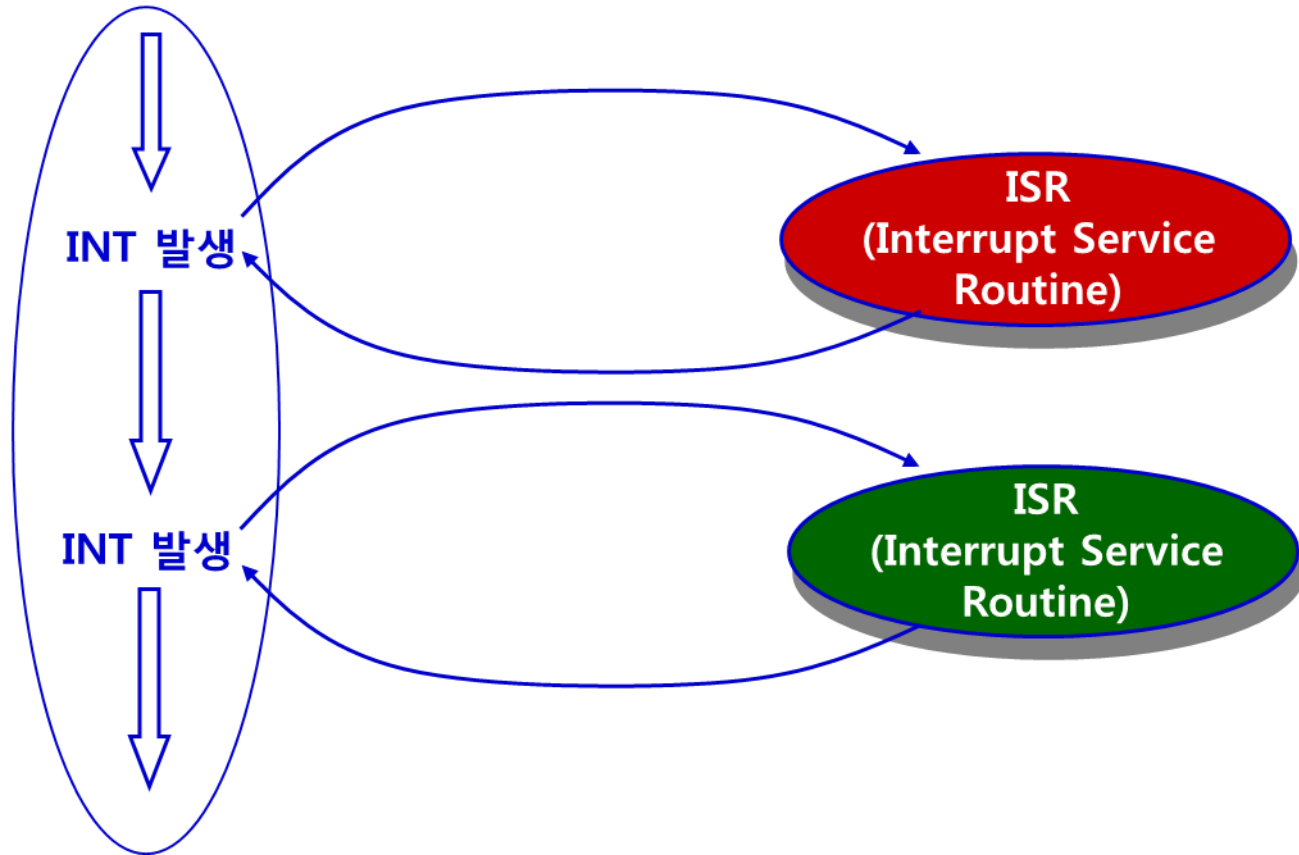
Question.

왜 address를 이렇게 표현했을까?

Architecture of ARM Processor

(5) Interrupt (Exception)

MAIN 프로그램 루틴



(5) Interrupt (Exception) Vector

□ 인터럽트 벡터

- ❖ 인터럽트 서비스 루틴을 처리하기 위한 명령 또는 위치가 저장된 메모리 공간

□ 인터럽트 벡터 주소 지정 방식

❖ 벡터 인터럽트(Vectored interrupt)

- 일반적인 마이크로프로세서 장치에서 여러 개의 주변 장치가 시스템 버스에 연결되어 사용되는 경우
- 주변장치가 인터럽트를 처리할 주소를 제공

Architecture of ARM Processor

(5) Interrupt (Exception) Vector

```
AREA      Init, CODE, READONLY
ENTRY
```

```
;-----
; Exception Vector Table
;-----
```

```
B ResetHandler   ; 0x00  Reset Exception
B HandlerUndef   ; 0x04  Undefined Exception
B HandlerSWI     ; 0x08  SWI Exception
B HandlerPabort  ; 0x0c  Prefetch Memory Abort
B HandlerDabort  ; 0x10  Data Memory Abort
B .              ; 0x14  Reserved
B HandlerIRQ     ; 0x18  IRQ Exception
B HandlerFIQ     ; 0x1c  FIQ Exception
```

```
;-----
; H/W Interrupt Vector Table
;-----
```

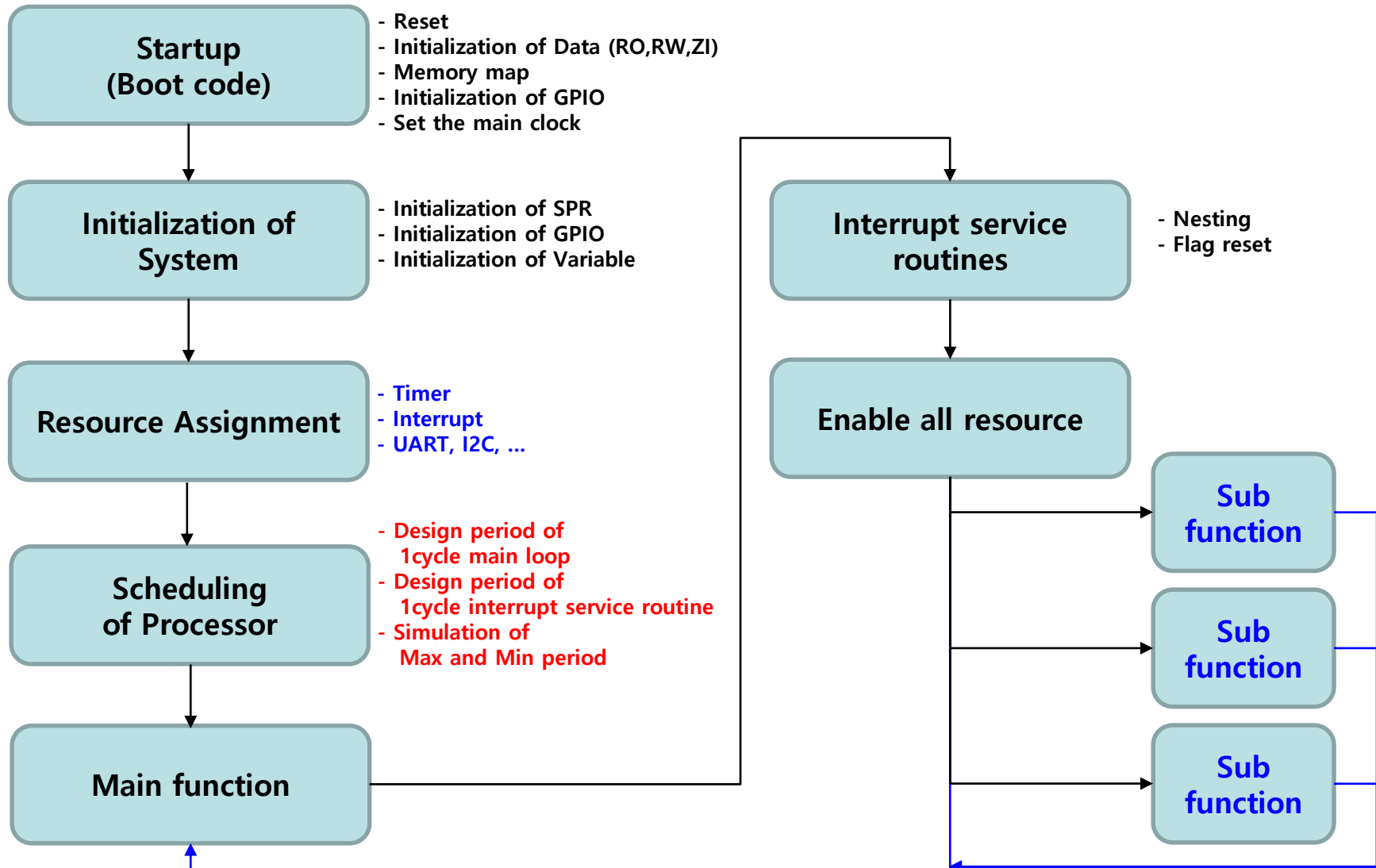
```
B HandlerEINT0   ; 0x20  EINT0
B HandlerEINT1   ; 0x24  EINT1
B HandlerEINT2   ; 0x28  EINT2
B HandlerEINT3   ; 0x2c  EINT3
B HandlerURX0    ; 0x30  INT_URX0
B HandlerUTX0    ; 0x34  INT_UTX0
B HandlerUERR0   ; 0x38  INT_UERR0
B HandlerURX1    ; 0x3c  INT_URX1
B HandlerUTX1    ; 0x40  INT_UTX1
B HandlerUERR1   ; 0x44  INT_UERR1
B HandlerTOF0    ; 0x48  INT_TOF0
B HandlerTMC0    ; 0x4c  INT_TMC0
B HandlerTOF1    ; 0x50  INT_TOF1
B HandlerTMC1    ; 0x54  INT_TMC1
B HandlerTOF2    ; 0x58  INT_TOF2
B HandlerTMC2    ; 0x5c  INT_TMC2
B HandlerTOF3    ; 0x60  INT_TOF3
B HandlerTMC3    ; 0x64  INT_TMC3
B HandlerTOF4    ; 0x68  INT_TOF4
B HandlerTMC4    ; 0x6c  INT_TMC4
B HandlerTOF5    ; 0x70  INT_TOF5
B HandlerTMC5    ; 0x74  INT_TMC5
B HandlerBT      ; 0x78  INT_BT
B HandlerSIO0    ; 0x7c  INT_SIO0
B HandlerSIO1    ; 0x80  INT_SIO1
B HandlerEINT4   ; 0x84  EINT4
B HandlerEINT5   ; 0x88  EINT5
B HandlerADC     ; 0x8c  INT_ADC
B HandlerEINT6   ; 0x90  EINT6
B HandlerEINT7   ; 0x94  EINT7
```

Reference : startup.s

Architecture of ARM Processor

Startup code

(1) Algorithm & Flowchart

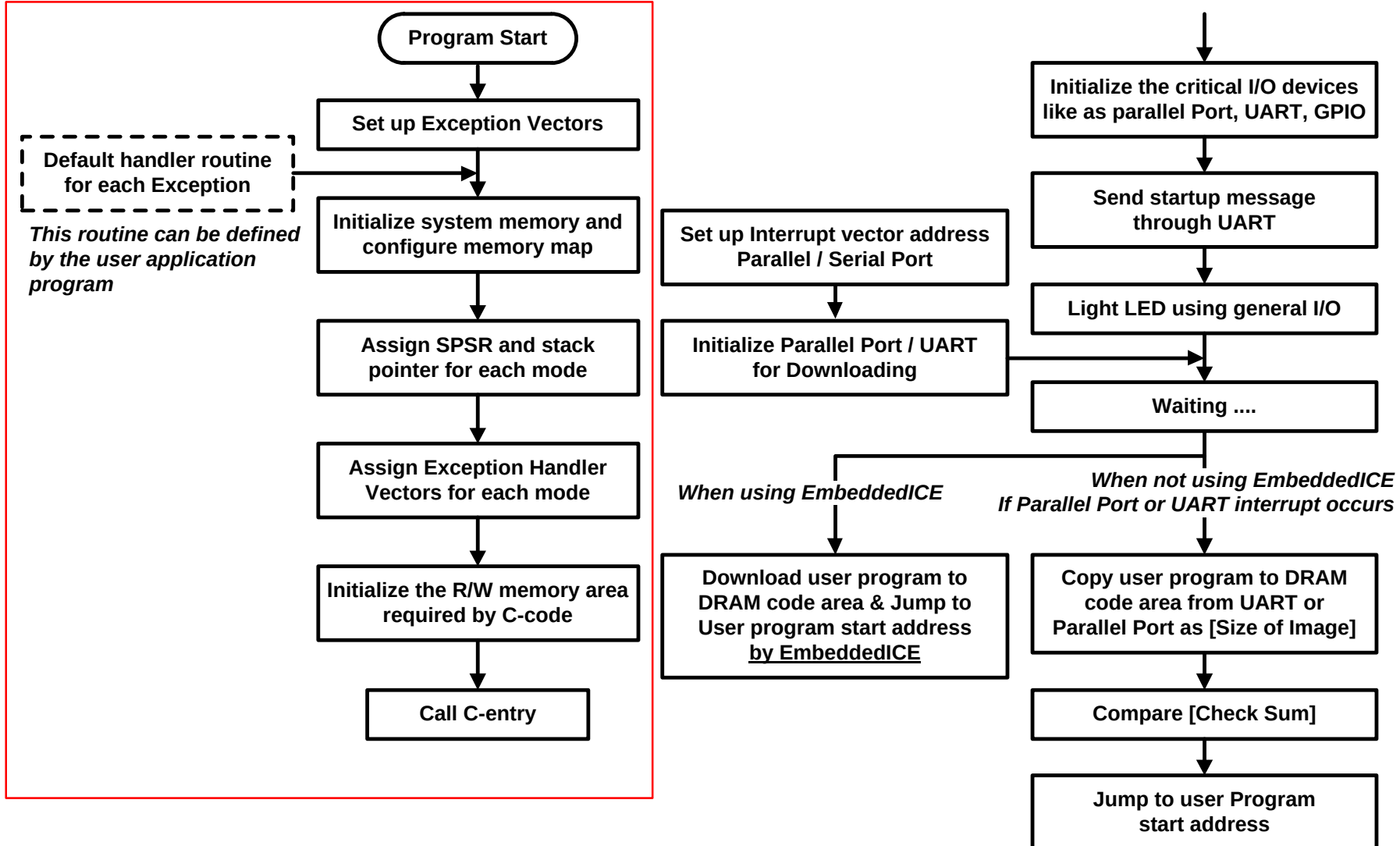


Architecture of ARM Processor

Startup code

Reference : startup.s

(2) Startup code – Structure of Boot program

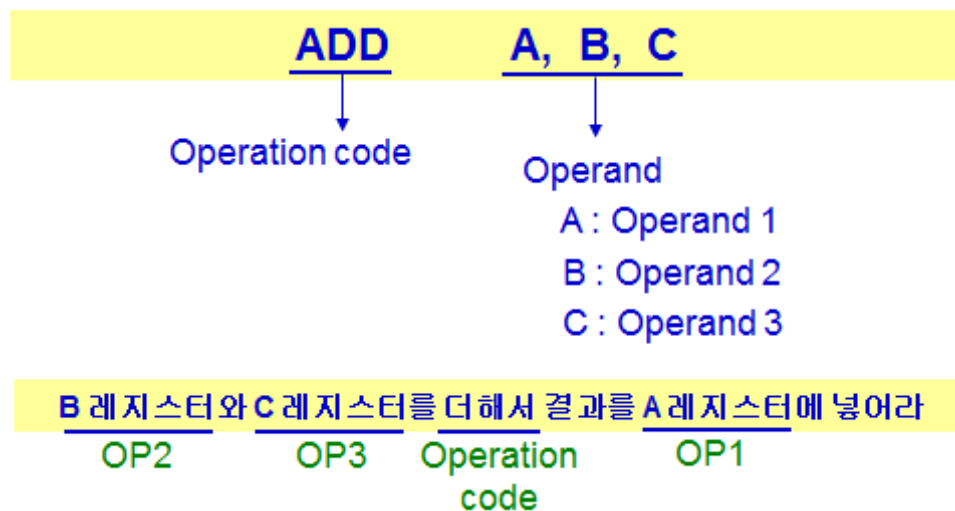


Architecture of ARM Processor

Startup code

(3) Startup code - Overview

- Examples are included **header program**, **the root program** and for additional main program modules.
- The Root Operation Structure is required to configure your system and must be coded in the **ARM assembly language**.
- It consists of the following function modules:
 - Set up **exception vectors** for each mode
 - Set up the **stack area** for each mode
 - Configure the **memory map**
- Instruction
 - Operation code
 - Operand (피연산자)



```
-----  
; H/W Interrupt Vector Table  
-----  
B HandlerEINT0 ; 0x20 EINT0  
B HandlerEINT1 ; 0x24 EINT1  
B HandlerEINT2 ; 0x28 EINT2  
B HandlerEINT3 ; 0x2c EINT3  
B HandlerURX0 ; 0x30 INT_URX0  
B HandlerUTX0 ; 0x34 INT_UTX0  
B HandlerUERR0 ; 0x38 INT_UERR0  
B HandlerURX1 ; 0x3c INT_URX1  
B HandlerUTX1 ; 0x40 INT_UTX1  
B HandlerUERR1 ; 0x44 INT_UERR1  
B HandlerTOF0 ; 0x48 INT_TOF0  
B HandlerTMC0 ; 0x4c INT_TMC0  
B HandlerTOF1 ; 0x50 INT_TOF1  
B HandlerTMC1 ; 0x54 INT_TMC1  
B HandlerTOF2 ; 0x58 INT_TOF2  
B HandlerTMC2 ; 0x5c INT_TMC2  
B HandlerTOF3 ; 0x60 INT_TOF3  
B HandlerTMC3 ; 0x64 INT_TMC3  
B HandlerTOF4 ; 0x68 INT_TOF4  
B HandlerTMC4 ; 0x6c INT_TMC4  
B HandlerTOF5 ; 0x70 INT_TOF5  
B HandlerTMC5 ; 0x74 INT_TMC5  
B HandlerBT ; 0x78 INT_BT  
B HandlerSIO0 ; 0x7c INT_SIO0  
B HandlerSIO1 ; 0x80 INT_SIO1  
B HandlerEINT4 ; 0x84 EINT4  
B HandlerEINT5 ; 0x88 EINT5  
B HandlerADC ; 0x8c INT_ADC  
B HandlerEINT6 ; 0x90 EINT6  
B HandlerEINT7 ; 0x94 EINT7
```

(3) Startup code
 – Set up Exception Vector

```
AREA Init, CODE, READONLY  
ENTRY  
-----  
; Exception Vector Table  
-----  
B ResetHandler ; 0x00 Reset Exception  
B HandlerUndef ; 0x04 Undefined Exception  
B HandlerSWI ; 0x08 SWI Exception  
B HandlerPabort ; 0x0c Prefetch Memory Abort  
B HandlerDabort ; 0x10 Data Memory Abort  
B . ; 0x14 Reserved  
B HandlerIRQ ; 0x18 IRQ Exception  
B HandlerFIQ ; 0x1c FIQ Exception
```

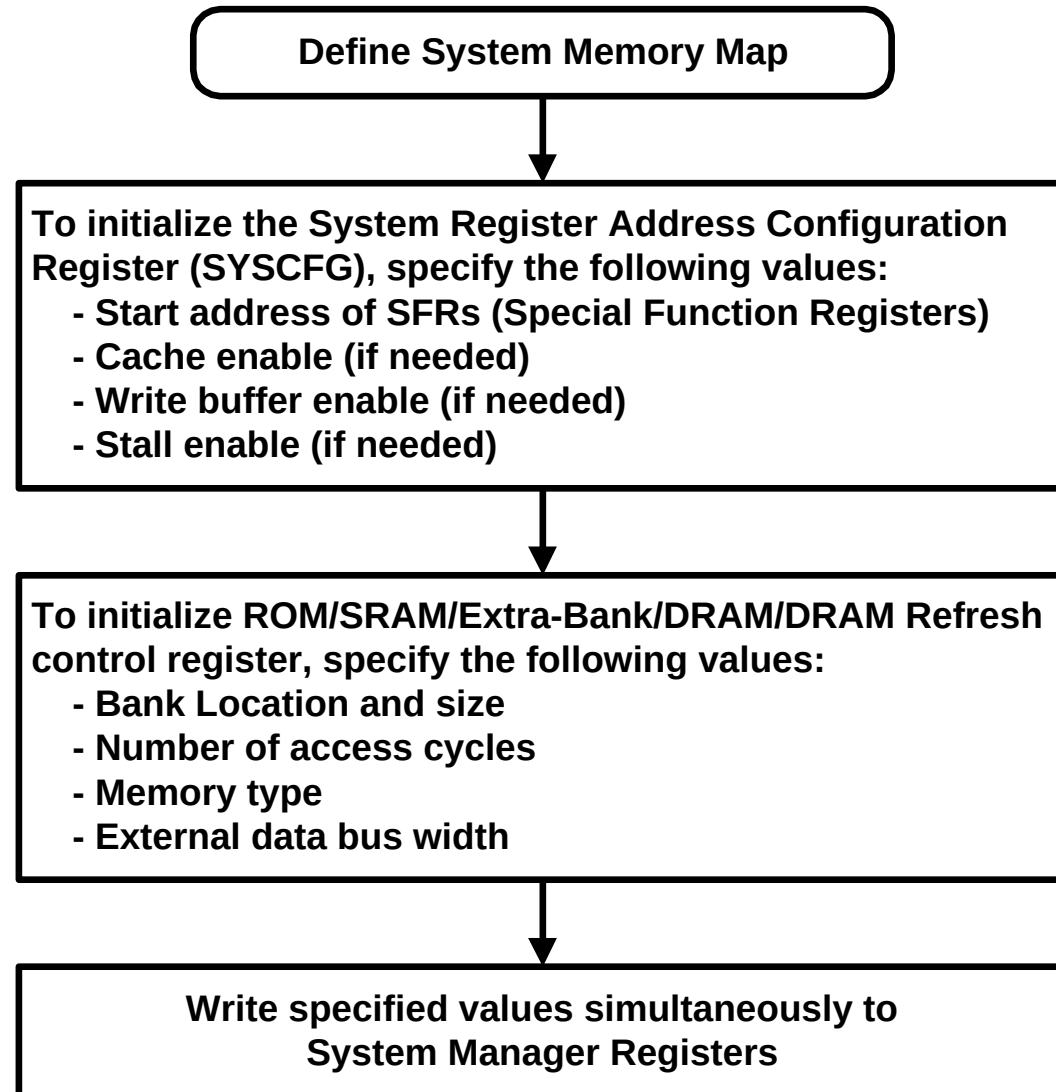
※ LTOrg : literal pool을 생성.
Arm mode,
Literal pool must be within 4KB of LDR to access it

Architecture of ARM Processor

(3) Startup code

Reference : startup.s

– System Manager Configuration



Architecture of ARM Processor

(3) Startup code – System Manager Configuration

Reference : startup.s

```
ResetHandler                                ;reset-> supervisor mode
                                           ;main.c -> usermode
                                           ;change usermode before main_jump

                                           ;Setup the SYSCFG register
                                           ;
LDR    r0,=0x01FF4000                      ;SFR start address
LDR    r1,SYSCFGDATA                      ;SYSCFG setting
STR    r1,[r0]
                                           ;
                                           ;Watch dog timer disable
LDR    r0,=BTCON
LDR    r1,=WDT_DISABLE                    ;BTCON 8~15bit = A5
STRH   r1,[r0]
```

```
SYSCFGDATA
DCD    STALL_OFF + SFR_STARTADDRESS

SMRDATA                                ;FLASH BOOTING
DCD    (CS0_EA<<21)+(CS0_BA<<12)+(CS0_TACC<<8)+(CS0_TCOH<<6)+(CS0_TACS<<4)+(CS0_TCOS<<2)+(CS0_MT<<1)+(CS0_DW<<0)
                                           ;Bank0=MD[2:0]=001 16bit Internal Flash
DCD    (CS1_EA<<21)+(CS1_BA<<12)+(CS1_TACC<<8)+(CS1_TCOH<<6)+(CS1_TACS<<4)+(CS1_TCOS<<2)+(CS1_MT<<1)+(CS1_DW<<0)
                                           ;nCS1 SRAM
DCD    0x00                                ;nCS2 Disable
DCD    0x00                                ;nCS3 Disable
DCD    0x00                                ;nCS4 Disable
DCD    0x00                                ;nCS5 Disable
DCD    0x00                                ;nCS6 Disable
DCD    0x00                                ;nCS7 Disable
```

Memory map

Architecture of ARM Processor

(3) Startup code – Stack Initialization

Reference : startup.s

```
InitStacks                                ;reset ==> supervisor mode
;                                          ;mode stack initialization
MRS    r0,cpsr
BIC     r0,r0,#MODEMASK                   ;modemask set_bit clear
ORR     r1,r0,#UNDEFMODE|NOINT            NOINT    EQU    0xc0
MSR     cpsr_cxsf,r1                     ;UndefMode
LDR     sp,=UndefStack                   → IRQ,FIQ disable
;
ORR     r1,r0,#ABORTMODE|NOINT            ;1=OR
MSR     cpsr_cxsf,r1                     ;AbortMode
LDR     sp,=AbortStack
;
ORR     r1,r0,#IRQMODE|NOINT
MSR     cpsr_cxsf,r1                     ;IRQMode
LDR     sp,=IRQStack
;
ORR     r1,r0,#FIQMODE|NOINT
MSR     cpsr_cxsf,r1                     ;FIQMode
LDR     sp,=FIQStack
;
BIC     r0,r0,#MODEMASK|NOINT
ORR     r1,r0,#SVCMode
MSR     cpsr_cxsf,r1                     ;SVCMode
LDR     sp,=SVCStack
;
MOV     pc,lr                            ;The LR register may be not valid for the mode changes.
;lr ==> supervisor mode
```

CPSR :

N	Z	C	V	.	.	.	.	.	.	.	.	I	F	T	M4	M3	M2	M1	M0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

Architecture of ARM Processor

(3) Startup code – C Variable Initialization

Reference : startup.s

```
; RO,RW region select
NOP
LDR    r0, =|Image$$RO$$Limit|      ; Get pointer to ROM data
LDR    r1, =|Image$$RW$$Base|        ; and RAM copy
LDR    r3, =|Image$$ZI$$Base|        ; Zero init base => top of initialised
;
00    CMP    r0, r1                    ; Check that they are different (r0-r1)
      BEQ    %F01
      CMP    r1, r3                    ; Copy init data
      LDRCC  r2, [r0], #4              ; post-indexed mode
      STRCC  r2, [r1], #4              ; CC=if carry clear (c=0 ==> borrow=1)
      BCC    %B00
;
01    LDR    r1, =|Image$$ZI$$Limit|   ; Top of zero init segment
;
02    MOV    r2, #0
      CMP    r3, r1                    ; Zero init
      STRCC  r2, [r3], #4
      BCC    %B02
```

RO, RW, ZI initialization

Errors & Warnings

0 0 14 Errors and warnings for ?est1.mcp

Image component sizes

	Code	RO Data	RW Data	ZI Data	Debug	
	8452	0	0	24	8408	Object Totals
	0	0	0	0	0	Library Totals

AXD - [ARM7TDMI0 - D:\ARM_PRJ\Projects\main.c]

File Search Processor Views System Views Execute Options Window Help

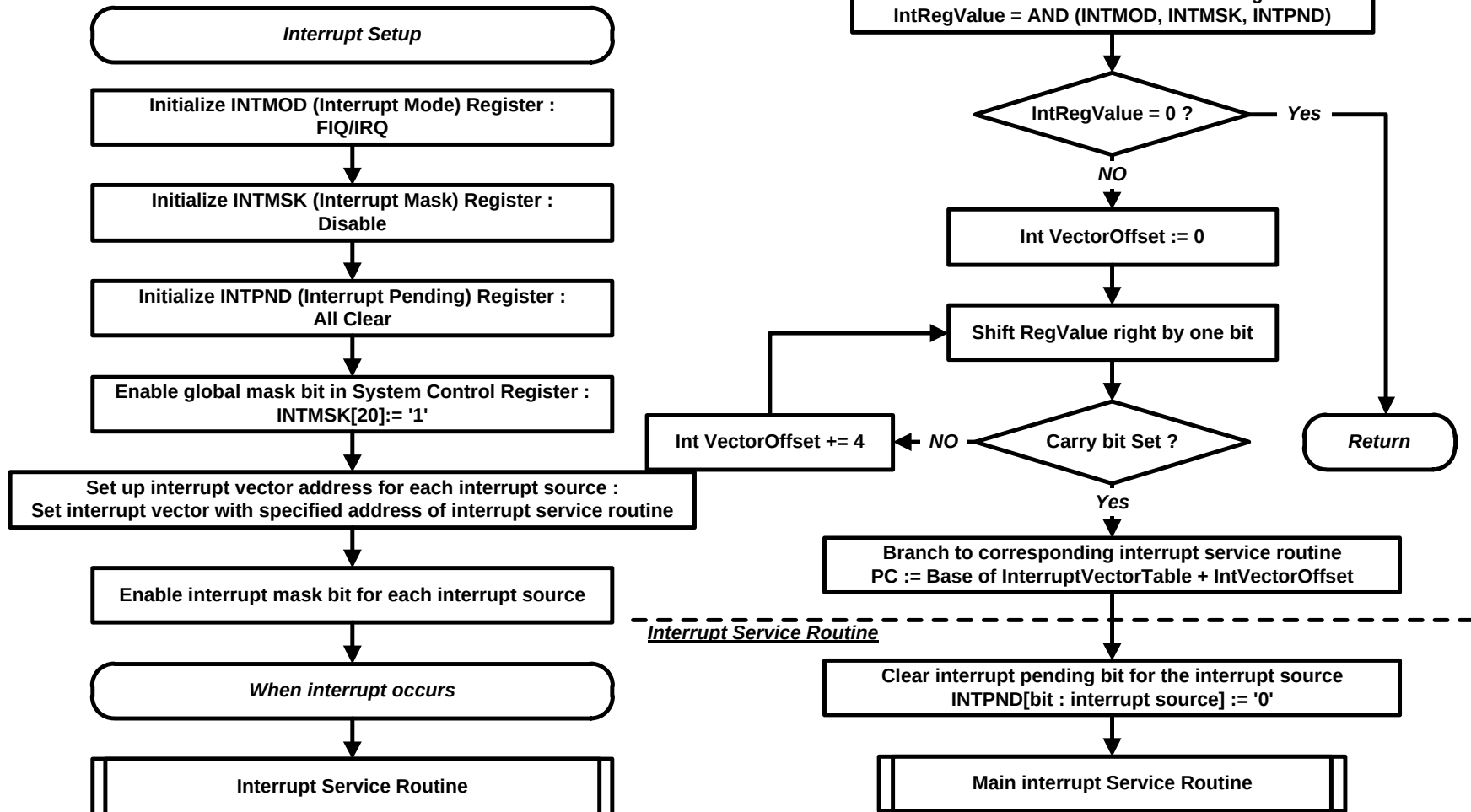
ARM7TDMI0 - Variables

Local	Global	Class
Variable	Value	
CAPT_VALUE	0x0000	
DCLK_EXT	.	
DCLK_VALUE	0x00000000	
F_400N_CNT	.	

```
1  /*****
2  /* Name : main.c
3  /* Modified and programmed by YONG SEOK CHI
4  /* Date : 2002. 10. 14 first edited
5  /* Description : main
6  /* SAMSUNG SDI PDP
7  /*****
8
9  #include "main.h"
```

(3) Startup code

– Exception Handling Procedure



Architecture of ARM Processor

(3) Startup code – Interrupt Handler Routine

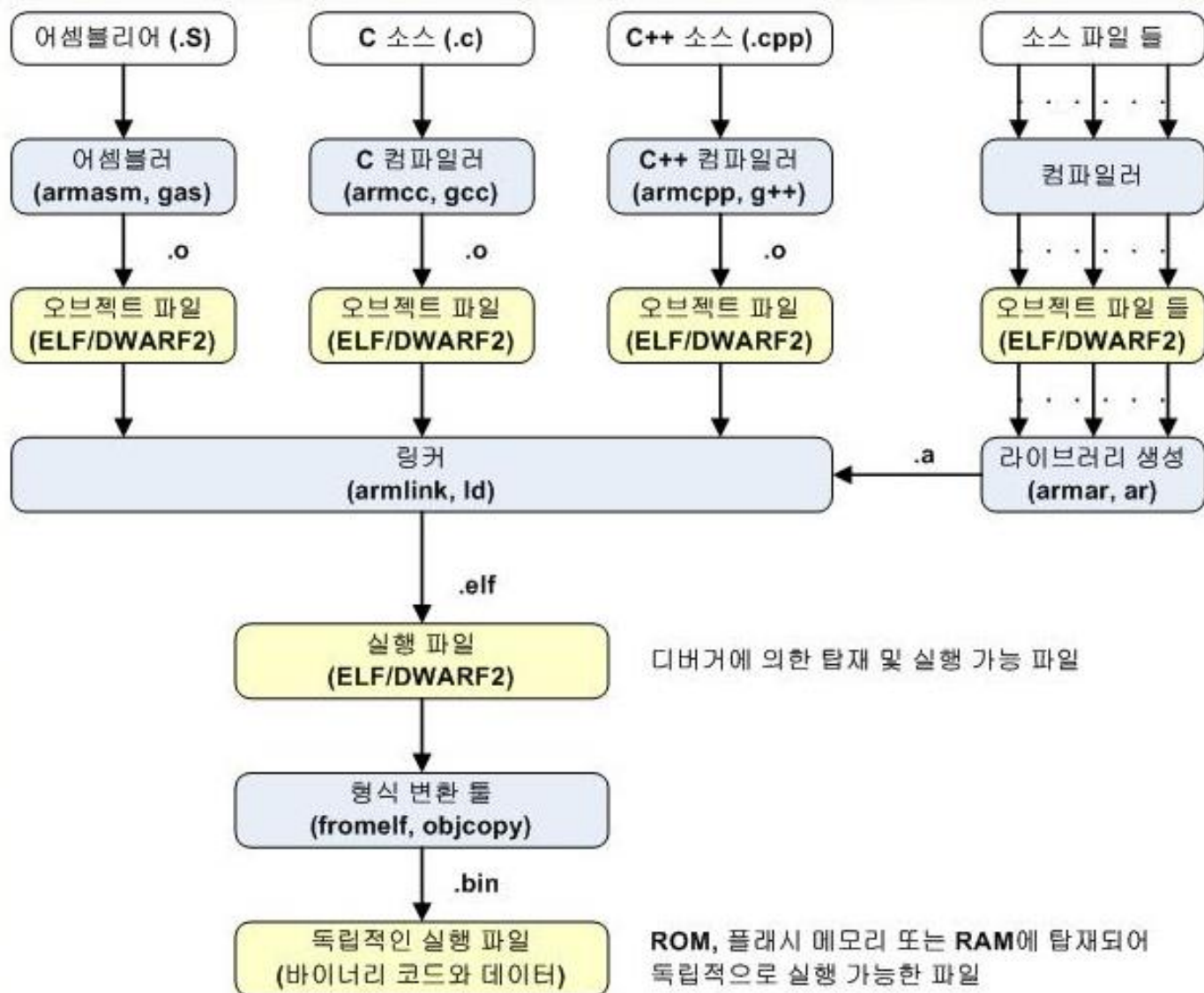
Reference : startup.s

```
IsrIRQ      ;IRQ handler routine
SUB         sp,sp,#4      ;Reserved for storing the start address of ISR later.
STMFD       sp!,{r8-r10}  ;Store r8-r10 value for using r8-r10.
;
LDR         r8,=INTMOD     ;r8 = INTMOD address
LDMIA       r8,{r8-r10}    ;r8=IntMode,r9=IntPend,r10=IntMask
MVN         r8,r8         ;r8= not r8
AND         r8,r8,r9       ;r8= r8 & r9
AND         r8,r8,r10      ;r8= r8 & r9 & r10
;                 ;r8 has IRQ,Unmasked,Pending bit
MOV         r9,#0         ;set r9 with internal vector offset
0  MOVSW     r8,r8,LSR #1   ;For checking the occurred INT Flag.
BCS         %F1           ;Check carry bit set
ADD         r9,r9,#4       ;If INT not occurred, add 4 to calculate the buffer number.
B           %B0
;
1  LDR         r10,=HandleEINT0 ;IntVectorTable must not be used.
ADD         r10,r10,r9      ;Calculate the buffer number that have the start address of occurred ISR.
LDR         r10,[r10]       ;Load the start address of ISR in buffer.
STR         r10,[sp,#12]    ;Store the start address of ISR to stack.
LDMFD       sp!,{r8-r10,pc} ;jump to appropriate interrupt service routine
B           .
```

Architecture of ARM Processor

Object file

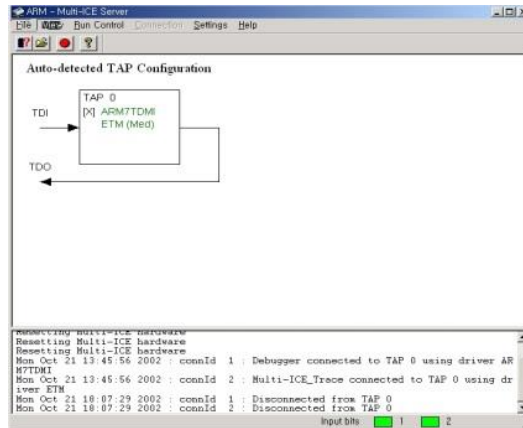
(1) Compile



Architecture of ARM Processor

Object file

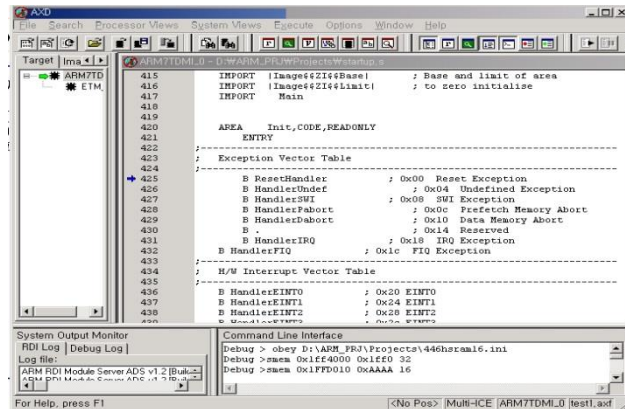
(2) ICE Tool (In-Circuit Emulator)



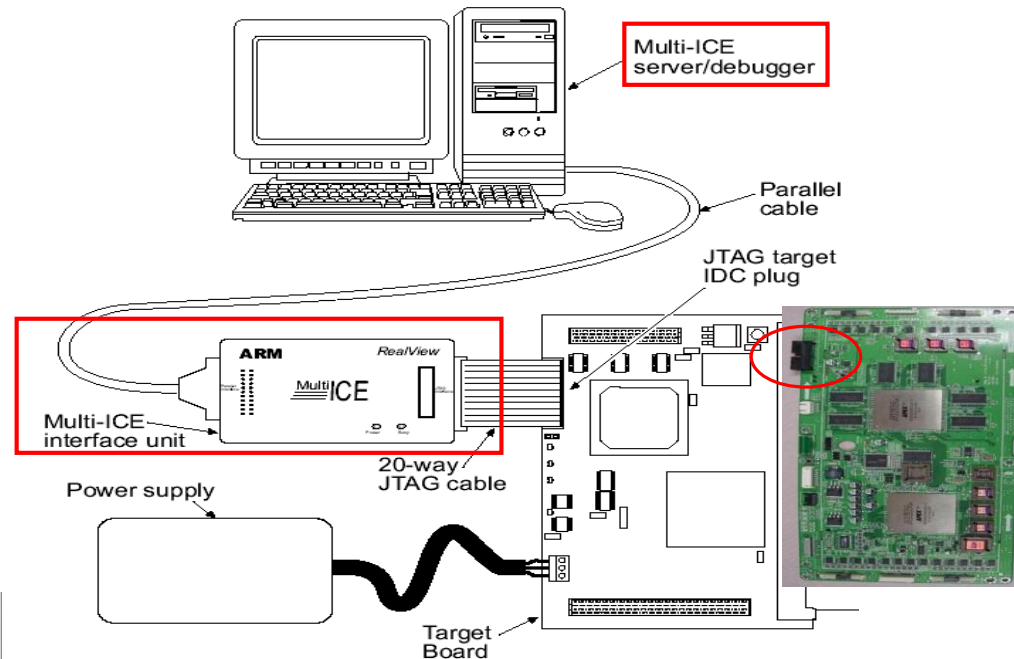
: Multi-ICE

: Real View ICE

(Real View Development Suite)



(3) Development flow



: ADS (Debugger)

: RVDS (compiler, debugger)

Architecture of ARM Processor

Object file

(4) Debugger

The screenshot displays the ARM7TDMI-L0 debugger interface. The main window shows the source code for `main.c`, which includes a `while` loop for LED control. The left sidebar contains panels for **Variables**, **Registers**, **Memory**, **System Output Monitor**, and **Command Line Interface**. The **Registers** panel shows the current state of the processor registers, with `r0` through `r11` listed. The **Memory** panel shows the memory dump starting at address `0x0`. The **System Output Monitor** panel shows the log of the debugger's interaction with the target hardware. The **Command Line Interface** panel shows the command `Debug > obey D:\ARM` being entered.

Legend:

- : Variable
- : Register
- : Memory
- : Command line

Debugger Controls:

- Go
- Stop
- Step In
- Step
- Step Out
- Run to Cursor

Source Code:

```
1  /*****  
2  /* Name : main.c  
3  /* Modified and programmed by YONG SEOK CHI  
4  /* Date : 2002. 10. 14 first edited  
5  /* Description : main  
6  /* SAMSUNG SDI PDP  
7  /*****  
8  
9  #include "main.h"  
10  
11 void Main(void)  
12  
13     INIT_PORT();  
14     TIMER1_SETT();  
15     TIMER3_SETT();  
16     TIMER4_SETT();  
17     INIT_ISR();  
18     _G_INT_E_;  
19  
20     while(1)  
21     {  
22         LED(1);  
23         LED(2);  
24         LED(4);  
25     } // end of while(...)  
26  
27 } // end of Main
```

Registers:

Register	Value
r0	0x60000000
r1	0x60000010
r2	0x00000000
r3	0x00015018
r4	0x01009BDC
r5	0x00000000
r6	0x00000000
r7	0x00000000
r8	0x00000000
r9	0xE67FDFD7
r10	0xEE7BCFFF
r11	0x00000000

Memory:

Address	0	1	2	3	4	5	6	7	8
0x00000000	8B	01	00	EA	23	00	00	EA	28
0x00000010	32	00	00	EA	FE	FF	FF	EA	36
0x00000020	40	00	00	EA	45	00	00	EA	4A
0x00000030	54	00	00	EA	59	00	00	EA	5E
0x00000040	68	00	00	EA	6D	00	00	EA	72

System Output Monitor:

```
Log file:  
ARM RDI Module Server ADS :  
ARM RDI Module Server ADS :  
ARM RDI 1.5.1 -> ASYNC RDI  
ARM Multi-ICE V2.2 (Build 1095)  
Connected to TAP 0, ARM7TD
```

Command Line Interface:

```
Debug > obey D:\ARM  
Debug > smem 0x1ff40  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0  
Debug > smem 0x1FFD0
```

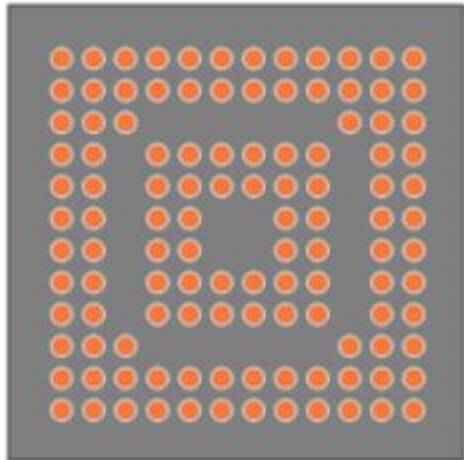
For Help, press F1

Line 12, Col 0 | Multi-ICE | ARM7TDMI-L0 | test1.axf

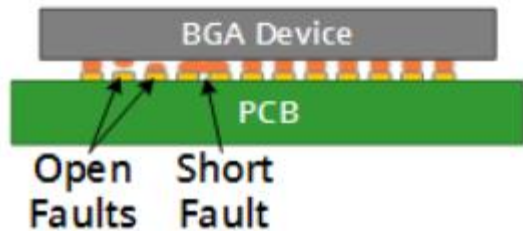
JTAG(Joint Test Action Group)

- IEEE STD 1149.1에 의 정의
- Boundary scan description language로 개발, 테스트 환경을 제공

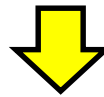
BGA (Bottom View)



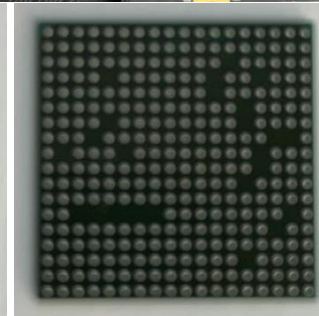
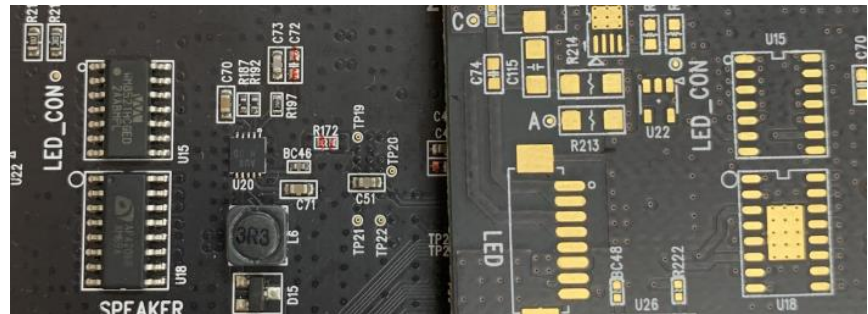
Mounted BGA with Faults
(Side View)



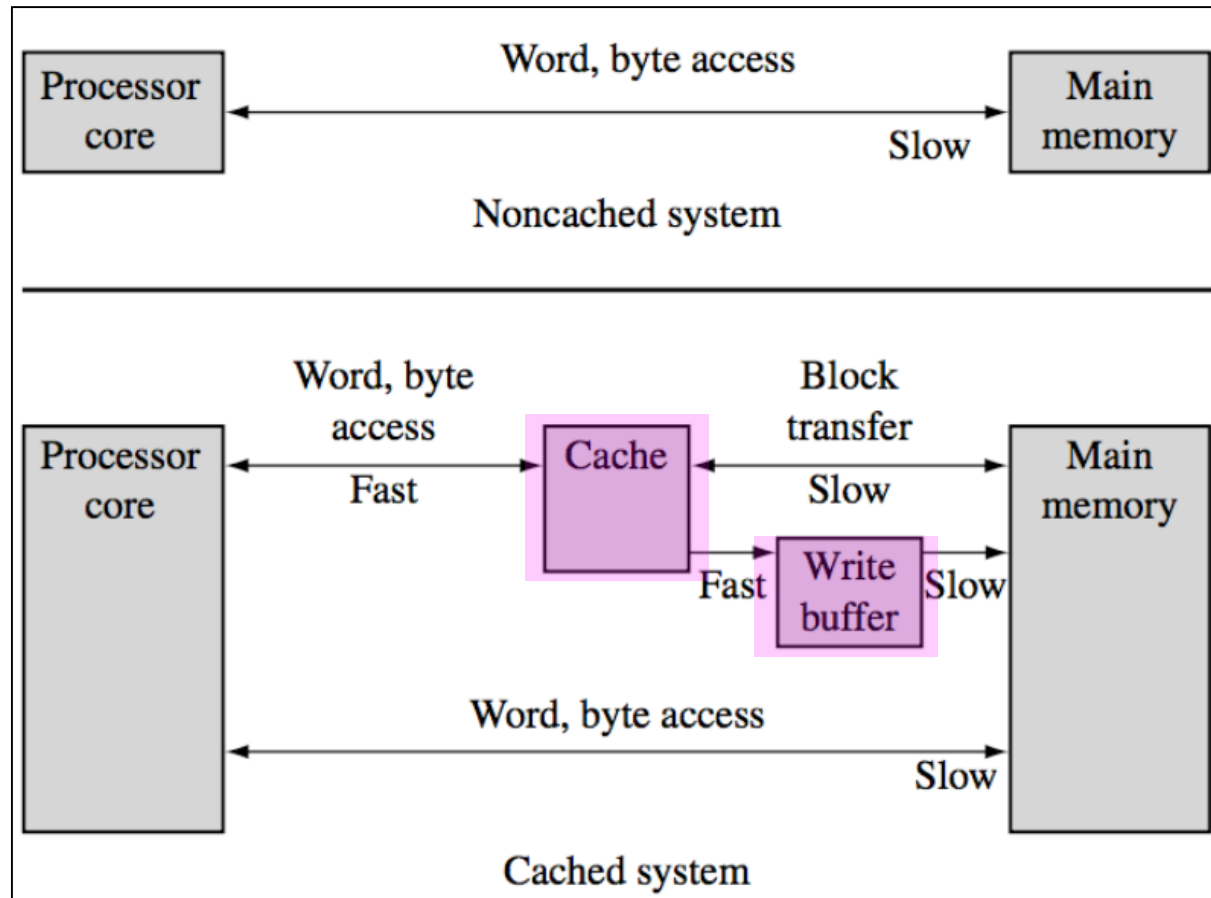
- IC내에 Boundary scan cell 내장되어 test사용



- 기능과 속도의 확장을 통해
저전력 시스템의
고속 제어 기능의 표준으로 활용되고 있음
- Embedded system의 debugging으로 활용

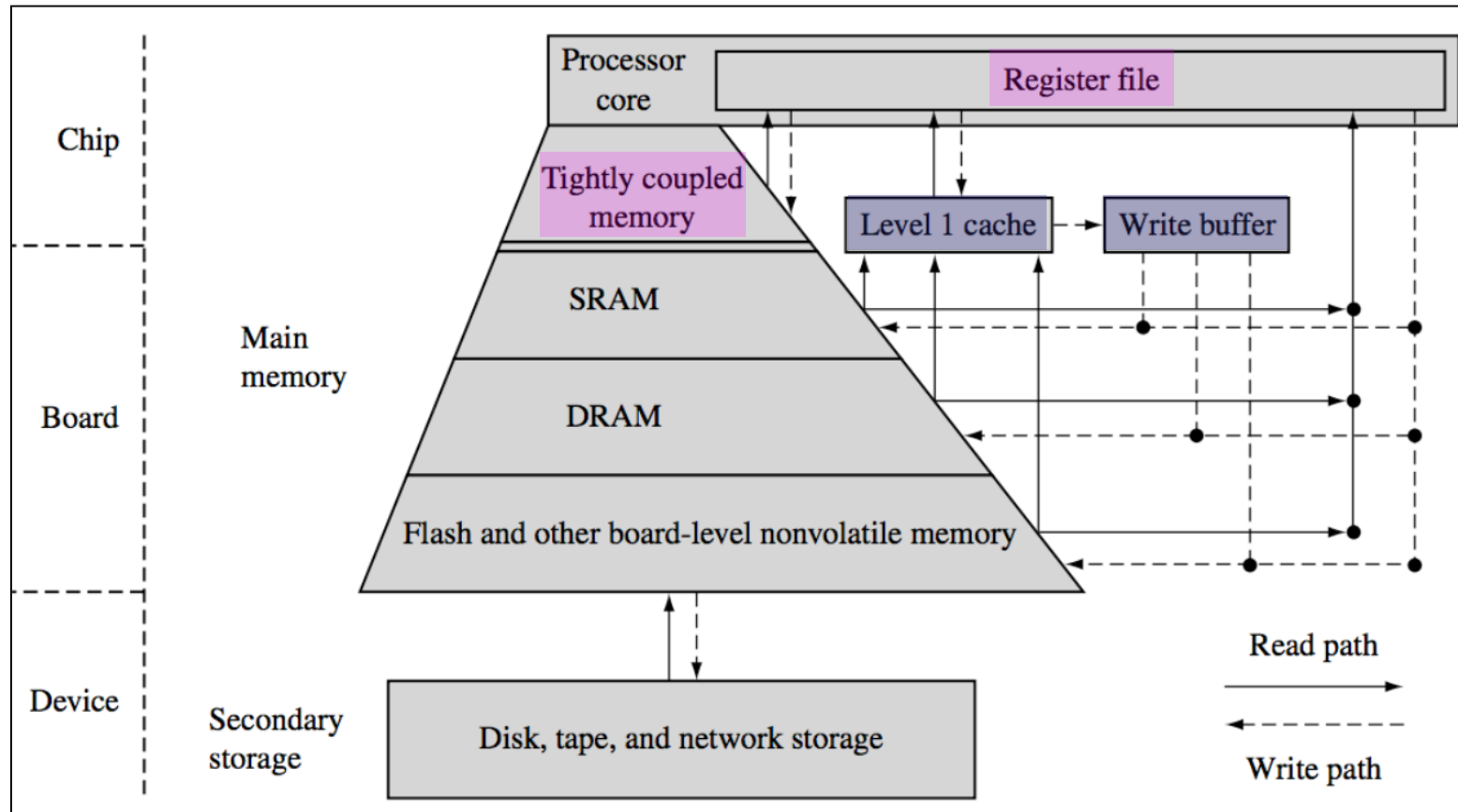


Memory Hierarchy



- **cache** : code와 data를 임시로 저장하는 고속의 on chip 메모리
- **write buffer** : cache로부터 main 메모리에 쓰기를 지원하는 FIFO 버퍼

Memory Hierarchy

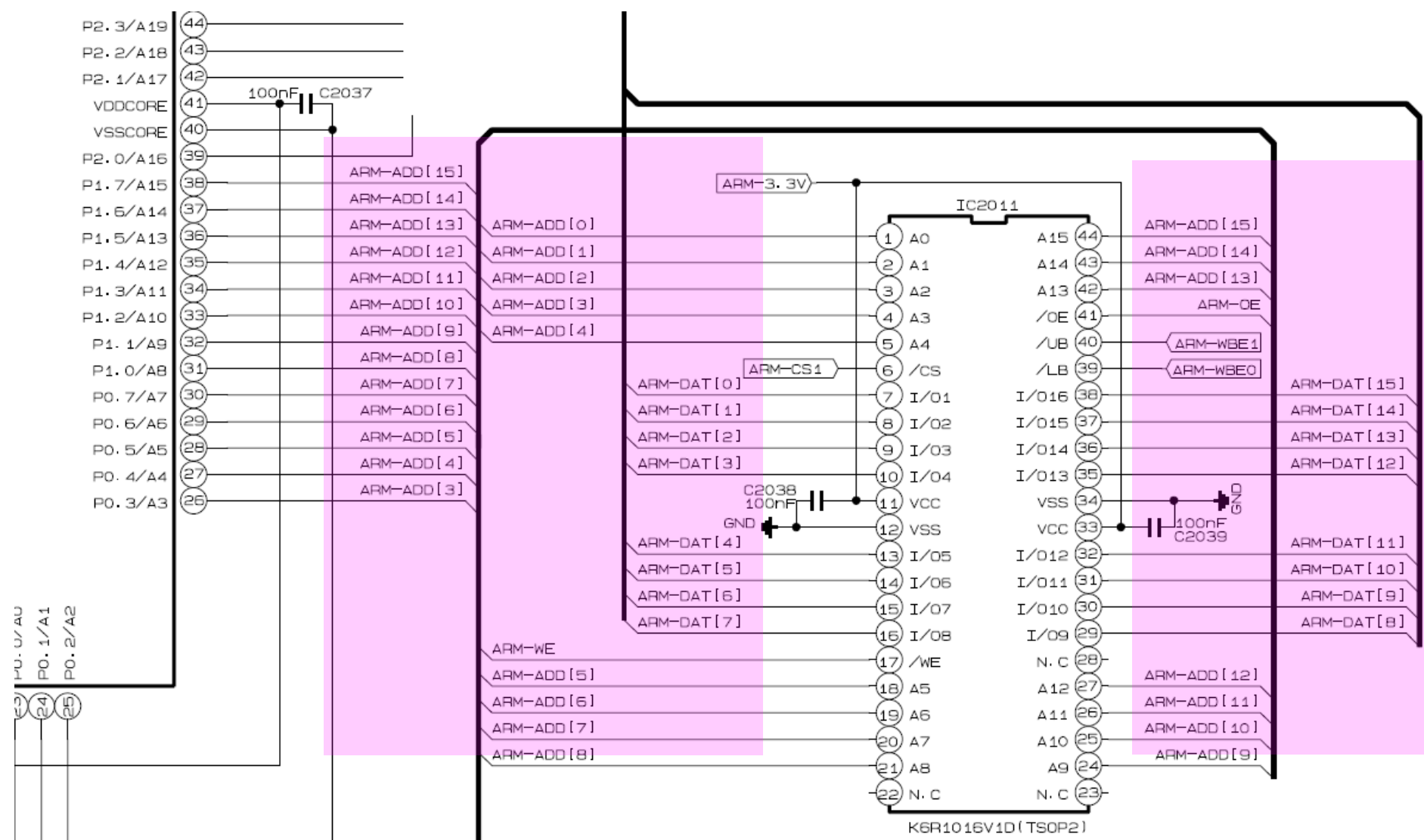


- **Register** : Core에 위치하여 시스템에서 가장 빠른 메모리 access를 제공
- **Tightly Coupled Memory (TCM)** : 고속으로 처리되어 시간이 보장되어야 하는 명령이나 데이터를 저장. CPU의 해당하는 어드레스 액세스 요청이 있으면 캐시나 외부 메모리를 access 하지 않고, 곧바로 이 고속 메모리를 access 하도록 사용하는 메모리시스템. TCM을 사용하여 실시간 작업, 인터럽트 처리 루틴 등의 중요 루틴을 보관

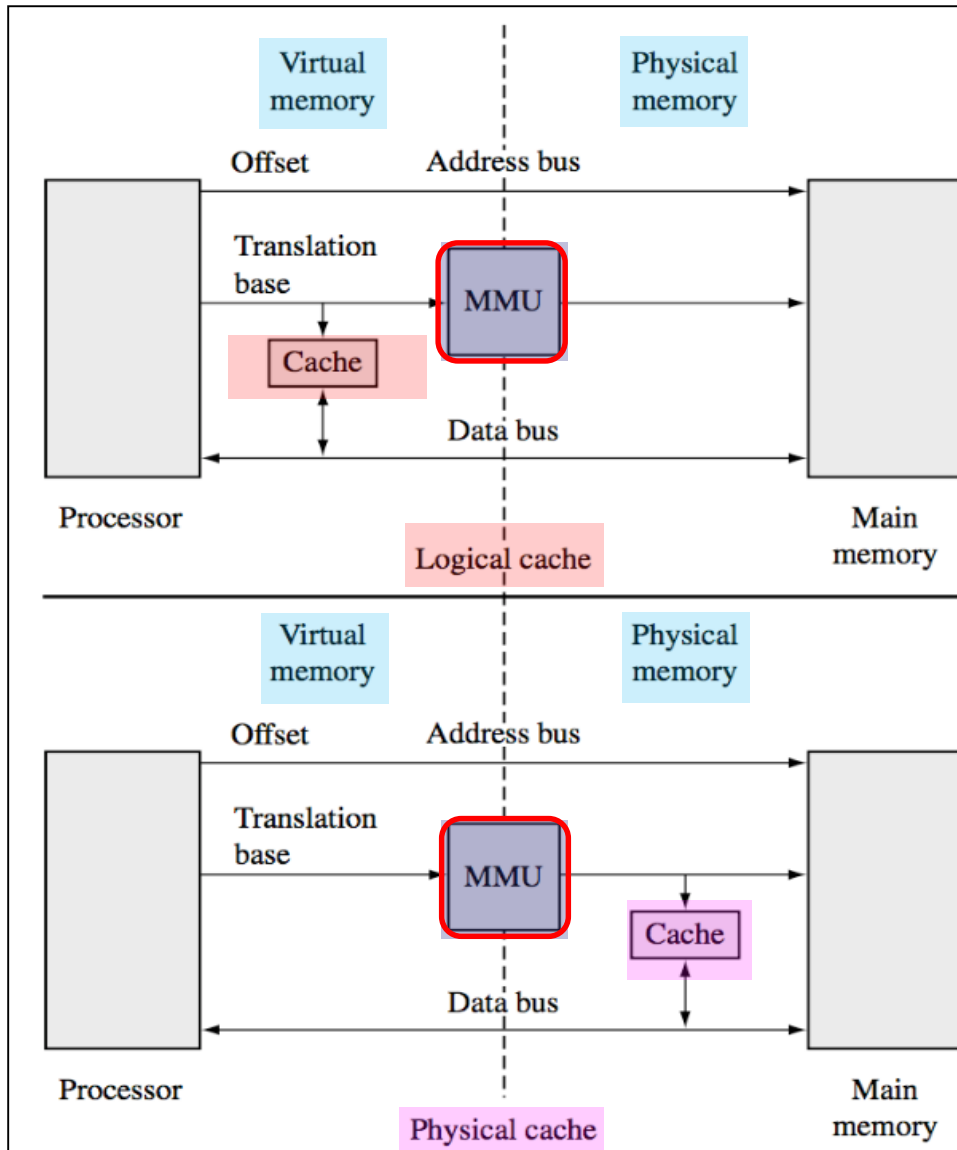
ARM System Developer's Guide

: Designing and Optimizing System Software

Non-cached system



Logical cache, Physical cache



- MMU
 - cache core
 - : core와 MMU사이에 위치하거나 MMU와 Physical memory 사이에 위치
 - logical cache
 - : core와 MMU사이에 위치하여 virtual memory에 data 저장
 - : ARM7~ARM9
 - physical cache
 - : MMU와 Physical memory 사이에 위치 physical memory에 data 저장
 - : ARM11
- ※ cache에 의한 성능향상 이유?
- : 예측 가능한 프로그램의 실행 (for 문 등의 반복문)

ARM Architecture

Ref. arm.com

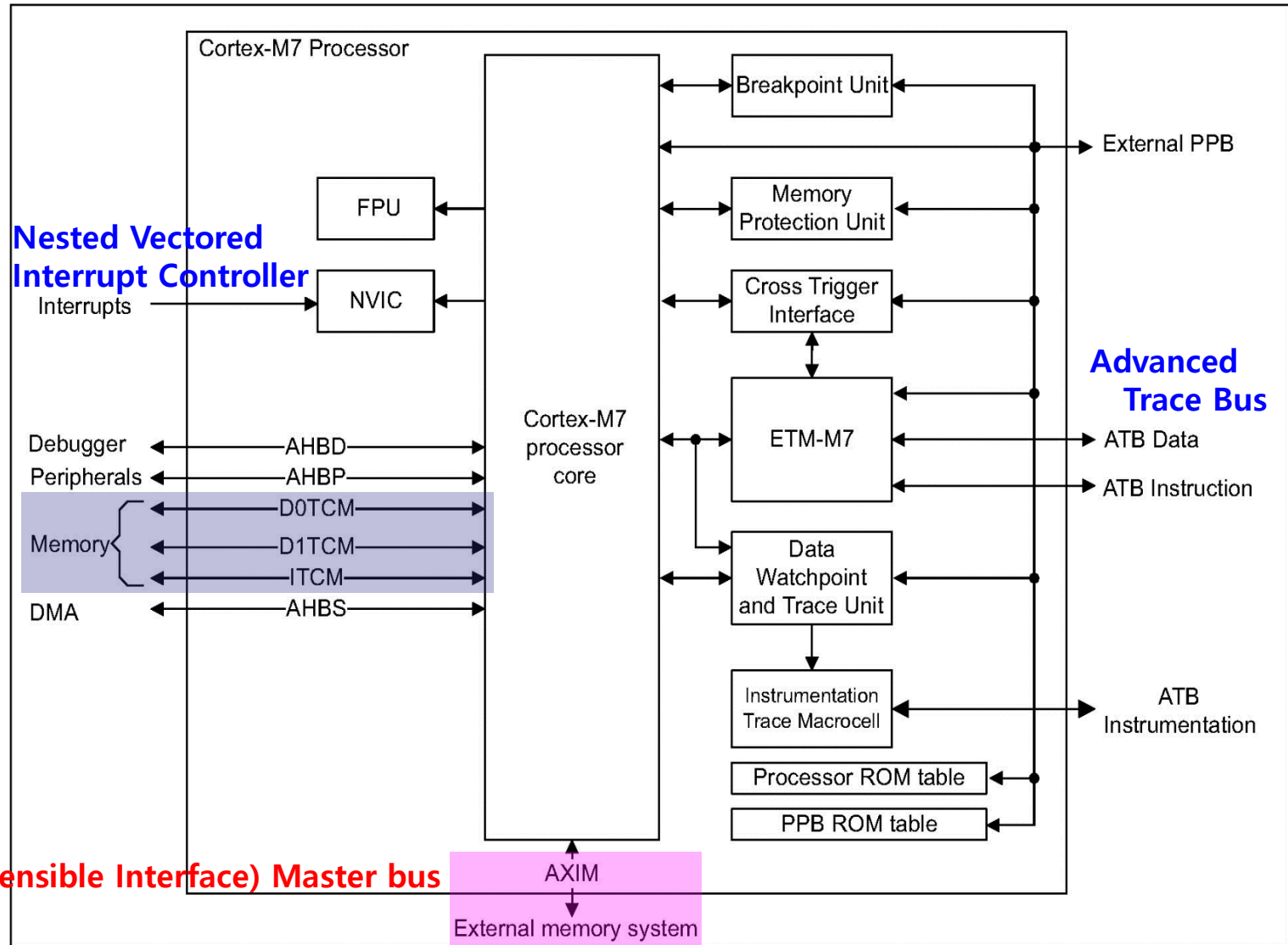
- **ARM® Cortex®-M7 Processor**
 - : Revision r1p0
 - : Technical Reference Manual
- **ARM®v7-M Architecture**
 - : Reference Manual
- **Cortex-M7 processor**는 **ARMv7E-M architecture**를 제공

Ref. st.com

STM32F767xx advanced ARM-based 32-bit MCUs

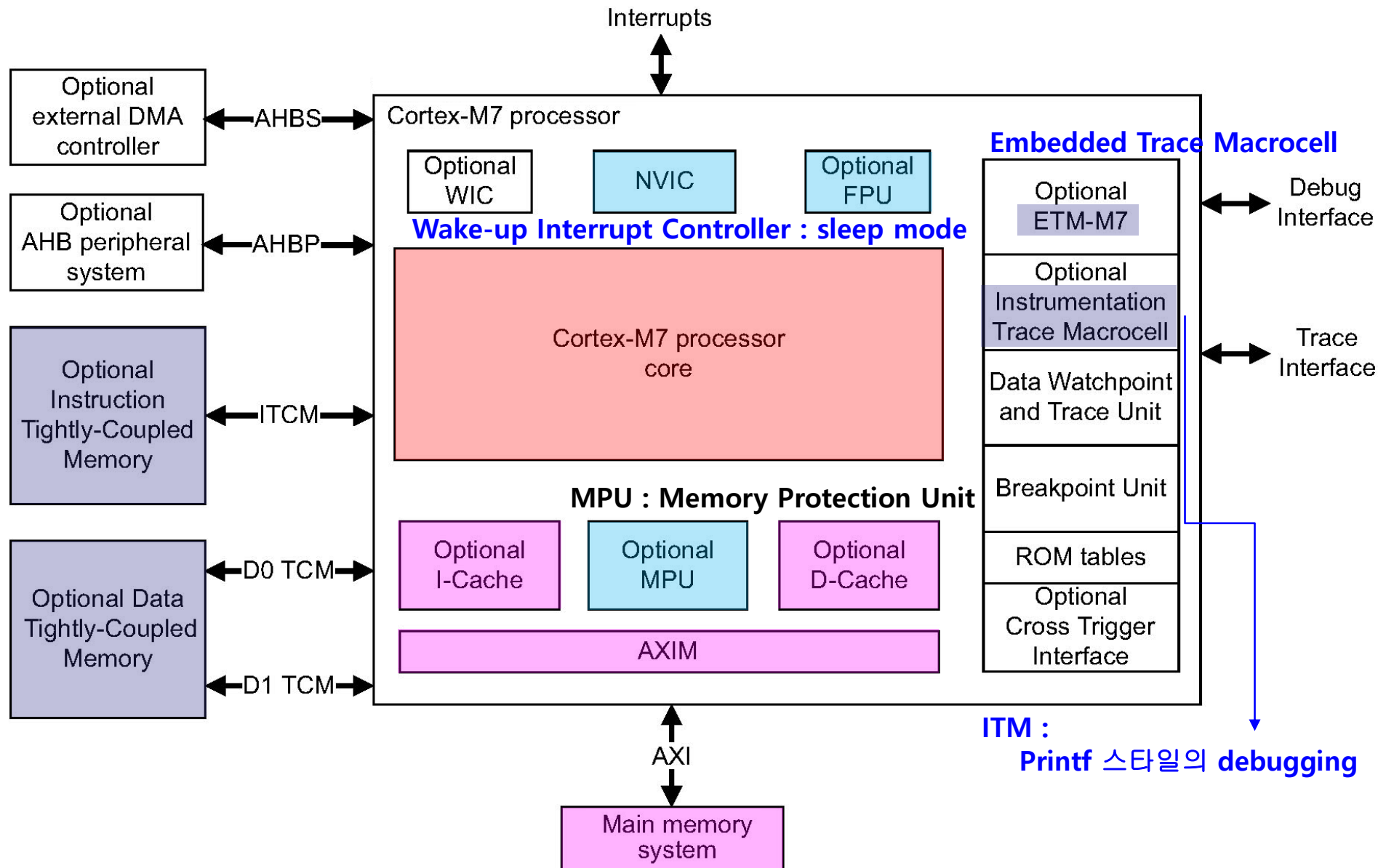
STM32F767VGT6의 Architecture

- TCM interface.
- Harvard instruction and data caches and AXI master interface.
- Dedicated low-latency AHB-Lite peripheral interface.



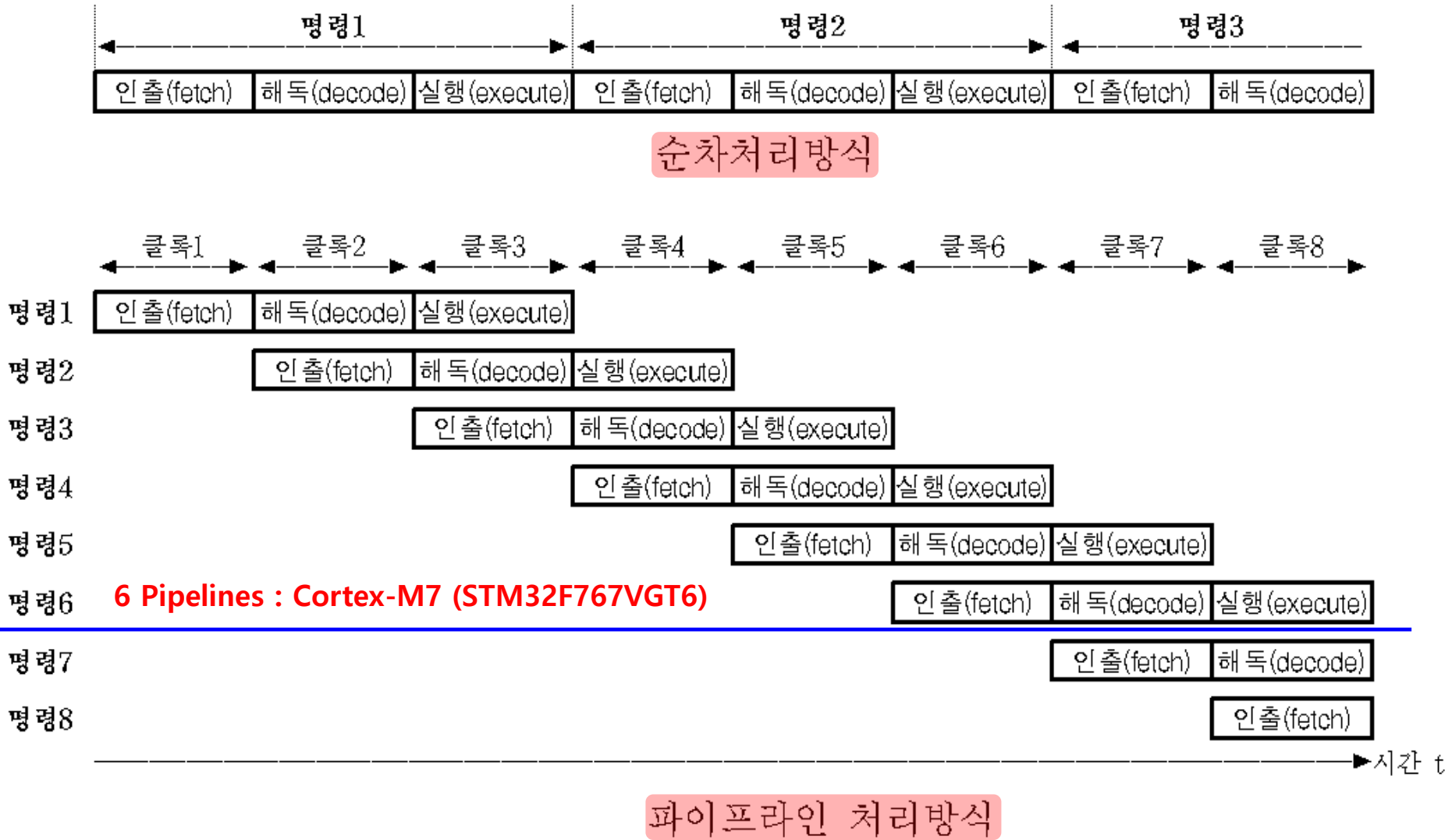
AXI (Advanced extensible Interface) Master bus

STM32F767VGT6의 Architecture



STM32F767VGT6의 Architecture

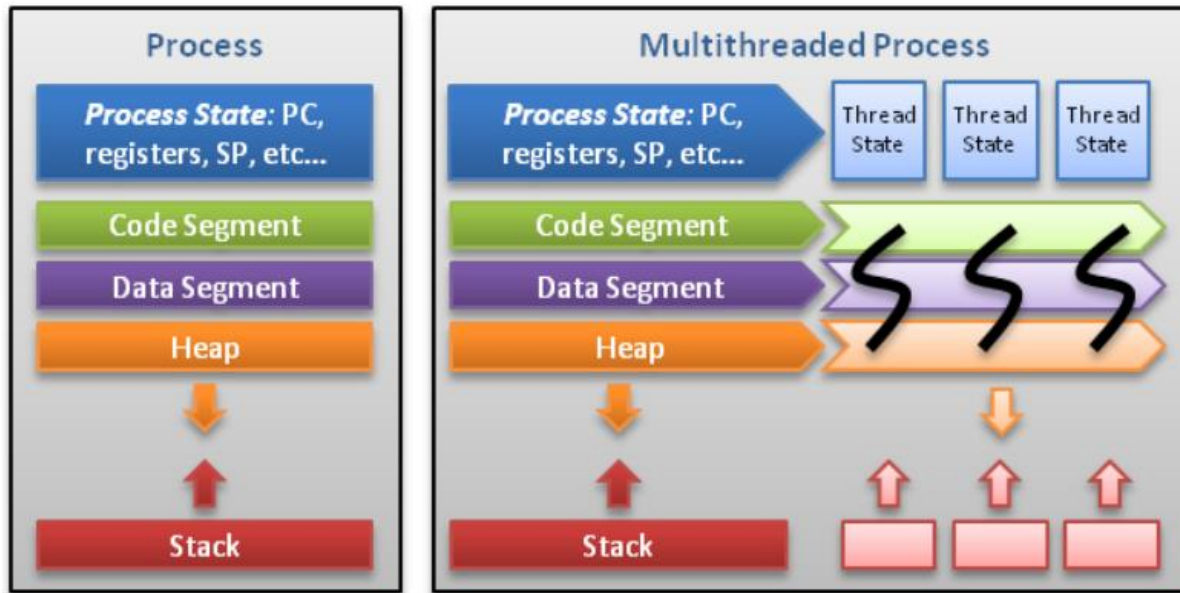
PIPE LINE



STM32F767VGT6의 Architecture

Thread

- : Process 내에서 실행되는 **일련의 흐름 단위**를 의미.
- : 한 프로세서는 하나의 **thread**를 가지고 있지만,
프로그램 환경에 따라 **2개** 이상의 **thread**를 동시에 실행할 수 있음 - **multithread**

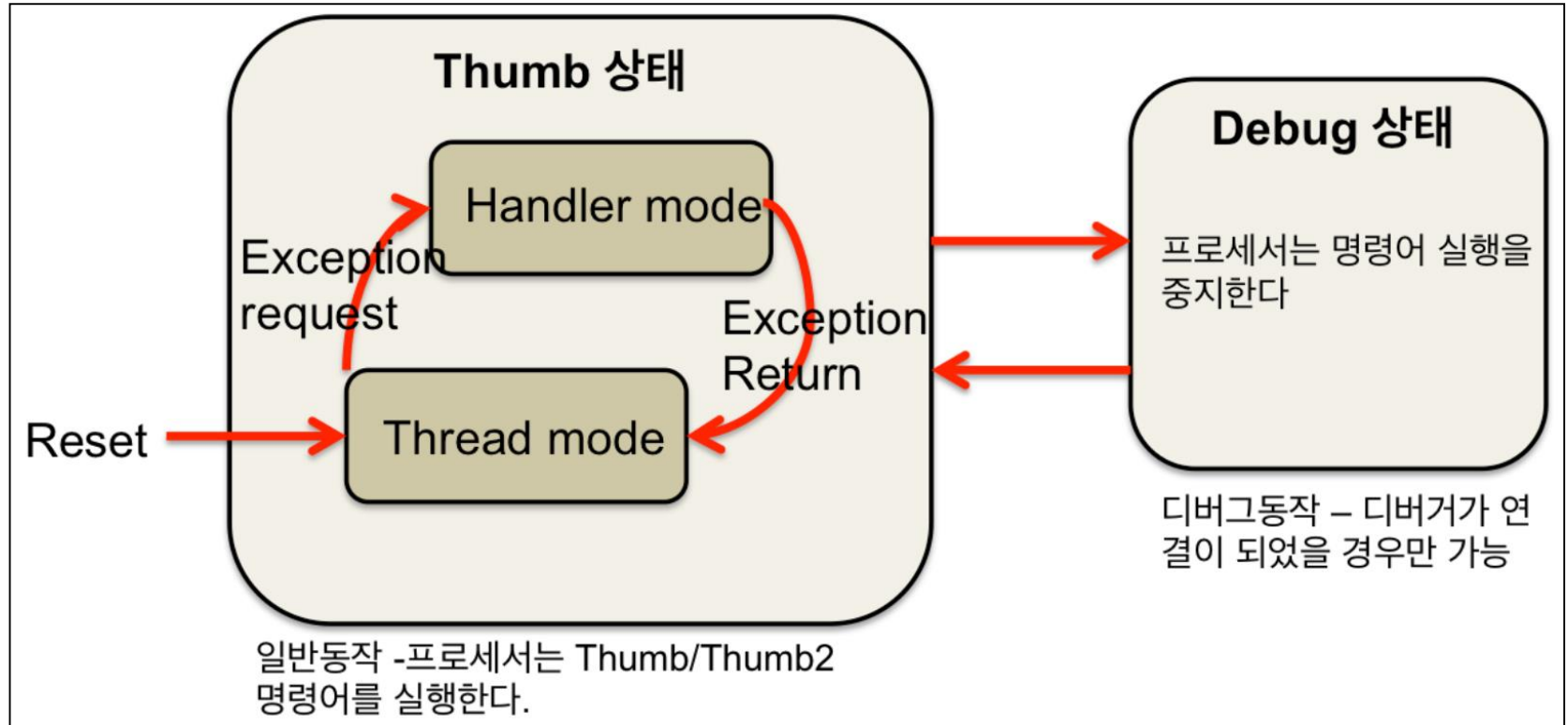


- **CORTEX-M7**

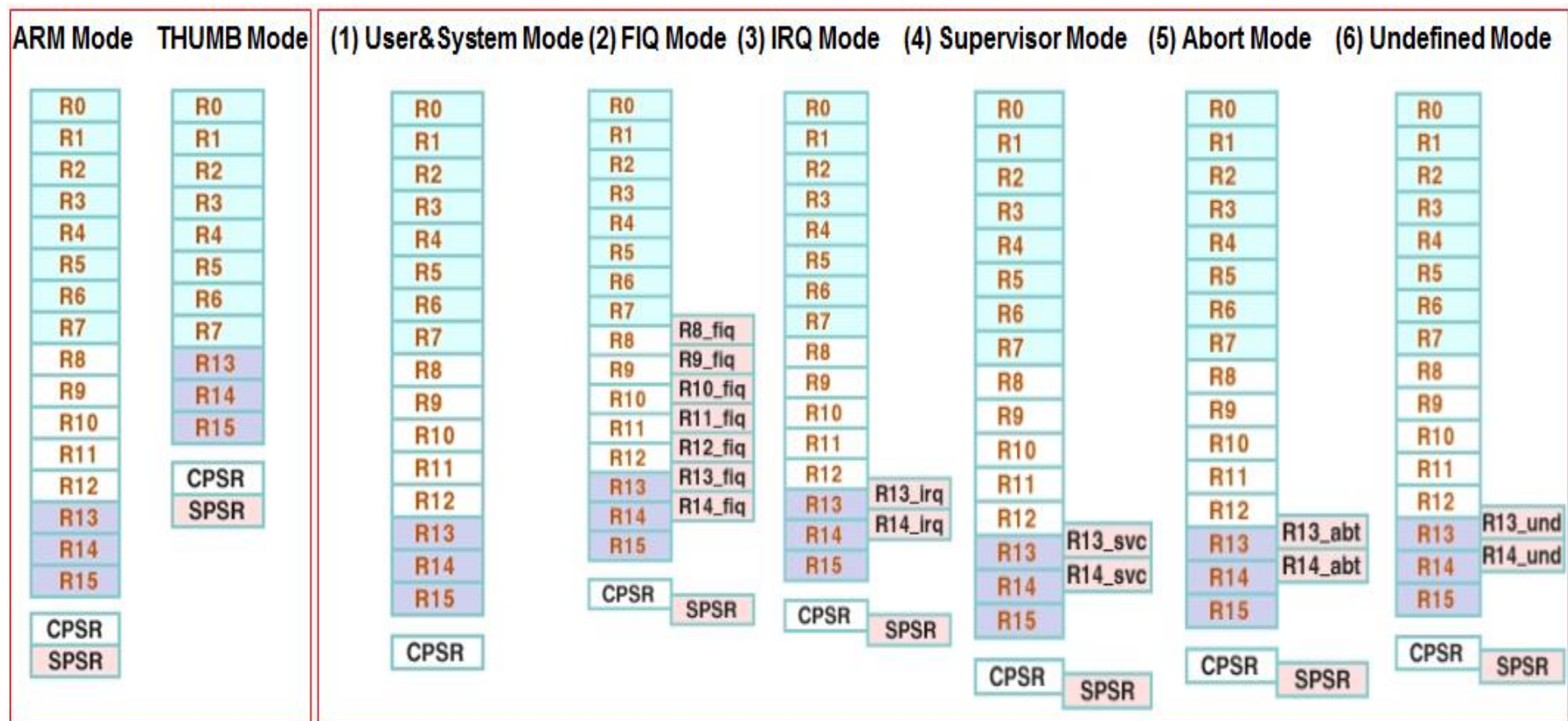
- : **Reset**후 **thread mode** 에서 동작
- : **Exception** 처리에서는 **Handler mode** 동작

STM32F767VGT6의 Architecture

Programmer's Model



Register : ARM과 Cortex-M 비교



R0-15 : General register

CPSR(Current Program Status Register)

SPSR(Saved Program Status Register)

R13 : SP(Stack Pointer)

R14 : LR(Link Register: PC값을 임시로 저장, "BL"의 실행시 사용)

R15 : PC(Program Counter: 다음에 수행이 되어야 할 명령어의 주소

❖ User Mode

- User task 또는 어플리케이션을 수행할 때의 프로세서의 동작 모드

❖ FIQ(Fast Interrupt Request) Mode

- 빠른 인터럽트의 처리를 위한 프로세서의 동작 모드

❖ IRQ(Interrupt Request) Mode

- 일반적으로 사용되는 인터럽트를 처리하기 위한 프로세서의 동작 모드

❖ SVC(Supervisor) Mode

- 시스템 자원을 관리할 수 있는 프로세서의 동작 모드
- Operating 시스템의 커널이나 드라이버를 처리하는 모드
- Reset이나 소프트웨어 인터럽트(SWI)가 발생하면 ARM은 Supervisor 모드가 된다.

❖ Abort Mode

- 명령이나 데이터를 메모리로부터 읽거나 쓸 때 오류가 발생하면 ARM은 Abort 모드가 된다.

❖ Undefined Mode

- ARM이 정의되지 않은 명령을 수행하려고 하면 수행되는 프로세서의 동작 모드

❖ System Mode

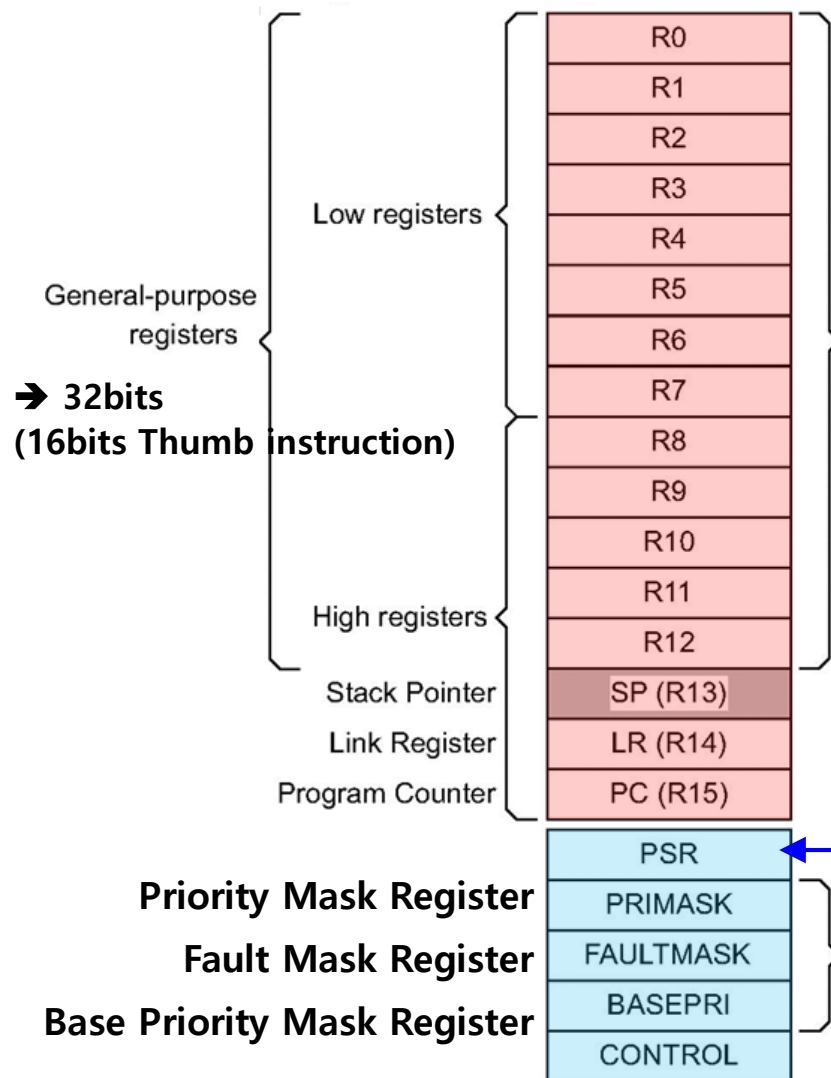
- User 모드와 동일한 용도로 사용
- ARM Architecture v4이후부터(ARM7) 지원된다.

STM32F767VGT6의 Architecture

Register :
ARM과 Cortex-M 비교

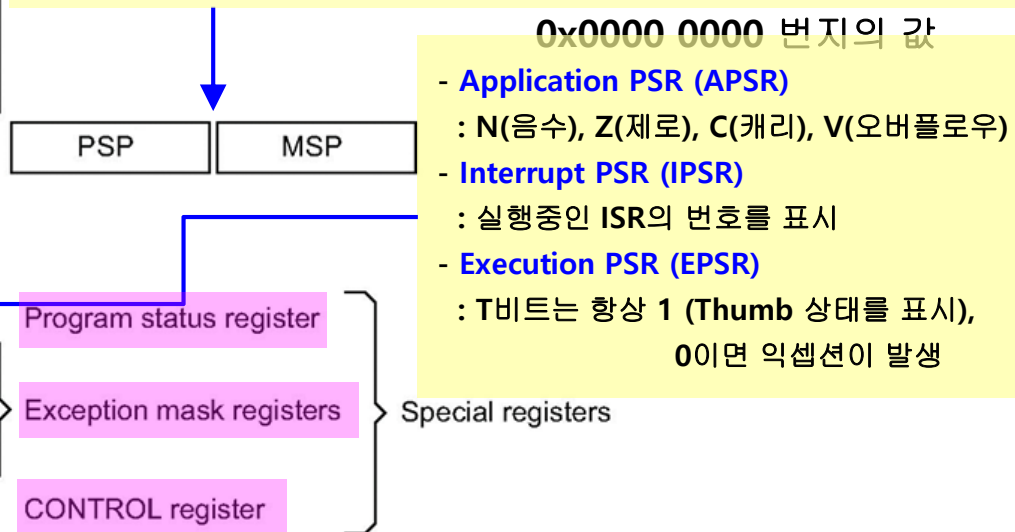
• CORTEX-M7

- : Reset후 **thread mode** 에서 동작
- : Exception 처리에서는 **Handler mode** 동작



• R13, SP (2개의 stack pointer)

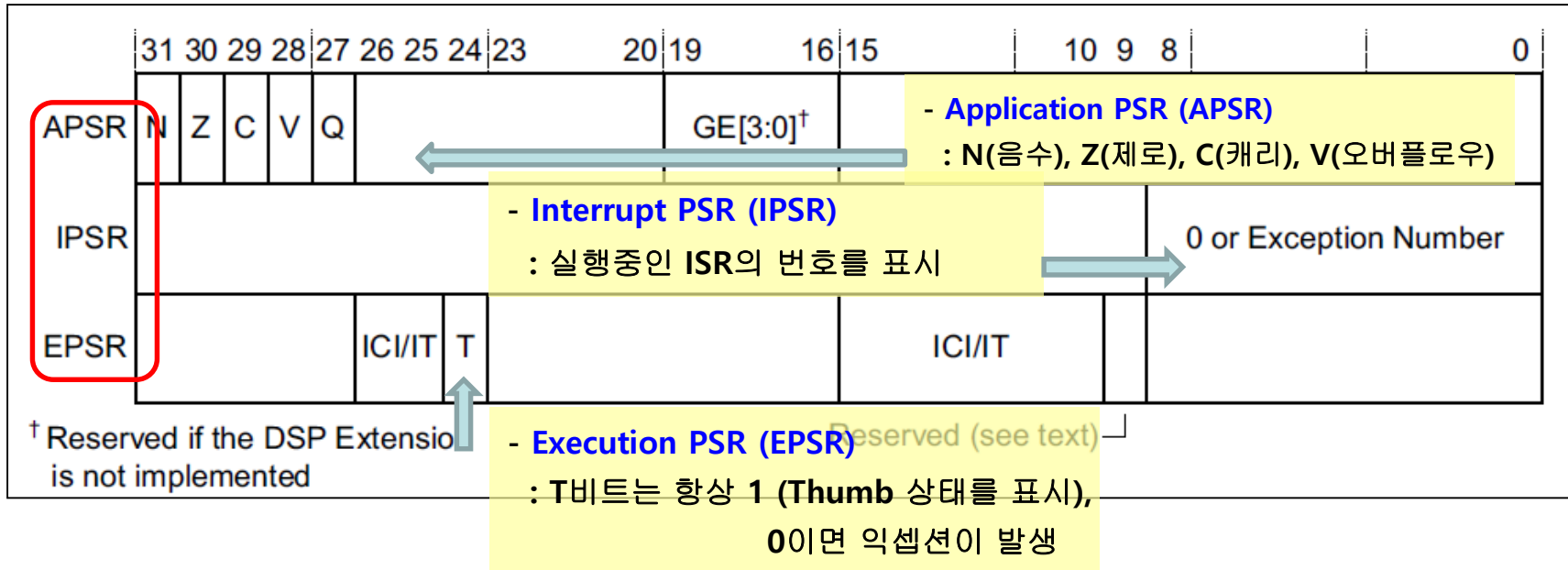
- **MSP**(main stack pointer), **PSP**(process stack pointer)
- **MSP** : Reset이후, 기본 **stack pointer**, **exception** 사용
- **PSP** : **thread mode**에서만 사용
- thread mode** 경우,
: **Control register [1]bit = 1, PSP / [1]bit = 0, MSP**
- handler mode** 경우
: **main stack pointer** 로 동작
- **reset** 직후, 항상 **process stack pointer**로 동작



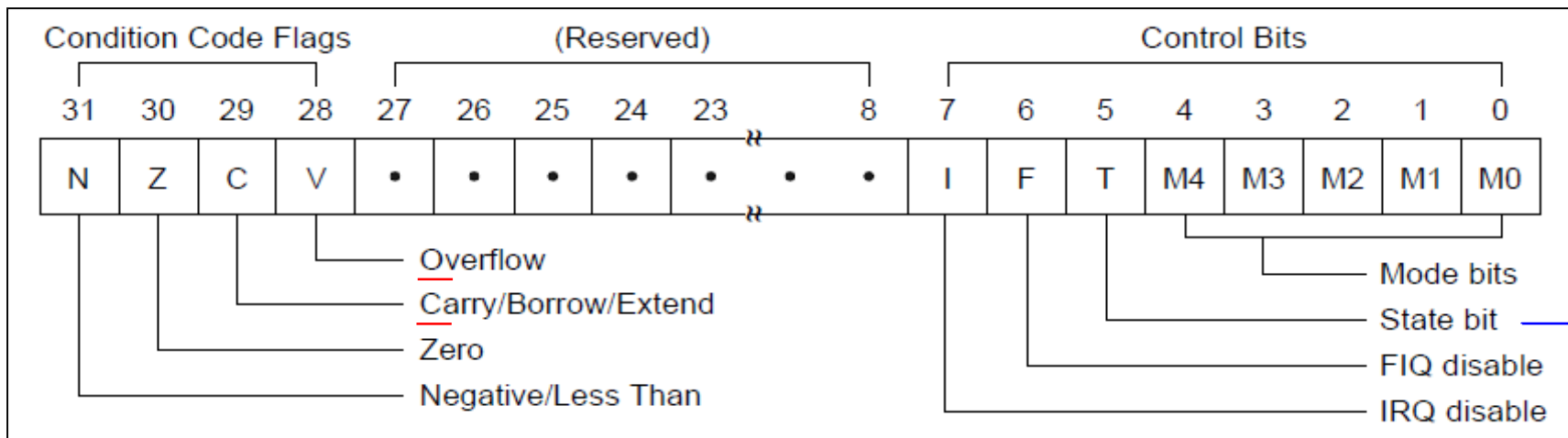
Architecture of ARM Processor

Register :
ARM과 Cortex-M 비교

PSR : Cortex-M7

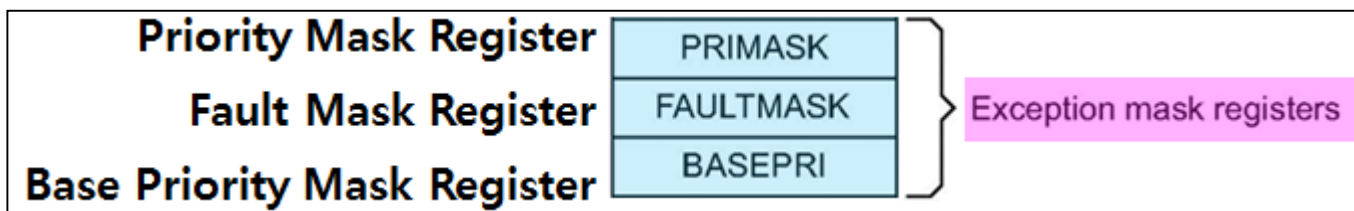


PSR : ARM7



ARM / Thumb mode

- Mask-abled Interrupt
 - : 처리 여부를 묻고, 처리하는 interrupt
- Non Mask-abled Interrupt (NMI)★
 - : 처리 여부를 묻지 않고, CPU가 무조건 처리하는 interrupt



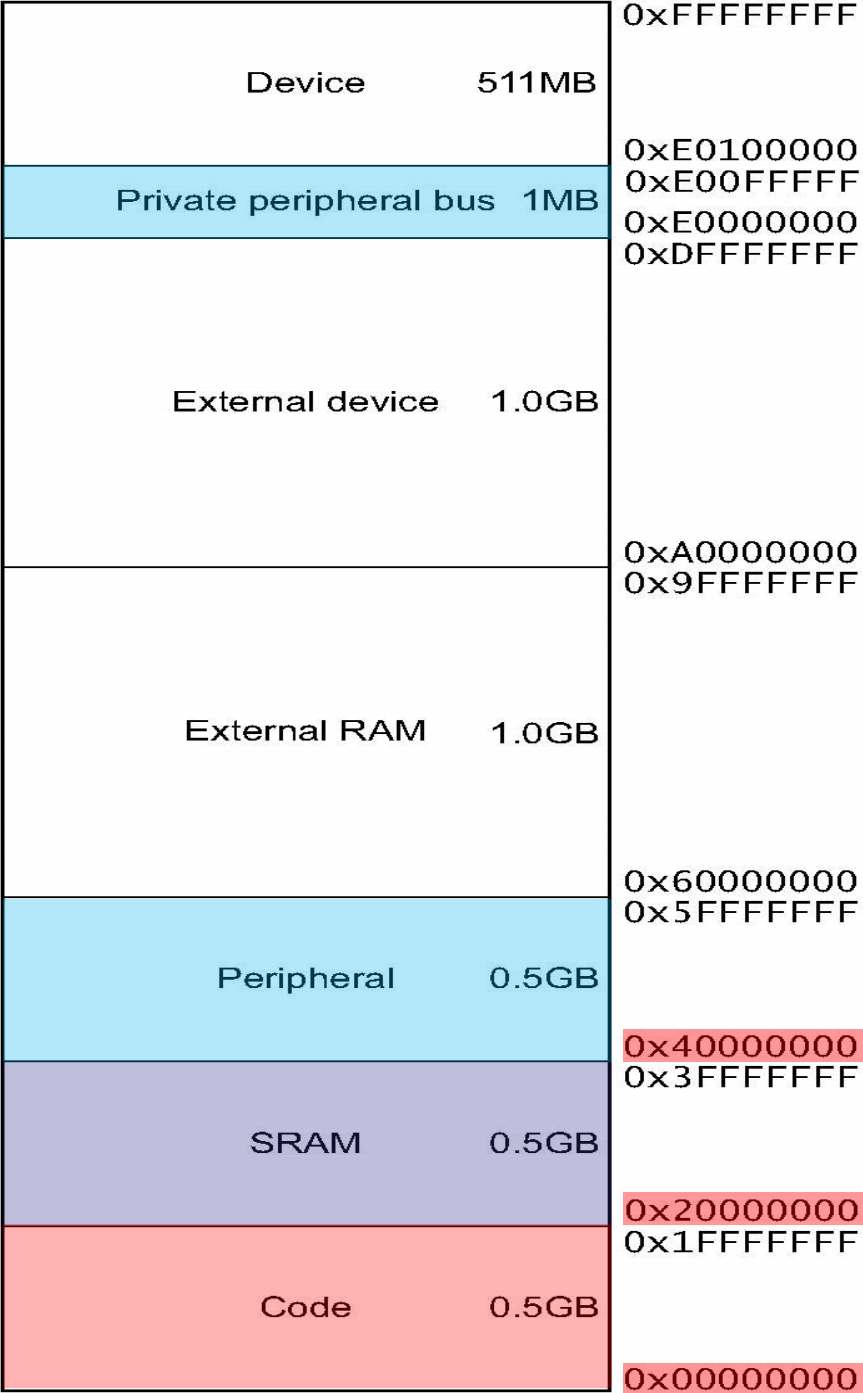
- Exception mask registers
 - Priority Mask Register
 - Power fail
 - : Non Mask-abled Interrupt과 hardware 결함 interrupt를 제외한 모든 interrupt를 차단한다
 - 현재 사용중인 interrupt 우선 순위를 가장 빠른 순서 "0"로 만든다
 - Fault Mask Register
 - : NMI를 제외한 모든 interrupt를 차단한다
 - Base Priority Mask Register
 - : 우선순위의 기준이 되는 값
 - 만일 기준값 = 7로 설정하면 7번째 우선순위보다 빠른 exception만이 허용된다.

STM32F767VGT6의 Memory map

Cortex-M7 계열

※ 총 4GB 용량

	STM32F767Vx	
	VG	VI
플래시 메모리 [KB]	1024	2048
SRAM [KB]	512	

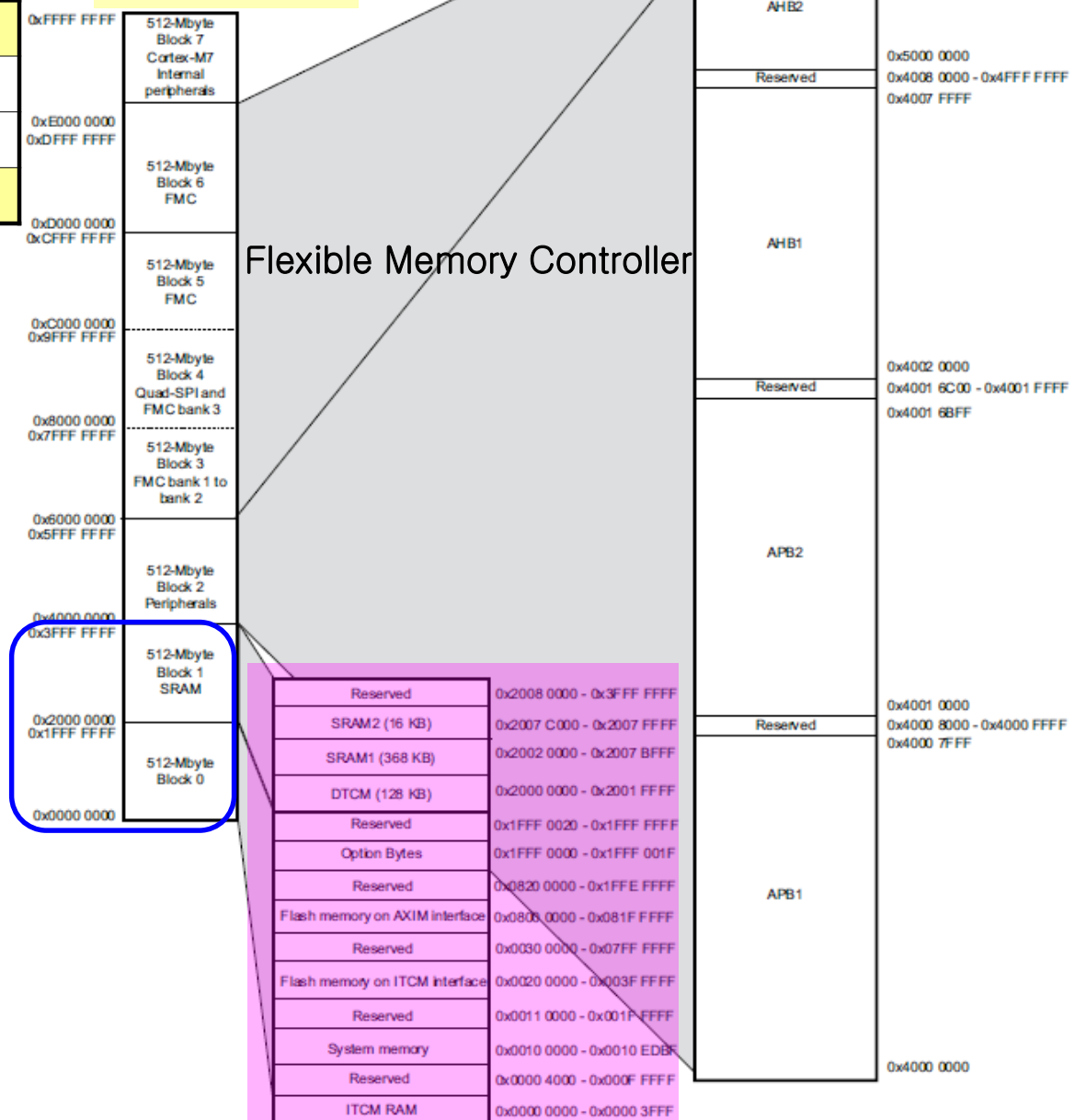


STM32F767VGT6의 Memory map

	STM32F767Vx	
	VG	VI
플래시 메모리 [KB]	1024	2048
SRAM [KB]	512	

STM32F767VGT6
flash : 1024kB
SRAM : 512kB

총 4GB 용량



STM32F767VGT6의 Memory map

• 512MB Block 1 (SRAM)

Reserved	0x2008 0000 - 0x3FFF FFFF
SRAM2 (16 KB)	0x2007 C000 - 0x2007 FFFF
SRAM1 (368 KB)	0x2002 0000 - 0x2007 BFFF
DTCM (128 KB)	0x2000 0000 - 0x2001 FFFF
Reserved	0x1FFF 0020 - 0x1FFF FFFF
Option Bytes	0x1FFF 0000 - 0x1FFF 001F
Reserved	0x0820 0000 - 0x1FFE FFFF
Flash memory on AXIM interface	0x0800 0000 - 0x081F FFFF
Reserved	0x0030 0000 - 0x07FF FFFF
Flash memory on ITCM interface	0x0020 0000 - 0x003F FFFF
Reserved	0x0011 0000 - 0x001F FFFF
System memory	0x0010 0000 - 0x0010 EDBF
Reserved	0x0000 4000 - 0x000F FFFF
ITCM RAM	0x0000 0000 - 0x0000 3FFF

• 512MB Block 0

- Flexible memory controller (FMC)
 - : NOR/PSRAM memory controller
 - : NAND/memory controller
 - : Synchronous DRAM controller (SDRAM/Mobile LPDDR SDRAM)

굵은 선: 64bits bus, 가는 선: 32bits bus

AXIM
AXI (Advanced
extensible
Interface)
Master Bus

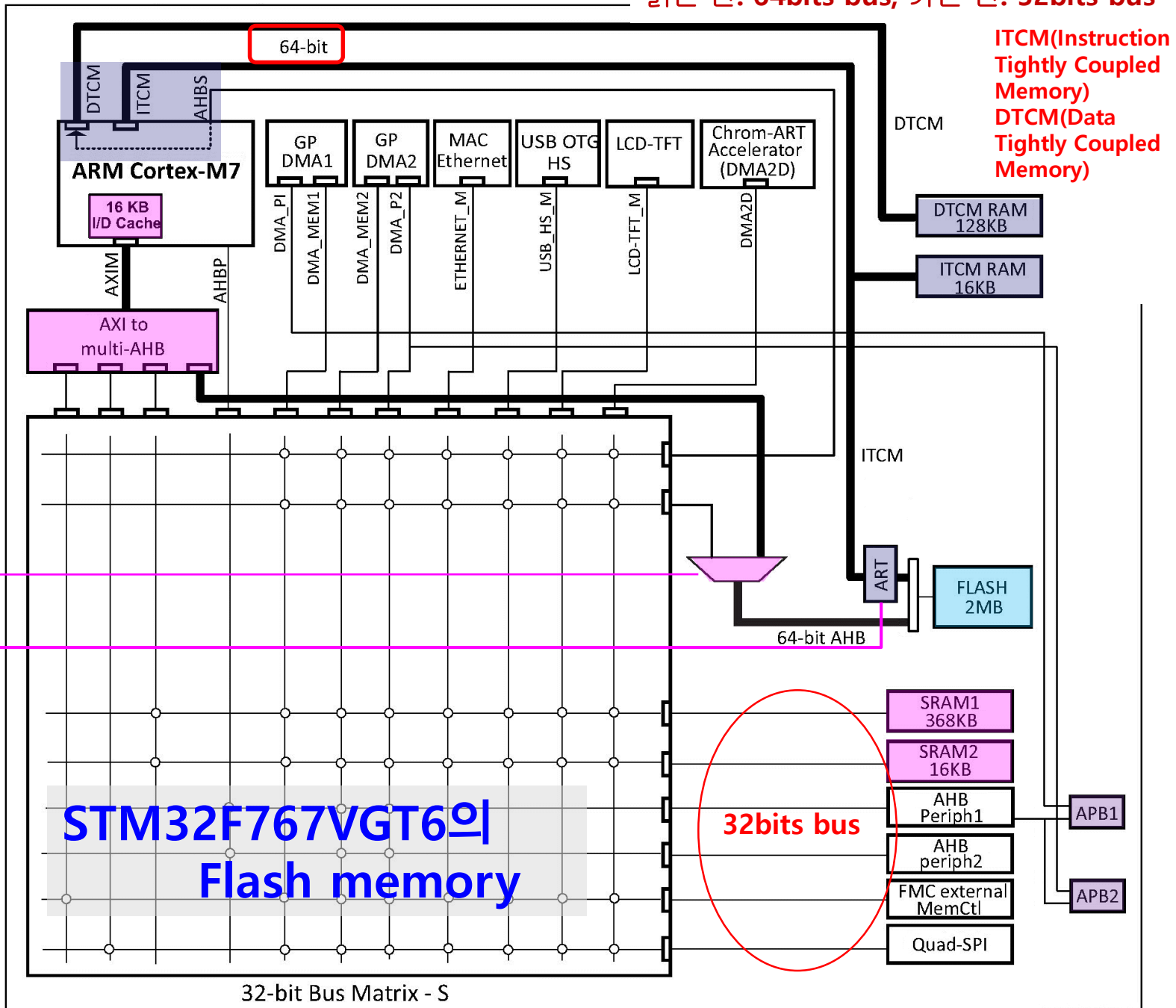
명령 캐시와
데이터 캐시를
사용 ←

ART(Adaptive
Real-Time)
Accelerator와

Prefetch buffer
사용

STM32F767VGT6의
Flash memory

ITCM(Instruction
Tightly Coupled
Memory)
DTCM(Data
Tightly Coupled
Memory)



32-bit Bus Matrix - S

32bits bus

STM32F767VGT6의 Flash memory

- **Prefetch buffer**

: CPU access speed > Flash memory

→ flash 동작속도가 느려 CPU는 wait

→ zero wait, one wait, two wait 에 해당되는 값만큼 latency를 인가.

→ 전체적으로 동작성능을 저하시킴.

→ **Prefetch buffer**

: Flash 메모리에서 명령어를 읽어 올 때 하나의 명령어만 읽어 오는 것이 아니라 한번에 16바이트씩 데이터를 읽어 오는 것.

- Flash memory interface 방법

① **ITCM** 사용(64bits) : instruction read, data read 가능하지만 write 불가능

→ **ART(Adaptive Real-Time) Accelerator** 사용하여 cache기능(write) 사용,
read는 Prefetch buffer 사용

② **AXIM bus**와 **AHB bus** 사용(64bits) : instruction read, data read/write 가능

③ **32bits AHB bus** 사용 : flash memory control register 사용

STM32F767VGT6의 SRAM

- Internal SRAM : 512KB

- 64비트의 DTCM 버스로 액세스하는 데이터 저장 : 128KB(0x2000 0000 번지부터 시작).
- 64비트의 ITCM 버스로 액세스하는 프로그램 저장 : 16KB(0x0000 0000 번지부터 시작).
- 32비트의 AHB 버스로 액세스하는 데이터 저장 : SRAM1 - 368KB(0x2002 0000 번지부터 시작)
: SRAM2 - 16KB(0x2007 C000 번지부터 시작)
- 최고 속도인 216MHz에서도 Zero access로 액세스

Reserved	0x2008 0000 - 0x3FFF FFFF
SRAM2 (16 KB)	0x2007 C000 - 0x2007 FFFF
SRAM1 (368 KB)	0x2002 0000 - 0x2007 BFFF
DTCM (128 KB)	0x2000 0000 - 0x2001 FFFF
Reserved	0x1FFF 0020 - 0x1FFF FFFF
Option Bytes	0x1FFF 0000 - 0x1FFF 001F
Reserved	0x0820 0000 - 0x1FFE FFFF
Flash memory on AXIM interface	0x0800 0000 - 0x081F FFFF
Reserved	0x0030 0000 - 0x07FF FFFF
Flash memory on ITCM interface	0x0020 0000 - 0x003F FFFF
Reserved	0x0011 0000 - 0x001F FFFF
System memory	0x0010 0000 - 0x0010 EDBF
Reserved	0x0000 4000 - 0x000F FFFF
ITCM RAM	0x0000 0000 - 0x0000 3FFF

STM32F767VGT6의 I/O 제어 레지스터의 비트 속성

I/O 제어 레지스터의 비트 속성

STM32F767VGT6에서 사용하는 I/O 제어 레지스터의 각 비트들은 고유한 속성을 가지고 있다

- **rw** : (read/write) 사용자 프로그램에서 read하거나 write할 수 있는 비트이다.
 - **r** : (read-only) 사용자 프로그램에서 read만 할 수 있는 비트이다.
 - **w** : (write-only) 사용자 프로그램에서 write만 할 수 있는 비트이다.
 - **rwo** : (read/write once) read 및 write가 가능하지만, write는 1번만 할 수 있는 비트이다.
-
- **rc_w1** : (read/clear) 사용자 프로그램에서 read하거나 1을 write하여 클리어할 수 있는 비트이다. 이 비트에 0을 write하면 아무 동작도 일어나지 않는다.
 - **rc_w0** : (read/clear) 사용자 프로그램에서 read하거나 0을 write하여 클리어할 수 있는 비트이다. 이 비트에 1을 write하면 아무 동작도 일어나지 않는다.
 - **rc_r** : (read/clear) 사용자 프로그램에서 read할 수 있고, read 후 자동으로 클리어되는 비트이다.
 - **rs** : (read/set) 사용자 프로그램에서 read하거나 1을 write하여 세트할 수 있는 비트이다. 이 비트에 0을 write하면 아무 동작도 일어나지 않는다.
-
- **rt_w** : (read-only write trigger) 사용자 프로그램에서 read만 할 수 있는 비트이다. 여기에 0이나 1을 write하면 이벤트가 발생하지만 비트 값에는 영향을 주지 않는다.
 - **t** : (toggle) 사용자 프로그램에서 1을 write하여 비트 값을 toggle할 수 있는 비트이다.
 - **Reserved** 또는 **Rev.** 또는 **-** : 사용하지 않는 비트이다.

microchip.com : atmega128a datasheet download



Products

click

Solutions

Tools and Software

Support

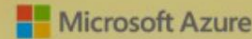
Education

About

Order Now



Learn How Microchip and Microsoft Solutions Simplify Your IoT Designs



REGISTER



Get Design Help



Events

Find a product



All



Enter keyword, item, model or part #

PRODUCTS

SOLUTIONS

TOOLS AND RESOURCES

SUPPORT

EDUCATION

Browse All Products

Product Selection Tools

Microcontrollers and Microprocessors

Analog

Amplifiers and Linear ICs

Clock and Timing

Data Converters

Embedded Controllers and Super I/O

FPGAs and PLDs

High-Speed Networking and Video

Interface and Connectivity

LED Drivers and Backlighting

Browse Microcontrollers and Microprocessors [view all](#)

8-bit MCUs

PIC® MCUs

AVR® MCUs

16-bit MCUs

PIC24F GU/GL/GP MCUs

PIC24F GA MCUs

PIC24F GB MCUs

PIC24F GC MCUs

dsPIC® DSCs

dsPIC33C DSCs

dsPIC33E DSCs

32-bit MCUs

PIC32 MCUs

SAM MCUs

AVR® and RISC MCUs

CEC MCUs

Microprocessors (MPUs)

SAMA5

SAMA7

SAM9

SiP and SOM

Wireless MCUs

microchip.com : atmega128a datasheet download

[PRODUCTS](#)[SOLUTIONS](#)[TOOLS AND RESOURCES](#)[SUPPORT](#)[EDUCATION](#)[Hide Filters](#)

Refine your search

[All Microchip Content](#)[Application Center](#)[Products](#)[Product Documents](#)[Data Sheets](#)[Errata](#)[Package Drawings](#)[Reference Manuals](#)[User Guides](#)[Application Notes](#)[Product Change Notification](#)[Corporate Information](#)

Results **1 - 5** of **5** for "atmega128a"

ATmega128A - 8-bit AVR Microcontrollers



The high-performance, low-power Microchip 8-bit AVR® RISC 128 KB flash memory with read-while-write capabilities, 4 Kbytes of SRAM, 232 I/O lines, 32 general purpose working registers,...

[Buy](#) [Samples](#) [Document](#)**Download****Atmel-8151-8-bit-AVR-ATmega128A_Datasheet**[ATmega128A - Complete Datasheet](#)[ATmega128A - Summary Datasheet](#)[AN_8166 - AVR525: Migration from ATmega128 to ATmega128A](#)

2. Atmel사의 8bit Micro-controller

AVR 패밀리

패밀리	장점	응용분야	주요 특징
32비트 AVR UC3 → ARM Processor와 대조됨	32비트 고성능 마이크로컨트롤러	범용	• 16~512[Kbyte]의 플래시 프로그램 메모리 • 48~144핀 • 66[MHz]까지 CPU 클럭 사용 • 1[MHz]당 최대 1.5[MIPS] 연산 속도 • 부동소수점 연산 장치 포함
8/16비트 AVR XMEGA	우수한 성능의 8 비트 마이크로컨트롤러	범용	• 16~384[Kbyte]의 플래시 프로그램 메모리 • 44~100핀 • 32[MHz]까지 CPU 클럭 사용 • 1[MHz]당 최대 1[MIPS] 연산 속도
8비트 megaAVR	풍부한 주변 장치가 포함된 마이크로컨트롤러	범용, 조명, LCD	• 4~256[Kbyte]의 플래시 프로그램 메모리 • 28~100핀 • 20[MHz]까지 CPU 클럭 사용 • 1[MHz]당 최대 1[MIPS] 연산 속도
8비트 tinyAVR	소형 마이크로컨트롤러	범용 소형 제어 장치	• 0.5~8[Kbyte]의 플래시 프로그램 메모리 • 6~32핀 • 20[MHz]까지 CPU 클럭 사용 • 1[MHz]당 최대 1[MIPS] 연산 속도
배터리 관리형 AVR	Li-Ion 배터리 관리형 마이크로컨트롤러	가스 측정, 인증, 회로 보호	• 1.8[V]~25[V]에서 동작 • 8~40[Kbyte]의 플래시 프로그램 메모리 • 28~48핀 • 8[MHz]까지 CPU 클럭 사용 • 1[MHz]당 최대 1[MIPS] 연산 속도
자동차형 AVR	지능형 제어	자동차	• 자동차 환경에 맞는 견고한 설계

2. Atmel사의 8bit Micro-controller

8비트 megaAVR 패밀리 소자(16MHz)

Million Instructions Per Second

소자명	Flash [KB]	ISP	EEP ROM [Byte]	SRAM [Byte]	8비트 타이머	16비트 타이머	PWM	SPI	TWI	USART	CAN	10 비트 A/D	10 비트 D/A	인터럽트 + 리셋 벡터	외부 인터럽트	MIPS	Vcc [V]	최대 I/O 핀
ATmega8	8	Yes	512	1K	2	1	3	1	1	1	0	8	0	19	2	16	2.7~5.5	23
ATmega128	128	Yes	4K	4K	2	2	7	1	1	2	0	8	0	35	8	16	2.7~5.5	53
AT90CAN128	128	Yes	4K	4K	2	2	7	1	1	2	1	8	0	37	8	16	2.7~5.5	53
AT90PWM3B	8	Yes	512	512	1	1	12	1	0	1	0	11	1	29	4	16	2.7~5.5	27

8비트 tinyAVR 패밀리 소자(요약)

소자명	Flash [KB]	ISP	EEP ROM [Byte]	SRAM [Byte]	8비트 타이머	16비트 타이머	PWM	SPI	TWI	USART	10 비트 A/D	10 비트 D/A	온도 센서	인터럽트 + 리셋 벡터	외부 인터럽트	MIPS	Vcc [V]	최대 I/O 핀
ATtiny85	8	Yes	512	512	2	no	4	1 USI	1 USI	0	4	1	있음	15	6	20	1.8~5.5	6
ATtiny4	0.5	Yes	0	32	—	1	2	0	0	0	0	1	—	11	4	12	1.8~5.5	4
ATtiny9	1	Yes	0	32	—	1	2	0	0	0	0	1	—	11	4	12	1.8~5.5	4
ATtiny40	4	Yes	0	256	1	1	2	1	1 slave	0	12	1	있음	18	18	12	1.8~5.5	18

2. Atmel사의 8bit Micro-controller

고 성능 저 전력 RISC 구조	133개의 명령어(대부분 한 클럭 사이클로 실행) 32X8 비트 개의 범용 작업 레지스터, 16Mhz의 명령 처리 속도
데이터 및 비휘발성 메모리	128K바이트의 프로그램용 플래시 메모리 내장 - 10,000번까지 R/W 4K 바이트의 SRAM, 4K 바이트의 EEPROM (100,000번 R/W) 외부 64K 바이트의 메모리 영역, 소프트웨어와 EEPROM 데이터 보안을 위한 락(Lock)기능
주변장치	내장 메모리(플래시, EEPROM)의 프로그래밍과 디버그 위한 JTAG인터페이스(IEEE std.11491) 53개의 I/O핀(A,B,C,D,E,F 포트 - 8비트, G 포트 - 5비트), 리셋을 포함한 35개의 인터럽트 소스 별도의 프리스케일러를 갖는 2개의 8비트 타이머/카운터(0,2) 별도의 프리스케일러를 갖는 2개의 16비트 타이머/카운터(1,3) 2개의 8비트 PWM 채널과 6개의 2~16비트 PWM채널 8채널 10비트 A/D 컨버터와 아날로그 비교기 SPI 시리얼 인터페이스, 2개의 프로그램이 가능한 USART, watchdog 타이머
특수 기능	6개의 슬립모드, 내부 RC 오실레이터와 외부 크리스탈을 연결하기 위한 발진회로 내장 외부, 내부 인터럽트(Interrupt) 소스
동작 전압	2.7-5.5V(ATmega128A), 4.5-5.5V(ATmega128)
동작 클럭	0-8MHz(ATmega128A), 0-16MHz(ATmega128)

2. Atmel사의 8bit Micro-controller

ATmega128 PIN MAP

PA, PB, PC, PD, PE, PF

: 8Ports

: 3 states @ reset

: Bi-direction

: Internal Pull_up

PG

: 5Ports

: 3 states @ reset

: Bi-direction

: Internal Pull_up

Serial IO

Analog
Comparator

Timer Counter
Comparator

Timer Counter
Clock

Programming
enable

ADC

JTAG

External memory

: Lower address

: Data 8bits

IO Port

: 40mA

External memory

: Upper address

IO Port

: 40mA

address 16개

→ 64 x 1024: 64K

Address Latch Enable
/Read, /Write

Serial 통신

External Interrupt

Crystal

1.5us

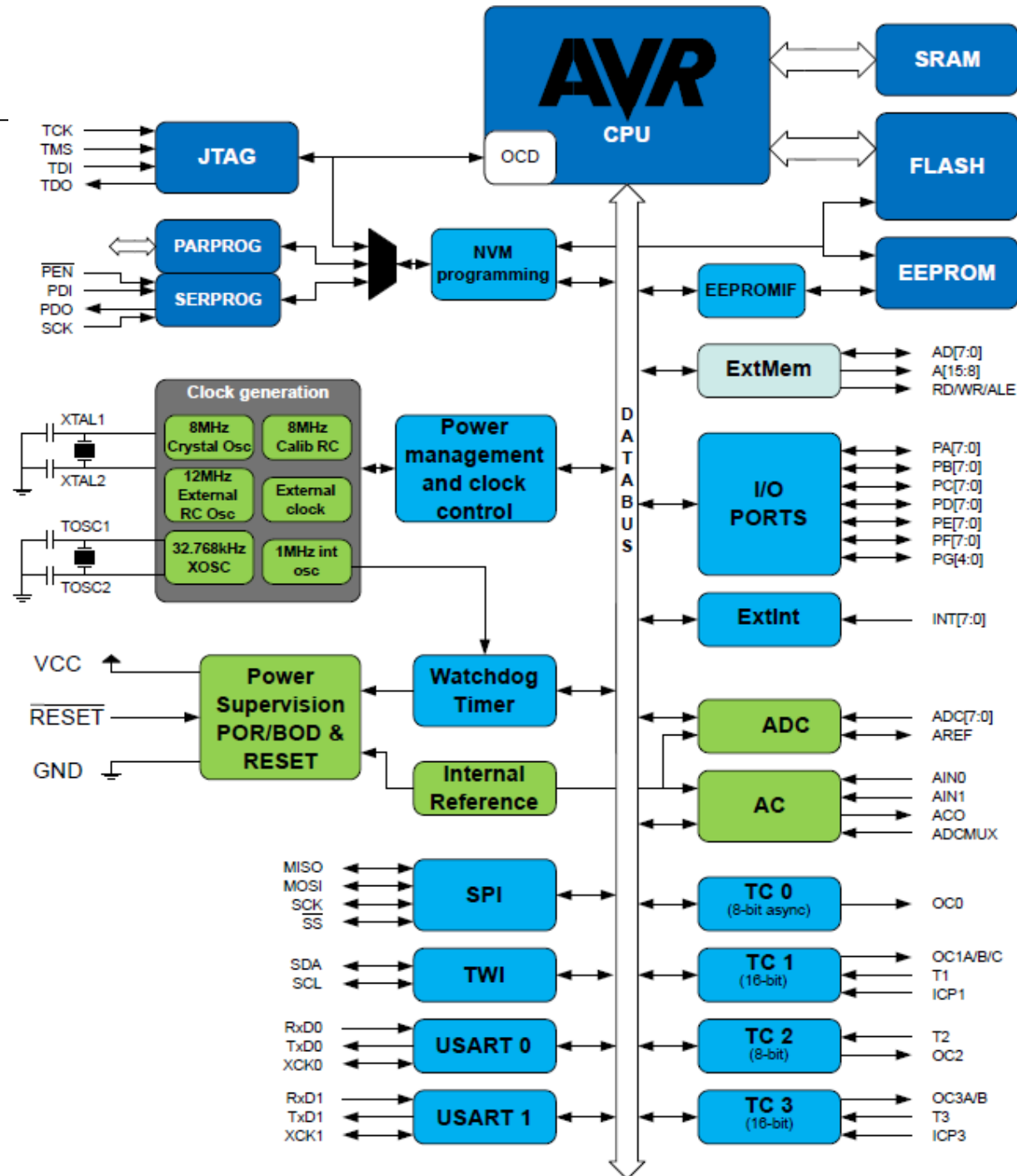
AVR[®]

Sink current : input

Source current : output

ATmega128

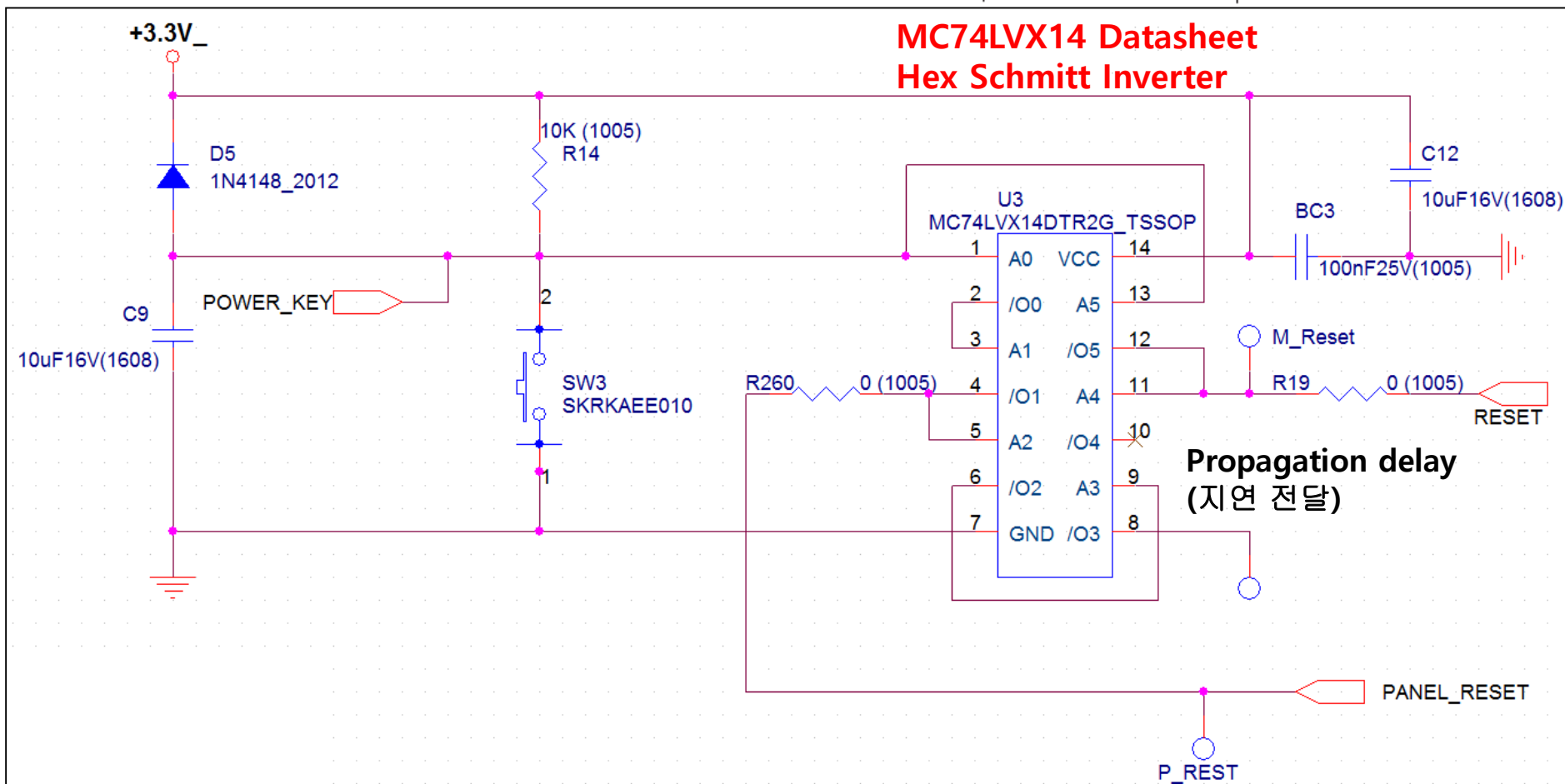
Block Diagram



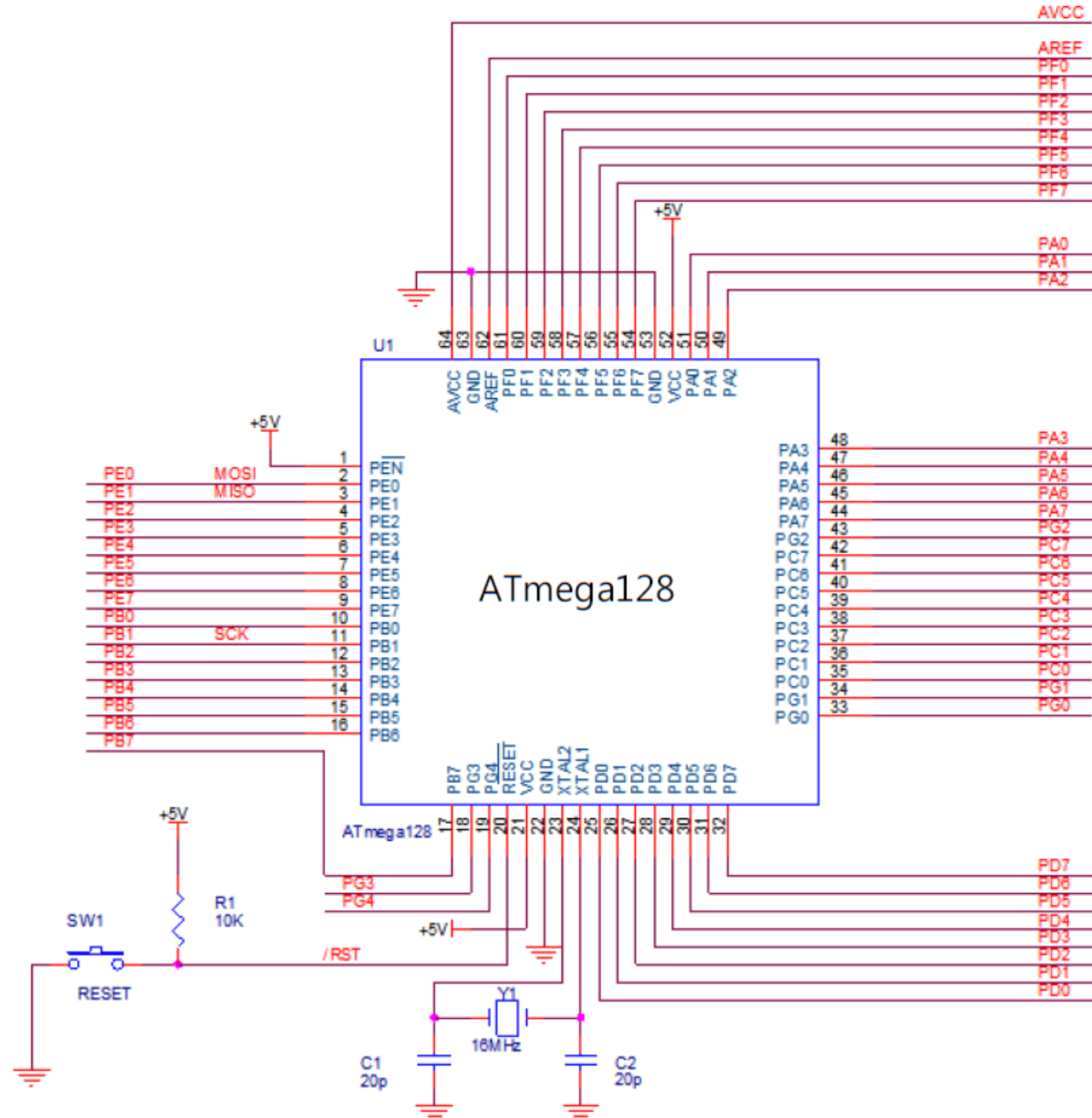
2. Atmel사의 8bit Micro-controller

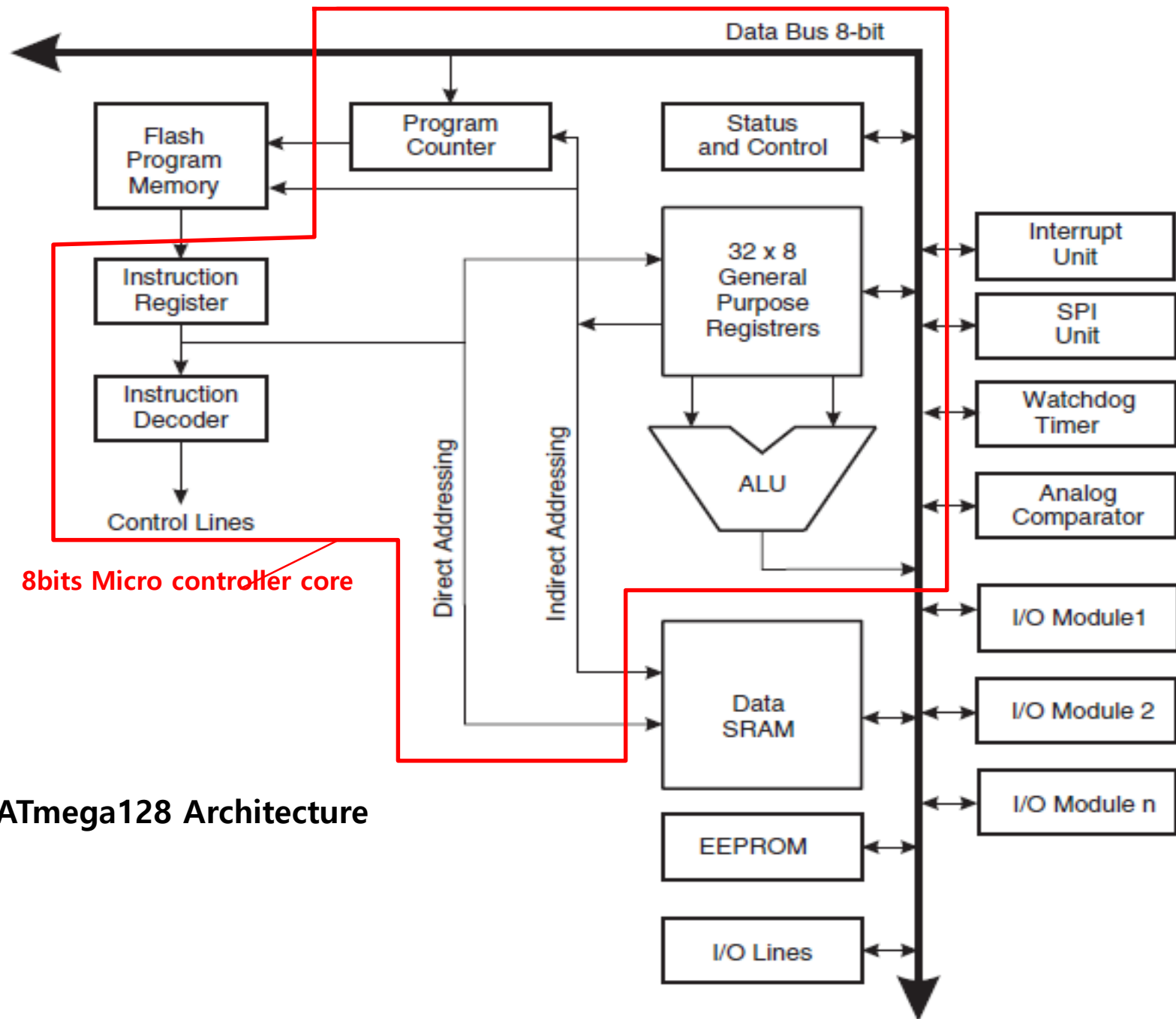
ATmega128 Reset

MCU Start-up



ATmega128 Reset





ATmega128 Architecture

2. Atmel사의 8bit Micro-controller

SREG(Status Register) 상태 레지스터

- : 가장 최근에 실행된 산술연산의 결과 정보를 저장
- : 결과에 따라 다음에 실행되는 프로그램의 흐름을 제어 명령어에 영향
- : 인터럽트가 발생했을 때
 - 상태 레지스터를 하드웨어가 자동으로 저장, 복귀 되지 않음
 - 소프트웨어적으로 저장, 복귀 동작을 수행해야 함

비트	7	6	5	4	3	2	1	0
	I	T	H	S	V	N	Z	C
읽기/쓰기	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
초깃값	0	0	0	0	0	0	0	0

- I (Global Interrupt Enable)
- T (Bit Copy Storage) : BLD(Bit Load)와 BST(Bit STore)
- H(Half Carry Flag) : 산술 연산에서 Half Carry/Half Borrow
- S(Sign Bit) : 부호비트, N과 V의 XOR 결과
- V(Two's Complement Overflow Flag)
- N(Negative Flag) : 연산결과가 음수
- Z(Zero Flag) : 연산결과가 0
- C(Carry Flag) : 연산결과에 Carry

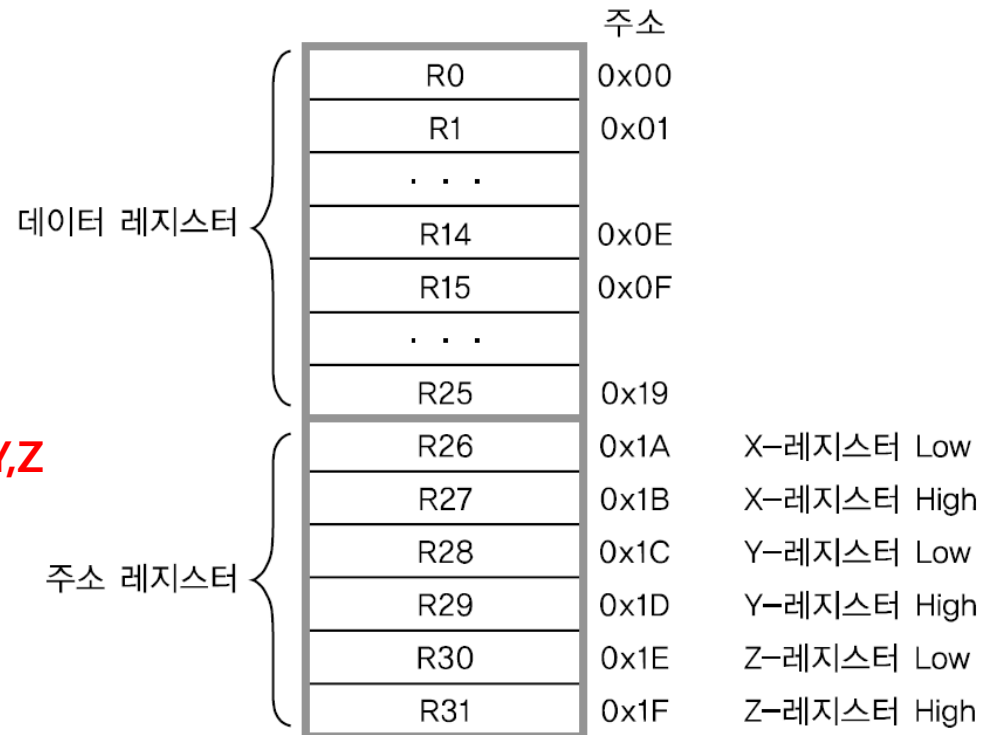
2. Atmel사의 8bit Micro-controller

비트명	읽기/쓰기	초깃값	설명
I	R/W	0	전역 인터럽트 활성화 비트 - 이 비트가 1이고, 개별적인 인터럽트 활성화 비트가 1이 되어야 인터럽트가 작동한다. - 인터럽트 서비스 루틴이 실행되면 자동으로 0이 되고, IRET 명령으로 인터럽트 서비스 루틴이 종료되면 자동으로 1이 된다. - SEI 명령으로 1, CLI 명령으로 0을 만들 수 있다.
T	R/W	0	비트 복사/저장 비트 - BST 명령으로 레지스터의 한 비트를 T 비트에 복사한다. - BLD 명령으로 T 비트를 레지스터의 한 비트에 복사한다.
H	R/W	0	반캐리(Half Carry) 비트 - 연산 결과 비트 3에서 비트 4로 캐리가 발생하면 1이 된다. - BCD(Binary-Coded Decimal) 연산에 유용하다.
S	R/W	0	부호 비트 $N \oplus V$, 음수 플래그와 2의 보수 오버플로 플래그의 배타적 OR
V	R/W	0	2의 보수 오버플로 플래그 연산 결과 $b7 \oplus b6$, 비트 7과 비트 6의 배타적 OR
N	R/W	0	음수 플래그 연산 결과 MSB가 1이면 음수 플래그가 1이 된다.
Z	R/W	0	제로 플래그 연산 결과 모든 비트가 0이면 제로 플래그가 1이 된다.
C	R/W	0	캐리 플래그 연산 결과 MSB에서 캐리가 발생하면 1이 된다.

2. Atmel사의 8bit Micro-controller

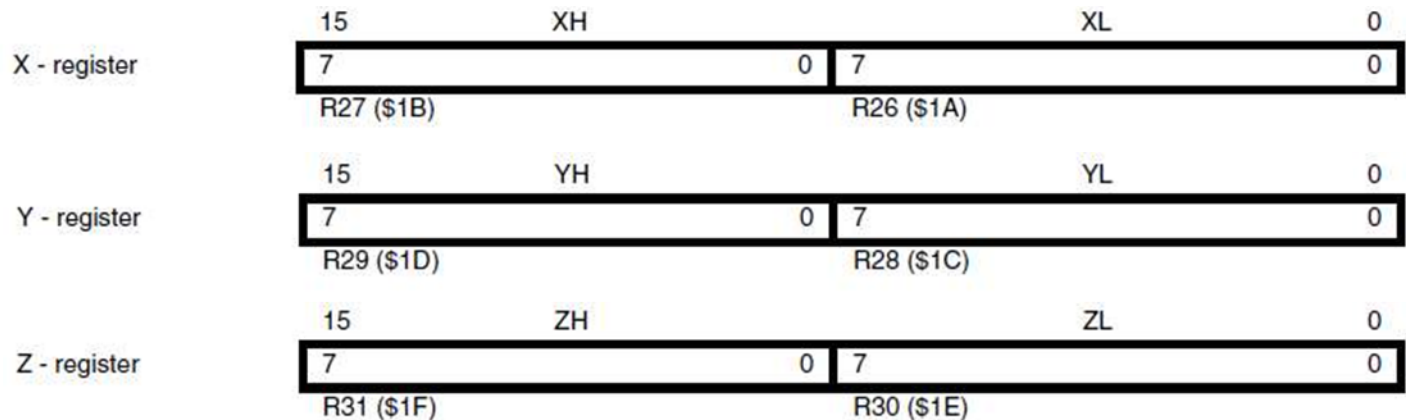
General Register(범용 레지스터)

- : **Data Register** 라고 명칭
- : **Register R0~R25**
- : 데이터를 저장하는 레지스터
- : 메모리 주소 **0x00~0x19**에 해당



Address Register (주소 레지스터) X,Y,Z

- : **Register R26~R31, 16bits Reg.**
- : **Register** 간접 주소방식을 위함
- : 메모리 주소 **0x1A~0x1F**에 해당



2. Atmel사의 8bit Micro-controller

Stack

: 데이터 메모리인 **SRAM** 사용.

: 서브루틴의 **호출(Call)** 주소나 인터럽트 처리 루틴의 복귀 주소 등을 임시 저장

: **LIFO(Last-In First-Out)** 방식 - 맨 나중에 저장한 데이터를 맨 처음 꺼내게 됨

Stack Pointer (SP) - 16bits

: **스택의 어드레스를 저장**하는 레지스터가 스택 포인터(**SP**)

: **SP**는 항상 스택의 상단을 가리킴

➔ 스택 포인터의 주소는 높은 값에서 낮은 값으로 감소

➔ **PUSH** 명령어 : 데이터가 스택에 저장. 이때 스택 포인터는 **1씩 감소**

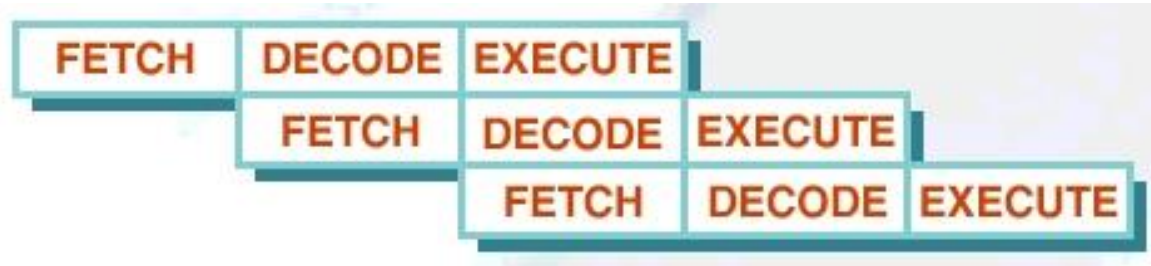
➔ **POP** 명령어 : 데이터를 스택의 **TOP**에서 꺼냄. 이때 스택 포인터는 **1씩 증가**

Bit	15	14	13	12	11	10	9	8	
	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SPH
	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	SPL
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

2. Atmel사의 8bit Micro-controller

Instruction Execution Timing of 32bits Processor

- ARM7 3steps pipeline



- ARM9 5steps pipeline



2. Atmel사의 8bit Micro-controller

Program Memory Map

: Internal Flash Memory Size 128KB

➔ Address size ?

(ATMEGA128 Address 수 16)

: 128K = 2 x 64K

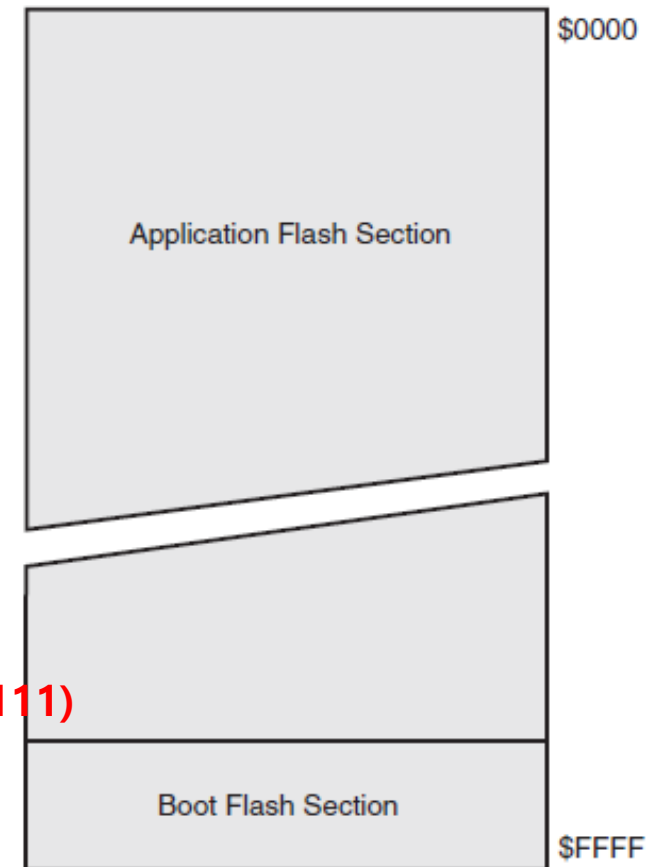
: 16bits ➔ 64K = $2^6 \times K$

➔ 0x0000 ~ 0x7FFF : RAMPZ0=0 LOWER

(0 ~ 0111 1111 1111 1111)

➔ 0x8000 ~ 0xFFFF : RAMPZ0=1 UPPER

(1000 0000 0000 0000 ~ 1111 1111 1111 1111)



Bit	7	6	5	4	3	2	1	0	
	-	-	-	-	-	-	-	RAMPZ0	RAMPZ
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

2. Atmel사의 8bit Micro-controller

Internal SRAM (4KB)

- : 32개의 범용 레지스터 영역
- : 64개의 I/O 레지스터
- : 160개의 확장 I/O 레지스터
- : 4KB의 내부 SRAM

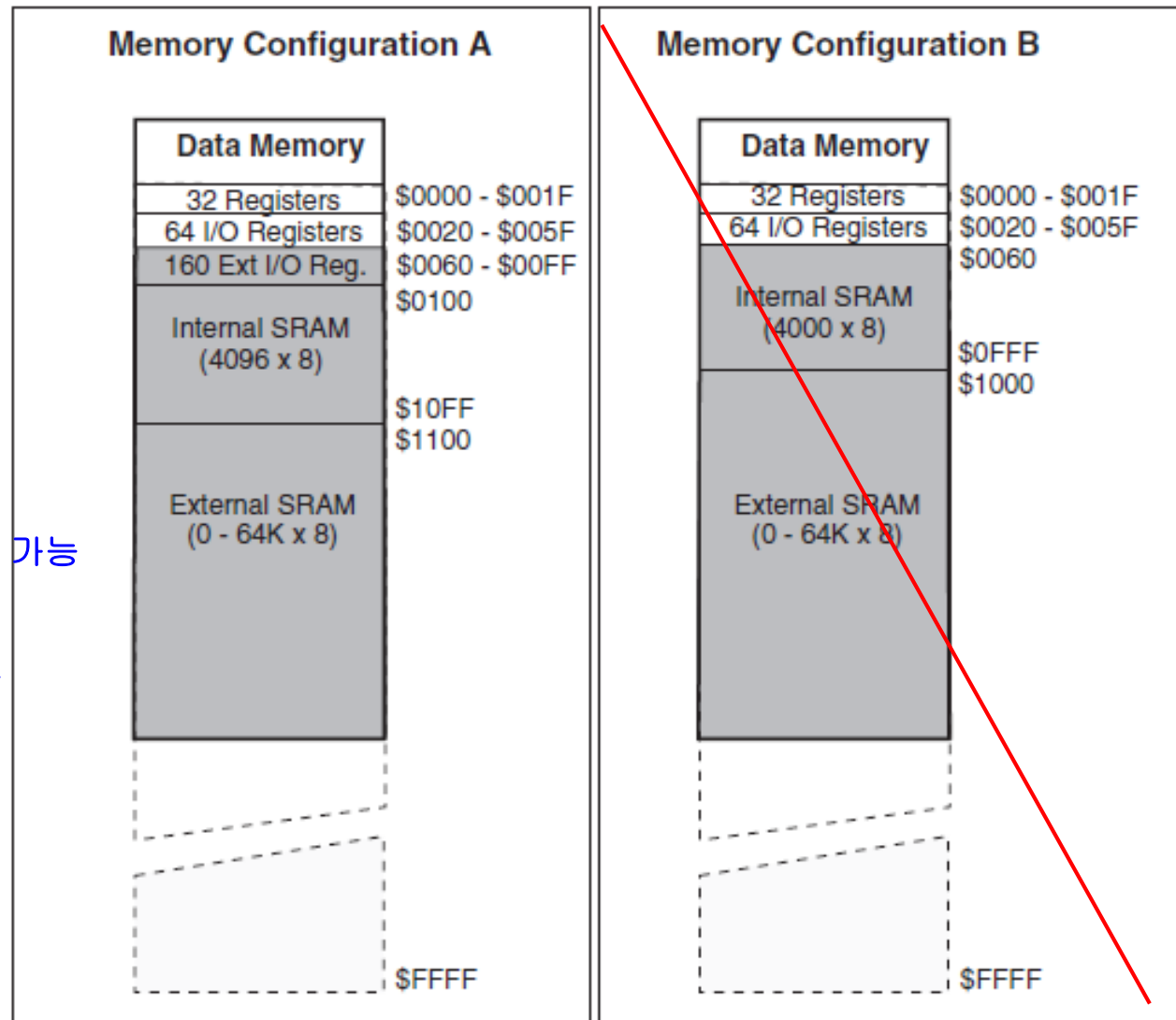
: C 프로그램 작성시

→ I/O 레지스터는

8bits, 16bits 읽기/쓰기 가능

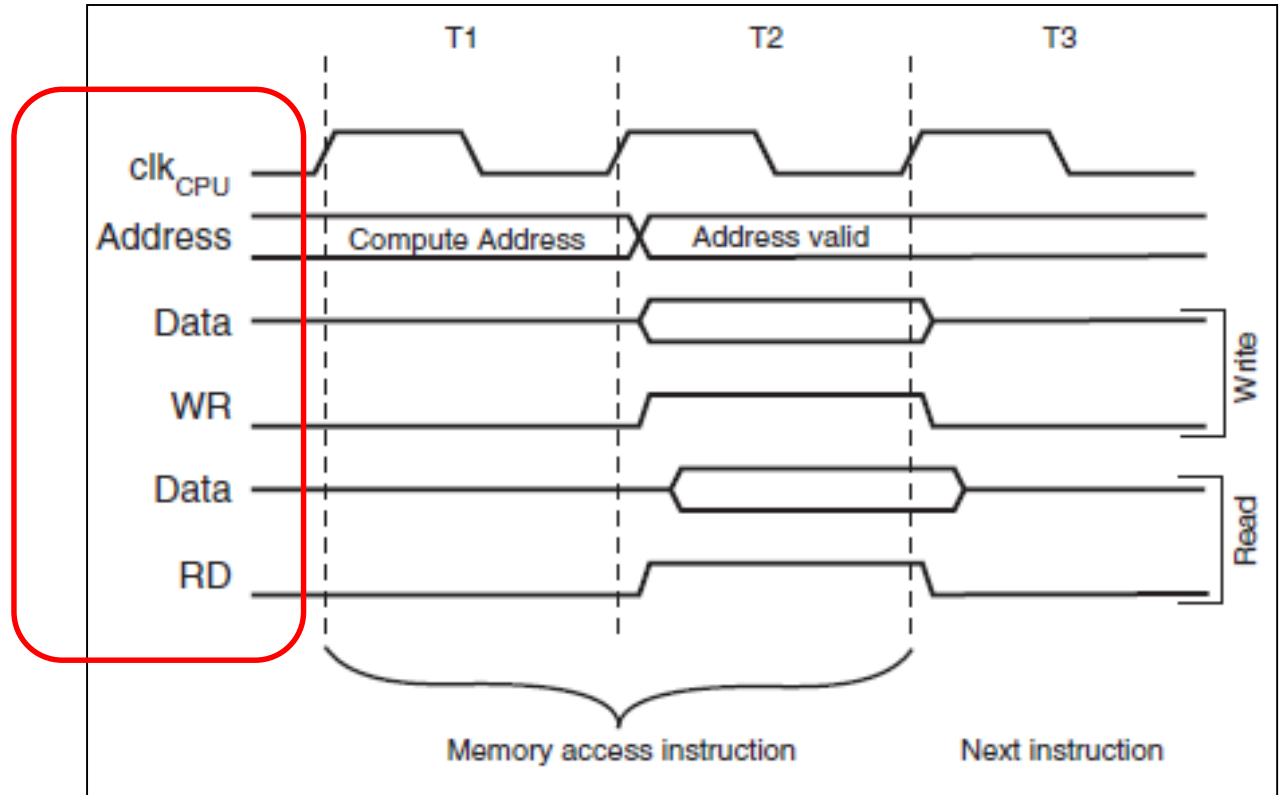
→ 확장 I/O 레지스터는

8bits 읽기/쓰기만 가능



2. Atmel사의 8bit Micro-controller

Internal SRAM Access Cycles



내부 SRAM은 0x0100 ~ 0x10FF번지의 4KB

-SRAM을 액세스하는 데는 2개의 시스템 clock 내에서 수행

-Address, data, WR, RD

2. Atmel사의 8bit Micro-controller

I/O

Register

[Question]

Data Register

: R0~ R25

Address Register

: R26 ~ R31

→ 0 ~ 0x1F

→ Offset 0x20

주소	레지스터명	기능
<u>0x00(0x20)</u>	PINF	포트 F 데이터 입력 레지스터
0x01(0x21)	PINE	포트 E 데이터 입력 레지스터
0x02(0x22)	DDRE	포트 E 데이터 방향 레지스터
0x03(0x23)	PORTE	포트 E 데이터 출력 레지스터
0x04(0x24)	ADCL	A/D 변환 결과 하위 레지스터
0x05(0x25)	ADCH	A/D 변환 결과 상위 레지스터
0x06(0x26)	ADCSRA	A/D 변환기 제어 및 상태 레지스터
0x07(0x27)	ADMUX	A/D 변환기 멀티플렉서 레지스터
0x08(0x28)	ACSR	아날로그 비교기 제어 및 상태 레지스터
0x09(0x29)	UBR0OL	USART 0 보우 레지스터(하위 바이트)
0x0A(0x2A)	UCSR0B	USART 0 제어 및 상태 레지스터 B
0x0B(0x2B)	UCSR0A	USART 0 제어 및 상태 레지스터 A
0x0C(0x2C)	UDR0	USART 0 I/O 데이터 레지스터
0x0D(0x2D)	SPCR	SPI 제어 레지스터
0x0E(0x2E)	SPSR	SPI 상태 레지스터
0x0F(0x2F)	SPDR	SPI I/O 데이터 레지스터
0x10(0x30)	PIND	포트 D 데이터 입력 레지스터
0x11(0x31)	DDRD	포트 D 데이터 방향 레지스터
0x12(0x32)	PORTD	포트 D 데이터 출력 레지스터
0x13(0x33)	PINC	포트 C 데이터 입력 레지스터

2. Atmel사의 8bit Micro-controller

I/O Register

주소	레지스터명	기 능
0x14(0x34)	DDRC	포트 C 데이터 방향 레지스터
0x15(0x35)	PORTC	포트 C 데이터 출력 레지스터
0x16(0x36)	PINB	포트 B 데이터 입력 레지스터
0x17(0x37)	DDRB	포트 B 데이터 방향 레지스터
0x18(0x38)	PORTB	포트 B 데이터 출력 레지스터
0x19(0x39)	PINA	포트 A 데이터 입력 레지스터
0x1A(0x3A)	DDRA	포트 A 데이터 방향 레지스터
0x1B(0x3B)	PORTA	포트 A 데이터 출력 레지스터
0x1C(0x3C)	EECR	EEPROM 제어 레지스터
0x1D(0x3D)	EDDR	EEPROM 데이터 레지스터
0x1E(0x3E)	EEARL	EEPROM 주소 레지스터(하위 주소)
0x1F(0x3F)	EEARH	EEPROM 주소 레지스터(상위 주소)
0x20(0x40)	SFIOR	특수 기능 I/O 레지스터
0x21(0x41)	WDTCSR	워치독 타이머 제어 레지스터
0x22(0x42)	OCDR	On-Chip 디버그 레지스터
0x23(0x43)	OCR2	타이머/카운터 2 출력비교 레지스터
0x24(0x44)	TCNT2	타이머/카운터 2 레지스터
0x25(0x45)	TCCR2	타이머/카운터 2 제어 레지스터
0x26(0x46)	ICR1L	타이머/카운터 1 입력캡처 레지스터 하위 바이트
0x27(0x47)	ICR1H	타이머/카운터 1 입력캡처 레지스터 상위 바이트

2. Atmel사의 8bit Micro-controller

I/O

Register

주소	레지스터명	기 능
0x28(0x48)	OCR1BL	타이머/카운터 1 출력 비교 B레지스터 하위 바이트
0x29(0x49)	OCR1BH	타이머/카운터 1 출력 비교 B레지스터 상위 바이트
0x2A(0x4A)	OCR1AL	타이머/카운터 1 출력 비교 A레지스터 하위 바이트
0x2B(0x4B)	OCR1AH	타이머/카운터 1 출력 비교 A레지스터 상위 바이트
0x2C(0x4C)	TCNT1L	타이머/카운터 1 하위 바이트
0x2D(0x4D)	TCNT1H	타이머/카운터 1 상위 바이트
0x30(0x50)	ASSR	비동기 상태 레지스터
0x31(0x51)	OCR0	타이머/카운터0 출력비교 레지스터
0x32(0x52)	TCNT0	타이머/카운터0 레지스터
0x33(0x53)	TCCR0	타이머/카운터0 제어 레지스터
0x34(0x54)	MCUCSR	MCU 제어 및 상태 레지스터
0x35(0x55)	MCUCR	MCU 제어 레지스터
0x36(0x56)	TIFR	타이머/카운터 인터럽트 플래그 레지스터
0x37(0x57)	TIMSK	타이머/카운터 인터럽트 마스크 레지스터
0x38(0x58)	EIFR	확장 인터럽트 플래그 레지스터
0x39(0x59)	EIMSK	확장 인터럽트 인에이블 레지스터
0x3A(0x5A)	EICRB	외부 인터럽트 제어 레지스터 B
0x3B(0x5B)	RAMPZ	확장 인터럽트 플래그 레지스터
0x3C(0x5C)	XDIV	XTAL Divide 제어 레지스터
0x3D(0x5D)	SPL	스택 포인터 하위 바이트
0x3E(0x5E)	SPH	스택 포인터 상위 바이트
0x3F(0x5F)	SREG	상태 레지스터

2. Atmel사의 8bit Micro-controller

확장 I/O

Register

주소	레지스터명	기 능
0x60	Reserved	—
0x61	DDRF	F 포트 데이터 방향 레지스터
0x62	PORTF	F 포트 데이터 출력 레지스터
0x63	PING	G 포트 데이터 입력 레지스터
0x64	DDRG	G 포트 데이터 방향 레지스터
0x65	PORTG	G 포트 데이터 출력 레지스터
0x66	Reserved	—
0x67	Reserved	—
0x68	S PMCSR	Store 프로그램 제어 및 상태 레지스터
0x69	Reserved	—
0x6A	EICRA	외부 인터럽트 제어 레지스터 A
0x6B	Reserved	—
0x6C	XMCRB	외부 메모리 제어 레지스터 B
0x6D	XMCRA	외부 메모리 제어 레지스터 A
0x6E	Reserved	—
0x6F	OSCCAL	오실레이터 캘리브레이션 레지스터
0x70	TWBR	TWI 비트 레이트 레지스터
0x71	TWSR	TWI 상태 레지스터
0x72	TWAR	TWI 어드레스 레지스터
0x73	TWDR	TWI 데이터 레지스터
0x74	TWCR	TWI 제어 레지스터

2. Atmel사의 8bit Micro-controller

주소	레지스터명	기 능
0x75	Reserved	—
0x76	Reserved	—
0x77	Reserved	—
0x78	OCR1L	타이머/카운터 1 출력비교 C 레지스터 하위 바이트
0x79	OCR1H	타이머/카운터 1 출력비교 C 레지스터 상위 바이트
0x7A	TCCR1C	타이머/카운터 1 제어 레지스터 C
0x7B	Reserved	—
0x7C	ETIFR	확장 타이머/카운터 인터럽트 플래그 레지스터
0x7D	ETIMSK	확장 타이머/카운터 인터럽트 마스크 레지스터
0x7E	Reserved	—
0x7F	Reserved	—
0x80	ICR3L	타이머/카운터 3 입력캡처 레지스터 하위 바이트
0x81	ICR3H	타이머/카운터 3 입력캡처 레지스터 상위 바이트
0x82	OCR3CL	타이머/카운터 3 출력비교 C 레지스터 하위 바이트
0x83	OCR3CH	타이머/카운터 3 출력비교 C 레지스터 상위 바이트
0x84	OCR3BL	타이머/카운터 3 출력비교 B 레지스터 하위 바이트
0x85	OCR3BH	타이머/카운터 3 출력비교 B 레지스터 상위 바이트
0x86	OCR3AL	타이머/카운터 3 출력비교 A 레지스터 하위 바이트
0x87	OCR3AH	타이머/카운터 3 출력비교 A 레지스터 상위 바이트
0x88	TCNT3L	타이머/카운터 3 레지스터 하위 바이트
0x89	TCNT3H	타이머/카운터 3 레지스터 상위 바이트

2. Atmel사의 8bit Micro-controller

주소	레지스터명	기 능
0x8A	TCCR3B	타이머/카운터 3 제어 레지스터 B
0x8B	TCCR3A	타이머/카운터 3 제어 레지스터 A
0x8C	TCCR3C	타이머/카운터 3 제어 레지스터 C
0x8D	Reserved	—
0x8E	Reserved	—
0x8F	Reserved	—
0x90	UBRR0H	USART 0 보우 레이트 레지스터(상위 바이트)
0x91	Reserved	—
0x92	Reserved	—
0x93	Reserved	—
0x94	Reserved	—
0x95	UCSR0C	USART 0 제어 및 상태 레지스터 C
0x96	Reserved	—
0x97	Reserved	—
0x98	UBRR1H	USART 1 보우 레이트 레지스터(상위 바이트)
0x99	UBRR1L	USART 1 보우 레이트 레지스터(하위 바이트)
0x9A	UCSR1B	USART 1 제어 및 상태 레지스터 B
0x9B	UCSR1A	USART 1 제어 및 상태 레지스터 A
0x9C	UDR1	USART 1 데이터 레지스터
0x9D	UCSR1C	USART 1 제어 및 상태 레지스터 C
0x9E~0xFF	Reserved	—

2. Atmel사의 8bit Micro-controller

External Memory Interface

: 외부 메모리 영역 0x1100~0xFFFF 번지, 60KB에 SRAM, Flash, LCD, A/D, D/A 등 사용.

: Register (MCUCR, XMCRA, XMCRB) → 실제 위치하는 주소는?

: 주요 특징

① 4개의 wait (delay의미) 설정 가능

② 외부 메모리 영역을 2개의 섹터로

나눌 수 있고, 독립적인 wait 설정 가능

③ 상위 어드레스는 필요한 비트만 동작 가능

④ 전류소비를 최소화하기 위해 data line에 Bus keeper 기능 설정 가능

: Interface signal

① AD0~AD7 : 어드레스 하위 및 데이터 신호

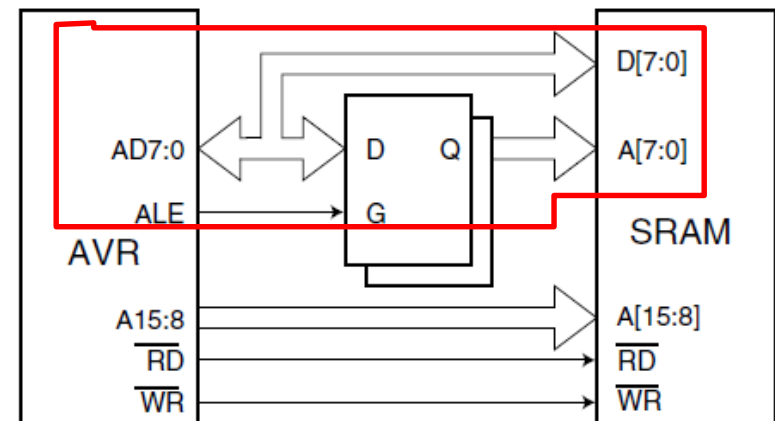
② A8~A15 : 어드레스 상위 신호

③ ALE : Address latch enable 신호

(address하위 8bit이 data bit와 공용이기 때문)

④ RD: 외부 메모리 읽기 신호

⑤ WR : 외부 메모리 쓰기 신호



2. Atmel사의 8bit Micro-controller

External Memory Interface : ALE (Address latch enable) 동작(D- FlipFlop)

: AD0~AD7이 address bus 일 때

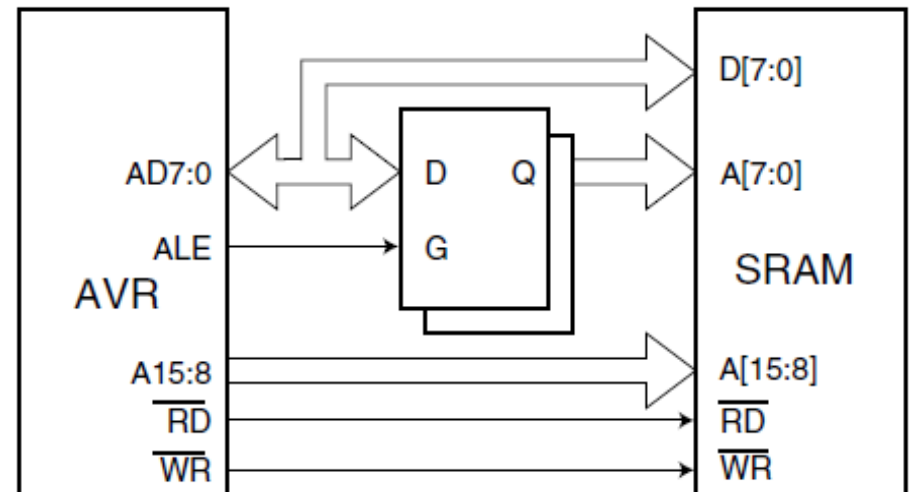
→ ALE='H'

AD0~AD7이 latch IC의 출력 Q를 통해 SRAM의 어드레스 신호인 A[7:0](A0~A7)에 연결됨

: AVR의 AD0~AD7이 data bus일 때

→ ALE='L' AD0~AD7이 latch(예, 74HCT573) datasheet

SRAM의 데이터 핀인 D[7:0](D0~D7)에 연결됨



2. Atmel사의 8bit Micro-controller

External Memory Interface

: **MCUCR** Register (MCU Control Register)

Bit	7	6	5	4	3	2	1	0
	SRE	SRW10	SE	SM1	SM0	SM2	IVSEL	IVCE
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

[7] SRE(External SRAM/XMEM Enable) : 외부 메모리 사용 Enable

→ if **SRE=1**, AD0~AD7, A8~A15, ALE, /RD, /WR enable됨 (**alternate pin**에서)

[6] **wait-state 선택 bit**

→ **XMCRA**의 [3,2,1]bit와 조합으로 **wait cycle** 설정

[5] Sleep Enable bit

[4,3,2] **Sleep mode**

[1] **Interrupt Vector Start Address** 설정

0 : Vectors는 flash memory 시작점에 위치

1 : Vectors는 boot loader in flash

[0] Interrupt Vector Select

1 : IVSEL bit의 변경을 하기 위해서

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby ⁽¹⁾
1	1	1	Extended Standby ⁽¹⁾

2. Atmel사의 8bit Micro-controller

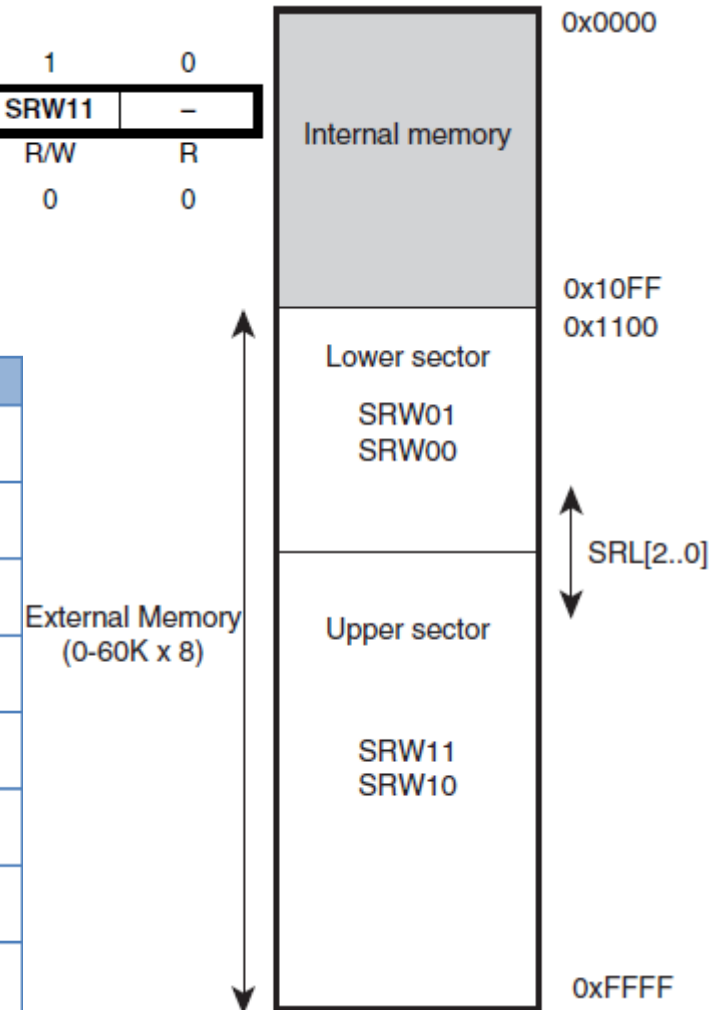
External Memory Interface

: **XMCRA** Register (eXternal Memory Control Register A)

Bit	7	6	5	4	3	2	1	0
	–	SRL2	SRL1	SRL0	SRW01	SRW00	SRW11	–
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R
Initial Value	0	0	0	0	0	0	0	0

[6,5,4] 외부 메모리 영역을 2개의 섹터로 분할

SRL2	SRL1	SRL0	Sector Limits
0	0	0	Lower sector = N/A Upper sector = 0x1100 - 0xFFFF
0	0	1	Lower sector = 0x1100 - 0x1FFF Upper sector = 0x2000 - 0xFFFF
0	1	0	Lower sector = 0x1100 - 0x3FFF Upper sector = 0x4000 - 0xFFFF
0	1	1	Lower sector = 0x1100 - 0x5FFF Upper sector = 0x6000 - 0xFFFF
1	0	0	Lower sector = 0x1100 - 0x7FFF Upper sector = 0x8000 - 0xFFFF
1	0	1	Lower sector = 0x1100 - 0x9FFF Upper sector = 0xA000 - 0xFFFF
1	1	0	Lower sector = 0x1100 - 0xBFFF Upper sector = 0xC000 - 0xFFFF
1	1	1	Lower sector = 0x1100 - 0xDFFF Upper sector = 0xE000 - 0xFFFF



2. Atmel사의 8bit Micro-controller

External Memory Interface

: XMCRA Register (eXternal Memory Control Register A)

Bit	7	6	5	4	3	2	1	0
	–	SRL2	SRL1	SRL0	SRW01	SRW00	SRW11	–
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R
Initial Value	0	0	0	0	0	0	0	0

[3,2,1] 외부 메모리 영역을 2개의 wait cycle 수 설정

: SRW01, SRW00은 Lower 섹터의 wait cycle 수 설정 비트

: SRW11, SRW10은 Upper 섹터의 wait cycle 수 설정 비트

SRWn1	SRWn0	Wait States
0	0	No wait-states
0	1	Wait one cycle during read/write strobe
1	0	Wait two cycles during read/write strobe
1	1	Wait two cycles during read/write and wait one cycle before driving out new address

2. Atmel사의 8bit Micro-controller

External Memory Interface

: **XMCRB** Register (eXternal Memory Control Register B)

Bit	7	6	5	4	3	2	1	0
	XMBK	-	-	-	-	XMM2	XMM1	XMM0
Read/Write	R/W	R	R	R	R	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

[7] XMBK (External Memory **Bus-keeper Enable**) = 1

: AD7~AD0 신호가

3-state로 되어야 하는 순간에 이전의 값(0 or 1)으로 출력 하도록 설정

[2,1,0] XMM

: 외부 데이터 메모리 어드레스의 **상위 바이트 중에**

어디까지 어드레스 기능으로 사용할지 설정

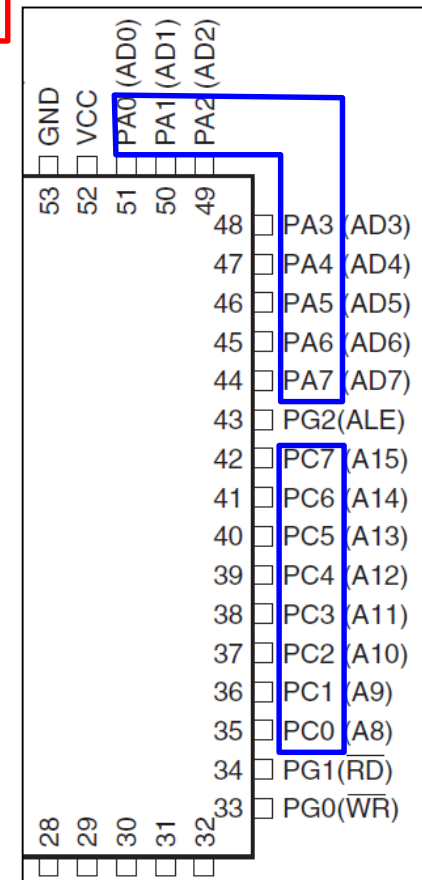
WHY?

: **address line 2^{16} → 메모리의 크기를 결정함.**

메모리 64KB - **address line 2^{16} - 1111 1111 1111 1111**

메모리 32KB - **address line 2^{15} - 111 1111 1111 1111**

메모리 4KB - **address line 2^{12} - 1111 1111 1111**



2. Atmel사의 8bit Micro-controller

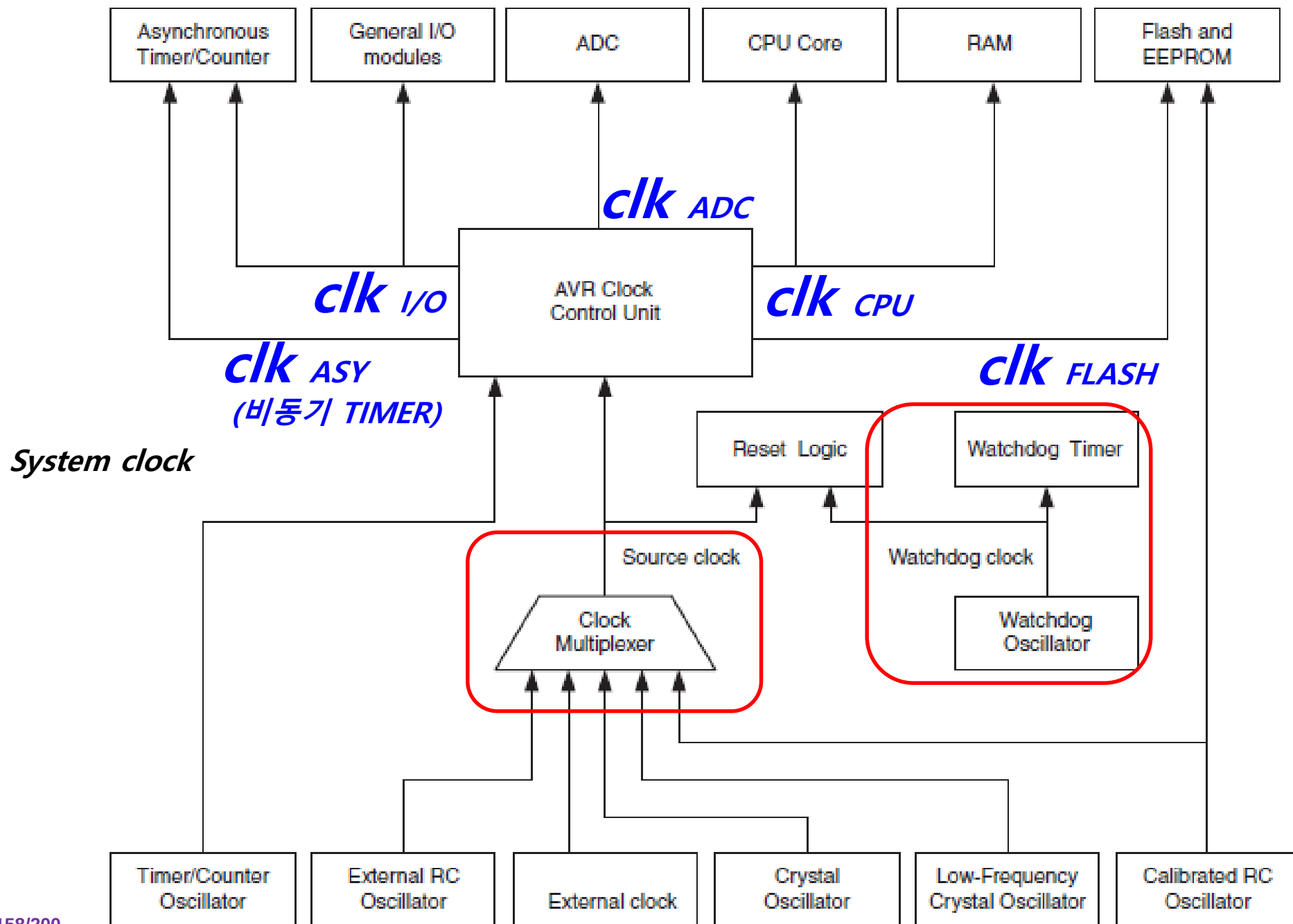
External Memory Interface

: XMCRB Register (eXternal Memory Control Register B)

Bit	7	6	5	4	3	2	1	0
	XMBK	-	-	-	-	XMM2	XMM1	XMM0
Read/Write	R/W	R	R	R	R	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

XMM2	XMM1	XMM0	외부 메모리 어드레스로 사용하는 비트 수 (어드레스 신호)	I/O포트로 사용할 수 있는 비트
0	0	0	8(A8-A15)	없음
0	0	1	7(A8-A14)	PC7
0	1	0	6(A8-A13)	PC6-PC7
0	1	1	5(A8-A12)	PC5-PC7
1	0	0	4(A8-A11)	PC4-PC7
1	0	1	3(A8-A10)	PC3-PC7
1	1	0	2(A8-A9)	PC2-PC7
1	1	1	0(없음)	PC0-PC7

Clock Distribution



Main processor

Memory 선정

Clock 선정

전원 부품 선정



[ENGLISH](#) [SITEMAP](#) [CONTACT US](#) [ADMIN](#)

[회사소개](#)

[사업영역](#)

[제품소개](#)

[투자정보](#)

[채용정보](#)

[고객지원](#)

[Duplexer](#) / [Filter](#) / [Filter Module](#) / [Isolator](#) / [Dielectric Chip Antenna](#) / [Patch Antenna](#) / [Intenna](#) / [NFC Antenna](#) /

[External Antenna](#) / [Crystal Device](#) / [RF Module](#) / [Camera Module](#) / [Optical Mouse](#) / [MicroPhone](#) / [Ceramic Capacitors](#) /

Click

[Home/제품소개/Crystal Device](#)

제품 소개

PRODUCT

[Duplexer](#)

[Filter](#)

[Filter Module](#)

[Isolator](#)

[Dielectric Chip Antenna](#)

[Patch Antenna](#)

[Intenna](#)

[NFC Antenna](#)

[External Antenna](#)

[Crystal Device](#)

[RF Module](#)

[Camera Module](#)

[Optical Mouse](#)

[MicroPhone](#)

[Ceramic Capacitors](#)

Crystal Device

경쟁력있고 빠른 제품으로 고객과 함께 발전하는 기업 파트론입니다.

[카테고리(이미지)를 클릭하시면 제품에 대한 자세한 SPEC 정보를 확인하실 수 있습니다.]

“파트론”은

각 기지간의 유기적이고 효율적인 협력체계의 구축으로 기술과 기능이 앞선 제품을 경쟁력있고 빠르게 공급하여 고객과 함께 발전할 것을 약속드립니다.

Click

Crystal Unit

Click

Crystal Oscillators

Click

TCXO

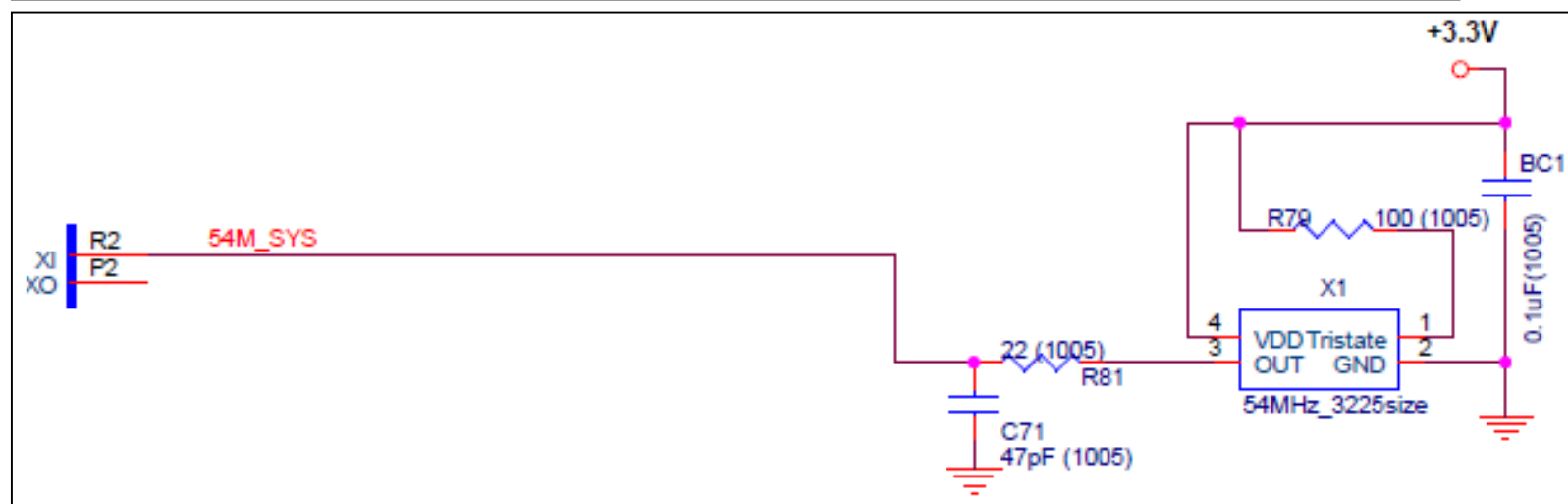
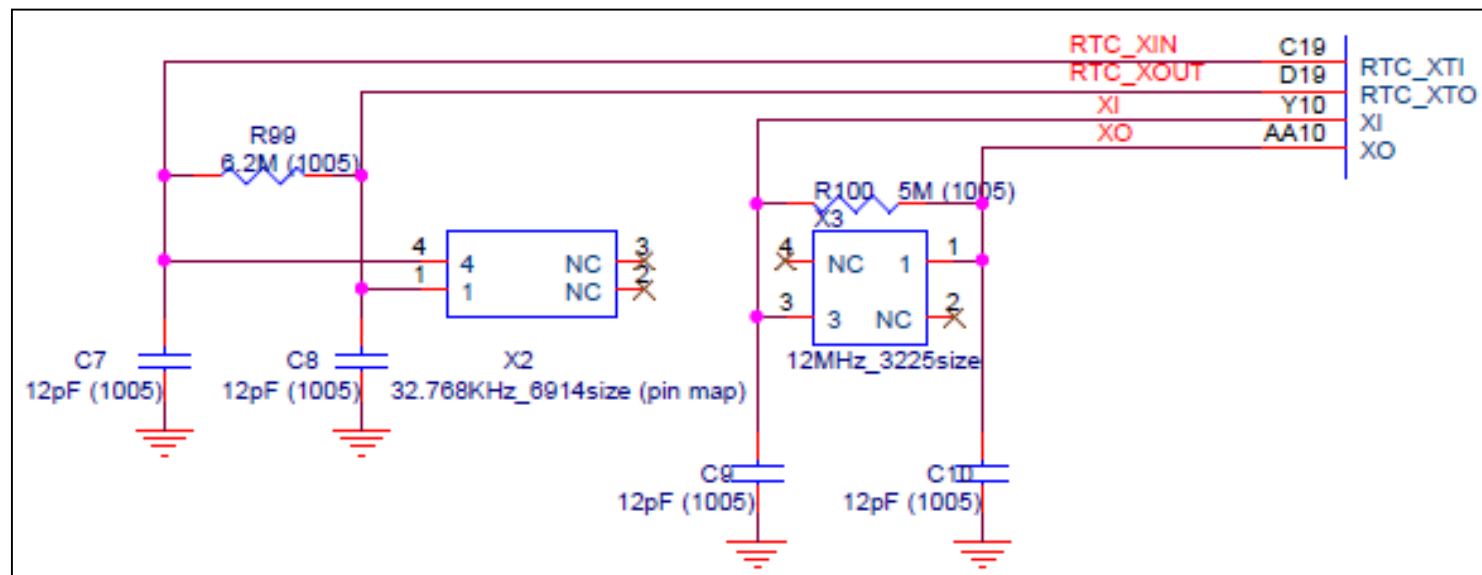
Click

VCXO



Packing





2. Atmel사의 8bit Micro-controller

Reset

- : MCU의 모든 I/O 레지스터를 초기화하고 **Reset Vector** 로부터 프로그램이 시작하게 됨
- : MCU가 **Reset** 후 일정한 **시간 지연이 이루어진 후에** 시스템이 재 시작됨
 - ➔ Vcc의 일정한 레벨로 도달하는 시간을 확보하기 위해 Watchdog 타이머에 의해 시간 delay.
 - ➔ 시스템 clock 의 안정된 발진을 위한 시간 delay
 - ➔ 시간 delay는 퓨즈 비트의 SUT1~0, CKSEL0에 의해 설정

Reset 형태

- : 파워-온 **Reset** → Vcc가 VPOT 이하일 때 MCU **Reset**
- : 외부 **Reset** → "RESET"핀에 최소한 $1.5\mu s$ 이상의 Low 레벨 펄스가 입력되면 MCU **Reset**
- : Watchdog **Reset** → Watchdog 타이머에서 지정된 주기 이상에서 MCU **Reset**
- : Brown-out **Reset** → Vcc가 VBOT 이하로 떨어져 $2\mu s$ 이상 지속될 때 MCU **Reset**
- : JTAG **Reset** → JTAG 시스템에서 **Reset** 레지스터에 1을 저장시키고
관련 하드웨어가 동작되어 MCU **Reset**

2. Atmel사의 8bit Micro-controller

Reset state

: Reset 이 발생한 원인을 MCUCSR 레지스터에서 확인.

: MCU Control and Status Register

Bit	7	6	5	4	3	2	1	0	
	JTD	–	–	JTRF	WDRF	BORF	EXTRF	PORF	MCUCSR
Read/Write	R/W	R	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	See Bit Description					

[4] JTRF(JTAG Reset Flag) : RESET=1

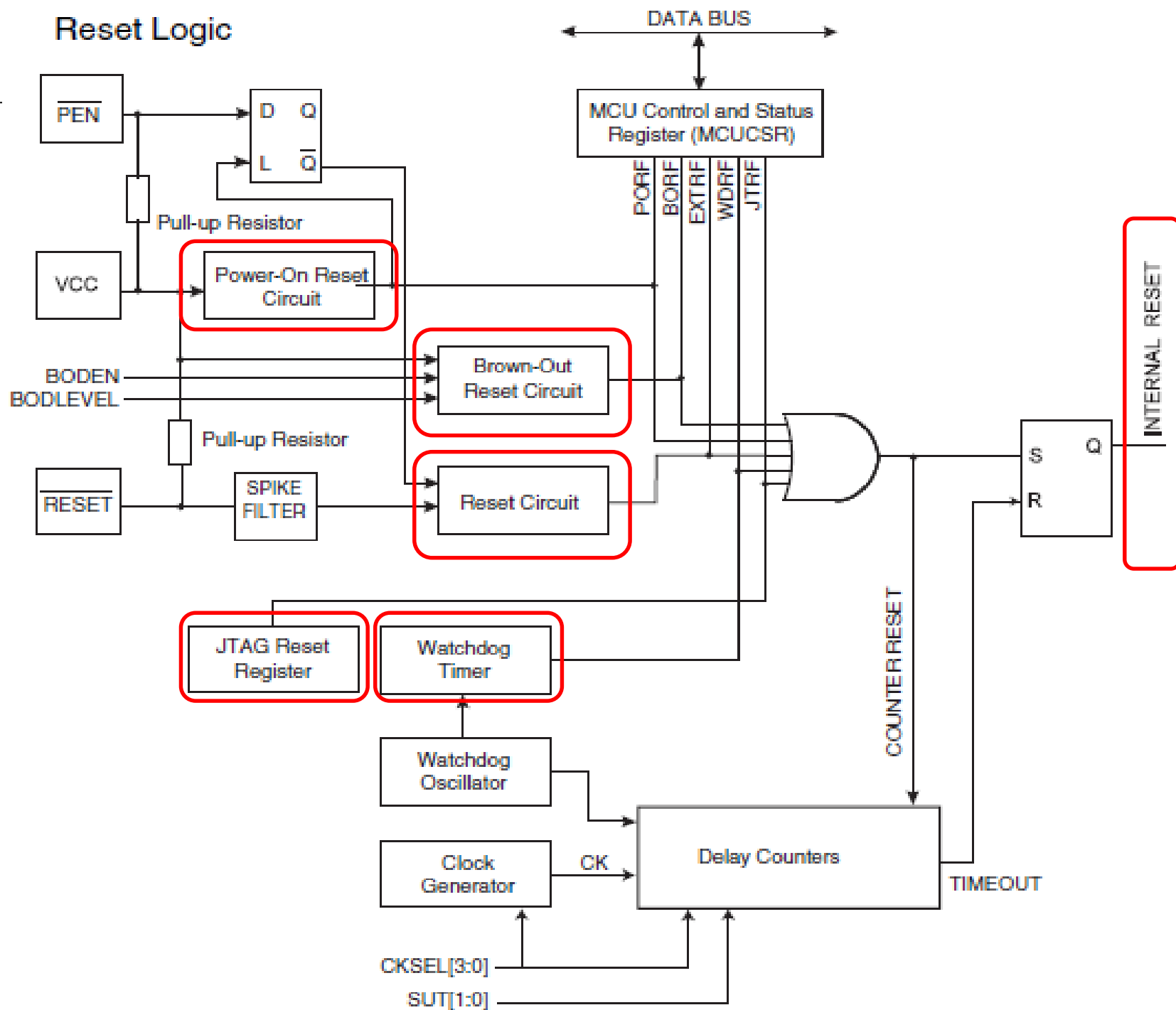
[3] WDRF(WatchDog Reset Flag)

[2] BORF(Brown-Out Reset Flag)

[1] EXTRF(EXTernal Reset Flag)

[0] PORF(Power-On Reset Flag)

Reset



2. Atmel사의 8bit Micro-controller

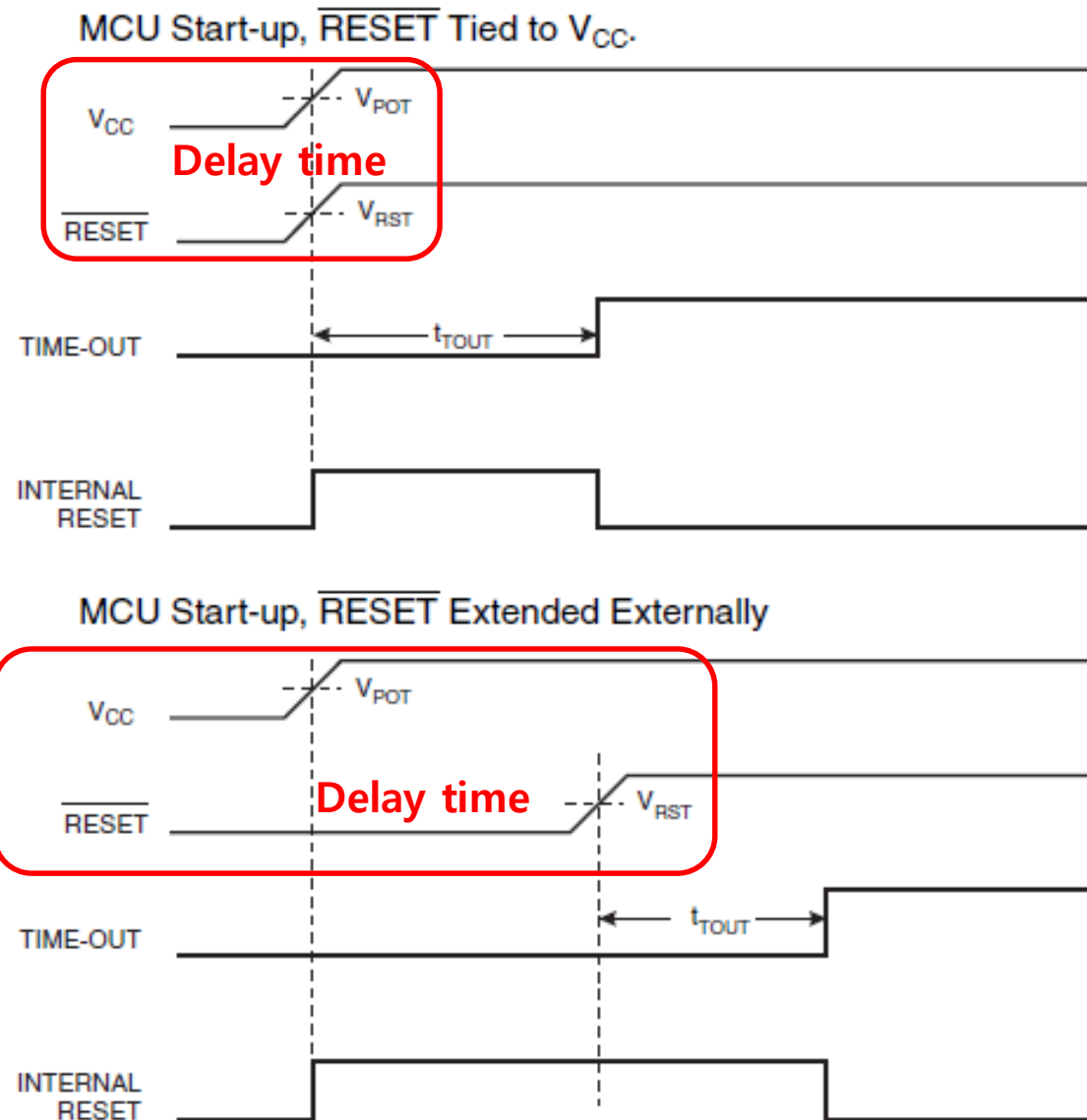
Reset

Reset Characteristics

Symbol	Parameter	Condition	Min	Typ	Max	Units
V_{POT}	Power-on Reset Threshold Voltage (rising)			1.4	2.3	V
	Power-on Reset Threshold Voltage (falling) ⁽¹⁾			1.3	2.3	
V_{RST}	$\overline{RESET}$ Pin Threshold Voltage		0.2 V_{CC}		0.85 V_{CC}	
t_{RST}	Pulse width on $\overline{RESET}$ Pin		1.5			μs
V_{BOT}	Brown-out Reset Threshold Voltage ⁽²⁾	BODLEVEL = 1	2.4	2.6	2.9	V
		BODLEVEL = 0	3.7	4.0	4.5	
t_{BOD}	Minimum low voltage period for Brown-out Detection	BODLEVEL = 1		2		μs
		BODLEVEL = 0		2		
V_{HYST}	Brown-out Detector hysteresis			100		mV

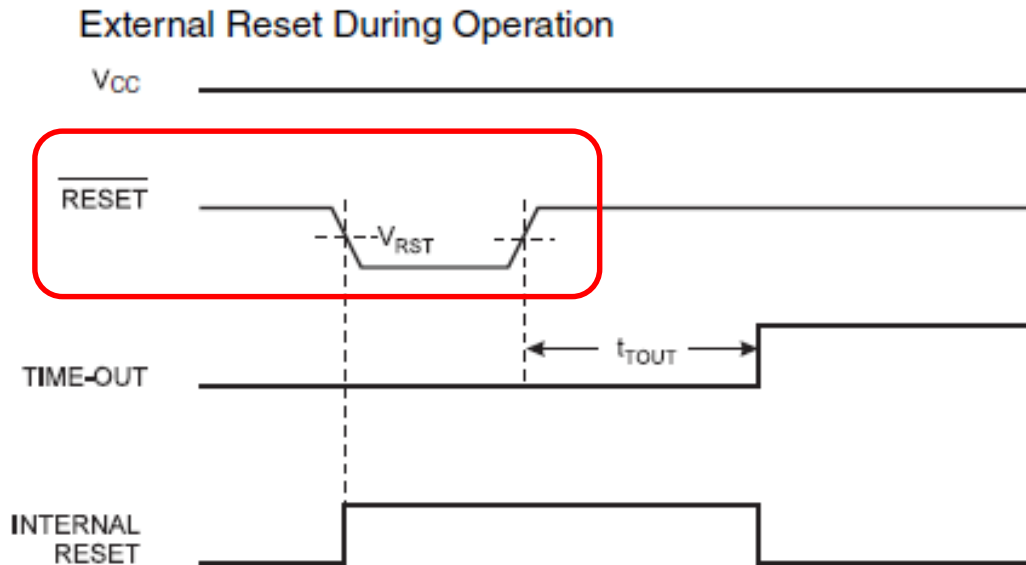
2. Atmel사의 8bit Micro-controller

Power on Reset



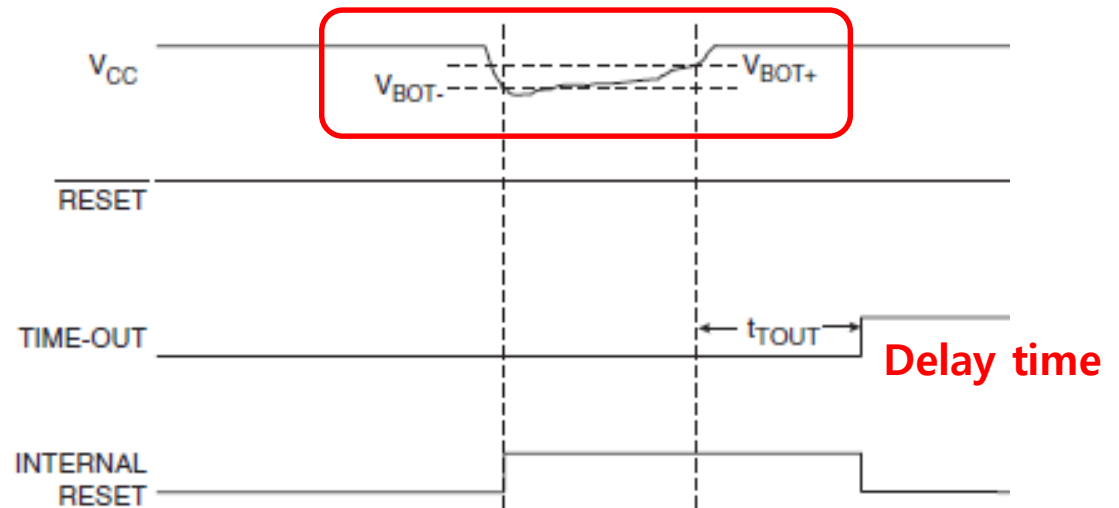
2. Atmel사의 8bit Micro-controller

External Reset



Brown out detection

Brown-out Reset During Operation



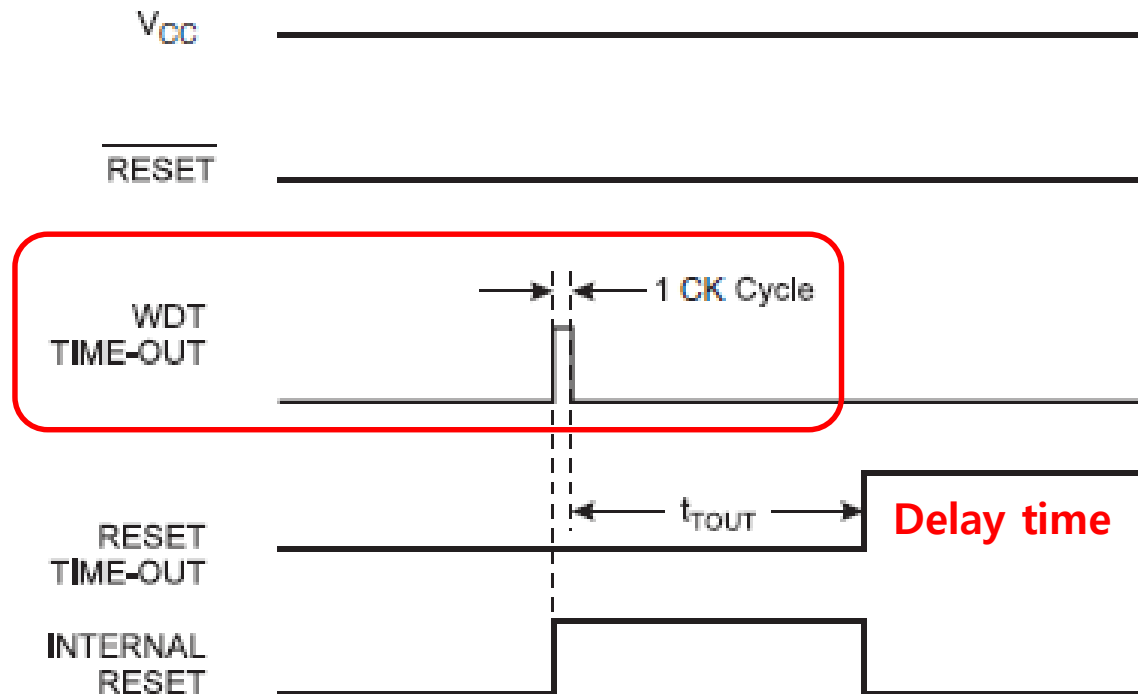
2. Atmel사의 8bit Micro-controller

Watchdog Reset

: Watchdog 타이머 인터럽트가 발생할 때 아래와 같이 1clock 리셋 펄스가 발생

: MCU가 무한 루프에 빠지거나 비정상적인 동작을 할 경우,
시스템을 **Reset** 시켜 정상 동작을 되돌리는 기능

: MCU가 무한 루프에 빠지거나 비정상적인 동작을 하는지 감시하는 타이머



About This Course

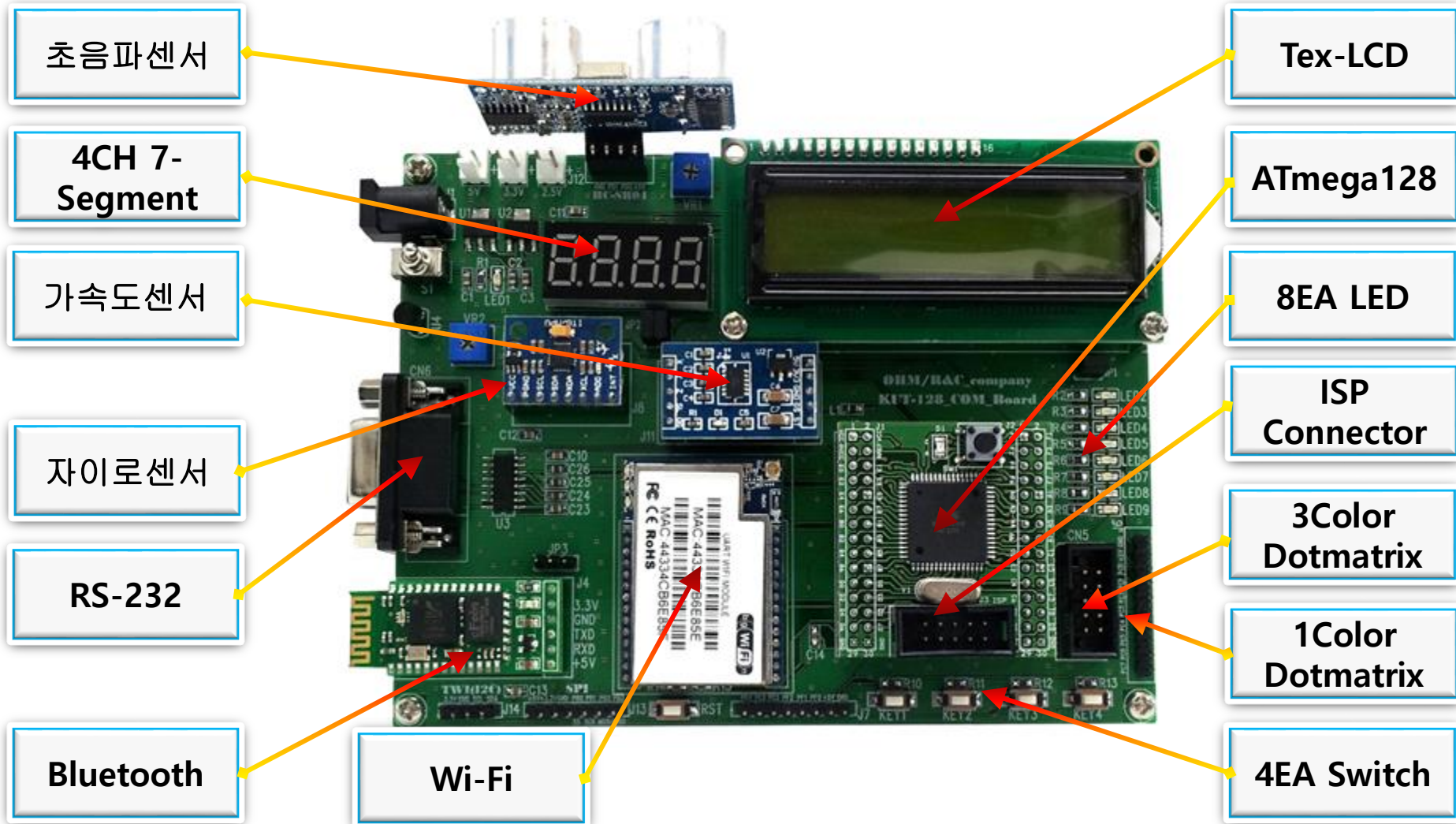
1. Micro Processor 원리
2. Atmel사의 8bit Micro-controller
3. KUT0128 Evaluation Board 기능과 특징
4. IO Port 제어
5. External Interrupt 제어
6. Timer counter 제어
7. UART 제어
8. AD Converter 제어
9. Comparator 제어
10. EEPROM 제어 (IIC, Parallel method)
11. SPI 제어

3. KUT0128 BORAD 기능과 특징



3. KUT0128 BORAD 기능과 특징

KUT-128_Com 실험 키트의 기능(1)



3. KUT0128 BORAD 기능과 특징

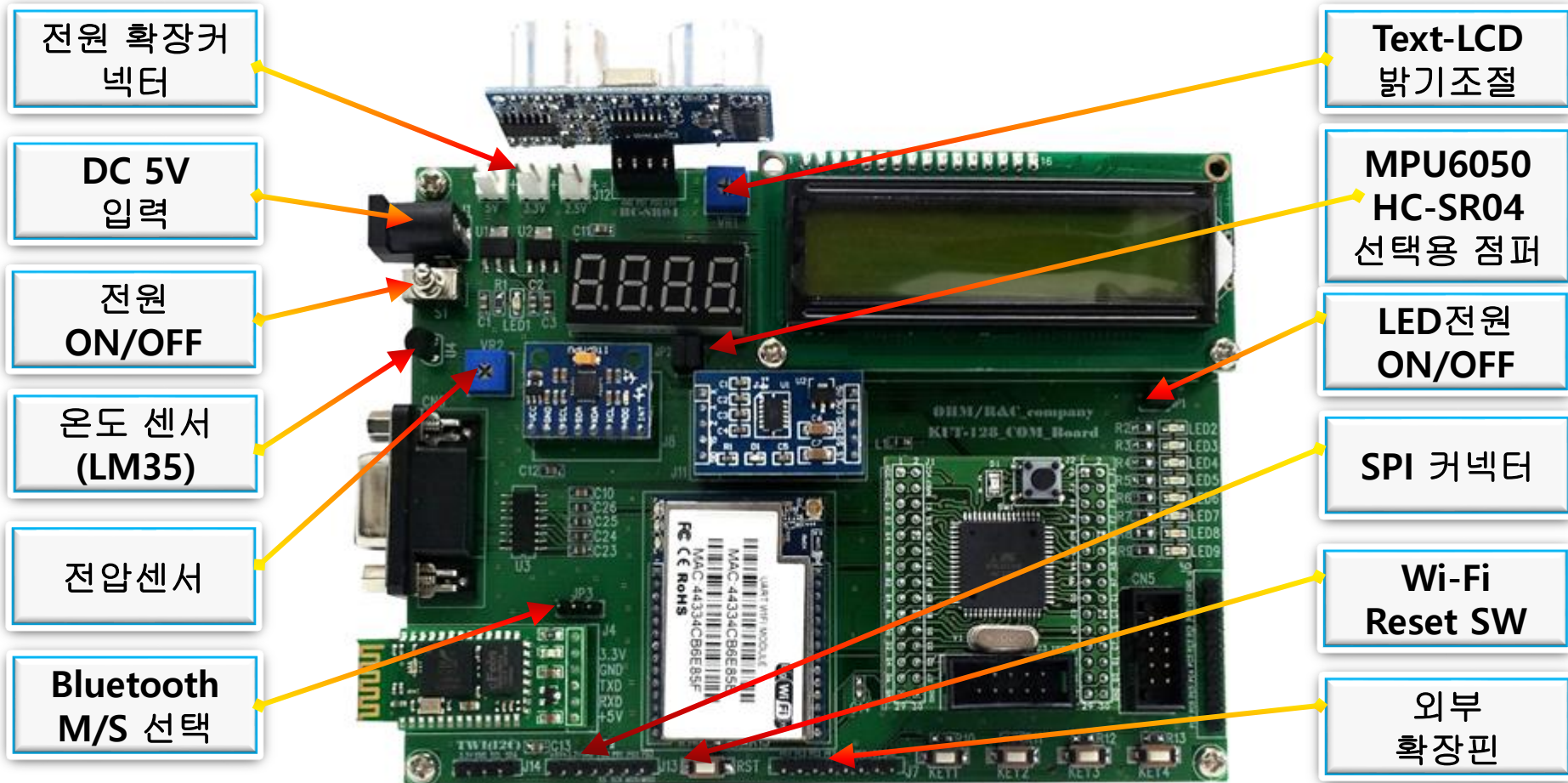
순번	기능	설명	PORT
1	초음파센서 HC-SR04	초음파를 발생하여 거리나 사물유무를 계산하는데 사용 : Ultrasonic (4m ~ 2cm)	PD0,PD1
2	4Digit 7-Segment	4개의 숫자표시가 가능	PD4~7,PB4~7 PG0~3
3	아날로그 가속도센서	X,Y,Z 3축의 아날로그 출력 : Accelerometer (MMA7361)	PF3~5
4	Digital 자이로센서	I2C통신이 가능, 가속도센서+자이로센서+온도측정가능 : Gyroscope + Accelerometer (MPU-6050)	PD0,PD1
5	Wi-Fi Module	Wi-Fi통신용 모듈(USART1) ※주의 : Bluetooth모듈과 동시 사용은 안되며 RS-232와는 가능함	PD2,PD3
6	RS-232용 커넥터	Max232를 사용한 RS-232통신 (USART0)	
7	Bluetooth Module	블루투스 통신용 모듈(USART1) ※주의 : Wi-Fi모듈과 동시 사용은 안되며 RS-232와는 가능함	PD2,PD3
8	2X16 Text LCD	2줄에 총 32개의영문 표시가능	PA0~2,PA4~7

3. KUT0128 BORAD 기능과 특징

순번	기 능	설 명	PORT
9	8bit Micro controller	ATmega128a-au	
10	LED Part	8개의 LED제어	PC0~7
11	4Ch Switch	4개의 Tact 스위치로 외부 입력 또는 외부인터럽트로 사용	PE4~7
12	3Color Dotmatrix	LED Part와 1Color Dotmatrix와 같이 사용 할 수 없음 (플랫케이블사용)	PC0~7
13	1Color Dotmatrix	LED Part와 3Color Dotmatrix와 같이 사용 할 수 없음 (점퍼케이블사용)	PC0~7

3. KUT0128 BOARD 기능과 특징

KUT-128_Com 실험 키트의 기능(2)



3. KUT0128 BORAD 기능과 특징

KUT-128_Com 실험 키트의 기능(2)

순번	기 능	설 명	기 타
1	전압 외부 출력	5V, 3.3V, 2.5V 외부전압	
2	DC 5V입력	DC 5V 아답터 Jack	
3	보드전원 ON/OFF	전원 On/Off	
4	온도센서 LM35	아날로그 온도센서	PF7
5	전압센서 VR(10K)	0~5V 전압 가변센서	PF6
6	블루투스 M/S 선택	블루투스모듈의 마스터 슬레이브 선택	점퍼or SW사용
7	TWI 커넥터	외부 I2C모듈 또는 장비 통신용 커넥터	

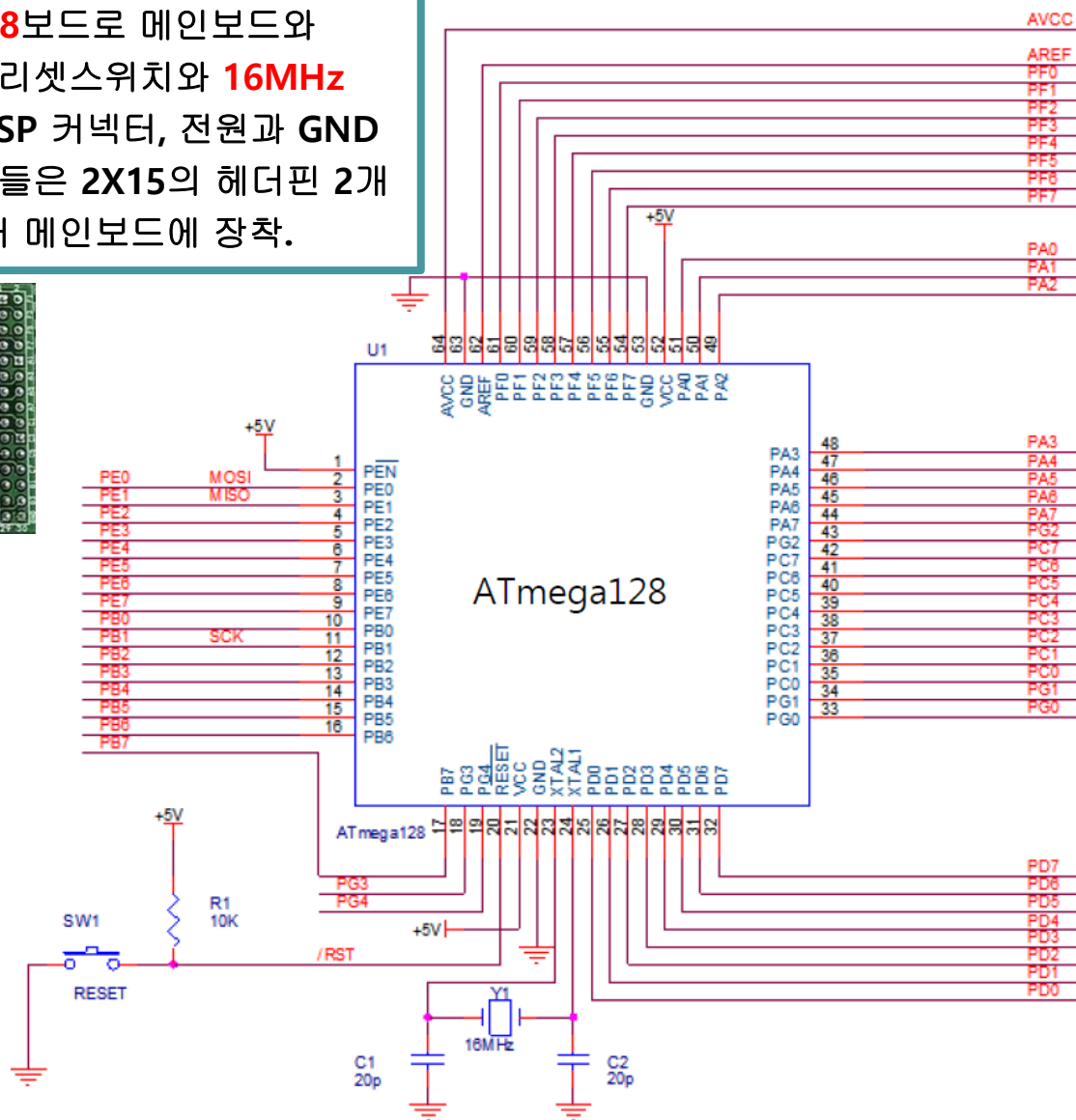
3. KUT0128 BORAD 기능과 특징

KUT-128_Com 실험 키트의 기능(2)

순번	기 능	설 명	기 타
8	Text-LCD 밝기 조절	Text-LCD 문자 밝기 조절용 가변저항	
9	MPU6050 or HC-SR04 선택용 점퍼	Close(점퍼연결) : MPU-6050 사용 Open(점퍼 제거) : TWI(I2C)확장 커넥터 또는 HC-SR04 사용	MPU-6050 : Gyroscope + Accelerometer HC-SR04 : Ultrasonic (PD0, PD1 공용 사용)
10	LED 전원 ON/OFF	Close(점퍼 연결) : LED부 사용 Open(점퍼 제거) : Dotmatrix 사용시	
11	SPI 커넥터	외부 SPI모듈 또는 장비 통신용 커넥터	PB0~PB3
12	Wi-Fi Reset 스위치	Wi-Fi 모듈의 리셋 또는 공장초기화용 스위치	
13	포트 확장	여분의 확장포트로 조이스틱이나 다양한 센서연결에 사용	PF0,1,2,PE2,3,PG4

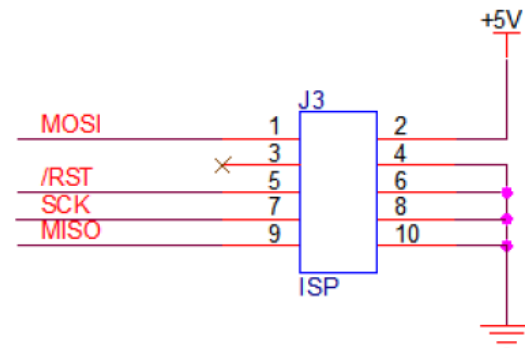
3. KUT0128 BORAD 기능과 특징

ATmega128보드로 메인보드와
탈착 가능, 리셋스위치와 **16MHz**
크리스탈, **ISP** 커넥터, 전원과 **GND**
가 연결. 핀들은 **2X15**의 헤더핀 2개
에 연결되어 메인보드에 장착.

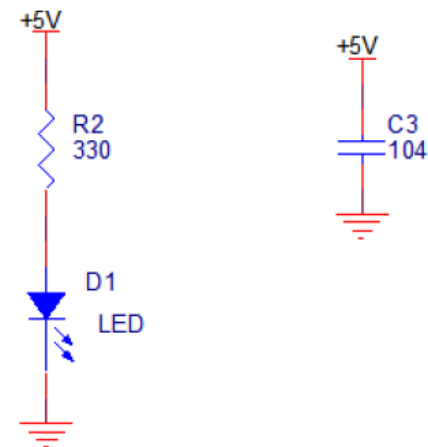


ATmega128 MCU

ISP PART



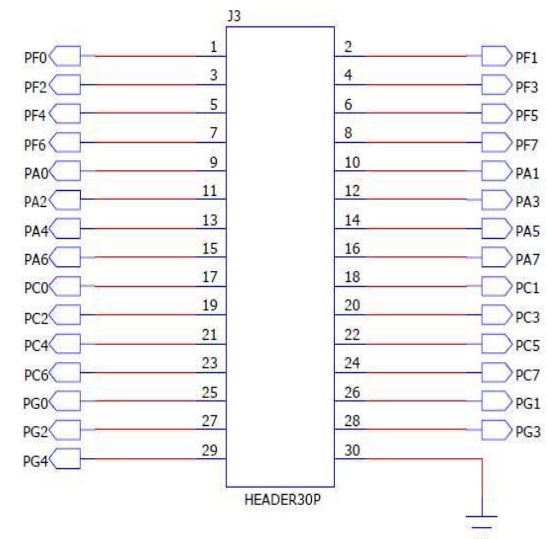
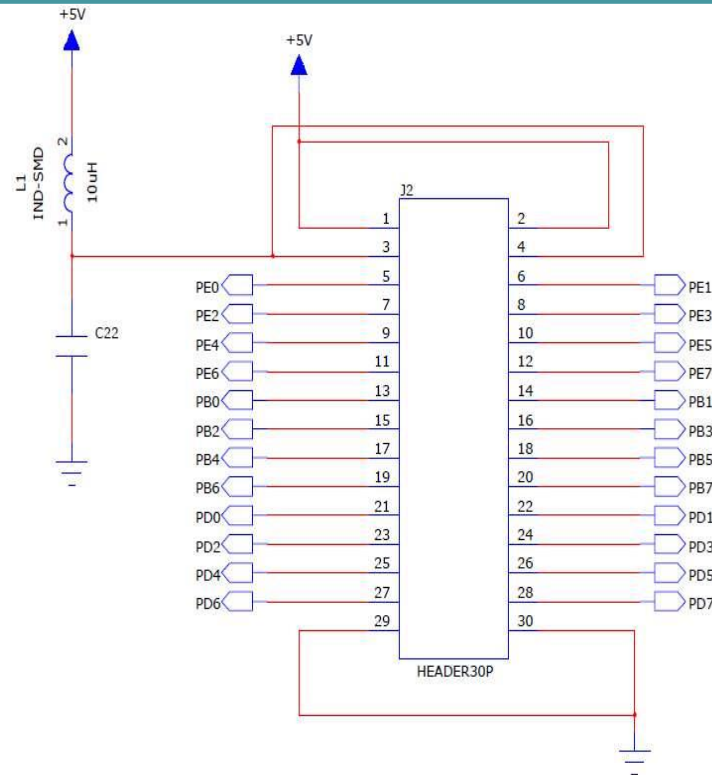
전원LED 및 바이패스 콘덴서 PART



3. KUT0128 BORAD 기능과 특징

ATmega128 모듈 연결 헤더소켓

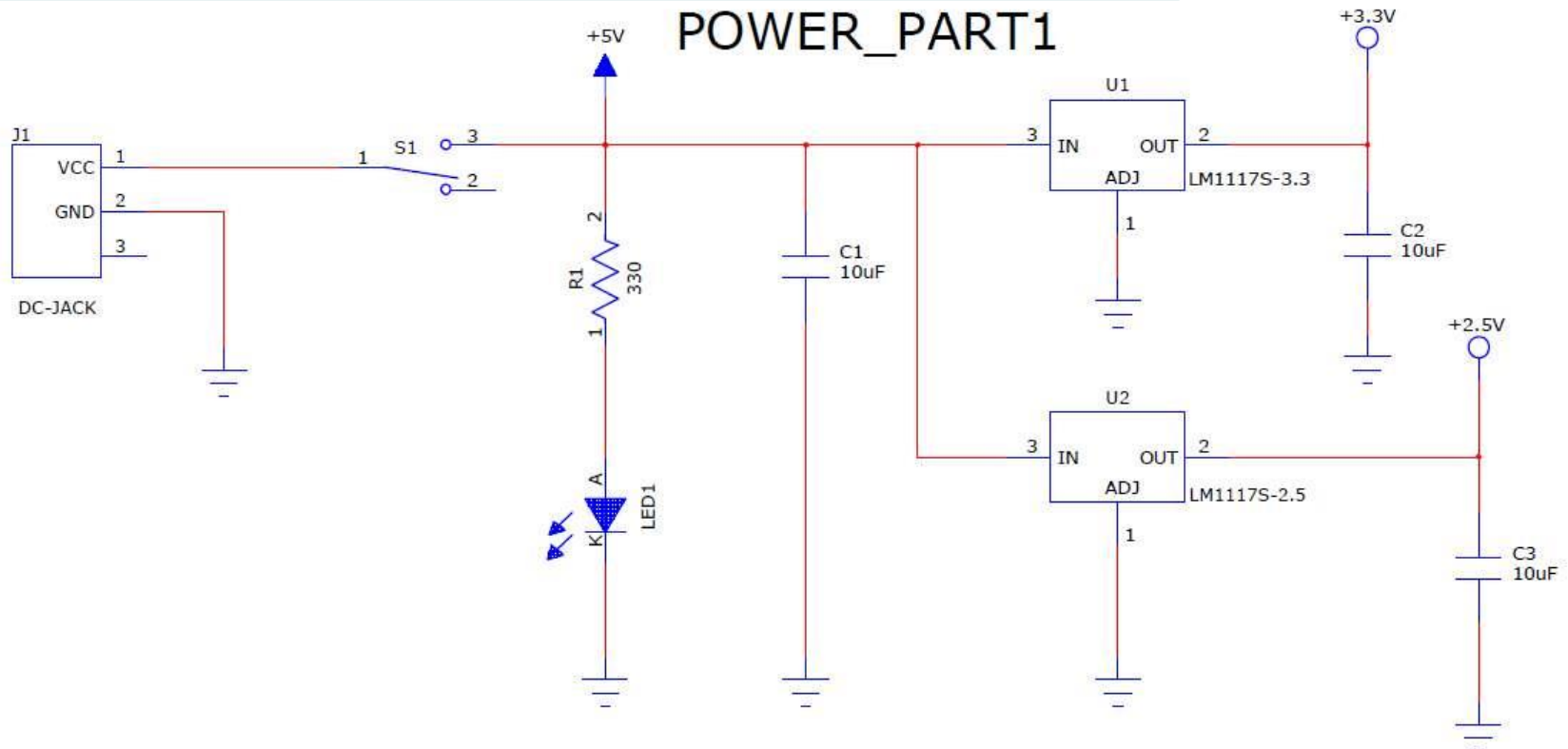
ATmega128 MCU보드가 장착되는 부분으로 2개의 2X15 헤더 소켓(30P)으로 메인보드를 통해 MCU로 전원이 공급되는 핀과 ATmega128에서 내려오는 신호가 메인보드로 공급될 수 있게 한다.



3. KUT0128 BORAD 기능과 특징

Power Part(1)

DC 5V 아답터를 연결하는 DC-Jack과 전원 On/Off용 스위치로 구성, 전원이 들어오면 LED에 불이 들어오고 2개의 레귤레이터 LM1117s-3.3과 LM1117s-2.5v를 거쳐 3.3v와 2.5v를 공급한다.



3. KUT0128 BORAD 기능과 특징

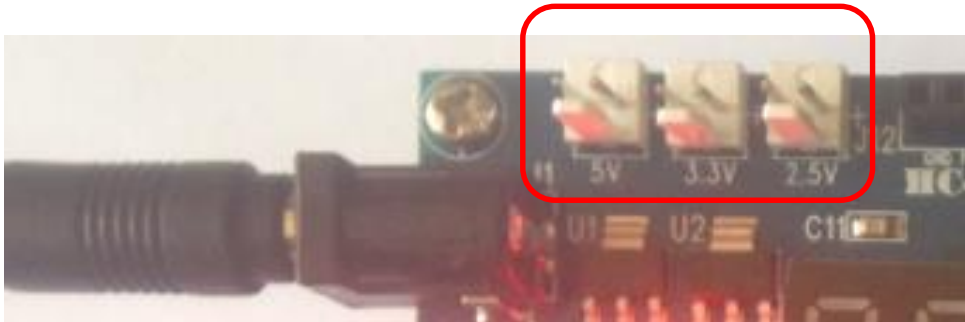
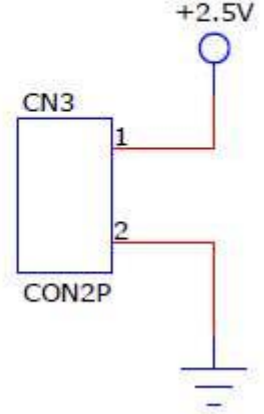
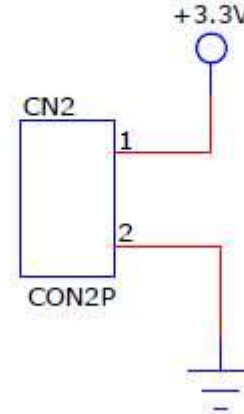
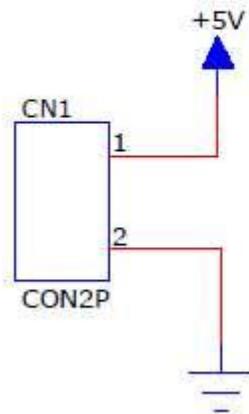
DC 5V 아답터를 연결하는 DC-Jack과 전원 On/Off용 스위치 **on**



3. KUT0128 BORAD 기능과 특징

Power Part(2)-외부전원

내부 전원을 외부로 공급하기
위한 커넥터로 5v, 3.3v, 2.5v
3가지 전압을 사용 할 수 있으며
레귤레이터 **lm1117s-2.5**를
lm1117s-1.8으로 변경하여
1.8v를 공급할 수 있다.

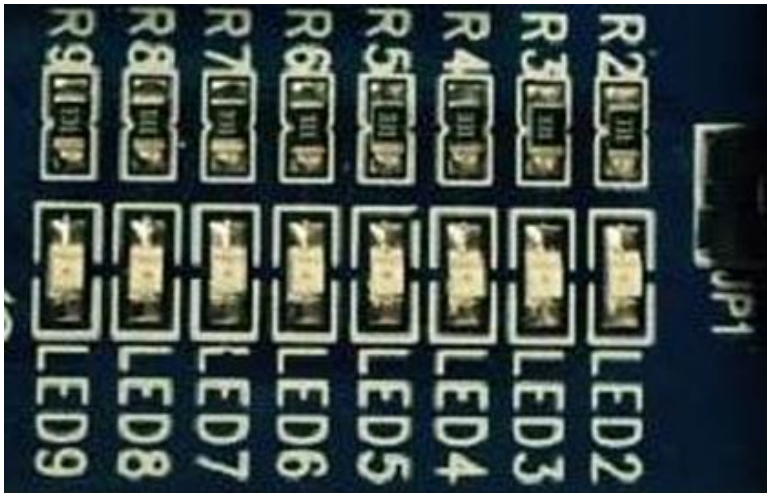


3. KUT0128 BORAD 기능과 특징

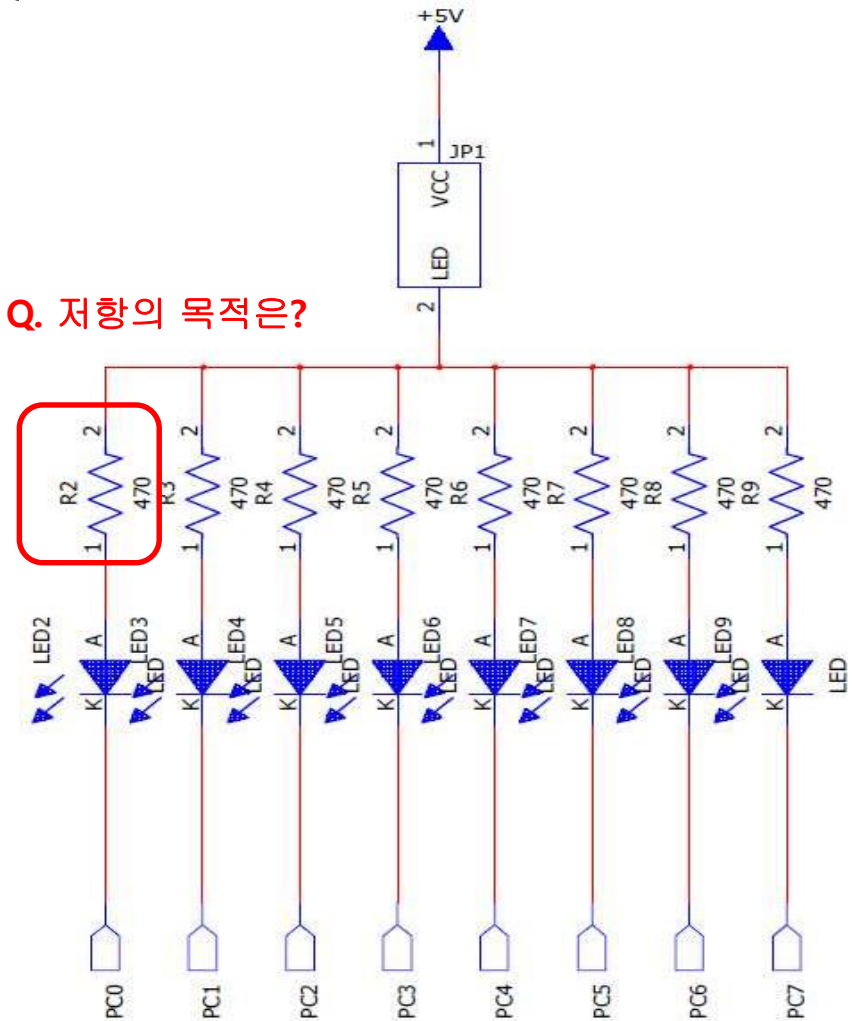
LED Part

LED 8개를 PORTC에 연결하여 PORT 제어를 실습할 수 있게 디자인.
각각의 LED를 제어.

PORT C Bit0~7중 0을 출력하면
On, 1을 출력하면 Off가 된다.



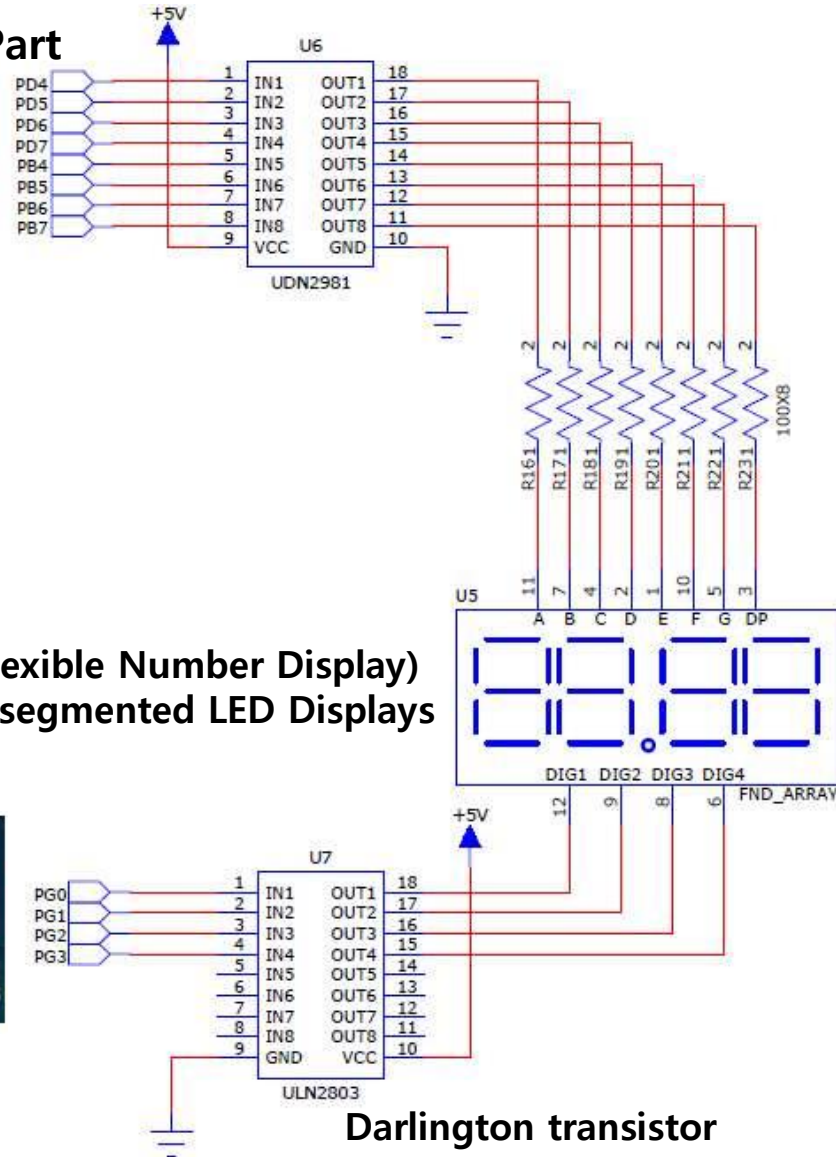
Q. 저항의 목적은?



3. KUT0128 BORAD 기능과 특징

7-SEGMENT Part

4채널의 7segment를 제어하기 위해
ULN2803 NPN TR어레이를 사용하여
숫자를 표시하고 싶은 채널을 선택할 수 있다
표시할 데이터는 포트 **UDN2981**를 제어해
PD4~PD7과 **PB4~PB7**포트에 연결되어
있는 7-Segment를 ON/OFF한다.



FND (Flexible Number Display)
: Multi-segmented LED Displays



UDN2981

ULN2803

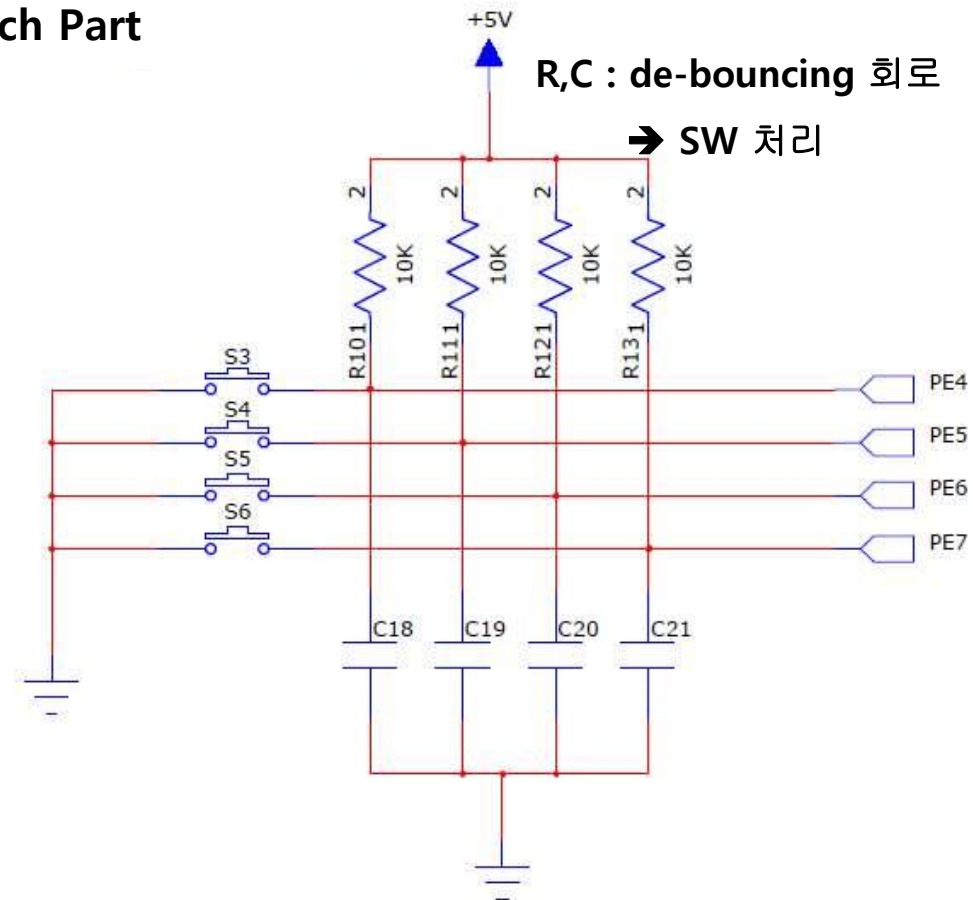
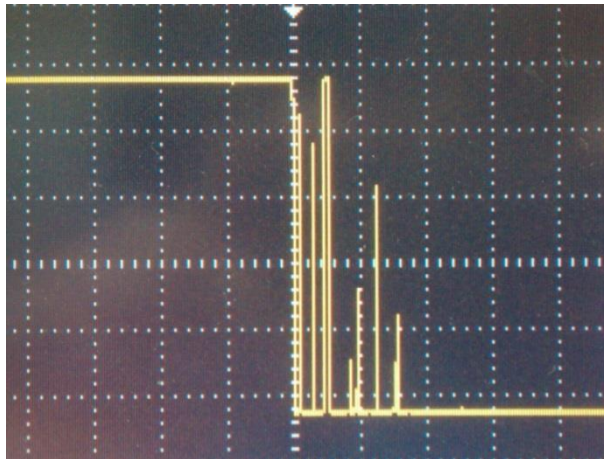
Darlington transistor

3. KUT0128 BORAD 기능과 특징

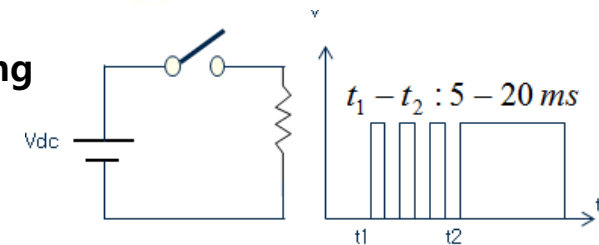
Switch Part

4개의 Tact스위치로 ATmega128의 설정을 어떻게 하느냐에 따라 외부 신호입력과 외부 인터럽트 방식으로 사용이 가능하다.

풀업 저항으로 인해 평상시는 5V상태를 유지하고 있으며 스위치가 ON되면 0V상태가 된다.



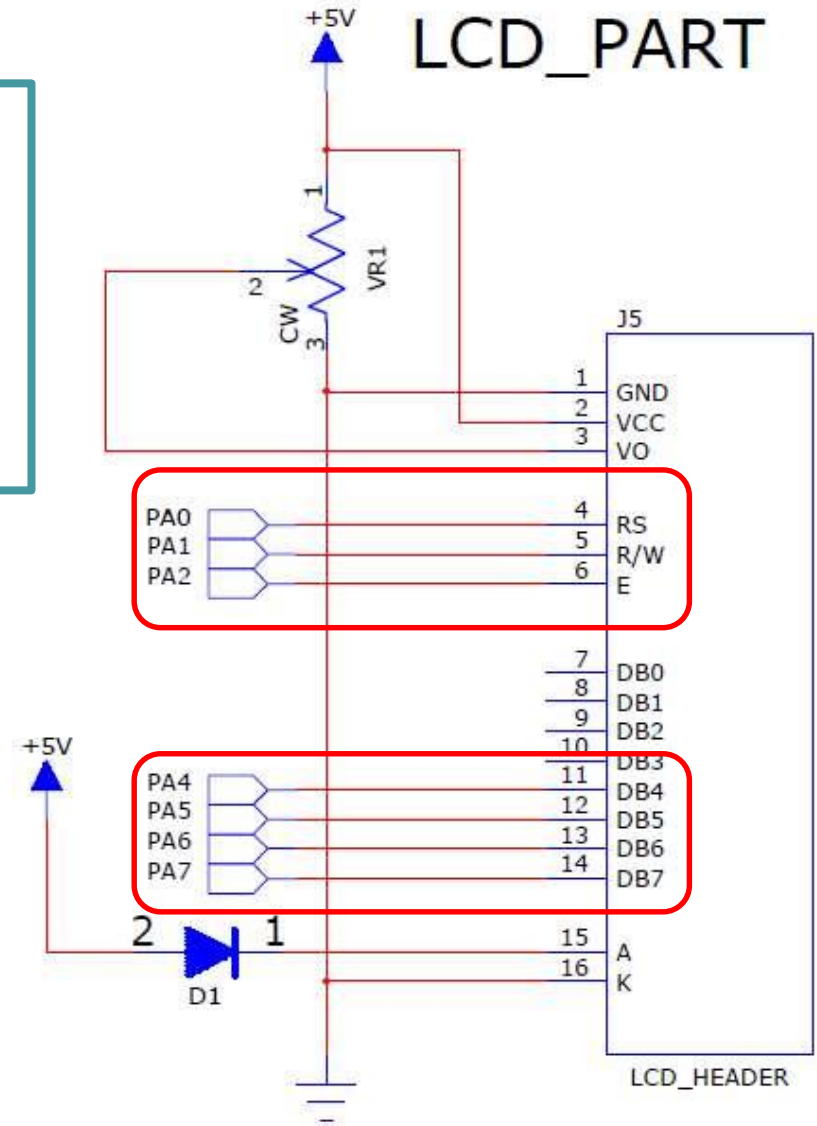
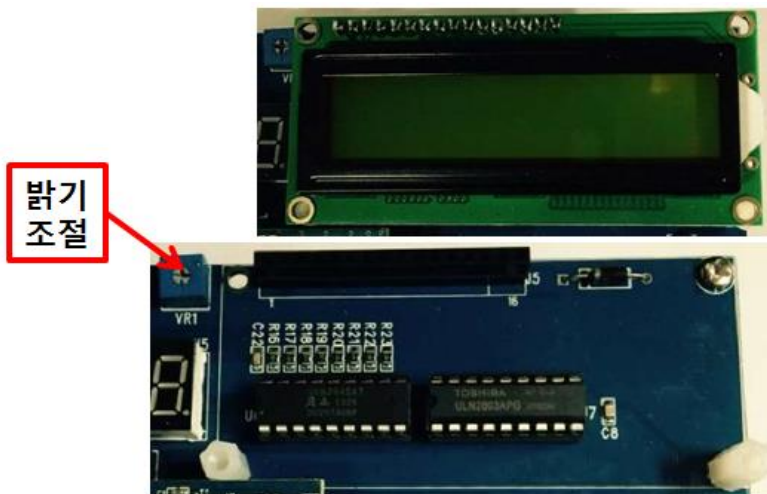
Chattering, Bouncing



3. KUT0128 BORAD 기능과 특징

LCD Part

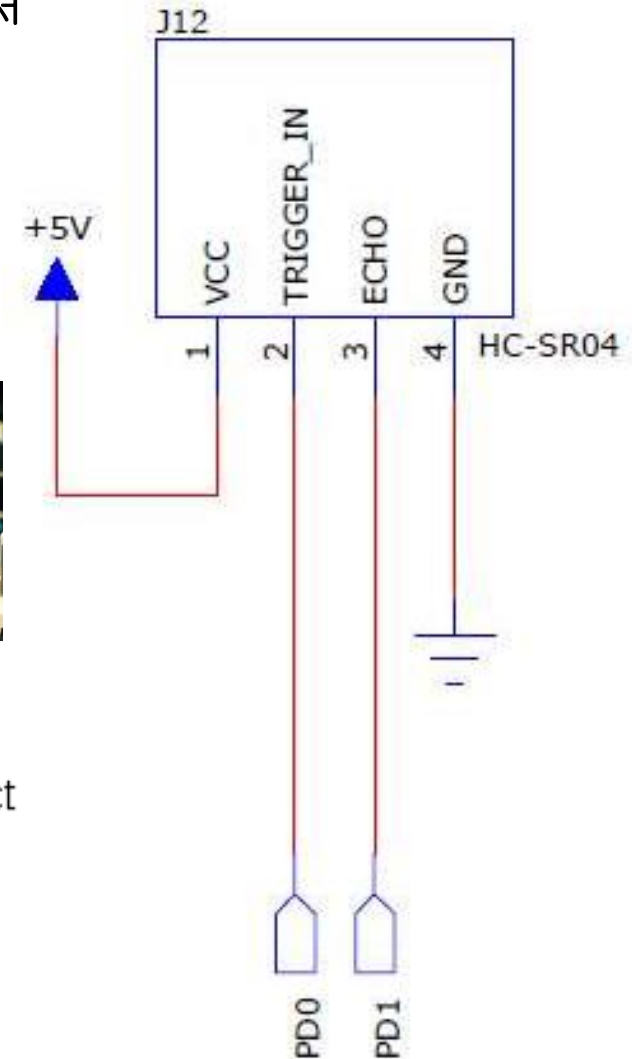
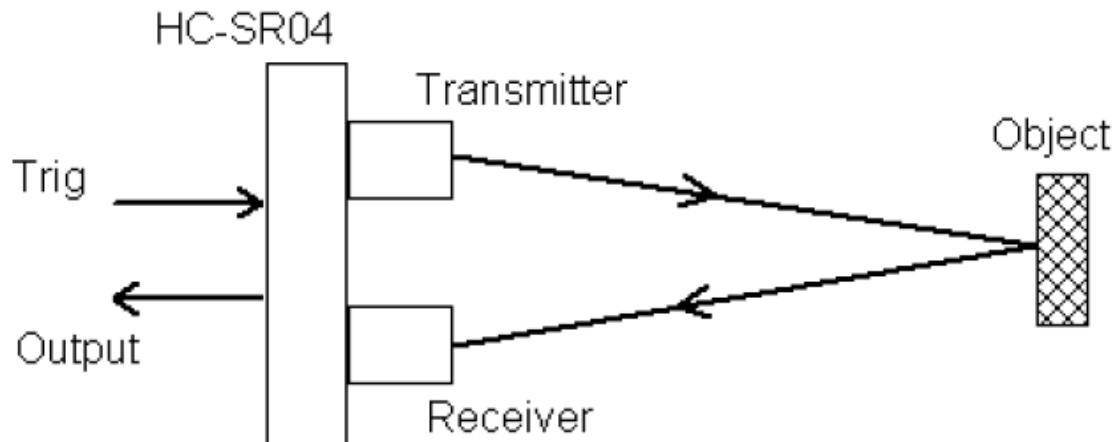
2 X 16개의 텍스트가 표시 가능한 LCD로
데이터 표시용으로 사용하며
8bit모드와 **4bit** 모드로 사용이 가능
하고 여기서는 **4bit** 모드로 사용된다.
VR1를 사용해 **Text의 밝기를 조절**할 수 있다.



3. KUT0128 BORAD 기능과 특징

초음파(Ultrasonic : 4m ~ 2cm) 센서

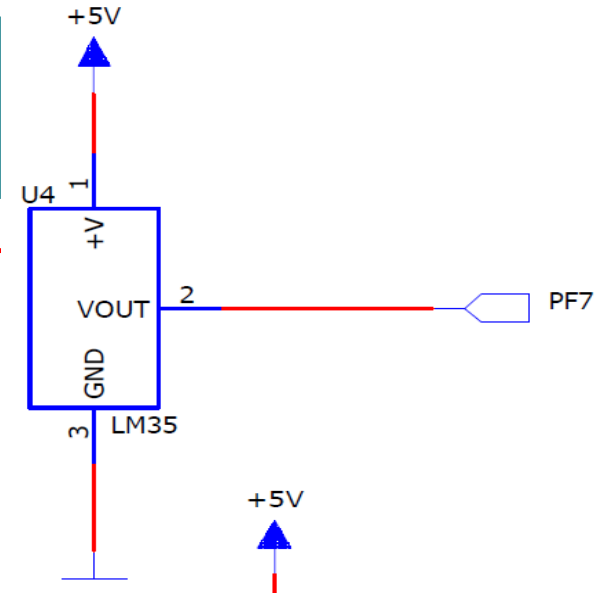
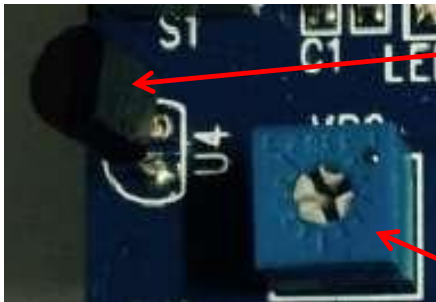
초음파를 이용하여 거리를 측정하거나
사물의 유무를 파악할 수 있는 센서로
2개의 신호 선을 사용 (**HC-SR04**) **30 degree angle**.



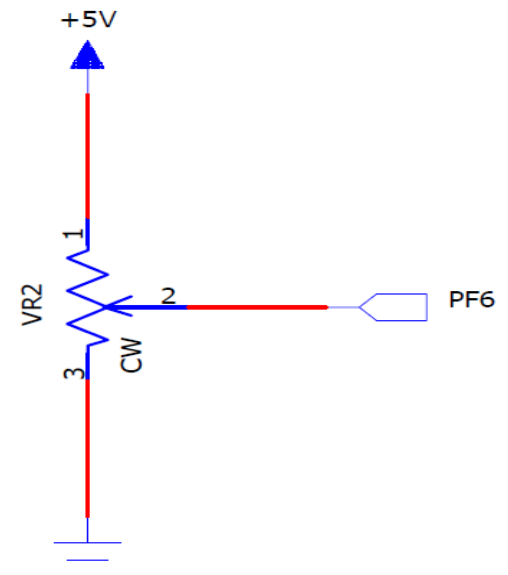
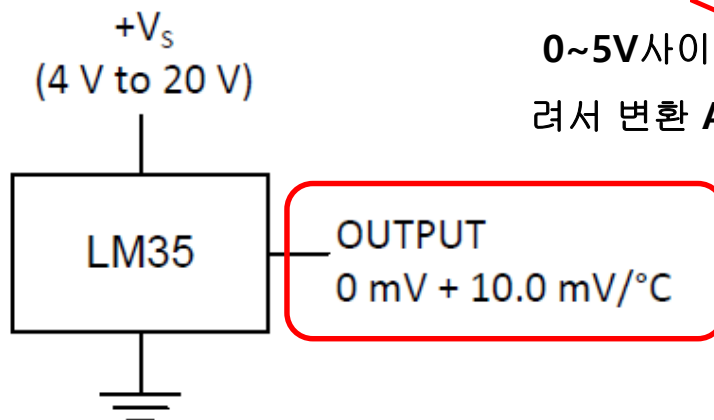
3. KUT0128 BORAD 기능과 특징

가변저항 및 온도 센서 부

LM35(온도센서) IC는 온도의 변화를 전압으로 출력해 주는 센서로 **0~5V전압이 출력**

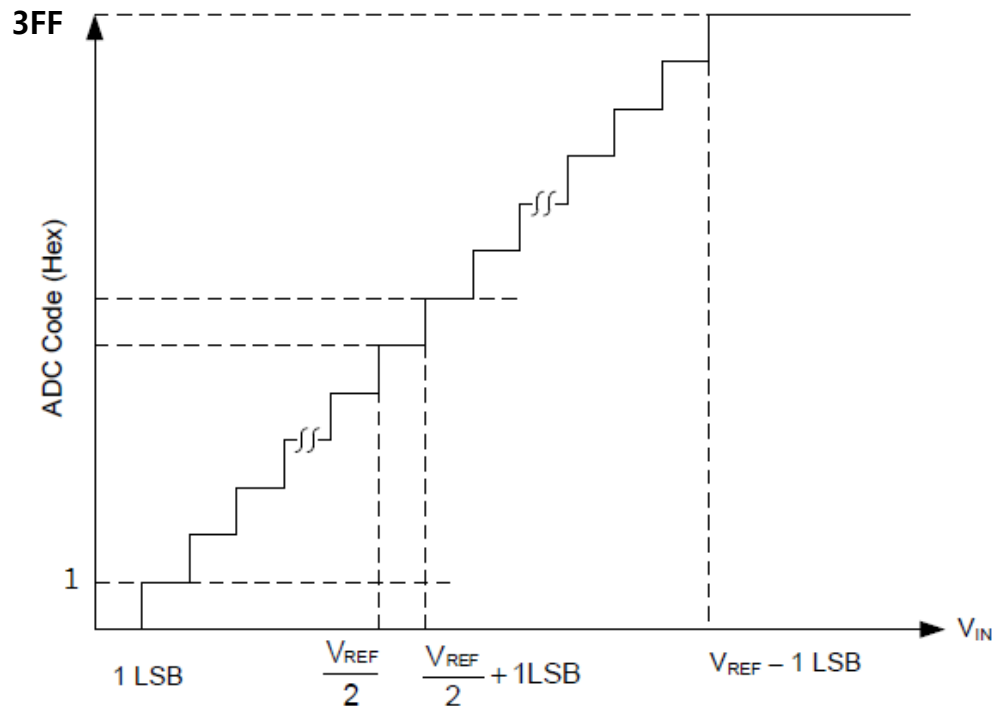


가변저항으로
0~5V사이의 전압을 수동으로 돌려서 변환 ADC TEST



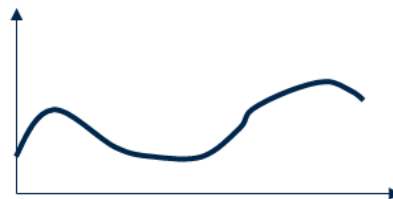
3. KUT0128 BORAD 기능과 특징

ADC (Analog to Digital Converter)

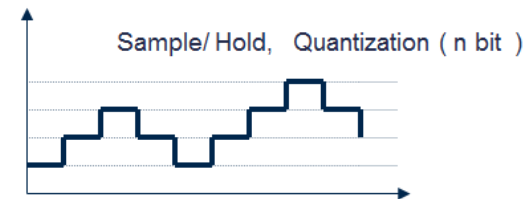


- 10-bit Resolution
- 0.5LSB Integral Non-linearity
- $\pm 2\text{LSB}$ Absolute Accuracy
- 13 - 260 μs Conversion Time

Analog : Continuous



Digital : Discrete



◆ ADC(Analog to Digital Conversion) : Binary (0, 1)
Sample/ Hold, Quantization (n bit)

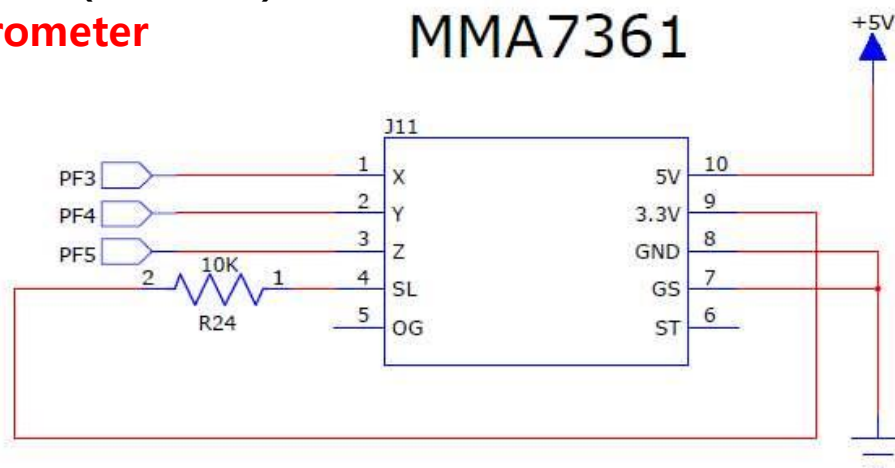
3. KUT0128 BORAD 기능과 특징

3축 가속도 센서부(아날로그)

Accelerometer

3개의 축 (X, Y, Z)의 가속도
변화량을 아날로그 전압값으로
출력해주는 모듈로 ATmega128의
AD변환기 PF3~5까지를 사용

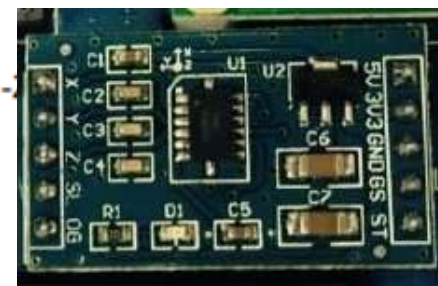
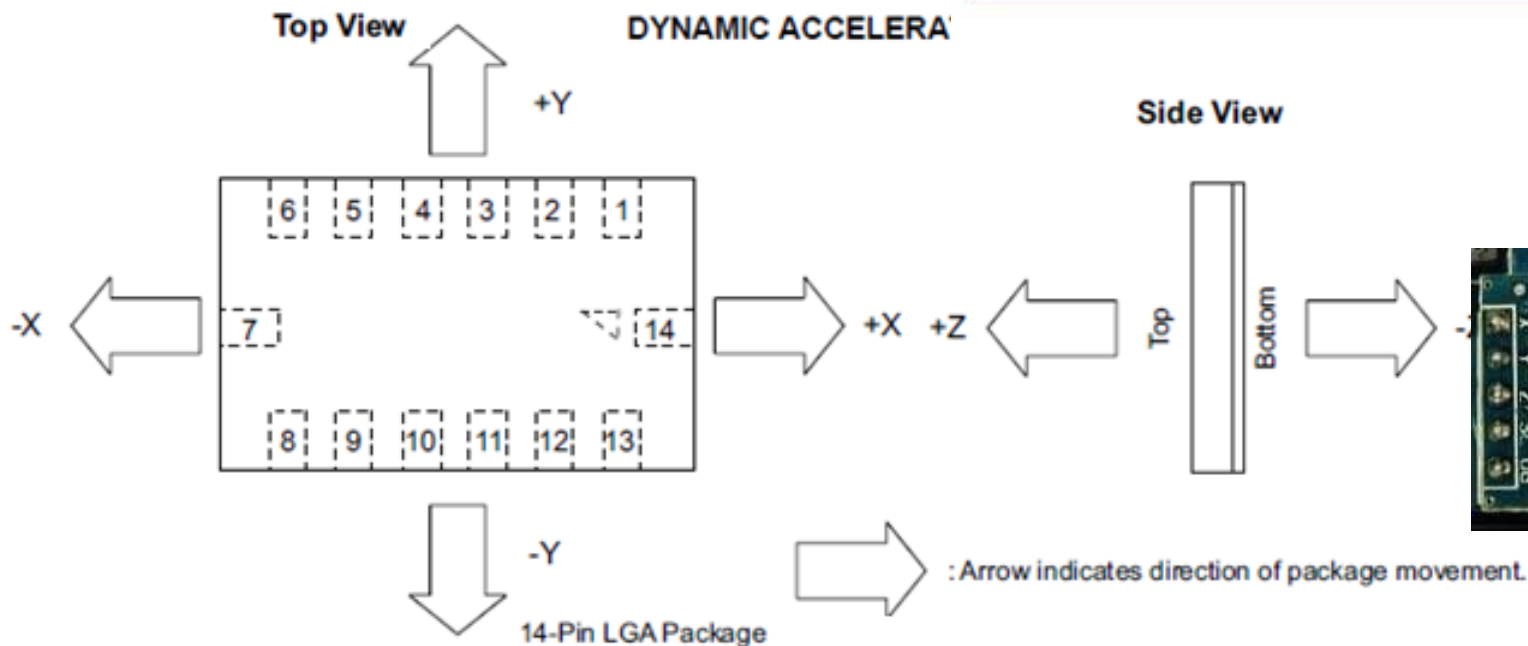
MMA7361



Top View

DYNAMIC ACCELERA

Side View

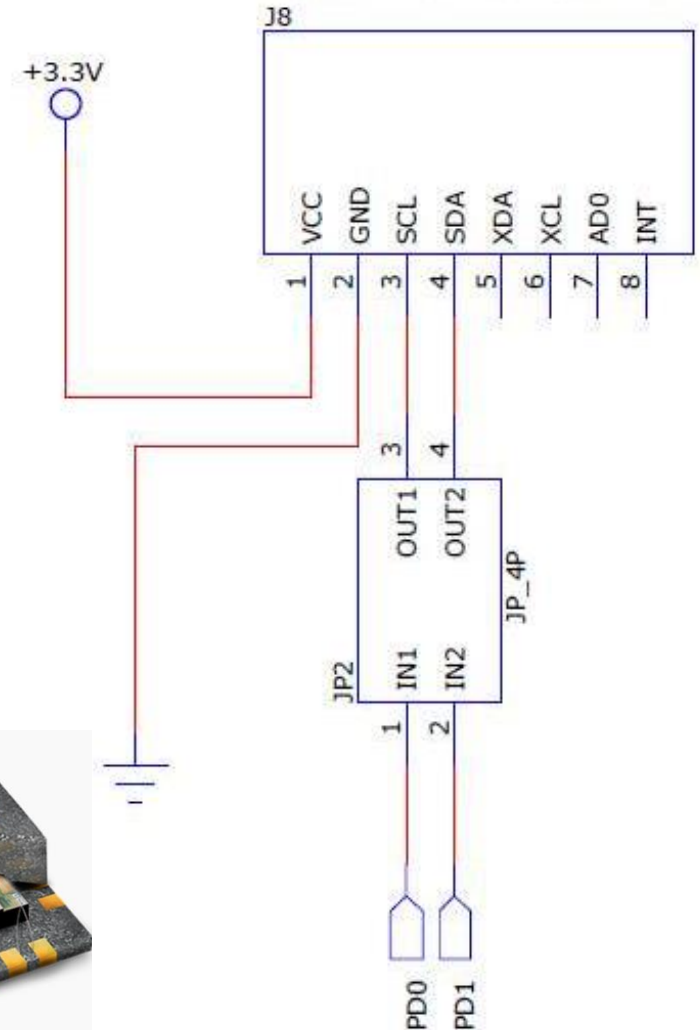
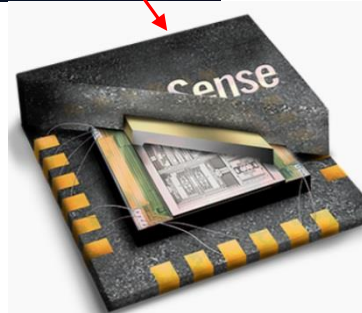
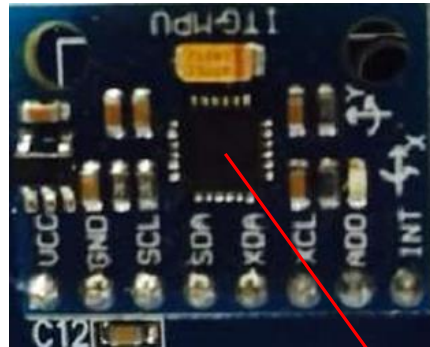
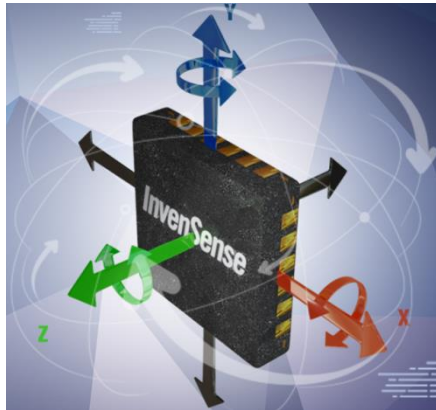


: Arrow indicates direction of package movement.

3. KUT0128 BORAD 기능과 특징

3축 자이로, 가속도 센서 부(디지털)
Gyroscope + Accelerometer (**MPU-6050**) : 6축

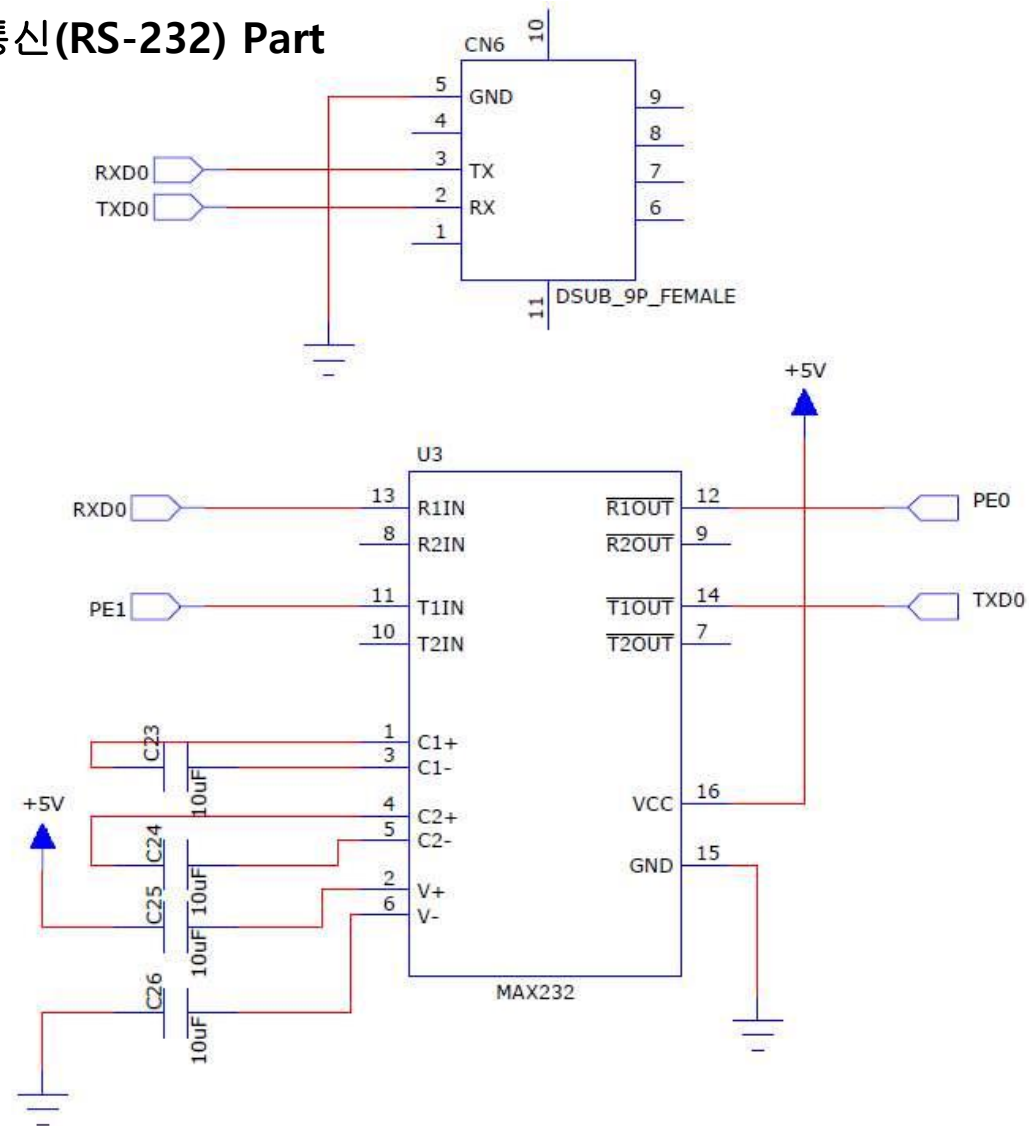
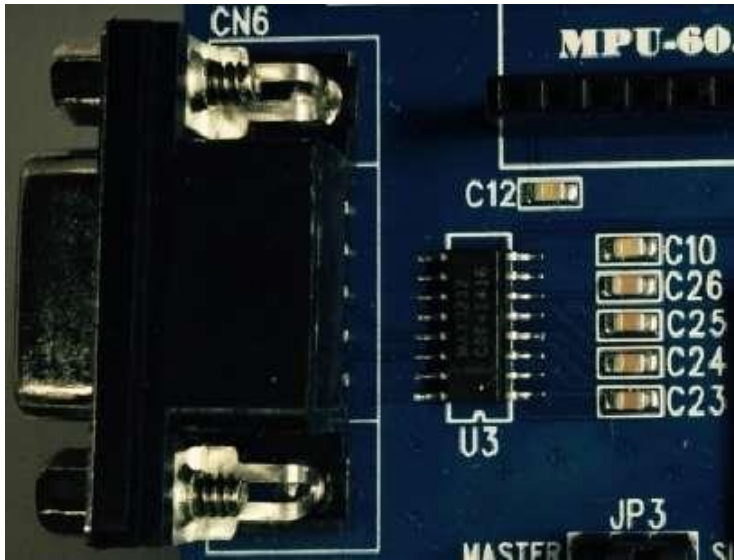
3축 가속도와 자이로 그리고 온도를
측정 할 수 있는 센서로 ATmega128과는
TWI(I2C)를 통해 데이터를 전송하며
칩은 **MPU-6050**을 사용



3. KUT0128 BORAD 기능과 특징

UART 통신(RS-232) Part

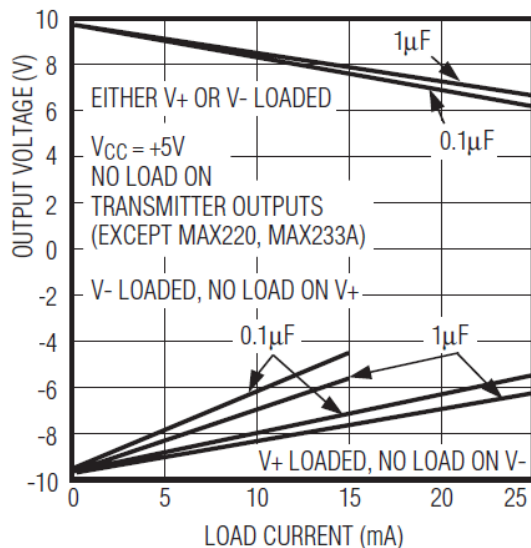
외부 장비 또는 PC등과 통신할 수 있는 기능으로 **USART0** 포트를 **Max232** IC에 연결하여 TTL레벨의 신호를 RS-232레벨로 변화



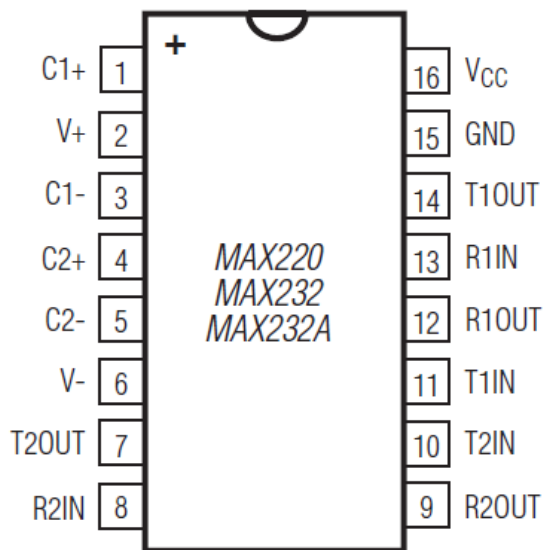
3. KUT0128 BORAD 기능과 특징

UART 통신(RS-232) Part

OUTPUT VOLTAGE vs. LOAD CURRENT

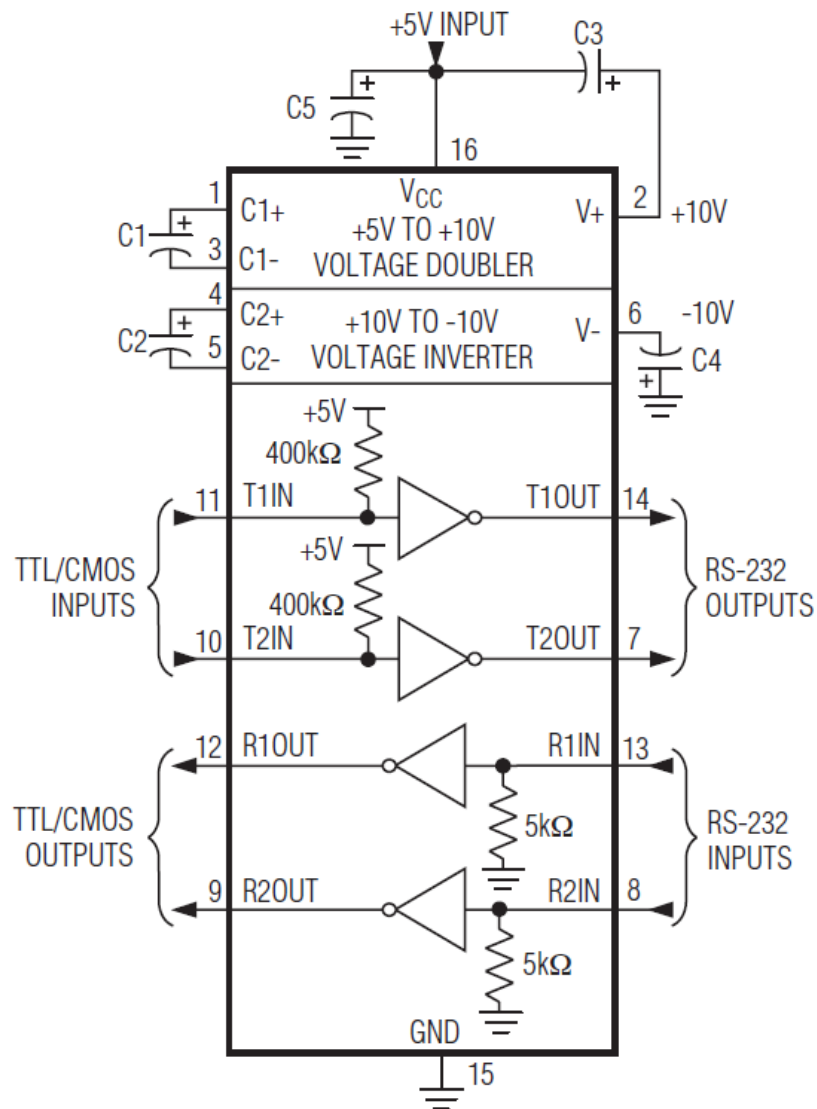


TOP VIEW



DIP/SO

CAPACITANCE (µF)					
DEVICE	C1	C2	C3	C4	C5
MAX220	0.047	0.33	0.33	0.33	0.33
MAX232	1.0	1.0	1.0	1.0	1.0
MAX232A	0.1	0.1	0.1	0.1	0.1



3. KUT0128 BORAD 기능과 특징

Bluetooth Part

(BC04B – Bluetooth 2.1, 2.4GHz~2.48GHz)

<https://www.bluetooth.org/en-us/specification/adopted-specifications>

Bluetooth 2.1 모듈로 USART 1을 통해

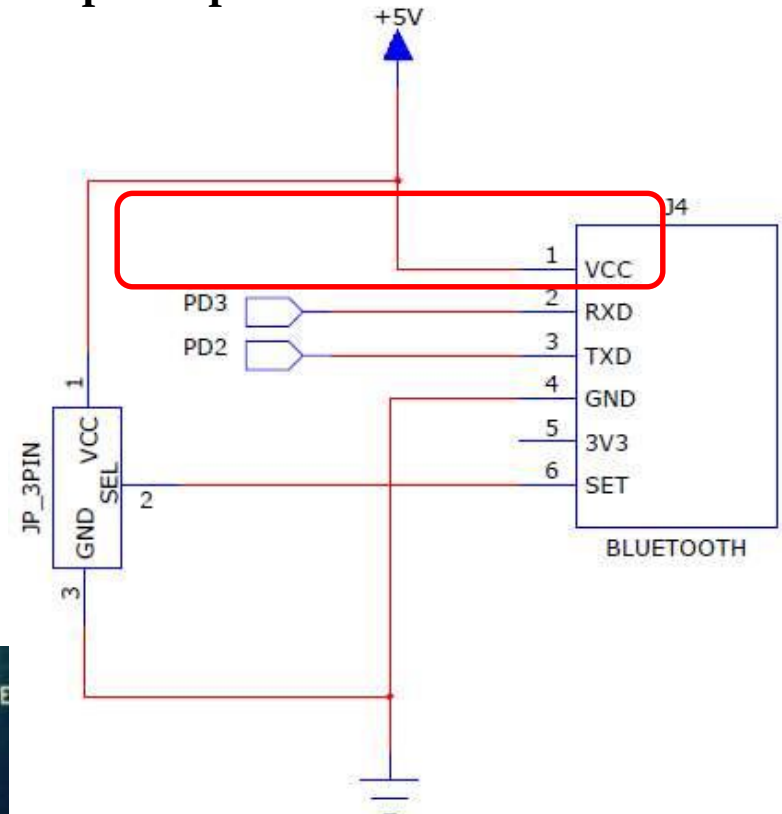
모듈의 **TXD, RXD**에 연결

Wi-Fi 모듈과 함께 사용하지 않는 것을

선호하며 블루투스모듈을 사용할 때는

Wi-Fi 모듈을 제거하는 것이 좋다.

JP3을 사용해 마스터와 슬레이브를 선택
할 수 있음



3. KUT0128 BORAD 기능과 특징

Wi-Fi Part (HLK-RM04: 2.4GHz)

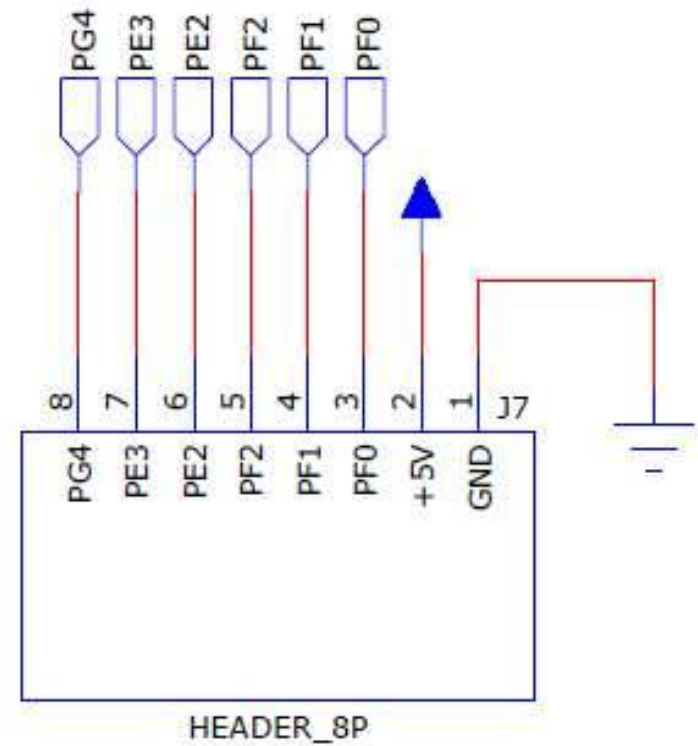
스마트기기 또는 노트북과 통신
하기 위한 **Wi-Fi** 모듈로 설정에
따라 **TCP, UDP** 모드를 사용할 수
있다. 블루투스 와 마찬가지로
와이파이를 사용시 블루투스는
제거하고 사용하기를 권장



3. KUT0128 BORAD 기능과 특징

확장 포트(1)

ATmega128의 64개의 핀 중 다양한 기능에 사용되고 남은 핀을 모아 놓은 커넥터로 **ADC0~3**과 타이머/카운터용 **PWM**이 나와 있어 다양한 모듈이 연결 가능



3. KUT0128 BORAD 기능과 특징

확장 포트(2)

SPI용 확장 모듈로

메인보드의 다양한

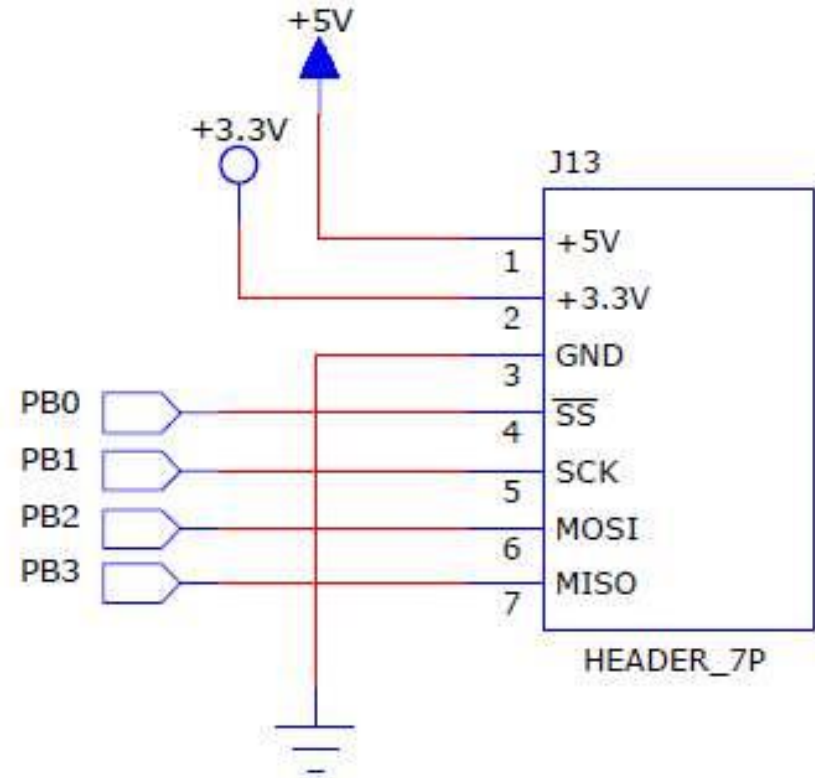
기능 이외에 센서모듈이나

메모리 또는 통신기능을

확장하기 위해 사용

SPI의 기능을 사용하지 않을

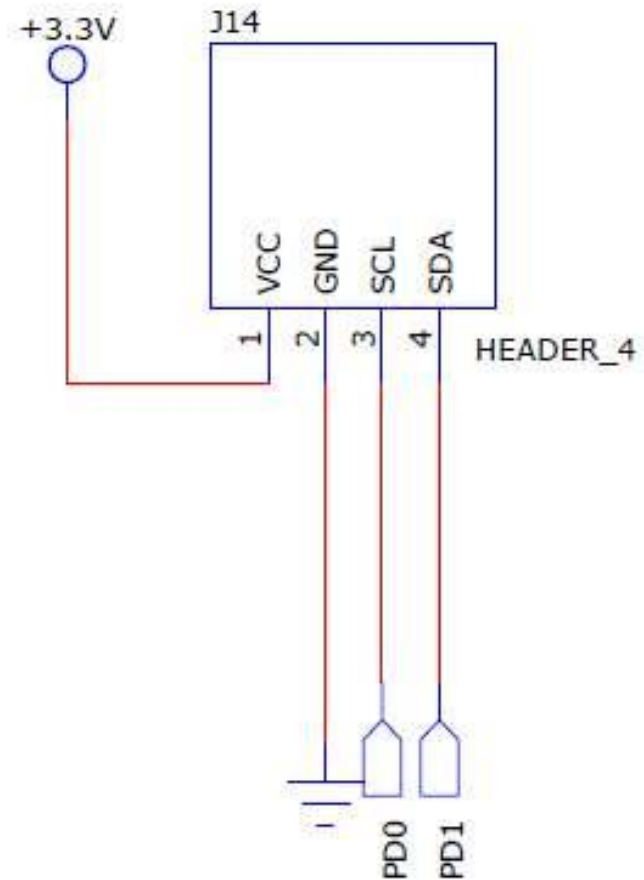
때는 일반 **I/O**포트 사용가능



3. KUT0128 BORAD 기능과 특징

확장 포트(3)

TWI(I2C)용 확장 모듈로
메인보드의 다양한 기능
이외에 센서모듈이나
메모리 또는 통신기능을
확장하기 위해 사용



3. KUT0128 BORAD 기능과 특징

Codevision 설치 및 프로젝트 생성 <http://www.hpinfotech.ro>



Home Products Purchase Contact

CodeVisionAVR V3.52

HP InfoTech presents a new version of the most popular (more microcontrollers.

CodeVisionAVR is the only Integrated Development Environment (CodeWizardAVR) for the AVR8, AVR8X, AVR DA, AVR DB, AVR CodeVisionAVR V3, besides it's own IDE, can now also be used

V3.52 adds support for the new AVR64EA28, AVR64EA32, AVR SSD1306, SSD1309, ST75765, ST7567, UC1608, UC 1701 and complete [revision history](#).

The CodeVisionAVR Advanced version features [Graphic Display](#) ILI9481, ILI9486, ILI9488, RA8875, S6D1121, SSD1289, SSD SED1530, SH1101A (OLED), SH1106 (OLED), SSD1303 (OLED), S1C501C, ST7565, ST7567, ST7735, ST7781, ST7793, T1608, T1609, UC1608, UC1610, UC1701, XG1700 and PCD8551.



HP InfoTech
Embedded tools that
make the difference™

click

Home Products Purchase Contact

CodeVisionAVR V3.52

HP InfoTech presents a new version of the most popular microcontrollers.

CodeVisionAVR is the only Integrated Development Environment (CodeWizardAVR) for the AVR8, AVR8X, AVR DA, AVR DB, AVR CodeVisionAVR V3, besides it's own IDE, can now also be used

ILI9481, ILI9486, ILI9488, RA8875, S6D1121, SSD1289, SSD SED1530, SH1101A (OLED), SH1106 (OLED), SSD1303 (OLED), S1C501C, ST7565, ST7567, ST7735, ST7781, ST7793, T1608, T1609, UC1608, UC1610, UC1701, XG1700 and PCD8551.



HP InfoTech
Embedded tools that
make the difference™

Home Products Purchase Contact

CodeVisionAVR

Integrated Development Environment for the 8-bit Microchip AVR, AVR8X, AVR DA, AVR DB, AVR DD, AVR EA and XMEGA Microcontrollers

Features

Libraries

CodeWizardAVR

LCD Vision

Chip Programmer

Arduino

Terminal

Supported Chips

Revision History

Download

click

- Application that runs under Windows® Vista, Windows 7, Windows 8 and Windows 10, 32-bit and 64-bit
- Easy to use Integrated Development Environment and **ANSI C compatible** Compiler
- Editor with auto indentation, syntax highlighting for both C and AVR assembler, function parameters and structure/union members



Download


[Current Version](#)
[Previous Versions](#)
[Examples](#)
[Documentation](#)

[CodeVisionAVR V3.52 Evaluation](#)

click

Free, 4kbytes code size limited version. Includes also Evaluation version of the [LCD Vision](#) font and image editor. Disabled saving of the generated C source code.

Evaluation Ver은 평가판으로 4Kbyte이하의 코드만 컴파일 이 지원됨.

[CodeVisionAVR V3.52 Commercial](#)

Password protected setup of the commercial version, including the full LCD Vision font and image editor.
Note: An Advanced license is required to use LCD Vision and the color TFT [graphic libraries](#).



필요시, ISP Driver 설치 <https://ftdichip.com/drivers/vcp-drivers/>



For ID and/or product ID or description string are used, it is the responsibility of the product manufacturer to maintain any subsequent WHCK re-certification as a result of making these changes.

[HOME](#)

[PRODUCTS](#) ▾

[APPLICATIONS](#) ▾

[DRIVERS](#) ▾

[SUPPORT](#)

For more detail on FTDI Chip Driver licence terms, please [click here](#).

Currently Supported VCP Drivers:

Subscribe to Our Driver Updates **밑으로 drag**

Operating System	Release Date	Processor Architecture						
		X86 (32-Bit)	X64 (64-Bit)	PPC	ARM	MIPSII	MIPSIV	SH4
Windows (Desktop)*	2021-07-15	2.12.36.4	2.12.36.4	–	2.12.36.4A****	–	–	–
Windows (Universal)***	2021-11-12	2.12.36.4U	2.12.36.4U	–	–	–	–	–

click

감사합니다